
CHIA Documentation

Release HEAD

Berkeley Architecture Research

Sep 01, 2026

GETTING STARTED

1	CHIA Basics	3
2	Installation	5
2.1	Optional extras	5
2.2	Core dependencies	6
2.3	Next steps	6
3	Quickstart: Say Hello (World) with CHIA	7
3.1	What we will do in this tutorial	7
3.2	Requirements	7
3.3	0. Project Setup	7
3.4	1. <code>print()</code> ing Hello from a worker	8
3.5	2. Print Hello in Verilator	11
3.6	3. Asking an LLM to say Hello	13
3.7	4. Editing RTL to say Hello	15
4	Architecture Overview	19
4.1	CHIA loops	19
4.2	CHIA clusters	20
4.3	Fault tolerance & reproducibility	20
4.4	Infrastructure	21
5	CLI Reference	23
5.1	Cluster management	23
5.2	Visualization	23
5.3	Cluster Status	24
5.4	Job Management	24
5.5	Fallback to Ray (<code>chia ray ...</code>)	25
5.6	FireSim	25
6	ChiaFunction	27
6.1	A few Ray concepts	27
6.2	Executing a CHIA node	28
6.3	Collecting results	30
6.4	Worker-side setup and cleanup hooks	31
6.5	Defining your own node	31
6.6	Wrapping actors (<code>chia_actor</code>)	31
6.7	Cross-cutting features	31
6.8	See also	32
7	ChiaTool	33

7.1	A few MCP concepts	33
7.2	Anatomy of a tool	34
7.3	The tool lifecycle	34
7.4	Bridging tools back to nodes	35
7.5	Defining your own tool	35
7.6	Tool placement and resources	36
7.7	Advanced	36
7.8	See also	37
8	Cluster Configuration Reference	39
8.1	Top-level structure	39
8.2	provider	41
8.3	auth	42
8.4	available_node_types	42
8.5	Cloud nodes	44
8.6	Tailnet (tailscale) clusters	47
8.7	A mixed on-prem + cloud example	49
8.8	Multiple heads on a single physical machine	51
8.9	Command execution order	51
9	Setting up logical workers	55
9.1	What a logical worker is	55
9.2	Anatomy of a worker definition	56
9.3	Dockerized vs. non-dockerized workers	56
9.4	Controlling the execution environment	58
9.5	Verifying the environment	59
9.6	Placement and scaling	60
9.7	Lifecycle	60
9.8	Common pitfalls	60
9.9	See also	61
10	Building a CHIA-compatible Docker image	63
10.1	What “CHIA-compatible” means	63
10.2	Extend a CHIA base image with your tool	64
10.3	Extend your tool image with CHIA	64
10.4	Wiring the image into a cluster	65
10.5	Pitfalls	66
10.6	See also	66
11	Authenticating Gemini backends: Antigravity and OpenCode + Vertex AI	67
11.1	Antigravity (agy)	67
11.2	OpenCode with Gemini on Vertex AI	69
11.3	Quick reference	70
12	Caching and Bypass	71
12.1	Tags	71
12.2	Bypass	71
12.3	Cache	73
12.4	Putting it together: populate, then replay	74
13	Profiling	75
13.1	Enabling profiling	75
13.2	What gets recorded	76
13.3	Attaching custom metadata	76
13.4	Visualizing a profile	76

14 Example: Agentic RoCC Accelerator (MemCpy)	77
14.1 What it does	77
14.2 Components	78
14.3 The accelerator contract	78
14.4 Running it	78
14.5 Tunable parameters	79
14.6 Debug feedback	79
14.7 Output and per-iteration diffs	80
15 gem5-BOOM Microarchitecture Alignment	81
15.1 Overview	81
15.2 How it works	81
15.3 Setup	83
16 RISC-V ISA Extension Implementation	87
16.1 Overview	87
16.2 How it works	88
16.3 Setup	90
17 BOOM Critical-Path Timing Optimization	93
17.1 Overview	93
17.2 How it works	94
17.3 Setup	96
18 Agentic Architectural Discovery with Evolutionary Coding Agents	99
18.1 Overview	99
18.2 How it works	100
18.3 Setup	103
19 Autonomous CIRCT Issue Solving	109
19.1 Overview	109
19.2 How it works	110
19.3 Setup	113
20 Base Runtime	115
20.1 Bypass	115
20.2 Cache	118
20.3 Chia Function	120
20.4 Chia Wait	122
20.5 Chia KV Store	123
20.6 Colocated	124
20.7 Dispatch Proxy	126
20.8 PID Registry	127
20.9 LLM Call	127
20.10 MCP tool servers	128
21 Chipyard Nodes	133
21.1 State definitions	133
21.2 Chisel build	146
21.3 Spike build	148
21.4 CIRCT	149
21.5 Macro compiler	154
21.6 Chipyard / Hammer	155
21.7 RISC-V build	157
21.8 RISC-V objdump	158

21.9	RISC-V DV generation	160
21.10	Torture	163
21.11	Verilator run	164
21.12	Spike cosim	167
22	ESP Support	169
22.1	The worker	169
22.2	Simulation	169
22.3	Vivado and the FPGA flow	170
22.4	Resources as license seats	170
22.5	API reference	171
23	Model Backends	181
23.1	Claude	181
23.2	Bedrock	183
23.3	Vertex	184
23.4	Antigravity	186
23.5	Codex	188
23.6	Copilot	189
23.7	Opencode	191
23.8	Openai Compat	194
23.9	Openai Providers	196
23.10	Ollama	197
23.11	vLLM	197
24	FireSim	199
24.1	Config	199
24.2	Chia Functions	205
24.3	State Def	206
25	VLSI	209
25.1	Hammer	209
25.2	SRAM characterization (CACTI)	212
26	Databases	217
26.1	Base	217
26.2	Postgres Node	219
26.3	Sqlite Node	223
27	GitHub	227
27.1	Github Client	227
27.2	Github Issues Node	228
27.3	Github Pulls Node	229
27.4	State Def	230
28	AWS	233
28.1	Config	233
28.2	EC2	233
28.3	Host	234
28.4	S3	235
29	Simulators	239
29.1	gem5	239
30	Analysis	245

30.1	Focal	245
30.2	PFAS	248
31	LLM servers, providers, and agent Nodes	251
31.1	Base class	251
31.2	Providers	251
32	Indices and tables	253
	Python Module Index	255
	Index	257

CHIA is an open-source framework for agile and principled hardware/software co-design research — where humans, agents, and existing hardware-design tools all work together.

New to CHIA? Start with the [CHIA Basics](#) page, or go straight to [Installation](#) to set things up.

The current release of CHIA is considered a beta. If you find any issues, please reach out to us via the [user forum](#) or [contact the lead developers / PI](#) (Angela Cui, Ferran Hermida-Rivera, Jack Toubes, and Sagar Karandikar).

CHIA stands for:

(Co-designing **H**ardware and software with **I**ntelligent **A**gents)

Originally CHIA stood for:

(**CH**ipyard's **I**ntelligent **A**gents)

(**CH**ipyard's **I**ntelligence **A**gency)

CHIA BASICS

CHIA is a framework for **fast and flexible orchestration of agentic hardware design workflows**, letting humans, agents, and hardware-design tools work together smoothly.

A hardware-design workflow is any sequence of steps you need to execute in the process of accomplishing some hardware-design task. For example, a simple workflow may include writing some Verilog, simulating the Verilog with Icarus Verilog or Verilator on some unit tests, and if it passes, synthesizing the RTL to get an area and timing estimate. An agentic workflow may include having an LLM write the Verilog or the unit tests.

CHIA boils a workflow down into two pieces:

- **Loops** — a CHIA loop is a Python script that describes the orchestration of a pipeline of tasks, and the data and control flow between them. These loops form a graph of sorts, so we often call tasks “*nodes*”, and control/data flow between them “*edges*”. Notably, any Python function can be a CHIA node. LLMs and agents are first-class participants — both as nodes in a loop and as orchestrators of the loop. Because loops are just code over a library of (often open-source) nodes, rearranging, scaling up, or completely reinventing a loop takes little more than changing some function calls and loop bounds.
- **Clusters** — a cluster is the collection of computers a loop executes on. You assemble a group of physical machines and, on top of them, define *logical workers* that expose virtualized hardware resources (CPU cores, GPUs, accelerators, FPGAs) and software resources (dependencies, environments, isolation, credentials). Nodes are scheduled onto workers that meet their requirements, and workers can be containerized for isolation, fast setup, and portability.

Beyond speed and flexibility, CHIA prioritizes **fault tolerance** — automatic rescheduling of failed tasks, on-the-fly cluster growth with no downtime, bypassing nodes with cached results for rapid restart, and process-leak prevention through subprocess tracking — and **good science**: thorough profiling, result caching, visualization, the ability to bypass nondeterministic nodes with cached results, and parallel execution of multiple instances of the same experiment.

CHIA is built on the [Ray](#) distributed-computing platform, chosen for its expressive control flow, fine-grained scheduling, and distributed execution and fault tolerance. It weaves in Docker for containerization, MCP (via FastMCP) for agent tools, Boto3 / Google Cloud client libraries for cloud integration, and TensorBoard, Weights & Biases, and GraphViz for profiling and visualization.

INSTALLATION

CHIA can be installed easily using pip. We recommend using CHIA in a conda environment (see the [Miniconda install guide](#)). If you plan on using any of our provided docker containers, you should pin your conda environment to Python version 3.10.19, in order to match the python version used inside the Dockerized workers.

The easiest way to get that is a dedicated conda environment. We recommend naming the conda environment the same thing on all machines you plan on using CHIA for, as it makes the setup of CHIA clusters easier. We will assume throughout these docs that this environment is called `chia_env`.

```
conda create -n chia_env python=3.10.19
conda activate chia_env
```

Clone the CHIA repository onto any machines you plan on using

```
git clone https://github.com/ucb-bar/chia.git
```

Then install the package in editable mode from your clone:

```
pip install -e /path/to/chia
```

This installs the `chia` package and its dependencies. We plan on releasing a PyPI CHIA release in the future.

Note

Users in the `ucb-bar` organization should also clone the internal node library `ucb-bar/chia-internal-nodes`. Install `ucb-bar/chia-internal-nodes` in `chia_env` using `pip install -e /path/to/chia-internal-nodes`. We encourage you to organize your project with the following structure:

```
your_repo
├── chia-internal-nodes
└── chia
```

2.1 Optional extras

Extra	Installs	Use for
tensorboard	tensorboardX, tensorboard	metrics logging
wandb	wandb	metrics logging
metrics	both of the above	metrics logging
postgres	psycopg[binary]	a Postgres-backed DatabaseNode

```
# For one extra
pip install -e "/path/to/chia[metrics]"

# For multiple extras
pip install -e "/path/to/chia[metrics,postgres]"
```

2.2 Core dependencies

Chia pins `ray[default]==2.54.0` along with, `mcp`, `pydantic`, `fastapi`, `boto3`, and the Google GenAI / Vertex client libraries. See `pyproject.toml` for the full pinned set.

2.3 Next steps

- *Quickstart* — run the Hello World! example.

QUICKSTART: SAY HELLO (WORLD) WITH CHIA

In this brief introduction to using CHIA, we explore 4 ways you can use CHIA to say Hello (World)!

3.1 What we will do in this tutorial

As we describe in *chia-basics*, a CHIA workflow consists of two parts: a loop and a cluster. The loop is a Python program which orchestrates tasks. The cluster is the compute substrate on which a loop executes.

In this tutorial, we will design a simple CHIA workflow (both loop and cluster) to say Hello.

First, we will build a very simple cluster (only requiring a single machine, with the option to use multiple) with a few worker nodes from [Chipyard](#), and write a loop which prints “Hello World!” from a worker in the cluster.

Second, we will use that simple cluster to build a verilog simulator of an in-order RocketChip core in Chipyard, compile a C program which prints “Hello World!” for the RocketChip core, and run the program in our Verilog simulator.

Third, we will add an OpenCode AI agent to the cluster and loop, and ask it to say “Hello World!”

Fourth, and finally, we will ask the AI agent to edit our RocketChip’s RTL using an MCP tool. We will ask it to add a line to the RocketChip source which prints to the Verilog output “Hello World!” after 1000 instructions have been committed by the processor.

3.2 Requirements

For this tutorial you need a Linux machine with an SSH server. It needs to be able to use public key authentication, and you will need a key set up for SSHing into the machine. This machine must have [Docker](#) installed. Additionally, this machine must have CHIA *installed*.

Optionally, if you have multiple machines available to you on the same LAN (for example, multiple research computers at a single university), there will be sections of this tutorial which allow you to see CHIA’s ability to spread compute across multiple physical machines. Each of these machines must have Docker installed, but does not need CHIA installed.

The `chia/examples/hello-world` directory contains checkpoints of the code after each step.

3.3 0. Project Setup

Start by creating a directory for the project on your machine. Open a shell in that directory and activate the conda environment where you have installed CHIA. We will assume in this tutorial that this environment is named `chia_env`.

If you have not yet set up the `chia_env` environment, create it and install CHIA. CHIA requires **Python 3.10.19** (matching the Python in the Docker images).

```
conda create -n chia_env python=3.10.19
conda activate chia_env
pip install -e /path/to/chia
```

Then create a directory for the project and activate the environment:

```
mkdir chia-hello-world
cd chia-hello-world
conda activate chia_env
```

You should assume that, unless otherwise specified, all commands in this tutorial are run from within the `chia_env` conda environment and in the project's directory.

3.3.1 Fix default `.bashrc`

We need various parts of the `~/.bashrc` file to execute even in non-interactive mode. To do so, edit your `~/.bashrc` file so that the following section is removed:

Edit your `~/.bashrc` file so that the following section is removed:

```
# If not running interactively, don't do anything
case $- in
    *i*) ;;
    *) return;;
esac
```

3.4 1. `print()` ing Hello from a worker

CHIA clusters are described in `.yaml` format. We will start by creating a file called `cluster.yaml`.

First, we need to add the `provider` key. The `provider` key is where we specify the head machine of the cluster. Provider only requires the subkey `head_ip`.

`head_ip` specifies the hostname or IP address of the physical machine from which the cluster will be managed. You should fill in this key with according to the hostname or private IP address of the machine on which you installed CHIA. CHIA always runs the head node on an existing machine; cloud workers are added via `aws_nodes` / `gcp_nodes` (not covered in this tutorial).

There is no `worker_ips` key: the machines eligible to host workers are taken from the `compatible_ips` you list on each node type below, so every worker machine is named exactly once next to the workers that run on it.

```
provider:
  head_ip: ${THIS_MACHINE}
```

CHIA fills variables specified with `${VARIABLE}` in the configuration from environment variables if they are defined, so you could copy this code block directly and just export a value for `${THIS_MACHINE}` as an environment variable.

Next, we need to add the authentication credentials we will use to bring the cluster up. The `auth` key needs two subkeys: `ssh_user` and `ssh_private_key`. `ssh_user` is the username you use to login to the physical machines in the cluster, and `ssh_private_key` is a key for authenticating your user. If your keys are saved in your SSH agent, you do not need to specify `ssh_private_key`. CHIA allows you to override the username and key on a per machine basis.

```
auth:
  ssh_user: ${USER}
  ssh_private_key: ~/.ssh/<ssh_key_name> # Comment if credentials are in SSH agent
```

(continues on next page)

(continued from previous page)

```
overrides: # Not needed for this tutorial
  <hostname>:
    ssh_user: otheruser
```

Now, we will specify our logical workers to map onto our physical machines. As described above, we will need a worker for Chipyard, so let's add that into our configuration.

```
available_node_types:
  hello_chipyard:
    resources: {"chipyard": 1}
    num_workers: 1
    compatible_ips: [${THIS_MACHINE}] # Or [Another_host]
    worker_setup_commands: ["source /home/ray/chipyard/env.sh"]
    docker:
      image: "ghcr.io/ucb-bar/chia-chisel-build:latest"
      container_name: "chia-chipyard-${USER}"
      run_options:
        - --shm-size=10.24gb # More shared memory
      pull_timeout: 7200 # default is 600 seconds but chia-chisel-build is large
```

Node types are specified under the `available_node_types` key, where subkeys are the names of different node types. Below, we introduce a node type named `hello_chipyard`. This node exposes a resource named “chipyard” broadcasting to the program that it has chipyard available. We will look at how this is leveraged later in this tutorial. We also specify the number of workers we want of this type (1), the IP addresses/hostnames of machines which can support this worker type (your head node machine or optionally another machine you have available to you), and a set of commands which are run before executing any tasks on this worker (in this case we source the chipyard environment shell script).

Finally, we also specify a docker image (we will use CHIA provided Docker images for Chipyard and Verilator), a `container_name`, arguments to pass to Docker during container construction (`run_options`), and in the Chipyard case, a large `pull_timeout` because the Chipyard container is fairly large. Container names are appended with numbers so that multiple workers can be constructed of a given type without any container naming conflicts. When multiple CHIA users are sharing a system, we recommend including your username in the container name.

Let's also add a Verilator node.

```
available_node_types:
  # ...

  hello_verilator:
    resources: {"verilator_run": 1}
    num_workers: 1
    compatible_ips: [${THIS_MACHINE}] # Or [Another_host]
    docker:
      image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
      container_name: "chia-verilator-${USER}"
      run_options:
        - --ulimit nofile=65536:65536
        - --shm-size=10.24gb
```

The final piece for the cluster is specifying commands which are run automatically at different moments when bringing up the cluster and running jobs on the cluster.

`head_env_commands` are run to set up the environment for all job's run on the head node.

`head_setup_commands` are run to set up the environment on the head only when bringing up the cluster

`head_start_ray_commands` are run to start Ray on the head node.

`worker_start_ray_commands` are run to start Ray on the worker nodes.

In general, these commands should source your shell environment scripts (like `~/ .bashrc`), and activate your conda CHIA environment as shown below. When activating Ray, setting `--dashboard-agent-listen-port=0` ensures that the Ray dashboard agent will choose an open port instead of failing to open an in-use port, avoiding conflicts when multiple CHIA clusters have workers on the same machines. Currently, CHIA only supports one head node per physical machine, but a machine can host workers in many separate CHIA clusters.

```
head_env_commands: ["source ~/.bashrc && conda activate chia_env"]
head_setup_commands: ["source ~/.bashrc && conda activate chia_env"]
head_start_ray_commands:
- "source ~/.bashrc && conda activate chia_env && ray stop"
- "source ~/.bashrc && conda activate chia_env && \
    ray start --head --port=6379 \
    --include-dashboard=True --dashboard-agent-listen-port=0"

worker_start_ray_commands:
- "ray stop"
- "ray start --address=$RAY_HEAD_IP:6379 --dashboard-agent-listen-port=0"
```

That's our cluster! With all of these keys in our `cluster.yaml`, we can now run the following command to create the cluster.

Since our configuration references `${THIS_MACHINE}`, first export it to the head machine's IP address:

```
export THIS_MACHINE=$(hostname -I | awk '{print $1}')
echo $THIS_MACHINE # This should be the IP address of the machine you are running on
```

Then bring up the cluster:

```
chia up cluster.yaml
```

Now let's write the simplest CHIA loop we could write: a loop that dispatches a `print()` statement onto any available worker in the cluster. Create a script called `hello-world.py`, and add the following lines:

```
from chia.base.ChiaFunction import ChiaFunction, get

@ChiaFunction()
def print_hello_world():
    print("Hello World (#1) from a remote call!")
```

Here we import the `ChiaFunction` decorator, and the `get` function. `@ChiaFunction` is how we declare that a function is a CHIA node. It gives us the ability to schedule the function onto the cluster, using `fn.chia_remote()`, as shown below. Additionally, any function annotated with `@ChiaFunction`, even when not called using `fn.chia_remote()`, is profiled by CHIA's *profiler*. Next, add this to the python file:

```
def main():
    get(print_hello_world.chia_remote())

if __name__ == "__main__":
    main()
```

`chia_remote()` is non-blocking and returns a tag which is used to eventually wait for the function to finish and collect its results. The easiest way to wait for and collect these results is with `get()`, as we do above.

That's the full script. We can schedule this script onto the cluster using the following command:

```
chia job submit --working-dir . -- python hello-world.py
```

In the output you should see lots of messages from the cluster, including a line towards the end of the run of the script which says:

```
(print_hello_world pid=979878) Hello World (#1) from a remote call!
```

3.5 2. Print Hello in Verilator

In this step, we will show how you can leverage CHIA to easily use existing hardware design tools. Our example uses Chipyard and Verilator. Specifically, we will add the following three things into our CHIA loop:

- (1) Compiling a single-file C program to run on RocketChip using a custom CHIA node.
- (2) Elaborating a RocketChip core and building a Verilator simulator for it using a provided CHIA node.
- (3) Running our C program on our RocketChip core in the Verilator simulator using a provided CHIA node.

We start by writing the simple C program we will use called `helloworld.c`, which just prints “Hello World (#2) from Verilator!”

```
#include <stdio.h>

int main(void)
{
    printf("Hello World (#2) from Verilator!\n");
    return 0;
}
```

We import CHIA's pre-made Chipyard Chisel Build and Verilator nodes. We will also import some other libraries we will use in this section.

```
from chia.base.ChiaFunction import ChiaFunction, get # Already here
from chia.chipyard.chisel_build_node import *
from chia.chipyard.verilator_run_node import *
from pathlib import Path
import subprocess
from ray import ObjectRef # Only for type annotations
```

Next, we create a new CHIA node for compiling single-file C programs to run on our RocketChip core. CHIA provides a node for doing this, but we include it here for instructive purposes.

```
@ChiaFunction(resources={"chipyard": 1})
def compile_program(src_contents : str) -> bytes:

    with open("temp.c", "w", encoding="utf-8") as src:
        src.write(src_contents)

    cmd = []
    cmd += ["/home/ray/chipyard/.conda-env/riscv-tools/bin/riscv64-unknown-elf-gcc"]
    cmd += ["-static"]
```

(continues on next page)

(continued from previous page)

```

cmd += ["-specs=htif_nano.specs"]
cmd += ["temp.c", "-o", "temp.riscv"]

subprocess.run(
    cmd,
    capture_output=True,
    text=True,
)

with open("temp.riscv", "rb") as elf:
    elf_data = elf.read()

return elf_data

```

`compile_program` is annotated with `@ChiaFunction`, indicating that we want to be able to schedule it onto our cluster. In addition, it specifies `resources={"chipyard": 1}`, meaning that it must be scheduled onto a worker with an available exposed resource named `chipyard`, and will consume 1 of that resource for the duration of it's running. Above, when creating our cluster, we defined our `hello_chipyard` worker with a resource called `chipyard`, so this node will be able to run on that worker.

`compile_program` takes as input the contents of a C source file. It writes the contents into a temporary file called `temp.c` on the worker, and then compiles it using a subprocess, into a file called `temp.riscv`, the contents of which are returned by the function.

Next, in our main function, we add the following code.

```

CHIPYARD_PATH = "/home/ray/chipyard" # Our Docker container's Chipyard path
CHIPYARD_CONFIG = "RocketConfig"

def main():
    get(print_hello_world.chia_remote()) # Already here

    # First new block
    c_src = Path("helloworld.c").read_text(encoding="utf-8")
    testBinFuture : ObjectRef[bytes] = compile_program.chia_remote(c_src)

    # Second new block
    cb_node = ChiselBuildNode(
        CHIPYARD_PATH, CHIPYARD_CONFIG, target = BuildTarget.VERILATOR
    )
    buildFuture : ObjectRef[BuildArtifact] = cb_node.build.chia_remote(cb_node)

    # Third new block
    verilator_node = VerilatorRunNode()
    run_output : RunResult = get(
        verilator_node.run.chia_remote(
            verilator_node, buildFuture, testBinFuture,
            "helloworld.riscv", "/home/ray"
        )
    )

    print("Output of Verilator run (run_output.log): ")
    print(run_output.log)

```

The first new block reads in our C program and compiles it using the `compile_program` node we just made.

The second creates a `ChiselBuildNode` for a `RocketConfig` targeting a Verilator simulator, and builds the Verilator simulator.

The third block takes the results from the first two blocks, and runs our Verilator simulation in the `home/ray` directory on our `hello_verilator` worker, naming the passed in test binary `helloworld.riscv` for profiling purposes. Finally, we print the output (returned in `run_output.log`).

You may notice that we don't call `get()` on the returns of the `chia_remote()` calls for compiling and building. This is because `chia_remote()` calls can wait for and collect the final results from past remote function executions. Any argument to a `chia_remote()` call can be of the parameter's type `T` or can be an `ObjectRef[T]`.

If you submit this script as above to the cluster, the output should now include the following (including our print from our C file):

```
(print_hello_world pid=996505) Hello World (#1) from a remote call!
Output of Verilator run (run_output.log):
Hello World (#2) from Verilator!
[UART] UART0 is here (stdin/stdout).
- /home/ray/chipyard/sims/verilator/generated-src/chipyard.harness.TestHarness.
↪RocketConfig/gen-collateral/TestDriver.v:179: Verilog $finish
```

Note that this script will take a few minutes to build the `RocketConfig` and run Verilator. If you are working on a machine where you can open a web browser, you can monitor progress using the Ray dashboard by opening <http://127.0.0.1:8265/#/overview>. Alternatively, you can forward this address to another computer and look at it in your browser there (VSCode provides support for this when working in it's Remote SSH extension).

3.6 3. Asking an LLM to say Hello

In this step, we add a new worker to the cluster which will provide the OpenCode CLI. `OpenCode` is a free and open source coding agent which comes with some free to use models. We will ask OpenCode to say "Hello World!".

First, we will add a new `hello_opencode` worker type to our `cluster.yaml` configuration which uses our CHIA provided OpenCode docker image. The following block should be sufficient.

```
available_node_types:
  # other worker types ...

  hello_opencode:
    resources: {"opencode_creds": 1}
    worker_setup_commands: ["source ~/.bashrc"]
    num_workers: 1
    compatible_ips: [${THIS_MACHINE}]
    docker:
      image: ghcr.io/ucb-bar/chia-opencode:latest
      container_name: "chia-opencode-${USER}"
```

We can expand our already running cluster with the following command.

```
chia up --add cluster.yaml
```

This version of `chia up` looks at the already running cluster, and compares it to the current configuration in `cluster.yaml`. CHIA will attempt to bring up any workers which were not yet instantiated or which have died.

Next, we will add a new import and a small block to our script which prompts OpenCode using the free `big-pickle` model.

```

from chia.base.ChiaFunction import ChiaFunction, get
from chia.chipyard.chisel_build_node import *
from chia.chipyard.verilator_run_node import *
from pathlib import Path
import subprocess
from ray import ObjectRef
from chia.models.opencode import * # New line
# ...

```

```

def main():

    get(print_hello_world.chia_remote()) # Already here

    # YOUR JOB: ADD THIS CODE BLOCK

    # ===== New code starts here =====
    llm = OpenCodeLLM(model="opencode/big-pickle")
    resp : QueryResult = get(llm.prompt.chia_remote(llm,
        "Can you please respond exactly \"Hello World (#3) from OpenCode!\"")
    ))
    print("LLM Responded:")
    print(resp.result)
    # ===== New code ends here =====

    # Already here
    c_src = Path("helloworld.c").read_text(encoding="utf-8")
    testBinFuture : ObjectRef[bytes] = compile_program.chia_remote(c_src)

```

You can now submit the job. You can let it run to completion again if you want, but if you want to stop it early, you can use the following. First, give the job a submission id when you submit it:

```
chia job submit --submission-id RUNSTEP3 --working-dir . -- python hello-world.py
```

Then, after you've seen the LLM's response, you can stop tailing the log of the run with CTRL+C and stop the job completely by running the following command:

```
chia job stop RUNSTEP3
```

Note that the job will continue running to completion unless you explicitly stop it.

In the output of this run, you should now see the following lines:

```

(print_hello_world pid=1057797) Hello World (#1) from a remote call!
LLM Responded:
Hello World (#3) from OpenCode!
Output of Verilator run (run_output.log):
Hello World (#2) from Verilator!
[UART] UART0 is here (stdin/stdout).
- /home/ray/chipyard/sims/verilator/generated-src/chipyard.harness.TestHarness.
↪RocketConfig/gen-collateral/TestDriver.v:179: Verilog $finish

```

3.7 4. Editing RTL to say Hello

Finally, we get to tie all of this together with a nice demonstration of a very natural use-case for CHIA: LLMs writing RTL. Specifically, we are going to ask an LLM to have our RocketChip core print “Hello World (#4) from instruction 1000!” when the core has retired it’s 1000th instruction.

In our example, let’s start with the following new import, the BashTool.

```
from chia.base.ChiaFunction import ChiaFunction, get
from chia.chipyard.chisel_build_node import *
from chia.chipyard.verilator_run_node import *
from pathlib import Path
import subprocess
from ray import ObjectRef
from chia.models.opencode import *
from chia.base.tools.BashTool import * # New line
# ...
```

In CHIA, just like any function can be a node, any function can be made into an MCP tool, by registering that function with an object of type ChiaTool. In this tutorial we do not create any new custom MCP tools. Any ChiaTool can be passed to any agent/LLM prompts in CHIA.

A BashTool is an MCP tool (its a child class of ChiaTool) which provides an agent/LLM with a bash interface to a specific worker.

We can instantiate the BashTool like this. We give it the name “chipyard_bash”, set it’s working directory to CHIPYARD_PATH, and set it’s task options so that it lands on our hello_chipyard worker by specifying that it requires a chipyard resource.

```
def main():
    get(print_hello_world.chia_remote())

    llm = OpenCodeLLM("opencode/big-pickle")
    resp : QueryResult = get(llm.prompt.chia_remote(
        llm, "Can you please respond just \"Hello World from OpenCode!\""))
    print("LLM Responded:")
    print(resp.result)

    # ===== New code starts here =====
    chipyBash : BashTool = BashTool(
        "chipyard_bash", CHIPYARD_PATH,
        task_options={"resources": {"chipyard": 1}}
    )
```

Let’s write a prompt for the LLM and query it, passing in our chipyTool to the tools argument:

```
LLM_RTL_PROMPT = "" \
"Can you use the chipyard_bash tool to " \
"edit chipyard's generators/rocket-chip " \
"Chisel source and add a new print to the " \
"Chisel which says \"Hello World (#4) from " \
"instruction 1000!\" when instret reaches " \
"1000? If any message already exists, add " \
"the word \"again\" at the end of the existing " \
"message. Please respond explaining your change."
```

(continues on next page)

(continued from previous page)

```
def main():
    # ...

    chipyBash : BashTool = BashTool(
        "chipyard_bash", CHIPYARD_PATH,
        task_options={"resources": {"chipyard": 1}}
    )
    resp : QueryResult = get(llm.prompt.chia_remote(
        llm, LLM_RTL_PROMPT, tools=[chipyBash]
    ))
    chipyBash.stop()
    print(resp.result)
```

Note that it's very important in this example to stop chipyBash because it consumes a chipyard resource. If you leave it running, the other nodes which use chipyard resources will be starved.

Finally, let's parse the output of the Verilator run to check for the LLM's change.

```
# Already Here
verilator_node = VerilatorRunNode()
run_output = get(verilator_node.run.chia_remote(verilator_node, buildFuture,
    ↳get(testBinFuture), "helloworld.riscv", "/home/ray/verilator/"))

# Already Here
print("Output of Verilator run (run_output.log): ")
print(run_output.log)

# ===== New =====
# First line of run_output.out is "testing $random ..."
# Next 1000 lines are committed instruction trace
# 1002 line should be "Hello from instruction 1000!"
print("Parsed lines from simulation (run_output.out): ")

print(run_output.out.split('\n')[1001])
print(run_output.out.split('\n')[1002]) # Should have LLM's addition
print(run_output.out.split('\n')[1003])
```

Now, if all goes well, you should see in your output the following

```
(print_hello_world pid=1076432) Hello World (#1) from a remote call!
LLM Responded:
Hello World (#3) from OpenCode!
```

Followed by many messages related to the tool MCP server

```
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO: Started server process [6063]
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO: Waiting for application startup.
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO: Application startup complete.
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO: Uvicorn running on http://172.17.0.
↳1:8000 (Press CTRL+C to quit)
chipyard_bash started at 172.17.0.1:8000 on node_
↳07348f9a70938e126620044cdale3f41554de0ad19fac5e54e9ead6e
```

(continues on next page)

(continued from previous page)

```
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    ${IP ADDRESS} - "POST /chipyard_
↳bash/mcp HTTP/1.1" 200 OK
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    ${IP ADDRESS} - "POST /chipyard_
↳bash/mcp HTTP/1.1" 202 Accepted
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    ${IP ADDRESS} - "GET /chipyard_bash/
↳mcp HTTP/1.1" 200 OK
...
```

Followed, eventually, by a response from the LLM, a tool shutdown message, and finally the verilator outputs. It should look something like this (though the LLM's response will vary).

```
The change has been made successfully and compiles without errors. Here's a summary of
↳what was done:

## Change Summary

**File modified:** `/home/ray/chipyard/generators/rocket-chip/src/main/scala/rocket/CSR.
↳scala`

**What was added:** A Chisel `printf` statement that fires when the architecture counter
↳`instret` (instruction retired count) reaches exactly 1000.

**The exact code added** (line 594, right after the `reg_instret` counter definition):

```scala
when (reg_instret === 1000.U) { printf("Hello World (#4) from instruction 1000!\n") }
```

**How it works:**
- The `reg_instret` signal is a `WideCounter(64, io.retire, ...)` that counts each
↳retired instruction (line 593).
- The `when` block checks if `reg_instret` equals exactly `1000.U` (unsigned 1000).
- When the condition is true, Chisel's `printf` (which prints during simulation/
↳emulation) outputs `"Hello World (#4) from instruction 1000!"`.
- Since no existing "Hello World" message was found in the file, the message was added
↳fresh (no "again" suffix needed).
```

```
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    Waiting for application shutdown.
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    Application shutdown complete.
(_ToolServerActor pid=6063, ip=172.17.0.1) INFO:    Finished server process [6063]
```

Output of Verilator run (run_output.log):

```
Hello World (#2) from Verilator!
[UART] UART0 is here (stdin/stdout).
- /home/ray/chipyard/sims/verilator/generated-src/chipyard.harness.TestHarness.
↳RocketConfig/gen-collateral/TestDriver.v:179: Verilog $finish
```

Parsed lines from simulation (run_output.out):

```
C0:          2574 [1] pc=[0000000080000562] W[r 8=00000000800016c7][1]
↳R[r10=00000000800016c0] R[r 0=0000000000000000] inst=[00750413] addi    s0, a0, 7
Hello World (#4) from instruction 1000!
C0:          2575 [1] pc=[0000000080000566] W[r 8=00000000800016c0][1] R[r
```

(continues on next page)

(continued from previous page)

```
→ 8=000000000800016c7] R[r 0=0000000000000000] inst=[00009861] c.andi s0, -8
```

Where the line “Hello World (#4) from instruction 1000!” indicates that the LLM succeeded.

And with that, you’ve gotten CHIA to say Hello in 4 different ways! Thanks!

ARCHITECTURE OVERVIEW

A **CHIA project** is a composition of two pieces: a **loop** and a **cluster**.

- A **loop** is an orchestration script that defines a pipeline of tasks — the *what to do*.
- A **cluster** is the collection of computers those tasks are scheduled onto — the *where and how it runs*.

4.1 CHIA loops

A loop can be described as a **graph**: the nodes are tasks (functions) and the edges are the data and control flow between them. CHIA treats both **programmatic** and **agentic** edges as first-class primitives, and any function can be driven either way.

4.1.1 Nodes

A **node** is a Python function — *any* Python function can be a node. Each node is tagged with the resources it needs from the worker it runs on. A node is held until (a) all of its inputs are ready and (b) a logical worker is available whose resources exceed the node's demands. Once it can be scheduled, its arguments are serialized and sent to that worker to execute.

4.1.2 Programmatic edges

A node meant to run programmatically is annotated with `@ChiaFunction(resources=...)` and dispatched with `fn.chia_remote(args)`. The call returns control to the caller immediately (asynchronous), handing back a reference you can later wait on and collect with `get()`, or pass directly as an argument to another node (an explicit edge). This enables asynchronous execution of many different nodes. A `@ChiaFunction` called directly — without `chia_remote` — just runs in the caller's own process and isn't a node.

4.1.3 Agentic edges with tools

A node can also be exposed to an agent as an **MCP tool**, registered on a `ChiaTool` object via `ChiaTool.mcp.add_tool(...)`. The `ChiaTool` stands up an MCP server that hosts its registered tools; the server itself runs on a worker chosen by the `task_options` given when it is created. By default a registered function runs on the worker hosting the tool server, but registering `ChiaTool.mcp.add_tool(fn.chia_remote_blocking)` lets the tool function execute remotely according to its *own* resources — so a tool server can live on one worker while its tools run on another. Every LLM/agent node exposes a query method that takes a list of `ChiaTools` to offer the model, and each tool's docstring becomes the description the model sees.

4.2 CHIA clusters

A loop needs diverse infrastructure working in tandem: different nodes need different *physical* hardware (FireSim needs FPGAs) and different *software* environments (dependencies, credentials, isolation). The cluster provides this.

4.2.1 Physical machines and logical workers

You start with a set of **physical machines** — these can be heterogeneous, with different operating systems, memory capacity, accelerators, etc. On top of them you define **logical workers** that expose virtualized hardware resources (CPU cores, GPUs, accelerators, FPGAs) and software resources (dependencies, credentials, isolation). An LLM worker, for instance, carries the dependencies for an agentic query (e.g. a Claude Code CLI and the provider’s credentials); a FireSim-runner worker would expose an FPGA resource and only map onto machines that have one. Workers are mapped onto machines when the cluster comes online — multiple workers can share a machine, and CHIA supports several allocation strategies for load balancing.

4.2.2 Containerization

Logical workers can run inside **containers**. This gives isolation between workers, front-loads environment setup, and makes clusters portable.

4.2.3 Cloud integration

Public-cloud machines can be folded directly into a cluster to add compute on demand. Spanning owned (on-prem) and borrowed (cloud) resources makes for cost-effective, efficient clusters. When machines can’t reach each other directly (the usual case across firewalls), CHIA connects them over a [tailscale](#) network by default — the recommended path for cloud workers, and fully unprivileged (userspace networking, no root or VPN setup). SSH reverse tunneling remains available as a fallback, with some small limitations on orchestration.

4.2.4 The cluster config and CLI

A cluster is specified in a YAML file: available physical machines under a `provider` section, logical-worker types (with their exposed resources and container images) under `available_node_types`, and any cloud instances under a provider section such as `aws_nodes`. Bring it up with `chia up <config>.yaml` — which spawns any cloud nodes, assigns workers to machines, and launches the workers — and tear it down with `chia down <config>.yaml`. See the [CLI Reference](#).

4.3 Fault tolerance & reproducibility

CHIA is built for long, expensive runs:

- **Worker / machine failure** is detected; no new work is scheduled there, and any tasks that were running are automatically re-queued onto another worker with the required resources. Clusters can grow at runtime — `chia up --add` adds resources to a live cluster (and reintegrates recovered nodes) with no downtime.
- **Process-leak prevention** — CHIA tracks the processes a node spawns and stops them when the node or loop is stopped or cancelled.
- **Caching & bypass** — a loop can reuse a node’s cached result instead of recomputing it, so it can restart quickly from anywhere after a crash and bypass nondeterministic nodes.
- **Profiling & visualization** record what ran where and for how long.

4.4 Infrastructure

CHIA curates a set of existing tools into a single fabric. The [Ray](#) distributed-computing platform is the substrate beneath CHIA's loops and clusters — providing scheduling, distributed execution, fault tolerance, and data collection. Ray was chosen over alternatives like LangGraph, the Microsoft Agent Framework, and Apache Airflow for its expressive control-flow semantics, fine-grained flexible scheduling, and distributed execution and fault tolerance. Around it, CHIA uses Docker for containerization, FastMCP for agent tools, Boto3 and the Google Cloud client libraries for cloud integration, and TensorBoard, Weights & Biases, and GraphViz for profiling and visualization.

A particular strength is how cleanly CHIA fits diverse components together. There are already CHIA nodes for hardware-design tools — Chipyard, FireSim, Hammer, CIRCT, gem5, ChampSim, and Verilator — for a wide range of LLM providers and local model serving (AWS Bedrock, Google Vertex, OpenAI and Anthropic APIs, Fireworks, Groq, OpenRouter, Ollama, vLLM) and agent CLIs (Claude Code, OpenAI Codex, GitHub Copilot, Opencode, Google Antigravity), and for supporting tasks like maintaining relational databases, compiling software, and collating GitHub issues.

CLI REFERENCE

The `chia` console script (`chia/cli/main.py`) manages clusters, visualizes flows, and proxies Ray's CLI (with optional per-cluster targeting).

```
chia <command> [options]
```

5.1 Cluster management

5.1.1 `chia up <config.yaml>`

Bring up a Chia/Ray cluster from a YAML config.

| Flag | Meaning |
|----------------------------|---|
| <code>-y, --yes</code> | Skip the confirmation prompt. |
| <code>-v, --verbose</code> | DEBUG logging. |
| <code>--dry-run</code> | Print the plan and generated scripts without executing. |
| <code>--add</code> | Only add new nodes to an existing cluster (skip ones already up). |

5.1.2 `chia down <config.yaml>`

Tear down the cluster. Flags: `-y/--yes`, `-v/--verbose`.

5.2 Visualization

5.2.1 `chia viz <source.py>`

Render the `@ChiaFunction` call graph of a flow **statically** from source.

| Flag | Meaning |
|-------------------------------------|---|
| <code>--func NAME</code> | Orchestrator function (auto-detected if omitted). |
| <code>--format {svg,png,pdf}</code> | Output format (default <code>svg</code>). |
| <code>--output-dir DIR</code> | Output directory (default: alongside the source). |

5.2.2 `chia viz-profile <log...>`

Visualize a `ChiaProfileCollector` JSONL log. See the [Profiling](#) guide for how to record one.

| Flag | Meaning |
|--|--|
| <code>--format {svg,png,pdf,html,table}</code> | Output format (default <code>svg</code>); <code>table</code> aggregates exec time to CSV. |
| <code>--output-dir DIR</code> | Output dir (ignored for <code>table</code>). |
| <code>--output PATH</code> | (<code>table</code>) CSV path; default <code>stdout</code> . |
| <code>--funcs ...</code> | (<code>table</code>) function names, or a file of names. |
| <code>--run N</code> | Run index to visualize (default: <code>last</code>). |
| <code>--gap-threshold SEC</code> | Gap that segments runs (default 600). |
| <code>--x-scale F</code> | Horizontal inches per second of wall-clock (default 0.5). |

5.3 Cluster Status

5.3.1 `chia status [...]`

Print cluster status. Proxies to `ray status`.

| Flag | Meaning |
|--|--|
| <code>--chia-cluster {path/to/cluster.yaml}</code> | Specify the Chia cluster YAML file to target. |
| <code>...</code> | all other args (including <code>--help</code>) go to Ray. |

5.3.2 `chia list [...]`

List all states of a given resource. Proxies to `ray list` (e.g. `chia list nodes`).

| Flag | Meaning |
|--|--|
| <code>--chia-cluster {path/to/cluster.yaml}</code> | Specify the Chia cluster YAML file to target. |
| <code>...</code> | all other args (including <code>--help</code>) go to Ray. |

5.4 Job Management

5.4.1 `chia job <subcommand> [...]`

Submit, stop, delete, or list Ray jobs.

Proxies to `ray job` — `submit`, `status`, `logs`, `list`, `delete`, etc. all forward, so `chia job --help` shows Ray's own job help. Supports `--chia-cluster`. The one exception is `stop`, which *chia overrides* with an augmented implementation; use `chia ray job stop` for Ray's unmodified behavior.

| Flag | Meaning |
|--|--|
| <code>--chia-cluster {path/to/cluster.yaml}</code> | Specify the Chia cluster YAML file to target. |
| <code>stop {job_id}</code> | Stop a job by its ID. |
| <code>...</code> | all other args (including <code>--help</code>) go to Ray. |

5.4.2 `chia job stop <job_id>`

By default a thin wrapper over `ray job stop`. With `--kill-tracked-pids`, it first asks the PID registry actor to kill every tracked subprocess (SIGTERM, then SIGKILL for any that linger past the grace period), then stops the job. The kill returns as soon as the processes have actually exited, so it normally completes in well under the grace period.

| Flag | Meaning |
|---------------------------------|--|
| <code>--kill-tracked</code> | Kill tracked subprocesses (via the PID registry) before stopping the job. Off by default. |
| <code>--grace-period SEC</code> | Seconds to wait for each tracked subprocess to exit after SIGTERM before escalating to SIGKILL (only used with <code>--kill-tracked-pids</code> ; default 25). |

5.5 Fallback to Ray (chia ray ...)

`chia ray <anything>` forwards to `ray <anything>` verbatim, so every current and future Ray command (`ray memory`, `ray timeline`, `ray attach ...`) is reachable — not just the promoted `status/list/job`. Using an explicit gateway also means chia's own `up/down` never shadow `ray up/ray down`, and `chia ray --help` shows Ray's complete command list.

Supports `--chia-cluster`. Unlike the promoted commands, the fallback applies **no** overrides — `chia ray job stop` runs Ray's `stop`, not chia's augmented one:

```
chia ray memory --chia-cluster=cluster_a.yaml
chia ray timeline
chia ray job stop <job_id>
```

5.6 FireSim

FPGA build/run orchestration:

- `chia firesim-build <config.yaml> --recipe NAME [--instance-type ...]`
- `chia firesim-run <config.yaml> [--hw-config ... --workload ... | --suite ...] [...]`
- `chia firesim-upload-workload [config.yaml] --marshal-config ... --images-dir ... --suite-name ...`
- `chia firesim-cleanup <config.yaml>` — terminate orphaned chia EC2 instances.

Run `chia <command> --help` for the full, current option list.

CHIAFUNCTION

`ChiaFunction` is the core primitive of a CHIA loop. The `@ChiaFunction` decorator turns *any* Python function into a **node**, a unit of work that can be scheduled onto a worker in the cluster, chained into a task graph, profiled, cached, and bypassed. This page explains the Ray concepts the rest of the docs lean on, then walks through every mode of execution a `ChiaFunction` supports.

- *A few Ray concepts*
- *Executing a CHIA node*
 - *Local call*
 - *Remote, asynchronous (`chia_remote`)*
 - *Remote, blocking (`chia_remote_blocking`)*
 - *Chaining refs into a task graph*
 - *Per-call option overrides (`.options`)*
 - *Functional form (`ChiaCallRemote`)*
- *Collecting results*
 - *Cancellation*
- *Worker-side setup and cleanup hooks*
- *Defining your own node*
- *Wrapping actors (`chia_actor`)*
- *Cross-cutting features*
- *See also*

6.1 A few Ray concepts

CHIA is built on the [Ray](#) distributed-computing platform, and the docs use Ray’s vocabulary throughout. You do not need to know Ray to use CHIA, but these terms recur:

- **Driver** — the process running your loop script (the `main()` you submit with `python ...` or `chia job submit ...`). It runs on the cluster’s head node, dispatches work, and collects results.
- **Worker** — a process that executes dispatched work. In CHIA a logical worker also advertises resources (CPU/GPU/FPGA counts, software capabilities) and only maps onto physical machines that can satisfy them. See [Architecture Overview](#) for how logical workers map onto machines.

- **Task** — a single asynchronous function invocation sent to a worker. Dispatching a `@ChiaFunction` with `fn.chia_remote(...)` creates a task. Tasks are stateless: the worker runs the function and returns the result.
- **Actor** — a stateful worker: a remote Python object whose methods run on the worker that holds it. CHIA uses actors for things that must persist across many calls, like the profile collector, the cache, and MCP tool servers.
- **Object reference** (`ObjectRef`) — a *future*: a handle to a result that may not exist yet. `chia_remote(...)` returns immediately with an `ObjectRef[R]` instead of blocking for the value. You resolve it later with `get()`, or pass it straight into another call as an argument.
- **Resources** — labels with quantities a task requires (`{"chipyard": 1}`) and a worker advertises. Ray holds a task until a worker with enough free resources is available, then leases that worker for the task's duration. This is how CHIA places work on the right machine.

6.2 Executing a CHIA node

CHIA provides a library of nodes decorated with `@ChiaFunction`. Most of them follow the same shape: a plain Python `class` that bundles the node's construction parameters, instance state, and private helpers, with one (or more) method decorated with `@ChiaFunction(...)` that is the actual dispatchable node. The decorated method comes pre-assigned with the resources it needs.

`chia.chipyard.chisel_build_node` is a representative example. The `ChiselBuildNode` class holds the node's build configuration (chipyard path, config, target, make flags) and a handful of private helpers. Its `build` method is the node, pre-assigned the `chipyard` resource:

```
class ChiselBuildNode:
    def __init__(self, chipyard_path, config,
                 target=BuildTarget.VERILATOR, ...):
        ...                                # instance state

    @ChiaFunction(resources={"chipyard": 1})
    def build(self) -> BuildArtifact:
        ...                                # runs `make` in sims/verilator
        return BuildArtifact(...)
```

When you call `build` you can choose how to execute it: locally in the driver, remotely on a worker, and with or without blocking for the result. The pre-assigned resource options can also be overridden per-call. The rest of this section walks through each mode using this node.

6.2.1 Local call

Calling the bound method directly runs it in the same process as made the call, exactly like an ordinary Python call:

```
cb_node = ChiselBuildNode("/home/ray/chipyard", "RocketConfig",
                          target=BuildTarget.VERILATOR)
artifact = cb_node.build()           # runs here, in the driver
```

This is handy for using ChiaFunction features like profiling, bypassing, and caching without dispatching the function to a worker.

6.2.2 Remote, asynchronous (`chia_remote`)

`fn.chia_remote(...)` dispatches the function as a task onto a worker that satisfies its resources. It is non-blocking and returns an `ObjectRef` immediately. Resolve and block on the ref with `get()` when you need the value:

```
ref = cb_node.build.chia_remote(cb_node)    # note: pass `cb_node` explicitly
# ... dispatch other work here; it runs concurrently ...
artifact: BuildArtifact = get(ref)          # blocks until the build finishes
```

6.2.3 Remote, blocking (chia_remote_blocking)

When you want the value synchronously and have no other work to overlap, `chia_remote_blocking` dispatches remotely and returns the unwrapped value:

```
artifact: BuildArtifact = cb_node.build.chia_remote_blocking(cb_node)
```

6.2.4 Chaining refs into a task graph

Any argument to a `chia_remote` call may be either a plain value of type `T` or an `ObjectRef[T]`. Passing a ref directly without `get()` tells Ray that this task depends on that one, forming an explicit edge in the task graph. Ray resolves the dependency for you and only starts the downstream task once its inputs are ready, so independent tasks run concurrently. Dispatch everything, wire refs together, and `get()` only the final result:

```
# No get() between these - refs flow straight in as arguments.
bin_ref = compile_program.chia_remote(c_src)
build_ref = cb_node.build.chia_remote(cb_node)

# run() depends on BOTH upstream tasks; Ray waits for them automatically.
result = get(
    verilator_node.run.chia_remote(
        verilator_node, build_ref, bin_ref, "helloworld.riscv", "/home/ray"
    )
)
```

The [Quickstart: Say Hello \(World\) with CHIA](#) builds up exactly this pattern step by step.

6.2.5 Per-call option overrides (.options)

To override the decorator-level options for a single dispatch, use `.options(...)`:

```
ref = cb_node.build.options(
    num_cpus=4, scheduling_strategy="SPREAD"
).chia_remote(cb_node)
```

Any keyword that Ray's `.options()` accepts is supported. The ones CHIA loops reach for in practice are:

- `resources` — the resource labels (and quantities) a worker must offer to run the node. E.g. tag a function with `@ChiaFunction(resources={"chipyard": 1})` needs a whole chipyard slot.
- `num_cpus` — how many CPUs the task reserves. Number of CPUs per machine is automatically discovered in cluster setup and advertised to the cluster. E.g. a chipyard node that needs 2 threads: `@ChiaFunction(num_cpus=2, resources={"chipyard": 1})`, with default value 1.
- `max_retries` — how many times a task is rerun if the worker it ran on dies. `0` disables automatic retry. E.g. `@ChiaFunction(num_cpus=0.1, max_retries=0)`, with default value 3.
- `scheduling_strategy` — how tasks are scheduled across nodes. Common values are "DEFAULT", which prioritizes locality then load balancing, and "SPREAD" to fan tasks out evenly. E.g. `run_synthesis.options(scheduling_strategy="SPREAD").chia_remote(design)`. This option also accepts a Ray scheduling-strategy *object*, most often a `PlacementGroupSchedulingStrategy`, which binds the task to a

placement group so a set of related `@ChiaFunction` calls gang-schedule onto the same (or co-located) logical worker(s) instead of being placed independently.

A bare `.options()` (no resources) can run on any worker.

6.2.6 Functional form (`ChiaCallRemote`)

`ChiaCallRemote(fn, *args, **kwargs)` is equivalent to `fn.chia_remote(*args, **kwargs)` but raises a clear `TypeError` if `fn` is not a `@ChiaFunction`. Use whichever reads better at the call site:

```
from chia.base.ChiaFunction import ChiaCallRemote

ref = ChiaCallRemote(compile_program, src_contents)
```

6.3 Collecting results

`get()` is the basic collector and is usually all you need. For loops that dispatch many tasks and want to react as each finishes, or that run long enough to hit a wedged worker, CHIA provides `chia_wait`, a drop-in replacement for `ray.wait` that operates on `TrackedRef` objects (an `ObjectRef` paired with a closure that can re-dispatch it). It returns the usual (ready, pending) split and can additionally detect tasks stuck in `PENDING_NODE_ASSIGNMENT` while the cluster has free resources, then cancel and resubmit them:

```
from chia.base.ChiaFunction import chia_wait, TrackedRef

tracked = [
    TrackedRef(compile_program.chia_remote(s),
               submit_fn=lambda s=s: compile_program.chia_remote(s),
               label=f"compile_{i}")
    for i, s in enumerate(sources)
]

# Return when at least num_returns tasks are ready, or after pending_timeout seconds of
# no progress.
ready, pending = chia_wait(tracked, num_returns=1,
                           pending_timeout=120, retry=True)

for tr in ready:
    result = get(tr.ref)
```

6.3.1 Cancellation

`chia_cancel(ref)` cancels a running task. Unlike a bare `ray.cancel`, it first looks up any subprocesses the task spawned and kills them on the correct remote nodes (process-group kill for `start_new_session` children) before cancelling the Ray task, part of CHIA's process-leak prevention:

```
from chia.base.ChiaFunction import chia_cancel

ref = build_megaboom.chia_remote(config)
chia_cancel(ref, force=True)
```

6.4 Worker-side setup and cleanup hooks

A `chia_remote` call may carry reserved kwargs that run side-effecting callables on the worker around the function: `_chia_setup` (runs *before* the function) and `_chia_cleanup` (runs *after*, in a `finally`, so it fires even if the function raises). Each takes an optional `_chia_setup_args` / `_chia_cleanup_args` tuple. Neither can see or replace the function's return value:

```
ref = run_sim.chia_remote(
    design,
    _chia_setup=mount_scratch, _chia_setup_args=(scratch_dir,),
    _chia_cleanup=unmount_scratch, _chia_cleanup_args=(scratch_dir,),
)
```

If setup raises, the function and cleanup are skipped and the task fails. If cleanup raises, the error is logged but never re-raised, so a failing teardown cannot mask the function's result or its own exception.

6.5 Defining your own node

Decorate any function with `@ChiaFunction`. The `resources` argument (and any other keyword) names what a worker must offer to run it:

```
from chia.base.ChiaFunction import ChiaFunction, get

@ChiaFunction(resources={"chipyard": 1})
def compile_program(src_contents: str) -> bytes:
    ...
    return elf_data
```

6.6 Wrapping actors (`chia_actor`)

A plain Ray actor handle can be given the same call surface as a `ChiaFunction` with `chia_actor`, so actor calls read like node dispatch:

```
from chia.base.ChiaFunction import chia_actor, get

store = chia_actor(some_actor)
n = get(store.size_bytes.chia_remote())
get(store.write.chia_remote(key, value))
```

We currently do not support profiling, bypass, or cache machinery on actors. Recover the raw Ray handle with `store.actor` (e.g. for `ray.kill`).

6.7 Cross-cutting features

The same `chia_remote` dispatch path also drives three of CHIA's framework features, all of which are transparent to your function's body:

- **Profiling.** Once a collector is running, every `ChiaFunction` function call is instrumented automatically. Execution time, the worker it ran on, and the dependency edges between calls are recorded. Call `get_profiler().add_info(...)` from inside a node body to attach domain metrics. See [Profiling](#).
- **Caching and bypass.** Pass a per-call `_chia_tag` to name a dispatch. A function marked `cache: true` then has its result persisted to disk under that tag, and a function marked `bypass: true` can replay a pre-recorded

value (or a registered provider's output) instead of running — while *still* dispatching through Ray so scheduling is exercised. See [Caching and Bypass](#).

```
# The _chia_tag names this call for both caching and bypass.
ref = run_verilator_test.chia_remote(design, _chia_tag=f"iter{i}_opt{j}")
```

These features compose: a single dispatch can be profiled and served from cache with no change to the decorated function.

6.8 See also

- [Quickstart: Say Hello \(World\) with CHIA](#) — a hands-on example that uses every mode above.
- [Architecture Overview](#) — how nodes, edges, workers, and clusters fit together.
- [Caching and Bypass](#) — the `_chia_tag` cache/replay workflow.
- [Profiling](#) — recording and visualizing a loop's execution.
- `chia viz-profile <log...>` — rendering a recorded profile from the CLI.

CHIATOOL

ChiaTool is the primitive CHIA uses to allow an **AI agent** to orchestrate control of the loop. Where `chia_remote(...)` creates a node that your flow dispatches programmatically — a programmatic edge in the task graph — a ChiaTool stands up an **MCP** server on a worker that an *agent* can call on its own initiative. That is an **agentic edge**: you give the agent a set of tools and a prompt, and the agent decides which tools to call, when, and with what arguments. This page explains the MCP concepts the tool layer leans on, walks through a tool’s lifecycle, shows how tools attach to an agent, and how to define your own tools.

- *A few MCP concepts*
- *Anatomy of a tool*
- *The tool lifecycle*
 - *Constructing a tool deploys it*
 - *Attaching a tool to an agent*
 - *Stopping a tool*
- *Bridging tools back to nodes*
- *Defining your own tool*
- *Tool placement and resources*
- *Advanced*
 - *The explicit initialization idiom*
 - *Intermediate base classes*
- *See also*

7.1 A few MCP concepts

CHIA tools speak the **Model Context Protocol (MCP)**, the open standard agents use to discover and call external tools over HTTP.

- **Tool (method)** — a single callable the agent can invoke, with a name and a docstring/type signature. The agent reads those to decide *when* to call the tool and *how* to fill in its arguments, so a tool method’s docstring is part of its interface, not just documentation.
- **Tool server** — a small HTTP server (FastAPI + uvicorn) that hosts one ChiaTool’s methods and speaks MCP. CHIA deploys it onto a worker as a Ray **actor** so it persists across many agent calls (see *ChiaFunction* for the actor/worker vocabulary). Its MCP endpoint is:

```
http://{host}:{port}/{name}/mcp
```

- **Resources / placement** — like a node, a tool server is pinned onto a worker that advertises the resources it needs. You pass these as `task_options` (see *Tool placement and resources*), so the agent's bash tool lands on, say, the worker that holds a Chipyard checkout.

7.2 Anatomy of a tool

CHIA provides a library of ready-made tools and a base class, `chia.base.tools.ChiaTool`, for writing your own. A tool is a plain Python **class** that subclasses `ChiaTool`, bundles the tool's configuration and state, and registers one or more **methods** as the callable tools the agent sees.

`chia.base.tools.BashTool` is a representative example: it gives an agent a bash interface to the worker it is deployed on. The class holds the working directory and registers a single `run_command` method:

```
class BashTool(ChiaTool):
    def setup(self, work_dir="/"):
        self.work_dir = work_dir
        self.mcp.add_tool(self.run_command, name=f"{self.name}_run_command")

    def run_command(self, command: str) -> str:
        """Run a bash command and return combined stdout/stderr."""
        ... # runs in the container

bt = BashTool(name="bash", work_dir="/home/", task_options={"resources": {"chipyard": 1}}
→)
```

To create a `ChiaTool`, you create a subclass which defines a `setup()` method where you register all of the tools with `self.mcp.add_tool(...)`.

The base class runs its own initializer behind the scenes — creating the `self.mcp` server object your tools attach to — then calls your `setup()` method to register the tools, then deploys the running server behind the scenes.

The constructor of the child class forwards arguments to the `setup` call (like `work_dir`) and takes two additional arguments: `name` and `task_options`. The former is a human readable string used by the agent to reference the tool, and the latter is used to determine where to place the tool server, and includes things like resource specifications and placement groups.

Alternatively, if you would rather write the constructor by hand, an *explicit initialization idiom* is supported and can be used instead of `setup()`.

7.3 The tool lifecycle

7.3.1 Constructing a tool deploys it

Unlike a node, which only runs when you call `chia_remote`, **instantiating** a `ChiaTool` immediately stands up its server on a worker: once `setup()` has registered the tools, the base deploys the server automatically. So construction is the deploy step:

```
from chia.base.tools.BashTool import BashTool

chipyard_bash = BashTool(
    name = "chipyard_bash", work_dir="/home/ray/chipyard",
    task_options={"resources": {"chipyard": 1}}, # land on a chipyard worker
```

(continues on next page)

(continued from previous page)

```
)
# The MCP server is now live at
# http://{chipyard_bash.hostname}:{chipyard_bash.port}/chipyard_bash/mcp
```

7.3.2 Attaching a tool to an agent

Pass the tool (or several) to any CHIA provided LLM or agent's `prompt` node via the `tools` argument. The agent receives each tool's MCP endpoint, discovers the methods, and calls them as needed to satisfy the prompt:

```
from chia.base.ChiaFunction import get

prompt = (
    "Use the chipyard_bash tool to add a printf to the Rocket core's "
    "Chisel source that fires when instret reaches 1000."
)
result = get(llm.prompt.chia_remote(llm, prompt, tools=[chipyard_bash]))
print(result.result)
```

Here the agent itself is a node — `llm.prompt` is dispatched with `chia_remote` — and the tool it was handed lets it reach out and act on the cluster while it runs. The *Quickstart: Say Hello (World) with CHIA* builds up exactly this pattern, ending with an agent editing RTL through a `BashTool`.

7.3.3 Stopping a tool

A tool server stays running until you stop it. Call `stop()` once the agent work that needs it is done; it shuts `uvicorn` down and kills the actor, freeing the port and the worker slot:

```
chipyard_bash.stop()
```

7.4 Bridging tools back to nodes

A tool method is ordinary Python, so it can dispatch `@ChiaFunction` nodes just like the rest of your flow. Because the agent expects a concrete return value (not an `ObjectRef`), use `chia_remote_blocking`, which dispatches remotely and returns the unwrapped result.

This is a composition point between the two primitives: the agent makes one tool call (an agentic edge), and that call fans out real, scheduled cluster work (programmatic edges) before returning a value the agent can reason about.

7.5 Defining your own tool

Subclass `ChiaTool` and define a `setup()` method that registers your tools, giving each method a clear docstring (which is given to the agent to describe the tool) and typed signature — the agent relies on both to call it correctly. `chia.base.tools.ChiaToolTemplate` is a copyable starting point.

Here's an example:

```
from chia.base.tools.ChiaTool import ChiaTool

class GreetingTool(ChiaTool):
    def setup(self, greeting="Hello"):
        self.greeting = greeting
        self.mcp.add_tool(self.greet, name=f"{self.name}_greet")
```

(continues on next page)

(continued from previous page)

```
def greet(self, who: str) -> str:
    """Return a greeting for *who*.
```

Args:

```
    who: The name to greet.
    """
    return f"{self.greeting}, {who}! This is {self.name}."
```

A tool method may return **any** type — MCP always serializes the result for the agent, falling back to a JSON (then `str`) rendering of arbitrary objects. Annotating the method's return type (as `greet` does with `-> str`) can allow MCP to provide the agent a schema for the return value in some cases. The following return types are formatted more cleanly and are preferred:

- `str` — handed to the agent as plain text.
- `dict`, a pydantic `BaseModel`, a `TypedDict`, or a dataclass — returned as structured, machine-readable JSON.
- `list` / `tuple`.
- `None` — an empty result.
- MCP Image / Audio content objects (from `mcp.server.fastmcp`) — for image or audio payloads, instead of raw bytes.

For tools that wrap long-running work, CHIA also provides asynchronous variants (`chia.base.tools.AsyncBashTool`, `chia.base.tools.AsyncJobTool`) that split a job into a `submit` method and a `status` method so the agent can poll rather than block.

7.6 Tool placement and resources

`task_options` is the dict of Ray scheduling constraints applied to the tool's server actor — the tool analog of a node's per-call `.options(...)`. The most common entry is `resources`, which pins the server onto a worker advertising the right capability:

```
# Land the bash tool on a worker that has a chipyard checkout.
BashTool("chipyard_bash", "/home/ray/chipyard",
        task_options={"resources": {"chipyard": 1}})
```

Because the server runs *on* that worker, anything the tool's methods do — run a command, read a file, dispatch a node — happens with that worker's filesystem and resources in reach. Omitting `task_options` lets the server land on any available worker.

7.7 Advanced

7.7.1 The explicit initialization idiom

Before the `setup()` hook existed, every tool was written with an explicit `__init__` that called the base class's construction steps by hand. This style is **still fully supported**: define `__init__` instead of `setup()` and CHIA leaves it untouched. You might prefer it when you want the tool's constructor signature to be explicit (for introspection or IDE help), or to match tools already written this way.

An `__init__` written this way follows a fixed three-step order, and the order matters:

1. ```super().__init__(name, task_options=...)``` creates the `FastMCP` instance (`self.mcp`) the tools attach to. It does not start a server yet.

2. `self.mcp.add_tool(self.method, name=...)`` registers each method as an MCP tool. This must happen after step 1 (so self.mcp exists) and before step 3 (so the server has its tools when it starts serving). Call it once per method you want to expose.`
3. `super().__post_init__()` deploys the server: it creates a Ray actor (constrained by `task_options`), which probes for a free port, starts uvicorn in a background thread, and registers the running tool. After this returns, `self.hostname` and `self.port` point at the live endpoint.

The same BashTool written explicitly:

```
class BashTool(ChiaTool):
    def __init__(self, name, work_dir="/", task_options=None):
        super().__init__(name, task_options=task_options)           # 1. init base
        self.work_dir = work_dir
        self.mcp.add_tool(self.run_command, name=f"{name}_run_command") # 2. register
        super().__post_init__()                                       # 3. deploy

    def run_command(self, command: str) -> str:
        """Run a bash command and return combined stdout/stderr."""
        ...
```

The two styles are interchangeable and can coexist in the same codebase: a subclass that defines `setup()` gets the automatic bracketing, while one that defines `__init__` keeps full manual control.

7.7.2 Intermediate base classes

An *intermediate* base sits between `ChiaTool` and a concrete tool, bundling functionality that several tools share — `chia.base.tools.AsyncJobTool` (the base for the async variants above) is the canonical example, adding a background-job runner its subclasses inherit.

Writing one correctly hinges on a single fact: the `setup()` idiom's generated `__init__` calls `ChiaTool.__init__` **directly**, skipping every intermediate `__init__` in between. So a `setup()`-style subclass of your intermediate never runs your intermediate's `__init__`. Make the intermediate work whether or not its own `__init__` runs:

Initialize state lazily, not in `** __init__`. E.g. provide an idempotent `_ensure_X()` that creates the state on first use, and call it at the top of every helper that needs it. This is the rule that makes the class work under both idioms with no per-subclass boilerplate:

```
def _ensure_job_state(self):
    if not hasattr(self, "_job_lock"):
        self._init_job_state()

def _job_start(self, work):
    self._ensure_job_state()    # works whether or not __init__ ran
    ...
```

This covers state that can be created lazily with sensible defaults. It does not transparently cover eager work that depends on constructor arguments (e.g. opening a connection from a `dsn` passed in). For that, you may require the leaf's `setup()` pass the arguments into the super class explicitly.

7.8 See also

- *ChiaFunction* — the programmatic counterpart; tools call nodes via `chia_remote_blocking`.
- *Quickstart: Say Hello (World) with CHIA* — a hands-on example that ends with an agent using a `BashTool` to edit RTL.

- *Architecture Overview* — how nodes, tools, agents, workers, and clusters fit together.
- *Profiling* — recording and visualizing a loop’s execution, including agent calls.

CLUSTER CONFIGURATION REFERENCE

A CHIA cluster is described by a single YAML file that you pass to `chia up` and `chia down`. This page is a complete reference for every key CHIA reads, a worked example that mixes on-premise and cloud machines, and a walk-through of the exact order in which CHIA runs your commands when it brings a cluster up and tears it down.

Note

CHIA's YAML deliberately resembles the [Ray cluster launcher](#) config so existing Ray configs feel familiar, but with additional support for heterogeneous on-premise setups as well as clusters split across on-premise and cloud providers. Chia currently does not support the following Ray autoscaler keys: `min_workers` / `max_workers`, `upscaling_speed`, `idle_timeout_minutes`, `cluster_synced_files`, `file_mounts_sync_continuously`, `provider.type`, `provider.external_head_ip`, and `provider.coordinator_address`.

8.1 Top-level structure

At the top level a config is organized into a handful of sections:

```
cluster_name: MyCluster           # identifier for this cluster

provider:                         # head machine (required)
  head_ip: ...

auth:                             # how to SSH into the machines
  ssh_user: ...
  ssh_private_key: ...

available_node_types:             # logical worker types + their resources
  my_worker:
    ...

aws_nodes:                       # optional: provision EC2 instances
  ...
gcp_nodes:                       # optional: provision GCP instances
  ...
tailnet:                         # recommended for cloud workers: join over
  ...                           # tailscale instead of SSH tunnels
tunnel_defaults:                 # optional: tuning for the SSH-tunnel fallback
  ...
```

(continues on next page)

(continued from previous page)

```
# lifecycle command hooks (see "Command execution order" below)
initialization_commands: [...]
head_env_commands: [...]
setup_commands: [...]
head_setup_commands: [...]
head_tearardown_commands: [...]
head_start_ray_commands: [...]
worker_start_ray_commands: [...]

# file syncing
file_mounts: {...}
rsync_exclude: [...]
rsync_filter: [...]
```

Any `${VAR}` reference in a string value is expanded from your environment when the config is loaded (e.g. `${USER}`). A bare `$VAR` is left as-is so it can be evaluated later on the remote shell.

8.1.1 Top-level keys

| Key | Default | Meaning |
|--|-----------------|---|
| <code>cluster_name</code> | "default" | Identifier for the head and workers of this cluster. |
| <code>provider</code> | <i>required</i> | Cluster head node. See provider . |
| <code>auth</code> | {} | SSH credentials for reaching the machines. See auth . |
| <code>available_node_types</code> | {} | Logical worker types, their Ray resources, and container images. See available_node_types . |
| <code>initialization_commands</code> | [] | Commands run first, each in its own SSH session, on the host (outside any container). They do not share an environment with each other or with later steps. |
| <code>setup_commands</code> | [] | Global setup commands run inside the main script session on every nod (head and workers). Inside the container for containerized workers. |
| <code>head_env_commands</code> | [] | Environment activation prepended to the head's main script on both <code>chia up</code> and <code>chia down</code> (e.g. <code>source ~/.bashrc && conda activate chia_env</code>). |
| <code>head_setup_commands</code> | [] | Head-only one-time setup, run during <code>chia up</code> in the head's main script. |
| <code>head_teardown_commands</code> | [] | Head-only commands run during <code>chia down</code> before <code>ray stop</code> . |
| <code>head_start_ray_commands</code> | [] | Commands that start Ray on the head (typically <code>ray stop</code> then <code>ray start --head ...</code>). |
| <code>worker_start_ray_commands</code> | [] | Commands that start Ray on each worker. CHIA injects <code>--resources</code> automatically (plus, for tailnet or tunneled workers, the pinned ports and any proxy env). |
| <code>file_mounts</code> | {} | { <code>remote_path</code> : <code>local_path</code> } directories rsync'd to each node before the main script. Paths are passed to rsync verbatim, so end <code>local_path</code> with <code>/</code> to copy the directory's <i>contents</i> into <code>remote_path</code> (without it rsync nests the directory: <code>remote_path/<dirname></code>). |
| <code>rsync_exclude</code> | [] | Patterns passed to rsync <code>--exclude</code> (e.g. <code>**/.git</code>). |
| <code>rsync_filter</code> | [] | Filter files (e.g. <code>.gitignore</code>) passed to rsync <code>--filter</code> . |
| <code>docker</code> | None | A cluster-wide default container config (see Container config), which individual node types can override. Specify at most one of the two. |
| <code>aws_nodes</code> | None | Provision EC2 instances and join them to the cluster — over the tailnet when a <code>tailnet</code> section is present (recommended), else via SSH tunnels. See Cloud nodes . |
| <code>gcp_nodes</code> | None | Provision GCP Compute Engine instances (same connect options as <code>aws_nodes</code>). See Cloud nodes . |
| <code>tunnel_defaults</code> | None | Tunnel/port-pinning defaults for the SSH-tunnel fallback path (ignored when cloud workers join over the tailnet). See Cloud nodes . |

8.2 provider

The `provider` section declares the head machine.

```
provider:
  head_ip: ${HEAD_IP}
```

| Key | Default | Meaning |
|---------|-----------------|---|
| head_ip | <i>required</i> | Hostname or IP of the machine that manages the cluster (runs the Ray head). |

8.3 auth

The `auth` section gives the SSH credentials CHIA uses to reach every machine, with optional per-host overrides.

```
auth:
  ssh_user: ${USER}
  ssh_private_key: /home/${USER}/.ssh/${USER}  # omit if your key is in ssh-agent
  overrides:
    some-host:
      ssh_user: ubuntu
      ssh_private_key: ~/.ssh/other_key
```

| Key | Default | Meaning |
|-------------------|---------|---|
| ssh_user | "" | Default SSH username for all machines. |
| ssh_private_key | None | Default private key path. Omit it if the relevant keys are already loaded into your SSH agent. |
| ssh_proxy_command | None | Default ssh ProxyCommand for reaching the machines (e.g. <code>nc -X 5 -x 127.0.0.1:1055 %h %p</code> to dial through a tailscale userspace SOCKS5 proxy, or a jump-host command). Applied to ssh, rsync, and SSH tunnels alike. |
| overrides | {} | Per-IP overrides, keyed by hostname/IP (or a <code>@node_type:index</code> placeholder). Each entry may set <code>ssh_user</code> , <code>ssh_private_key</code> , <code>ssh_proxy_command</code> , <code>tailnet: true / manage_tailscale: true</code> (see Tailnet (tailscale) clusters), or a <code>tunnel</code> block (see Cloud nodes). For provisioned cloud nodes CHIA populates these automatically — <code>tailnet</code> overrides when a <code>tailnet:</code> section is present, <code>tunnel</code> overrides otherwise. |

8.4 available_node_types

Each entry under `available_node_types` defines a *logical worker type*: the Ray resources it advertises, how many of them to run, where they may run, and the container (if any) they run in.

```
available_node_types:
  verilator_run:
    resources: {"verilator_run": 8}
    num_workers: 4
    compatible_ips: [machine9, machine10, machine11, machine12]
    worker_env_commands: ["source ~/.bashrc && conda activate chia_env"]
    docker:
      image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
      container_name: "chia-verilator-run-${USER}"
      run_options:
        - --ulimit nofile=65536:65536
        - --shm-size=10.24gb
```

| Key | Default | Meaning |
|------------------------------------|---------------------------------------|---|
| <code>resources</code> | <code>{}</code> | Custom Ray resources advertised by each worker of this type, e.g. <code>{"verilator_run": 8}</code> . A <code>@ChiaFunction</code> requesting these resources is scheduled onto a matching worker, consuming the amount it requests. |
| <code>num_workers</code> | <code>1</code> | How many workers of this type to launch. For Ray-config familiarity, if <code>num_workers</code> is absent CHIA falls back to <code>max_workers</code> , then <code>min_workers</code> , then <code>1</code> . |
| <code>max_workers</code> | <i>optional</i> | Ray-config alias for <code>num_workers</code> , used only when <code>num_workers</code> is absent (and it takes precedence over <code>min_workers</code>). CHIA does not autoscale — this sets a single fixed worker count, not an upper bound. |
| <code>min_workers</code> | <i>optional</i> | Ray-config alias for <code>num_workers</code> , used only when both <code>num_workers</code> and <code>max_workers</code> are absent. Not a lower bound; if you set <code>min_workers</code> and <code>max_workers</code> to different values, <code>min_workers</code> is ignored and <code>max_workers</code> wins. |
| <code>compatible_ips</code> | <i>required if num_workers > 0</i> | The machines this type's workers may run on. Accepts <code>@node_type:index</code> placeholders. |
| <code>worker_env_commands</code> | <code>[]</code> | Per-type environment activation prepended to the worker's main script on both <code>chia up</code> and <code>chia down</code> . Runs inside the container for containerized types. |
| <code>worker_setup_commands</code> | <code>[]</code> | Per-type one-time setup run during <code>chia up</code> in the worker's main script. |
| <code>docker</code> | <code>None</code> | Container config for this type, overriding any cluster-wide default. Specify at most one. See Container config . |
| <code>balance_level</code> | <code>"cluster"</code> | How this type spreads across its eligible IPs: <code>cluster</code> packs around whatever other types already placed (fewest nodes globally); <code>worker</code> distributes this type's own workers as evenly as possible across its IP pool, regardless of if the IP may be shared with another type. |

8.4.1 Container config

A `docker:` block may appear cluster-wide at the top level or inside any node type; the node-type block overrides the cluster-wide one.

```
docker:
  image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
  container_name: "chia-verilator-run-${USER}"
  pull_before_run: True
  pull_timeout: 3600
  run_options:
    - --ulimit nofile=65536:65536
    - --shm-size=10.24gb
    - "-v $SSH_AUTH_SOCK:/ssh-agent"
  run_setup_commands:
    - cd /home/ray/ && git pull
```

| Key | Default | Meaning |
|--------------------|------------------|--|
| image | <i>required</i> | Container image URI |
| container_name | "chia_container" | Base container name. CHIA appends the worker index (-0, -1, ...) so multiple workers of a type don't collide. Include \${USER} on shared machines. |
| pull_before_run | True | Pull the image before running. Set False to use a cached image. |
| pull_timeout | 600 | Seconds to allow for the pull. Raise it for large images. |
| run_options | [] | Extra flags passed to <code>docker run</code> (ulimits, shm size, volume mounts, <code>--user</code> , env vars, ...). |
| run_setup_commands | [] | Commands run inside the container after it starts, before the worker's main script (e.g. <code>clone/pull a repo</code> , <code>fix up /etc/passwd</code>). |

Note

Scripts run over SSH as a **non-interactive login shell** (`bash --login`): `/etc/profile` and `~/.bash_profile` are sourced, but `~/.bashrc` is not (and many `~/.bashrc` files bail out early for non-interactive shells). If you rely on `conda/venv` set up in `~/.bashrc`, source it explicitly in `head_env_commands` / `worker_env_commands`, e.g. `source ~/.bashrc && conda activate chia_env`.

8.5 Cloud nodes

CHIA can provision public-cloud machines and join them to the cluster. Declare them under `aws_nodes` (EC2) and/or `gcp_nodes` (Compute Engine); everything downstream of provisioning is provider-agnostic.

How cloud workers connect. When the config has a top-level `tailnet:` section (the **recommended** approach), cloud workers join over the tailnet — CHIA installs userspace tailscale on each instance, joins it, and routes Ray through the per-machine CONNECT proxy. No SSH tunnels, no reverse forwards, no GatewayPorts, no iptables, and worker-worker traffic is a full mesh. See *Tailnet (tailscale) clusters* for the full picture.

Without a `tailnet:` section, cloud workers fall back to **reverse SSH tunnels** to the head (the original path, documented below): each worker's Ray/tool ports are reverse-tunnelled so the head can reach them, and traffic between workers is routed through the head. Tunnels are still fully supported, but tailnet is preferred — it scales better (per-machine port allocation, no head-as-hub bottleneck) and needs no `sshd` changes.

8.5.1 aws_nodes

```
aws_nodes:
  region: us-east-1
  verilator_run_aws:
    KeyName: my-keypair           # an EC2 key pair in your account
    InstanceType: c5.9xlarge
    count: 3
    ImageId: ami-0ec10929233384c7f
    ssh_user: ubuntu
    ssh_private_key: /home/${USER}/my-keypair.pem
    setup_commands:
      - "echo ${GITHUB_TOKEN} | docker login ghcr.io -u myuser --password-stdin"
    BlockDeviceMappings:         # passed through to EC2 RunInstances
      - DeviceName: /dev/sda1
```

(continues on next page)

(continued from previous page)

```
Ebs:
  VolumeSize: 500
  VolumeType: gp3
```

region is a section-level key (default us-west-2). Every other key lives under a named node type:

| Key | Default | Meaning |
|--------------------|------------------|---|
| KeyName | <i>required</i> | EC2 key pair name (must already exist in the account). |
| InstanceType | <i>required</i> | EC2 instance type (e.g. c5.9xlarge). |
| count | <i>required</i> | Number of instances to launch for this type. |
| ImageId | Ubuntu 22.04 AMI | AMI to launch. |
| ssh_user | None | SSH user for the AMI (e.g. ubuntu). Injected into the auth override for each provisioned IP. |
| ssh_private_key | None | Private key path matching KeyName. Injected into the auth override. |
| skip_default_setup | False | Skip CHIA's default setup (git/conda/docker install) and run only your setup_commands. |
| setup_commands | [] | Commands run on the EC2 host before it joins the cluster (appended to the defaults unless skipped). |
| setup_timeout | 1800 | Seconds allowed for setup. |
| ssh_timeout | 120 | Seconds to wait for SSH to come up. |
| join_tailnet | auto | Join the cluster over the tailnet instead of reverse SSH tunnels. Defaults to true when a top-level tailnet: section exists, else false. See Tailnet (tailscale) clusters . |
| (anything else) | | Unknown keys (e.g. BlockDeviceMappings, UserData) are passed straight through to the EC2 RunInstances call. |

AWS API access (always required). CHIA picks up credentials by default from ~/.aws/credentials / ~/.aws/config. Override these paths with the environment variables AWS_CONFIG_FILE=/path/to/aws/config and AWS_SHARED_CREDENTIALS_FILE=/path/to/aws/credentials before calling chia up. Instances launch into the account's default VPC.

8.5.2 gcp_nodes

gcp_nodes is the Compute Engine analog of aws_nodes.

```
gcp_nodes:
  project: your-gcp-project          # required
  zone: us-central1-a                # default
  gcp_worker:
    machine_type: n1-standard-1
    count: 2
    ssh_user: chia                    # local user created on the VM
    ssh_public_key: ${HOME}/.ssh/id_ed25519.pub
    ssh_private_key: ${HOME}/.ssh/id_ed25519
    spot: true
    disk_size_gb: 100
```

project (required), zone (default us-central1-a), and network / subnetwork (default to the project's default VPC) are section-level keys. Every other key lives under a named node type:

| Key | Default | Meaning |
|---------------------------------|-----------------|--|
| <code>machine_type</code> | <i>required</i> | GCE machine type (e.g. <code>n1-standard-1</code>). |
| <code>count</code> | <i>required</i> | Number of instances to launch for this type. |
| <code>image</code> | Ubuntu image | Boot image (family or full image URL). |
| <code>zone</code> | section zone | Per-type zone override. |
| <code>disk_size_gb</code> | image default | Boot disk size in GB. |
| <code>spot</code> | False | Launch as Spot/preemptible VMs (cheaper, can be reclaimed). |
| <code>ssh_user</code> | None | Login user CHIA connects as (see Authentication below). |
| <code>ssh_private_key</code> | None | Private key path CHIA's SSH client uses. Recommended (otherwise it falls back to your <code>ssh-agent</code> / <code>~/.ssh</code> defaults). |
| <code>ssh_public_key</code> | None | Public key injected into the VM (metadata method). |
| <code>use_os_login</code> | False | Use OS Login instead of metadata SSH keys (see Authentication below). |
| <code>skip_default_setup</code> | False | Skip CHIA's default host setup (<code>git/conda/docker</code>) and run only your <code>setup_commands</code> . |
| <code>setup_commands</code> | <code>[]</code> | Commands run on the host before it joins the cluster (appended to the defaults unless skipped). |
| <code>setup_timeout</code> | 1800 | Seconds allowed for setup. |
| <code>ssh_timeout</code> | 120 | Seconds to wait for SSH to come up. |
| <code>join_tailnet</code> | <i>auto</i> | Join the cluster over the tailnet instead of reverse SSH tunnels. Defaults to true when a top-level <code>tailnet:</code> section exists, else false. See Tailnet (tailscale) clusters . |
| <i>(anything else)</i> | | Merged into the instance definition sent to the Compute API. |

Authentication. A GCP bring-up uses two distinct credentials at two layers:

- **GCP API access (always required).** Set it up once with `gcloud auth application-default login` (or point `GOOGLE_APPLICATION_CREDENTIALS` at a service-account JSON). A default VPC network must already exist (or set `network`).
- **SSH into the instance**, chosen per node type by `use_os_login`:
 - **Metadata SSH keys** (default). CHIA connects to the GCP instance as `ssh_user` with the matching private key to the public key `ssh_public_key`. This is an ordinary keypair (the GCP analog of an AWS `KeyName`), not tied to any Google identity, and it is silently ignored if the project or org enforces OS Login.
 - **OS Login** (`use_os_login: true`). Ties access to a GCP identity via IAM. You must register your `ssh` key manually (`gcloud compute os-login ssh-keys add`) and set `ssh_user` to the derived posix username. Use this when your org enforces OS Login.

8.5.3 Referencing cloud nodes (@ placeholders)

Because cloud IPs aren't known until provisioning, you refer to them by placeholder of the form `@<node_type>:<index>`, where `<node_type>` is a key under `aws_nodes` / `gcp_nodes` and `<index>` is 0-based. Placeholders are valid anywhere an IP is — in a node type's `compatible_ips` and in `auth.overrides` keys:

```
available_node_types:
  verilator_run_aws:
    resources: {"verilator_run": 32}
    num_workers: 2
    compatible_ips: ["@verilator_run_aws:0", "@verilator_run_aws:1"]
    docker: {...}
```

8.5.4 tunnel_defaults

This applies only to the **SSH-tunnel fallback** (no `tailnet:` section, or `join_tailnet: false` on a type); it is ignored when cloud workers join over the tailnet. For every tunneled cloud IP, CHIA automatically adds an `auth.overrides` entry with a `tunnel` (a per-IP `auth.overrides[ip].tunnel` you set yourself still wins). `tunnel_defaults` overrides the default ports/behavior for *all* auto-tunneled nodes. It accepts any tunnel field except `tunnel_ip` (which CHIA assigns per-worker). Common ones:

```
tunnel_defaults:
  ray_worker_port_min: 20000
  ray_worker_port_max: 20001
  head_worker_port_min: 21000
  head_worker_port_max: 21001
```

Other tunnel fields (with defaults) include `gcs_tunnel_port` (16379), `ray_node_manager_port` (16800), `ray_object_manager_port` (16801), `tool_port_min/max` (18000/18010), `head_tool_port_min/max` (8000/8010), `head_node_manager_port` (29800), `head_object_manager_port` (29801), `kill_orphaned_tunnels` (true), and `pre_tunnel_commands` (sshd GatewayPorts + file-limit setup, run once per physical cloud IP). A typo in any field name fails loudly at load time.

8.6 Tailnet (tailscale) clusters

CHIA can form a cluster across machines whose only mutual connectivity is a [tailscale](#) network, including tailscaled in **userspace-networking mode** (no root, no TUN device, no sudo on any machine). Unlike the SSH-tunnel path for cloud machines, tailnet mode uses no SSH tunnels, no reverse port forwards, no sshd configuration, and no iptables — and workerworker traffic between any 2 hosts works (full mesh).

Under the hood:

Userspace tailscaled delivers *inbound* tailnet TCP to `127.0.0.1:<port>` and requires *outbound* dials to go through its local SOCKS5 proxy. The head and every logical worker register in Ray under a unique loopback IP (bindable even in userspace mode, unlike the real tailnet IP). CHIA runs one small stdlib-Python relay per machine that carries all of Ray's gRPC through a single **HTTP CONNECT proxy**: Ray is pointed at it via `grpc_proxy`, the proxy reads the destination from each CONNECT request, maps the peer's loopback IP to its tailnet address, and forwards through the SOCKS5 proxy. Because there are no per-port outbound listeners, port blocks need only be unique **per machine** — two workers on different machines reuse the same ports, so port consumption does not grow with cluster size. (ChiaTool traffic is plain HTTP and keeps small per-port SOCKS listeners for peer tool ports.) Workerworker traffic between hosts works (full mesh).

The presence of the `tailnet:` block opts the cluster in — every worker IP that is not the head machine is automatically treated as a tailnet machine, and SSH to it automatically dials through the SOCKS5 proxy (`nc -X 5 -x <socks_proxy> %h %p`; the head needs OpenBSD netcat). No per-IP overrides are required:

```
tailnet:
  head_tailnet_ip: 100.64.0.1      # required: the head's tailscale IP
  socks_proxy: 127.0.0.1:1055     # tailscaled --socks5-server on every machine

provider:
  head_ip: 10.0.0.1               # how CHIA SSHes to the head (real IP)

auth:
  ssh_user: ${USER}

available_node_types:
  tailscale_worker:
```

(continues on next page)

(continued from previous page)

```
resources: {"tailscale_worker": 4}
num_workers: 1
compatible_ips: [100.64.0.2] # workers by their tailscale IPs
```

`auth.overrides.<ip>` entries still win for special cases — a different `ssh` user/key for one host, or a custom `ssh_proxy_command` when the head's `nc` is not OpenBSD `netcat`.

Cloud workers. When a `tailnet:` section is present, `aws_nodes` and `gcp_nodes` workers join the cluster over the tailnet **by default** instead of reverse SSH tunnels. This requires `tailnet.auth_key` — use a **reusable** (ideally ephemeral, pre-authorized) key, referenced as `${TS_AUTHKEY}` so it stays out of the file.

Under the hood: On-prem hosts can opt into the same managed lifecycle with `manage_tailscale: true` in their `auth.overrides` entry or by setting `manage_all: true` in the `tailnet:` section; by default CHIA assumes on-prem tailnet hosts already run their own `tailscaled`. CHIA handles the whole lifecycle: the userspace `tailscale` binaries are installed during instance setup (static tarball, no root), `tailscaled --tun=userspace-networking` is started with the SOCKS5 proxy from `socks_proxy`, the machine is joined with `tailscale up --auth-key=<tailnet.auth_key>`, and its tailnet IP is discovered and wired into the relay mesh. Orchestration SSH continues over the instance's public IP.

Fully managed clusters. Set `manage_all: true` in the `tailnet:` section and CHIA manages `tailscale` on **every** machine, including the head — no manual `tailscaled` anywhere, and `head_tailnet_ip` may be omitted (it is discovered at bring-up). The constraint: `tailscale` cannot be bootstrapped over `tailscale`, so under `manage_all` every worker must be addressed by an ordinary SSH-reachable IP/hostname; a worker listed by a `100.64.0.0/10` address fails loudly at config load. Opt individual machines out with `manage_tailscale: false` in their `auth.overrides` entry (address those by their tailnet IP and run `tailscaled` there yourself). On `chia down`, CHIA-managed daemons are stopped (the head's last); their state persists in `tailscale_dir`, so re-ups rejoin without consuming the auth key unless the directory was cleaned.

Additional `tailnet:` fields for managed machines: `auth_key` (as above), `tailscale_version` (pinned tarball version) and `tailscale_dir` (install/state directory on managed machines, default `/tmp/<cluster_name>/tailscale` — per-cluster, so cluster daemons never collide with each other or a personally-run `tailscaled`; pair with a distinct `socks_proxy` port for full isolation. Keep the path short: the `tailscaled` control socket lives under it and Unix socket paths are limited to ~107 characters — checked at config load. Note `/tmp` state may not survive reboots, so the next `chia up` rejoins using the reusable auth key).

Optional `tailnet:` port fields (defaults in parentheses): `head_advertise_ip` (127.200.0.1), `gcs_port` (6379 — must match `--port` in `head_start_ray_commands`), `connect_proxy_port` (13129 — the relay's HTTP CONNECT listener that Ray's `grpc_proxy` points at), `head_node_manager_port` (23744), `head_object_manager_port` (23745), `head_tool_port_min/max` (23760/23770), `head_worker_port_min/max` (23808/23935), `worker_block_base` (24000), `worker_block_size` (256), `tool_port_count` (11), and `worker_port_count` (128). Port blocks are indexed **per machine** (reused across machines), and grow upward from `worker_block_base` meeting no head port (the head owns the block just below it), so up to 162 workers fit **on a single machine** at the defaults before allocation refuses — cluster size is unbounded. A machine's Ray worker-port range must exceed its CPU count (Ray prestarts one worker process per CPU). CHIA injects `--node-ip-address`, the pinned ports, and the `grpc_proxy` env into the head's and workers' `ray start` commands automatically, starts the relays before the workers, and stops them on `chia down`.

Constraints: every worker must be a tailnet worker or colocated on the head machine (the head advertises a loop-back IP that LAN workers cannot route to); mixing with SSH-tunneled/cloud workers is rejected at config load; `chia up --add` is not yet supported (re-run `chia up` — existing workers are detected and skipped). See `examples/tailscale/` for a complete working example.

8.7 A mixed on-prem + cloud example

This config keeps the head and several worker types on owned machines while bursting Verilator simulation onto AWS, connecting everything over a **fully tailnet** cluster (the recommended setup for cloud workers). To run purely on-prem, delete the `aws_nodes` section and the `@verilator_run_aws:*` placeholders; to add more cloud capacity, raise count and add matching placeholders.

Note

The `tailnet: {manage_all: true}` block makes this an all-tailnet cluster: CHIA installs and joins userspace tailscale on **every** machine — the head, the on-prem workers, and the provisioned cloud instances — and routes Ray through per-machine CONNECT proxies. No SSH tunnels, no `tunnel_defaults`, no `sshd` changes. Every machine must be addressed by an ordinary SSH-reachable name/IP (as they all are here), since tailscale can't be bootstrapped over tailscale. `head_tailnet_ip` is omitted — `manage_all` discovers it at bring-up. Note this routes on-premon-prem traffic over userspace WireGuard too; if your on-prem workers already share a fast LAN and you only need to burst to cloud, the SSH-tunnel path (a `tunnel_defaults` block, no `tailnet:` section) keeps that local-network traffic direct. See *Tailnet (tailscale) clusters*.

```
cluster_name: ChiaClusterExample
```

```
available_node_types:
```

```
    # On-prem Verilator workers, pinned to specific machines.
```

```
    verilator_run:
```

```
        resources: {"verilator_run": 8}
```

```
        num_workers: 4
```

```
        compatible_ips: [machine0, machine1, machine2, machine2]
```

```
        worker_env_commands: ["source ~/.bashrc && conda activate chia_env"]
```

```
        docker:
```

```
            image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
```

```
            container_name: "chia-verilator-run-{$USER}"
```

```
            run_options:
```

```
                - --ulimit nofile=65536:65536
```

```
                - --shm-size=10.24gb
```

```
    # Cloud Verilator workers - provisioned by the aws_nodes block below.
```

```
    verilator_run_aws:
```

```
        resources: {"verilator_run": 32}
```

```
        num_workers: 3
```

```
        compatible_ips:
```

```
            - "@verilator_run_aws:0"
```

```
            - "@verilator_run_aws:1"
```

```
            - "@verilator_run_aws:2"
```

```
        docker:
```

```
            image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
```

```
            container_name: "chia-verilator-run-{$USER}"
```

```
            pull_before_run: True
```

```
            run_options:
```

```
                - --ulimit nofile=65536:65536
```

```
                - --shm-size=10.24gb
```

```
    # On-prem VLSI workers (no container; uses the host conda env).
```

(continues on next page)

(continued from previous page)

```

vlsi:
  resources: {"VLSI": 1, "syn": 1, "cacti": 4}
  num_workers: 6
  compatible_ips: [machine1, machine2, machine3, machine4, machine5, machine6]
  worker_env_commands:
    - "source ~/.bashrc && source /ecad/tools/vlsi.bashrc && conda activate chia_
↪env"

# Provision the cloud half of the cluster.
aws_nodes:
  region: us-east-1
  verilator_run_aws:
    KeyName: my-keypair
    InstanceType: c5.9xlarge
    count: 3
    ImageId: ami-0ec10929233384c7f
    ssh_user: ubuntu
    ssh_private_key: /home/${USER}/my-keypair.pem
    setup_commands:
      - "echo ${GITHUB_TOKEN} | docker login ghcr.io -u myuser --password-stdin"
    BlockDeviceMappings:
      - DeviceName: /dev/sda1
        Ebs:
          VolumeSize: 500
          VolumeType: gp3

# Join every machine (head + on-prem + cloud) to the tailnet.
# manage_all: CHIA installs/starts/joins userspace tailscale itself;
# head_tailnet_ip is discovered at bring-up.
tailnet:
  manage_all: true
  auth_key: ${TS_AUTHKEY}      # reusable tailscale auth key

provider:
  type: local
  head_ip: machine7
  # No worker_ips: the worker pool is the union of every node type's
  # compatible_ips below (on-prem hosts + the cloud @-placeholders).

auth:
  ssh_user: ${USER}
  ssh_private_key: /home/${USER}/.ssh/${USER}

head_env_commands: ["source ~/.bashrc && conda activate chia_env"]

head_start_ray_commands:
  - ray stop
  - ray start --head --port=6379 --include-dashboard=True --dashboard-agent-listen-
↪port=0

worker_start_ray_commands:
  - ray stop

```

(continues on next page)

(continued from previous page)

```
- ray start --address=$RAY_HEAD_IP:6379 --dashboard-agent-listen-port=0
```

8.8 Multiple heads on a single physical machine

On shared lab machines it is common for two users (or two clusters) to want a Ray *head* on the same host. A head binds several fixed TCP ports, so the second cluster must move every one of them off the defaults or `ray start` (or the first cluster) will fail. Four ports matter — the GCS port (default 6379), the dashboard (8265), the Ray client server (10001), and the head’s dashboard agent (52365) — and the worker join address must follow the new GCS port. Pick replacements that are free on the host, and note that an explicitly assigned port must **not** fall inside Ray’s worker-port range (10002–19999 by default): `ray start` rejects the overlap, which is why the client-server port below jumps to 20101 rather than 10101.

head_start_ray_commands:

```
- ray stop
- ray start --head --port=6479 --include-dashboard=True
  --dashboard-port=8365 --ray-client-server-port=20101
  --dashboard-agent-listen-port=52465
```

worker_start_ray_commands:

```
- ray stop
- ray start --address=$RAY_HEAD_IP:6479 --dashboard-agent-listen-port=0
```

Nothing else needs to move: worker nodes already use `--dashboard-agent-listen-port=0` (OS-assigned) in the examples above, and the remaining head ports (object manager, node manager, metrics export, ...) are randomized by default. Containerized workers coexist regardless. `ray stop` is also safe on a shared host — it can only signal processes owned by the invoking user, so it never touches the other cluster.

Two operational consequences of non-default ports:

- **Drivers must be pinned to the cluster address** — connect with `ray.init(address="<head_ip>:6479")` (or the `RAY_ADDRESS` environment variable), never `address="auto"`: with several Ray instances alive on one machine, auto-discovery picks one arbitrarily, and it may be the other user’s cluster.
- **Job submission must name the dashboard** — `chia job submit --address http://127.0.0.1:8365` ... (the dashboard listens on localhost on the head, so submit from the head or tunnel to it).

8.9 Command execution order

When you run `chia up`, CHIA sets up the head node, assigns each declared worker to a machine (constrained `compatible_ips` types first, then unconstrained — see `assign_nodes` in `chia/cluster/config.py`), connects any cloud nodes into the cluster (joining them to the tailnet and starting the per-machine relays, or establishing SSH tunnels on the fallback path), and then sets up the workers (in parallel across machines, sequentially within a machine).

8.9.1 chia up — head node

All head commands run on the host over SSH (the head node is never containerized):

| | |
|--|-------------------------------|
| 1. initialization_commands | ← each in its own SSH session |
| 2. file_mounts rsync | ← separate rsync processes |
| 3. — single SSH session (env persists) — | |
| head_env_commands | e.g. conda activate |
| setup_commands | global |

(continues on next page)

(continued from previous page)

```
head_setup_commands
head_start_ray_commands
```

```
ray stop; ray start --head
```

8.9.2 chia up — worker node

Without a container, everything runs on the host:

1. initialization_commands ← each in its own SSH session
2. file_mounts rsync ← separate rsync processes
3. — single SSH session (env persists) —

| | |
|---|---|
| <pre><type>.worker_env_commands setup_commands <type>.worker_setup_commands export RAY_HEAD_IP=... worker_start_ray_commands (--resources injected)</pre> | <pre>per node type global per node type</pre> |
|---|---|

With a container, the host pulls/starts the container first, then the main script runs **inside** it:

1. initialization_commands ← HOST, each in its own SSH session
2. file_mounts rsync ← HOST, separate rsync processes
3. docker setup ← HOST (pull, run, run_setup_commands inside)
4. — single session INSIDE CONTAINER (env persists) —

| | |
|---|--|
| <pre><type>.worker_env_commands setup_commands <type>.worker_setup_commands export RAY_HEAD_IP=... worker_start_ray_commands (--resources injected)</pre> | |
|---|--|

Cloud (and other tailnet/tunneled) workers get some extra steps before the main script, and the Ray ports pinned in `worker_start_ray_commands`:

- **Tailnet workers** (the recommended cloud path): CHIA joins the machine to the tailnet if it manages tailscale there, starts the per-machine relay, and injects the `grpc_proxy` env.
- **Tunneled workers** (SSH-tunnel fallback): CHIA runs `pre_tunnel_commands` once per physical cloud IP and brings up the reverse SSH tunnel.

8.9.3 chia down

Workers are torn down first (in parallel), then the head:

Workers (with container):

1. docker exec:


```
<type>.worker_env_commands
ray stop
```
2. docker stop <container>
3. docker rm -f <container>

Workers (no container):

1. — single SSH session —

| | |
|--|--|
| <pre><type>.worker_env_commands ray stop</pre> | |
|--|--|

Head (after all workers):

— single SSH session —

(continues on next page)

(continued from previous page)

```
head_env_commands
head_tearardown_commands
ray stop
```

Note

`head_env_commands` and the per-type `worker_env_commands` run on **both** `chia up` and `chia down` — they are for environment activation. Use the `*_setup_commands` hooks for one-time setup. When `head_ip` is also listed in a node type's `compatible_ips` (so the head also hosts a worker) and that worker isn't containerized, CHIA skips `ray stop` on the worker so it doesn't kill the head's Ray process.

SETTING UP LOGICAL WORKERS

A CHIA cluster starts from a set of **physical machines**, but the things that actually execute your `@ChiaFunction` nodes are **logical workers** that advertise a set of *resources* (virtualized hardware and software capabilities) and are mapped onto compatible machines when the cluster comes up (see [Architecture Overview](#)). This guide walks through defining a logical worker in your cluster YAML, choosing between a **dockerized** and a **non-dockerized (bare)** worker, and — most importantly — using CHIA’s command hooks to make sure the environment your remote functions execute in is the one you intended.

For the full YAML schema this guide references, see [Cluster Configuration Reference](#).

- *What a logical worker is*
- *Anatomy of a worker definition*
- *Dockerized vs. non-dockerized workers*
- *Controlling the execution environment*
- *Verifying the environment*
- *Placement and scaling*
- *Lifecycle*
- *Common pitfalls*
- *See also*

9.1 What a logical worker is

A logical worker is one Ray worker process, launched by `chia up` on some physical machine, that advertises the resources you gave it. Its definition lives under `available_node_types` in the cluster YAML and answers four questions:

- **What can it do?** — `resources`, e.g. `{"verilator_run": 8}`. A node decorated `@ChiaFunction(resources={"verilator_run": 1})` waits until a worker advertising that resource has a free unit, then runs there. Quantities matter: a worker advertising 8 units can concurrently run 4 nodes requesting 2 units.
- **How many of them?** — `num_workers`.
- **Where may it run?** — `compatible_ips`, the machines this type can be placed on. The cluster’s worker pool is the union of every type’s `compatible_ips`; multiple workers (of the same or different types) can share one machine.

- **What environment does it execute in?** — either a container image (`docker: block`) or the host's own environment, plus the command hooks described below.

A worker type is *logical* because it says nothing about a specific machine: the same `verilator_run` definition can put three workers on one big on-prem box, or spread them across a mix of on-prem and cloud machines, without any change to the loop code that dispatches onto it.

9.2 Anatomy of a worker definition

```
available_node_types:
  verilator_run:
    resources: {"verilator_run": 8}      # what this worker advertises
    num_workers: 4                      # how many workers of this type
    compatible_ips: [machine9, machine10] # where they may be placed
    worker_env_commands:                # environment activation (up AND down)
      - "source ~/.bashrc && conda activate chia_env"
    worker_setup_commands: []           # one-time setup (up only)
    docker:                             # omit this block for a bare worker
      image: "ghcr.io/ucb-bar/chia-verilator-run:latest"
      container_name: "chia-verilator-run-${USER}"
      run_options:
        - --ulimit nofile=65536:65536
        - --shm-size=10.24gb

worker_start_ray_commands:
  - ray stop
  - ray start --address=$RAY_HEAD_IP:6379 --dashboard-agent-listen-port=0
```

CHIA injects `--resources` (and, for tailnet or tunneled workers, pinned ports) into the `ray start` line automatically — you never write the resources into `worker_start_ray_commands` yourself.

9.3 Dockerized vs. non-dockerized workers

Both dockerized and non-dockerized workers join the same Ray cluster and are indistinguishable to the loop; they differ in *where the worker's environment comes from* and *what CHIA has to do to create it*.

| | Dockerized (docker: block) | Bare (no docker: block) |
|-----------------------|---|---|
| Environment source | The container image. Tools, Ray, Python, and the chia package are baked in; every machine runs the identical environment. | The host itself. Every compatible machine must already provide the tools, a matching Ray/Python, and the chia package (typically via a shared conda env). |
| Where commands run | Host: <code>initialization_commands</code> , <code>rsync</code> , container start. Everything else (env/setup hooks, <code>ray start</code> , your functions) runs inside the container. | Everything runs directly on the host over SSH. |
| Isolation | Own filesystem and PID namespace. An agent node cannot read files you did not mount, and <code>ray stop</code> in one worker cannot kill another. | None — workers share the host's filesystem and processes. CHIA skips <code>ray stop</code> for the second and later bare workers on a host so they don't kill each other, but nothing stops your functions from interfering through the filesystem. |
| First-time setup cost | An image pull (seconds to minutes). E.g. CHIA's Chipyard image replaces an ~hour-long from-source setup. | Whatever it takes to install the toolchain on every machine, by hand or via <code>worker_setup_commands</code> . |
| Host access | Nothing from the host is visible unless you mount it with <code>-v</code> in <code>run_options</code> (CHIA adds <code>--net=host</code> and <code>--shm-size=8g</code> , and no volume mounts, itself). | Full access to the host — needed for host-attached hardware (FPGAs), license servers, and site tool trees (e.g. <code>/ecad</code>). |

Use a dockerized worker when you can. It front-loads environment setup, makes the cluster portable across heterogeneous machines, and isolates agentic workers — an LLM worker that can only see its own container cannot copy your reference implementation or read a proprietary PDK. The image must be CHIA-compatible (Ray and Python matching the head, chia package installed, bash login shell, `openssh-client/rsync`); building one is covered in [Building a CHIA-compatible Docker image](#).

Use a bare worker when the environment can't be containerized: FireSim workers that drive physical FPGAs, vendor EDA tools tied to host-mounted installs and license daemons, or machines where Docker isn't available. A bare worker's YAML is the same minus the `docker: block` — but now *you* are responsible for the host environment, usually by activating it in `worker_env_commands`, which are run before starting the worker process, meaning that all nodes execute in a process started from that environment (e.g. the `$PATH` from that environment). We recommend that bare workers have the same conda environment as the head, to ensure that there are no Ray or CHIA version mismatches. To do this, export the head's environment to a YAML file with `conda env export --no-builds > env.yml` (`--no-builds` drops platform-specific build strings so the file ports across machines) and copy it to the worker host. There, recreate it with `conda env create -f env.yml` — the new environment keeps the name recorded in the file (`chia_env`), so the `conda activate chia_env` in `worker_env_commands` works unchanged.

The process is started as the SSH user from the auth block — the global `ssh_user` / `ssh_private_key` by default, overridable **per machine** under `auth.overrides` (keyed by hostname/IP, or an `@node_type:index` placeholder for cloud nodes). Because a bare worker runs as its SSH login user, the override decides which Unix account your functions execute under on that machine — and therefore whose `~/ .bashrc` is sourced and which files, groups, and tool licenses they can reach:

```
available_node_types:
  vlsi:
    resources: {"VLSI": 1, "syn": 1}
    num_workers: 6
    compatible_ips: [machine1, machine2, machine3]
    worker_env_commands:
      - "source ~/.bashrc && source /usr/licensed/scripts/ecad.bashrc && conda_
```

(continues on next page)

(continued from previous page)

```
↪activate chia_env"

auth:
  ssh_user: ${USER}                # default for every machine
  # ssh_private_key: ~/.ssh/key    # omit if your key is in ssh-agent
  overrides:
    machine3:                      # per-machine: different account here
      ssh_user: vlsi-svc
      ssh_private_key: ~/.ssh/vlsi_svc_key
```

Here workers landing on machine1 / machine2 run as `${USER}`, while the one on machine3 runs as the `vlsi-svc` service account. CHIA fills in `auth.overrides` entries automatically for cloud-provisioned machines (SSH user, key, and tailnet or tunnel config), so you normally only write them by hand for on-prem hosts.

A `docker:` block may also be given once at the top level as a cluster-wide default that individual node types override. The head node is never containerized.

9.4 Controlling the execution environment

The central fact to understand: **the shell session that runs your env/setup hooks is the same session that runs ray start**. The Ray worker processes it spawns inherit that session's environment — and those processes are exactly where your `@ChiaFunction` bodies execute. So “making the remote environment correct” means putting the right commands in the right hooks, in this order (full walk-through in [Cluster Configuration Reference](#)):

- | | |
|--|--|
| 1. <code>initialization_commands</code> | ← HOST, each in its own SSH session |
| 2. <code>file_mounts rsync</code> | ← HOST |
| 3. <code>docker setup</code> | ← HOST (dockerized only: pull, run, then <code>run_setup_commands</code> inside) |
| 4. – single session – on the host (bare) or INSIDE the container – | |
| <code><type>.worker_env_commands</code> | per-type env activation |
| <code>setup_commands</code> | global, all nodes |
| <code><type>.worker_setup_commands</code> | per-type one-time setup |
| <code>export RAY_HEAD_IP=...</code> | |
| <code>worker_start_ray_commands</code> | (--resources injected) |

Which hook to use for what:

- `worker_env_commands` — **environment activation**: conda envs, tool `env.sh` scripts, exports your functions need. These run on both `chia up` and `chia down` (teardown needs ray on the PATH too), so they must be idempotent and side-effect-free. This is the hook that makes `source ~/chipyard/env.sh` visible to every node that runs on the worker.
- `worker_setup_commands` — **one-time setup** during `chia up` only: cloning a repo, generating a config, warming a cache. Runs after activation, in the same session.
- `setup_commands` — like `worker_setup_commands` but global: runs on the head and every worker (inside the container for dockerized types).
- `docker.run_setup_commands` — dockerized only; runs inside the container right after it starts, *before* the main session (e.g. `git pull` a mounted checkout, fix up `/etc/passwd` for an arbitrary `--user`).
- `initialization_commands` — host-side, outside any container, each command in its own SSH session (no shared environment). Use for host prerequisites like `docker login`.

Note

All of these run in a **non-interactive login shell** (`bash --login`): `/etc/profile` and `~/.bash_profile` are sourced but `~/.bashrc` is **not** — and many `~/.bashrc` files exit early for non-interactive shells anyway. If your conda setup lives in `~/.bashrc`, source it explicitly: `source ~/.bashrc && conda activate chia_env`. In a dockerized worker, prefer baking the `PATH` into the image with `ENV` (see [Building a CHIA-compatible Docker image](#)).

9.5 Verifying the environment

Before trusting a worker with real work, check it from the outside in.

1. Preview the generated scripts. `--dry-run` prints the node assignments and the exact per-worker scripts CHIA would run, without touching any machine:

```
chia up my_cluster.yaml --dry-run
```

2. Confirm the workers registered. After `chia up`, on the head node (in the same env as `head_env_commands`):

```
ray status          # every worker and its advertised resources
```

or, for the raw resource totals:

```
python -c "import ray; ray.init(address='auto'); print(ray.cluster_resources())"
```

Your custom resources (`verilator_run`, `chipyard`, ...) should appear with the expected totals (`num_workers` × the per-worker quantity). A worker whose Ray or Python *minor* version doesn't match the head fails to join — recheck image/conda pins if a resource is missing.

3. Inspect a worker's shell directly. For a dockerized worker, exec into its container on the machine it landed on (CHIA appends the worker index to `container_name`):

```
ssh machine9
docker exec -it chia-verilator-run-${USER}-0 bash --login
which ray && ray --version      # matches the head?
python --version
which verilator                # your tool on the login-shell PATH?
```

For a bare worker, SSH to the host, run your `worker_env_commands` by hand, and make the same checks. Remember to test with `bash --login` (not your interactive shell) so you see what CHIA's scripts see.

4. Smoke-test with a real dispatch. The ground truth is a node that runs on the worker and reports what it found. Request exactly one unit of the worker's resource and return the environment:

```
import getpass, os, shutil, socket
from chia.base.ChiaFunction import ChiaFunction, get

@ChiaFunction(resources={"verilator_run": 1})
def check_env() -> dict:
    return {
        "host": socket.gethostname(),
        "user": getpass.getuser(),
        "verilator": shutil.which("verilator"),
        "ray_head": os.environ.get("RAY_HEAD_IP"),
```

(continues on next page)

(continued from previous page)

```

    "path": os.environ.get("PATH"),
}

print(get(check_env.chia_remote()))

```

If `shutil.which` comes back `None` for a tool you sourced in `worker_env_commands`, the activation didn't take (most often the `~/ .bashrc` pitfall above). Because this dispatches through the normal scheduler, it also confirms placement: the reported host must be one of the type's `compatible_ips`.

9.6 Placement and scaling

Workers are assigned to machines when the cluster comes up. Each type is constrained to its `compatible_ips`; within them, `balance_level` picks the strategy: `cluster` (default) places each worker on the machine with the fewest workers globally, `worker` spreads this type's own workers evenly across its IP pool regardless of what other types placed. Listing a machine twice in `compatible_ips` biases more workers onto it.

Cloud machines fold into the same mechanism: declare instances under `aws_nodes` / `gcp_nodes` and reference them in `compatible_ips` with `@<node_type>:<index>` placeholders. CHIA provisions the instances, joins them to the cluster over tailscale by default (the recommended path; SSH reverse tunnels are an opt-in fallback), and pins the worker's Ray ports automatically — the logical-worker definition itself is unchanged. See [Cloud nodes](#).

9.7 Lifecycle

```

chia up my_cluster.yaml           # provision cloud nodes, assign + start workers
chia up my_cluster.yaml --add     # add newly-declared workers / restart dead ones
chia down my_cluster.yaml         # stop workers (and containers), then the head

```

`chia up --add` compares the YAML against the live cluster and only brings up workers that aren't already registered, so you can grow a running cluster (or reintegrate a machine that died) without downtime. On `chia down`, dockerized workers get `ray stop` inside the container followed by `docker stop` / `docker rm`; bare workers get `ray stop` on the host (skipped when the worker shares the head's machine, so it can't kill the head's Ray).

9.8 Common pitfalls

- **Missing tools at run time** — activation lives in `~/ .bashrc`, which login shells don't source. Move it to `worker_env_commands` or the image's `ENV`.
- **Worker never joins the cluster** — Ray/Python minor-version mismatch with the head. Pin both to the head's versions (see [Building a CHIA-compatible Docker image](#)).
- **Container name collisions on shared machines** — include `${USER}` in `container_name`; CHIA only appends the worker index.
- **Locally-built image + ``pull\_before\_run: true``** (the default) — the pull fails. Set `pull_before_run: false`.
- **Files missing inside a container** — CHIA adds no volume mounts; add `-v` entries to `run_options`, or bake the files into the image (mounts can't work for cloud workers — the files don't exist on the remote host).
- **One bare worker kills another** — don't put `ray stop` in `worker_setup_commands`; leave it in `worker_start_ray_commands` where CHIA knows when to skip it.

9.9 See also

- *Architecture Overview* — how loops, nodes, workers, and clusters fit together.
- *Cluster Configuration Reference* — every YAML key and the exact command execution order.
- *Building a CHIA-compatible Docker image* — building a CHIA-compatible worker image.
- *ChiaFunction* — how nodes request the resources workers advertise.

BUILDING A CHIA-COMPATIBLE DOCKER IMAGE

CHIA can run worker nodes inside Docker containers (see [Cluster Configuration Reference](#)). When you bring a cluster up with `chia up`, CHIA starts each worker's container, then runs your tool inside it. This guide shows how to build an image that works as a CHIA worker — starting from a CHIA base image, and (for the cases that need it) from scratch.

- *What “CHIA-compatible” means*
- *Extend a CHIA base image with your tool*
- *Extend your tool image with CHIA*
- *Wiring the image into a cluster*
- *Pitfalls*
- *See also*

10.1 What “CHIA-compatible” means

At cluster start, `chia up` (over SSH to the worker's host) does roughly this:

```
docker run -d --net=host --shm-size=8g <your run_options> <image> sleep infinity
docker exec -i <container> bash --login # <- your worker_setup_commands + `ray start ..
↪ .`
```

The container is kept alive by `sleep infinity`; **Ray is not started by the image** — CHIA execs a login shell into the running container and runs the `ray start --address=...` command from your cluster YAML there. So a compatible image only has to provide the right *environment* for that command to succeed. Concretely it must:

- have **Ray ``2.54.0``** importable on the login-shell PATH, on a **Python 3.10.19** interpreter;
- have the **chia package installed** (so worker-side `chia` imports and MCP *tool servers* work);
- provide a **``bash`` login shell**;
- ship **``openssh-client``** and **``rsync``** for file transport.

The Ray and Python versions must **match the head** — Ray refuses to connect a worker whose Ray or Python *minor* version differs from the cluster head's. The quickest way to satisfy all of the above is to start from a CHIA base image, which already does.

10.2 Extend a CHIA base image with your tool

ghcr.io/ucb-bar/chia is the canonical base. It is rayproject/ray:2.54.0-cpu with the chia package pip-installed on top, so it already provides the right Python, Ray 2.54.0, the chia package (and its MCP/FastAPI dependencies), and a ray user with HOME=/home/ray. You just add your tool:

```
FROM ghcr.io/ucb-bar/chia:latest

# Install your tool's system dependencies as root, then drop back to `ray`.
USER root
RUN apt-get update \
    && apt-get install -y --no-install-recommends my-tool openssh-client rsync \
    && rm -rf /var/lib/apt/lists/*
USER ray

# (Optional) make your tool discoverable on the login-shell PATH.
ENV PATH="/opt/my-tool/bin:${PATH}"
```

That image is ready to be a CHIA worker — no EXPOSE, ENTRYPOINT, or ray start is needed in the Dockerfile; CHIA supplies those at run time.

Note

A login shell (bash --login) sources /etc/profile and ~/.bash_profile / ~/.profile but **not** ~/.bashrc. Put anything your tool needs on the PATH via ENV in the Dockerfile, or activate it in worker_env_commands / worker_setup_commands in the cluster YAML — don't rely on ~/.bashrc.

10.3 Extend your tool image with CHIA

When you already have a working image — a vendor EDA tool, your own build environment, anything not based on rayproject/ray — you don't rebuild it from scratch. You **layer a self-contained CHIA interpreter on top**, leaving the image's own Python and tools untouched. CHIA only needs Ray, the chia package, and Python 3.10.19; keeping those in their own venv means they can't disturb whatever Python your tool already depends on.

The pattern (the same one CHIA's chia-circt image uses, since its base ships a different Python): install a relocatable Python 3.10.19, make a dedicated venv for Ray + chia, and put it first on the PATH.

```
FROM your-registry/your-tool-image:latest

USER root
# Add transport + download tools if the base lacks them.
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl ca-certificates openssh-client_
    && rsync \
    && rm -rf /var/lib/apt/lists/*

# A relocatable Python 3.10.19 (glibc 2.17), independent of the image's own
# Python. Pin these to match every other worker on the cluster.
ARG PYTHON_VERSION=3.10.19
ARG PBS_TAG=20260203
RUN curl -fsSL \
    "https://github.com/astral-sh/python-build-standalone/releases/download/${PBS_TAG}/
```

(continues on next page)

(continued from previous page)

```

→ cpython-${PYTHON_VERSION}+${PBS_TAG}-x86_64-unknown-linux-gnu-install_only.tar.gz" \
  | tar -xzf - -C /opt \
&& mkdir -p /home/ray/anaconda3/envs \
&& /opt/python/bin/python3 -m venv /home/ray/anaconda3/envs/py_worker \
&& /home/ray/anaconda3/envs/py_worker/bin/pip install --no-cache-dir \
    "ray[default]==2.54.0" pyyaml

# Install the chia package into that venv. chia isn't on PyPI, so install from
# your chia checkout (here copied in from the build context).
COPY chia /tmp/chia
RUN /home/ray/anaconda3/envs/py_worker/bin/pip install --no-cache-dir /tmp/chia \
    && rm -rf /tmp/chia

# Put CHIA's interpreter first so `ray`, `python`, and `chia` resolve to it.
ENV PATH="/home/ray/anaconda3/envs/py_worker/bin:${PATH}"
ENV HOME=/home/ray
WORKDIR /home/ray
# Allow an arbitrary UID - chia up may launch with --user $(id -u):$(id -g).
RUN mkdir -p /home/ray && chmod -R a+rwX /home/ray && chmod a+rw /etc/passwd

```

Note

Prepending the venv to PATH makes python / pip resolve to CHIA's interpreter for the whole container. If your image's tools rely on *their* own python being first on the PATH, either invoke them by absolute path, or skip the global ENV PATH and instead prepend /home/ray/anaconda3/envs/py_worker/bin only in the node type's worker_env_commands — that is enough for CHIA's ray start to find Ray.

The same steps on a bare ubuntu (or python) base build a worker entirely from scratch. `dockerfiles/ChiaCircBaseDockerfile` is a complete worked example of this pattern (CHIA layered onto a CIRCT toolchain on ubuntu:24.04), and `dockerfiles/SimWorkerMinimalDockerfile` shows the smallest viable image.

10.4 Wiring the image into a cluster

Reference the image from a node type's docker: block, and start Ray with the cluster's worker_start_ray_commands:

```

available_node_types:
  my_tool_worker:
    resources: {"my_tool": 1}           # what this worker advertises
    num_workers: 1
    compatible_ips: [${THIS_MACHINE}]  # required when num_workers > 0
    worker_setup_commands: ["source /opt/my-tool/env.sh"]
    docker:
      image: "ghcr.io/you/chia-my-tool:latest"
      container_name: "chia-my-tool-${USER}"
      pull_before_run: true             # set false for a locally-built image
      run_options:
        - --shm-size=10.24gb

worker_start_ray_commands:

```

(continues on next page)

(continued from previous page)

```
- ray stop
- ray start --address=$RAY_HEAD_IP:6379
```

A few practical notes on the `docker: block` (full reference: [Cluster Configuration Reference](#)):

- `pull_before_run` — defaults to `true`. Set it to `false` when the image was built locally and isn't in a registry, or CHIA's `docker pull` will fail.
- `run_options` — flags spliced verbatim into `docker run`. CHIA adds **no** volume mounts itself; add `-v` here. The SSH-agent convention is `-v $SSH_AUTH_SOCK:/ssh-agent -e SSH_AUTH_SOCK=/ssh-agent` (CHIA `chmods /ssh-agent` for you). `--net=host` and `--shm-size=8g` are always added by CHIA.
- `resources` — the labels this worker advertises; `@ChiaFunction` nodes and tools land here by requesting them (see [ChiaFunction](#)).
- `compatible_ips` — the machines this worker type may run on; required for any type with workers.

10.5 Pitfalls

- **Version mismatch.** A worker whose Ray or Python *minor* version differs from the head silently fails to join the cluster. Match the head's `2.54.0 / 3.10.x` exactly.
- `~/.bashrc` **is not sourced.** Use `ENV/profile` files or `worker_env_commands` to put tools on the `PATH` (see the note above).
- **No automatic mounts.** Anything the container needs from the host must be a `-v` entry in `run_options`.
- **Local image + `pull_before_run: true`.** CHIA will try to pull and fail; set `pull_before_run: false` for images that only exist locally.
- **Tool-server ports.** Because containers run with `--net=host`, an MCP tool server binds a port directly on the host (probing from `8000` by default). Make sure that range is free on machines that host tool-serving workers.

10.6 See also

- [Cluster Configuration Reference](#) — the full `docker: / node-type` config schema.
- [Quickstart: Say Hello \(World\) with CHIA](#) — brings up a cluster with CHIA's Chipyard and Verilator images.
- [ChiaTool](#) — MCP tool servers, which run inside these worker containers.
- [ChiaFunction](#) — how nodes request the `resources` a worker advertises.

AUTHENTICATING GEMINI BACKENDS: ANTIGRAVITY AND OPENCODE + VERTEX AI

We recommend reaching Gemini through Google credentials that live on the *host* and are bind-mounted into the LLM worker containers:

- `chia.models.antigravity.AntigravityLLM` — Google’s **Antigravity CLI** (agy). OAuth only; credentials under `~/.gemini` by default.
- `chia.models.opencode.OpenCodeLLM` with the `google-vertex` provider — the **OpenCode CLI** calling Gemini on **Vertex AI**. Google Application Default Credentials (ADC) under `~/.config/gcloud` by default.

Both can be set up once on the host and then mounted into the credential directory in the corresponding docker container in your cluster config yaml. `examples/memcpy/cluster.yaml` and `examples/circt_issue_solver/cluster_*.yaml`

show this.

- *Antigravity (agy)*
- *OpenCode with Gemini on Vertex AI*
- *Quick reference*

11.1 Antigravity (agy)

11.1.1 Install and sign in

```
curl -fsSL https://antigravity.google/cli/install.sh | bash    # installs ~/.local/bin/agy
agy                                                         # first run: interactive
↪ sign-in
```

agy prints a Google OAuth URL — open it in a browser and sign in with the Google account whose Antigravity / Gemini Code Assist entitlement you want to use (on a headless host, copy the URL to your laptop’s browser). You’ll need to pick a **location**: `global`, `us` or `eu`. The creds get stored in:

- `~/.gemini/antigravity-cli/antigravity-oauth-token` — the refresh token (there is no API-key path);
- `~/.gemini/antigravity-cli/settings.json` — the project and location:

```
{"gcp": {"project": "<your-project-id>", "location": "global"}}
```

Recommendation: **Pick ``global``**. Gemini *Pro* models are served from the global endpoint; with us some `gemini-*-pro*` requests fails with “*Selected model is not supported in the selected location*”. You can change it later by editing `settings.json` or by signing in again.

To switch accounts or projects, run `agy` interactively and use its `/logout` then `/login` slash commands (the license selector appears again).

You confirm what you have:

```
agy models                                # ids you can pass as model=...
agy --print "Reply with exactly PONG"      # sanity check
agy --model gemini-3.1-pro-high --print "Reply PONG" # Pro reachable? (needs global)
grep -A3 '"gcp"' ~/.gemini/antigravity-cli/settings.json
```

The model id suffix (`-high` / `-low`) is `agy`’s reasoning-effort tier.

11.1.2 Mounting into the container

The `ghcr.io/ucb-bar/chia-antigravity` image has `agy` installed and reads its state from `/home/ray/.gemini`. Bind-mount your signed-in directory there and run the container as your uid so the mounted files are readable **and** **writable** — `agy` refreshes its token and writes logs and the per-conversation SQLite databases into `antigravity-cli/` on every call. (Chia never edits your `~/.gemini/config`: each run gets a private HOME with its own MCP server list and a symlink to the shared `antigravity-cli`.)

```
antigravity:
  resources: {"antigravity_creds": 1}
  docker:
    image: "ghcr.io/ucb-bar/chia-antigravity:latest"
    run_options:
      - "--user $(id -u):$(id -g)"
      - "-v ${HOME}/.gemini:/home/ray/.gemini"
    run_setup_commands:
      - echo "user:x:$(id -u):$(id -g):/home/ray:/bin/bash" >> /etc/passwd
```

Notes:

- Recommended to use `${HOME}` (or an absolute path), not `~`
- Mount the whole `~/.gemini` tree, not just the token: project/location come from `settings.json` next to it. Editing `settings.json` on the host takes effect on the next call inside the container (no restart).
- The `/etc/passwd` line gives your uid a home of `/home/ray` so `$HOME` resolves correctly inside the container. (`AntigravityLLM` resolves `~/.gemini` on the worker that runs the prompt, not where the object was built, so it is safe to construct it on a node whose HOME is `/root`.)
- The host directory must exist before `chia up`; Docker creates a missing mount source as an empty root-owned directory, which shows up as `mkdir /home/ray/.gemini/antigravity-cli: permission denied`.

Verify from the host after `chia up`:

```
docker exec <antigravity-container> agy --print "Reply with exactly PONG"
```

If it prints an OAuth URL instead, the mounted directory is not the signed-in one (or is empty).

11.2 OpenCode with Gemini on Vertex AI

OpenCode is provider-agnostic — the model is `<provider>/<model>` and each provider brings its own credentials. For Gemini on Vertex AI the provider is `google-vertex`, which needs two things: **Google Application Default Credentials (ADC)** and a **GCP project**. Setup is three steps: sign in on the host, mount + configure the container, and tell the driver which model/project to use.

11.2.1 1. Host: sign in and pick the project

```
gcloud auth application-default login           # writes ~/.config/gcloud/
↪ application_default_credentials.json
gcloud auth application-default set-quota-project <project> # required for user
↪ credentials, else 403
gcloud services enable aiplatform.googleapis.com --project <project>
```

Check the identity you just created — ADC is a **separate credential from the gcloud CLI account** (`gcloud auth list` can show a different user):

```
curl -s "https://www.googleapis.com/oauth2/v3/tokeninfo?access_token=$(gcloud auth
↪ application-default print-access-token)" | grep email
```

That identity needs the *Vertex AI User* role on `<project>`. For unattended hosts you can use a service-account key file instead of a login (step 2 shows how to point at it).

11.2.2 2. Container: mount the credentials, set project + location

The `ghcr.io/ucb-bar/chia-opencode` image needs no `gcloud`: OpenCode's Google auth library reads the ADC file directly. Mount the `gcloud` config directory and pass the project/location as environment variables — the `google-vertex` provider reads them from the `opencode` process's environment, not from `gcloud config`:

```
opencode:
  resources: {"opencode_creds": 1}
  docker:
    image: "ghcr.io/ucb-bar/chia-opencode:latest"
    run_options:
      - "-v ${HOME}/.config/gcloud:/home/ray/.config/gcloud" # ADC file
      - "-e GOOGLE_CLOUD_PROJECT=${GOOGLE_CLOUD_PROJECT}"   # project
      - "-e VERTEX_LOCATION=global"                          # Gemini Pro is
↪ global-only
      # service-account key instead of a login:
      # - "-v /path/to/sa-key.json:/home/ray/sa-key.json:ro"
      # - "-e GOOGLE_APPLICATION_CREDENTIALS=/home/ray/sa-key.json"
```

`export GOOGLE_CLOUD_PROJECT=<project>` **before** `chia up` so the `${GOOGLE_CLOUD_PROJECT}` substitution has a value, and make sure `~/.config/gcloud` exists (Docker would otherwise create an empty root-owned directory). The mount is read-only in practice, so no uid tricks are needed.

Verify from the host:

```
docker exec <opencode-container> opencode run -m google-vertex/gemini-3.1-pro-preview
↪ "Reply with exactly PONG"
```

11.2.3 3. Driver: choose the model and pass the project through

In CHIA code, select a Vertex model as `google-vertex/<vertex model id>` and pin the project/location in OpenCode's config with an `AdditionalModelProvider` (this also registers the model id in case OpenCode's catalog lags Vertex):

```
from chia.models.opencode import OpenCodeLLM, AdditionalModelProvider

vertex = AdditionalModelProvider(
    id="google-vertex", npm="@ai-sdk/google-vertex", name="Google Vertex AI",
    models=["gemini-3.1-pro-preview"],
    options={"project": "<project>", "location": "global"},
)
llm = OpenCodeLLM(model="google-vertex/gemini-3.1-pro-preview",
    additional_providers=[vertex])
```

Config values take precedence over the container's `GOOGLE_CLOUD_PROJECT` / `VERTEX_LOCATION`; those env vars remain as a fallback (e.g. for the manual `opencode run` check above). Credentials are never in the config — they always come from the mounted ADC.

The `memcpyy` example does this (`examples/memcpy/llm.py`) whenever `MEMCPY_OPENCODE_MODEL` starts with `google-vertex/`, reading the project from `GOOGLE_CLOUD_PROJECT` on the **driver**. A `chia job submit driver` does not inherit your shell, so forward both values through the job's runtime env:

```
chia job submit \
  --runtime-env-json '{"env_vars": {"MEMCPY_OPENCODE_MODEL": "google-vertex/gemini-
  3.1-pro-preview", "GOOGLE_CLOUD_PROJECT": "$GOOGLE_CLOUD_PROJECT"}}' \
  -- python $PWD/examples/memcpy/memcpy_loop.py --llm opencode
```

11.3 Quick reference

| | Antigravity (agy) | OpenCode + Vertex AI |
|--------------------|---|---|
| Sign in | agy (interactive OAuth; /logout + /login to switch) | gcloud auth application-default login + set-quota-project |
| Project / location | chosen at sign-in; <code>~/.gemini/antigravity-cli/settings.json</code> | <code>GOOGLE_CLOUD_PROJECT</code> / <code>VERTEX_LOCATION</code> (+ pinned via <code>AdditionalModelProvider</code>) |
| Credential on disk | <code>~/.gemini/antigravity-cli/antigravity-oauth-token</code> | <code>~/.config/gcloud/application_default_credentials.json</code> (or a SA key) |
| Mount | <code>-v \${HOME}/.gemini:/home/ray/.gemini (+ --user, writable)</code> | <code>-v \${HOME}/.config/gcloud:/home/ray/.config/gcloud (read-only is fine)</code> |
| Gemini Pro | location global | <code>VERTEX_LOCATION=global</code> |
| Model id | agy models (e.g. <code>gemini-3.1-pro-high</code>) | <code>google-vertex/<vertex id></code> (e.g. <code>gemini-3.1-pro-preview</code>) |

CACHING AND BYPASS

CHIA's **cache** mechanism persists a function's real output to disk, and **bypass** replaces a function's computation with pre-recorded data while still dispatching the call through Ray. These mechanisms enable redundant and costly work to be skipped on reruns, easy testing of cluster orchestration, and improved fault tolerance by storing in-progress results. Both are built around `ChiaFunction`, keyed by a per-call tag (`_chia_tag`), and configured in the same YAML file passed to your loop.

At a high level, the cache is the write path (populate from a real run) and bypass is the read path (replay it). They share the tag.

- *Tags*
- *Bypass*
- *Cache*
- *Putting it together: populate, then replay*

12.1 Tags

Every dispatch can carry an optional `_chia_tag` that names this specific call — e.g. `f"iter{i}_opt{j}"`. Both caching and bypass use it as the key, and both can be restricted to tags matching a regex:

```
# The tag identifies this call for both the cache and bypass.
for i in range(N_ITERS):
    for j in range(N_PARALLEL):
        ref = run_verilator_test.chia_remote(design, _chia_tag=f"iter{i}_opt{j}")
```

12.2 Bypass

When a function is bypassed it is still dispatched according to its resource requirements, but the real computation is replaced by a provider that returns pre-recorded data. This lets you test cluster orchestration without paying for the underlying work (builds, simulations, LLM calls, ...).

12.2.1 Configuration

List functions under a `bypass:` section. `bypass:` `true` opts a function in; an optional `tags:` list (regex patterns) restricts it to matching calls; an optional `data:` path supplies a file to serve.

```
bypass:
  # Bypass every call to this function.
  simple_add:
    bypass: true

  # Bypass only calls whose _chia_tag matches one of these patterns.
  run_verilator_test:
    bypass: true
    tags: ["iter0_.*"]

  # Shorthand: bypass: true with no extra options.
  simple_multiply: true

  summarize_perf:
    bypass: true
    data: /path/to/recorded_perf.md  # served to the worker as a string
```

A function not listed (or `bypass: false`) runs normally. If no YAML is loaded at all, bypass is a complete no-op.

12.2.2 Setup and providers

Create one Bypass instance as part of loop setup and register a provider for each bypassed function. The provider runs on the worker (with the same scheduling as the real function) and returns the replacement value:

```
from chia.base.bypass import Bypass
from chia.base.ChiaFunction import ChiaFunction, get

# If yaml_path is None, bypass does nothing.
bypass = Bypass(yaml_path=args.bypass_config)

# Required function signature:
# tag: the function's _chia_tag
# data_path: the YAML "data:" path or None
# *args, **kwargs: the original call's
def provider_42(tag, data_path, *args, **kwargs):
    return 42

# Remote ChiaFunction
@ChiaFunction(resources={"adder": 1})
def simple_add(x, y):
    return x + y

bypass.set_provider("simple_add", provider_42)

# Dispatched to a node with "adder" resource,
# but returns 42 from the provider instead of running.
result = get(simple_add.chia_remote(1, 2))  # -> 42
```

A function is only bypassed when it is `bypass: true` and has either a registered provider or a `data: path`.

12.2.3 Serving a file instead of a provider

If you give a function a `data: path` but no provider, CHIA serves that file's contents as the result. The read is routed through a Ray actor pinned to the node that constructed the Bypass (the head), so workers on other nodes can read it **without a shared filesystem**:

```
bypass:
  summarize_perf:
    bypass: true
    data: /path/to/recorded_perf.md  # served to the worker as a string
```

12.2.4 Gating with a condition

Besides a provider, a function may register a *condition* — an extra gate that decides, at dispatch time, whether the bypass actually happens. It runs on the caller as the last step of the bypass decision (after the `bypass: true` flag, the provider/data check, and the tag patterns) and returns a bool:

```
# cond(tag, data_path, *args, **kwargs) -> bool (same args as a provider)
def cache_hit(tag, data_path, *args, **kwargs):
    return get(get_active_cache().has.chia_remote(tag))

bypass.set_cond("run_verilator_test", cache_hit)
```

A falsy return means the call is **not** bypassed and runs for real; a truthy return lets the bypass proceed. When no condition is registered the default is True (no extra gate). Use it to make the decision depend on runtime state, most usefully to only replay from the cache when the value is actually present (see *Putting it together: populate, then replay* below).

12.3 Cache

The cache is an LRU key/value store of pickled (tag, data) files on disk that warm-starts by scanning the cache directory, so cached values survive across loop runs. Since the cache is implemented as a remote Ray actor located on the head node, it must be accessed using `chia_remote`. Start it once on the driver after `ray.init()`:

```
from chia.base.cache import start_cache

start_cache(size=4, units="GB",
            cache_dir_path="/data/chia_cache",
            yaml_path="path/to/yaml.yaml")

# Access via chia_remote and get
cache_hit = get(get_active_cache().has.chia_remote(tag))
```

12.3.1 Writing is automatic

In the yaml, mark a function `cache: true` in the `cache:` section (same file as `bypass:`). Its output is then written to the cache automatically, keyed by the call's `_chia_tag`. An optional `tags:` list gates which calls are written.

```
cache:
  run_verilator_test:
    cache: true
    tags: ["iter.*"]  # only cache calls whose tag matches
```

(continues on next page)

(continued from previous page)

```
# shorthand (cache, no tag filter):  
build_megaboom: true
```

```
# With run_verilator_test cache:true, the real result is written under "iter0".  
result = get(run_verilator_test.chia_remote(design, _chia_tag="iter0"))
```

12.3.2 Reading is manual (via bypass)

To replay a cached value, register a bypass provider that reads it back:

```
from chia.base.cache import get_active_cache  
from chia.base.ChiaFunction import get  
  
def cache_provider(tag, data_path, *args, **kwargs):  
    hit, value = get(get_active_cache().read.chia_remote(tag))  
    if not hit:  
        raise KeyError(f"cache miss for tag {tag!r}")  
    return value  
  
bypass.set_provider("run_verilator_test", cache_provider)  
  
# "iter0" was written on the real run above; this serves it without rerunning.  
served = get(run_verilator_test.chia_remote(design, _chia_tag="iter0"))
```

12.4 Putting it together: populate, then replay

The common workflow is one real run that *populates* the cache, then later runs that *replay* it so you can iterate on the loop/orchestration without recomputing the expensive steps.

See `examples/bypass_cache/bypass_cache_loop.py` for a runnable version (run it twice: cold then warm).

PROFILING

CHIA can profile a loop's execution (how long each `ChiaFunction` call took, which worker ran it, and how calls depend on one another) and render the result as a dependency graph, an interactive timeline, or a CSV table. Profiling is built around `ChiaProfiler`, a singleton that instruments every `ChiaFunction` call automatically once a collector is running.

Start the collector on the driver, run your loop, then point the `chia viz-profile` CLI at the JSONL log it wrote.

- *Enabling profiling*
- *What gets recorded*
- *Attaching custom metadata*
- *Visualizing a profile*

13.1 Enabling profiling

Start the `ChiaProfileCollector` from the driver, after `ray.init()` and before any profiled `@ChiaFunction` calls:

```
import ray
from chia.trace.profiler import start_collector

ray.init(address="auto")

# Defaults to logging under /tmp/ray/{job_id}.
start_collector(log_dir="/data/chia_profiles")

# ... run your @ChiaFunction loop ...
```

The `ChiaProfileCollector` is cleaned up automatically on `ray.shutdown()` or process exit; you can also stop it explicitly:

```
from chia.trace.profiler import stop_collector

stop_collector()  # idempotent
```

13.2 What gets recorded

Once the collector is running, `ChiaFunction` instruments itself with no changes to your code. Every call emits events keyed by a globally unique `call_id`:

- `dispatch` — a remote call (`chia_remote()`) started on a worker. Carries the worker IP / id, node id, Ray resources, and `obj_ref_deps` (the dependency edges, see below).
- `complete` — a remote call's result was retrieved via `get()`. Carries `exec_time_s` (high-resolution execution time) plus any `add_info` metadata.
- `local_start` / `local_end` — a `ChiaFunction` that ran locally rather than on a worker. `local_end` also carries `exec_time_s`.

13.2.1 Dependency edges

The profiler reconstructs the task graph by tracking object identity: when a call returns a value, it records `id(value) -> call_id`; when a later call receives that same object as an argument, it emits a dependency edge (`obj_ref_deps`). Small interned values (`int`, `float`, `str`, `bytes`, `bool`, `None`) are deliberately skipped to avoid false-positive edges.

13.3 Attaching custom metadata

From inside any `@ChiaFunction` body, call `add_info` to merge extra fields into that call's `complete` (or `local_end`) event. It is thread-local, so concurrent tasks don't cross metadata:

```
from chia.trace.profiler import get_profiler

@ChiaFunction
def run_verilator_test(design):
    cycles = ... # run the sim
    get_profiler().add_info({"simulation_cycles": cycles})
    return result
```

This is how CHIA's library nodes surface domain metrics — e.g. simulation cycles and waveform sizes from Verilator runs, or token counts and cost from LLM model calls — into the profile. The metadata lands under the event's `extra` key.

For events that aren't tied to a single call, use `log_event`:

```
get_profiler().log_event("checkpoint", iteration=3, note="post-merge")
```

Each custom event is timestamped and tagged with the current `worker_id` automatically.

13.4 Visualizing a profile

Render a recorded log with the `chia viz-profile` CLI — a dependency graph (`svg/png/pdf`), an interactive HTML timeline, or an aggregated CSV table. See [chia viz-profile <log...>](#) in the CLI Reference for the full list of formats and flags.

EXAMPLE: AGENTIC ROCC ACCELERATOR (MEMCPY)

A worked, end-to-end example of an agentic hardware-design loop: an LLM (Claude Code by default; OpenCode or Google Antigravity via `--llm`) designs a RISC-V RoCC accelerator in Chisel, CHIA builds it into a MegaBoom SoC, runs it against a bare-metal test, and, on any failure, feeds the error back to the LLM to debug and retry until the test passes.

The example lives at `examples/memcpy/` and is essentially self-contained: it depends only on the installed `chia` package and the shared `DRAMSim2` ini files in `examples/common/dramsim.ini`. It is a good starting point for building your own generate, build, simulate, debug loops on top of CHIA's chipyard nodes.

- *What it does*
- *Components*
- *The accelerator contract*
- *Running it*
- *Tunable parameters*
- *Debug feedback*
- *Output and per-iteration diffs*

14.1 What it does

The accelerator is a hardware `memcpy`: it copies an array of 64-bit elements from a source region to a destination region, driven by two custom instructions on opcode `custom1`. The target design is `MegaBoomV3HumanCommitLogConfig` (MegaBoom V3 with the human-readable commit-log harness, so a failing run yields a readable instruction trace for the debugger).

The loop runs two nodes in parallel, then builds, runs, and debugs:

```
test build (parallel)      implement (parallel)
  copy memcpy.c into       Claude writes memcpy.scala
  $chipyard/tests, run cmake (the RoCC accelerator) and
  -> build/memcpy.riscv    wires it into the target
  build/memcpy.dump        config, via chipyard_bash

  chisel build (ChiselBuildNode)    target: MegaBoomV3HumanCommitLogConfig
```

(continues on next page)

(continued from previous page)

```

verilator run (VerilatorRunNode)      memcpy.riscv, +loadmem +verbose

build failed / sim failed / incorrect?
  | yes ( NUM_DEBUG_ATTEMPTS)          | no

debug (Claude) — rebuild + rerun      DONE (passed)

```

14.2 Components

| File | Role |
|----------------|--|
| memcpy_loop.py | Main orchestration: parallel test-build + implement, then the build → run → debug loop. Dumps all collateral to out/. |
| test_build.py | build_test <i>ChiaFunction</i> — copies memcpy.c into \$chipyard/tests, registers a CMake target, builds build/memcpy.riscv + build/memcpy.dump, and reads them back. Runs on the chipyard container. |
| llm.py | The implement + debug LLM nodes (Claude Code, OpenCode or Antigravity, chosen with --llm) and the failure-feedback formatters. LLM calls are dispatched onto the chosen backend's node (llm / opencode / antigravity); Claude Code and Antigravity share one resumable session across the calls. |
| helpers.py | Run-outcome classification (classify_run), the out/ dumper, the chipyard git-diff node (collect_diff), and dramsim.ini loading. |
| prompts/ | The implement (implement.md) and debug (debug.md) prompt text, with \${VAR} placeholders filled from constants at load time. |
| constants.py | Every tunable knob (loop counts, configs, paths, timeouts, resources). |
| cluster.yaml | Minimal cluster: one chisel-build, one verilator, and one node per LLM backend (llm, opencode, antigravity) — keep the ones you use. |
| memcpy.c | The bare-metal test: issues the two RoCC instructions and checks the copy. |

14.3 The accelerator contract

memcpy.c drives the accelerator at opcode custom1 with two instructions; the implement prompt specifies exactly this so the generated design matches the test:

- `funct == 0` — `rs1` = source base address, `rs2` = destination base address. Latch both; write `rd = 1`.
- `funct == 1` — `rs1` = array length (number of 64-bit elements). Copy the whole array source → destination via the RoCC memory port; write `rd = 1` when done.

Correctness is judged from the MEMCPY Num Correct: N line the program prints: the run passes iff N == DATA_SIZE (constants.DATA_SIZE, kept in sync with memcpy.c).

14.4 Running it

The bundled cluster.yaml brings up a chisel-build (chipyard) node, a verilator (verilator_run) node, and one node per LLM backend. The implement/debug LLM can be **Claude Code** (llm node), **OpenCode** (opencode node) or **Google Antigravity** (antigravity node, Gemini via the agy CLI) — keep them up and pick per run with --llm (default claude), or comment out the ones you don't use. Each backend reads its credentials from the mounts shown at the top of cluster.yaml (Antigravity mounts your ~/.gemini sign-in; choose the global location at sign-in so Gemini Pro is available). See *Authenticating Gemini backends: Antigravity and OpenCode + Vertex AI* for the sign-in and mount details of the two Gemini backends.

```
export THIS_MACHINE="your_machine_ip"
chia up examples/memcpy/cluster.yaml
# pick the backend with --llm (claude is the default); run from the repo root:
chia job submit -- python $PWD/examples/memcpy/memcpy_loop.py --llm claude
chia job submit -- python $PWD/examples/memcpy/memcpy_loop.py --llm opencode
chia job submit -- python $PWD/examples/memcpy/memcpy_loop.py --llm antigravity
chia down examples/memcpy/cluster.yaml
```

Ray jobs do not inherit your shell environment, so MEMCPY_* knobs go through the job's runtime env. OpenCode is provider-agnostic (MEMCPY_OPENCODE_MODEL is any provider/model); Gemini on Vertex AI is the wired-up example — it needs Google ADC on the host and a GCP project:

```
export GOOGLE_CLOUD_PROJECT=<project>      # before `chia up` (the opencode node forwards_
↪ it)
chia job submit \
  --runtime-env-json "{\"env_vars\": {\"MEMCPY_OPENCODE_MODEL\": \"google-vertex/gemini-
↪ 3.1-pro-preview\", \"GOOGLE_CLOUD_PROJECT\": \"${GOOGLE_CLOUD_PROJECT}\"}}\" \
  -- python $PWD/examples/memcpy/memcpy_loop.py --llm opencode
```

Pass the absolute path to memcpy_loop.py. The driver runs on the cluster head, where the repo lives, so out/ is written into the real examples/memcpy/out.

All backends edit the chipyard checkout through the same chipyard_bash MCP tool. prompt is a ChiaFunction on each backend, so the loop dispatches it onto the chosen worker (llm, opencode or antigravity). Claude Code and AntigraVity thread their session transcript from each call into the next so the debugger resumes the implement conversation; session persistence for OpenCode is in development (each OpenCode call is currently independent).

14.5 Tunable parameters

All knobs live in constants.py; container paths and cluster knobs are MEMCPY_* environment-overridable, so nothing is hardcoded into the loop.

| Constant | Meaning |
|-------------------------------------|--|
| NUM_DEBUG_ATTEMPTS | Max debug-and-retry rounds after the first failure (default 3). |
| NUM_PERF_OPT_ITERS | Reserved for a future post-correctness performance-optimization phase (defined, not yet used). |
| BUILD_CONFIG | Chisel config to build (MegaBoomV3HumanCommitLogConfig). |
| DATA_SIZE | Element count; must match memcpy.c. |
| VERILATOR_TIMEOUT_CYCLES / _SECONDS | Simulation caps so a hung design fails fast. |
| *_RESOURCE | Ray scheduling tokens, sized so the persistent chipyard_bash actor coexists with the builds. |

14.6 Debug feedback

On a failure the debug node (the same LLM session, resumed for Claude Code and AntigraVity) receives:

- **Build failure** — build stderr tail plus stdout windowed on the first error.
- **Simulation failure** (runtime / timeout / incorrect) — the simulator stdout (the commit log) and spike-dasm output tails, plus the last COMMIT_LOG_TAIL_LINES lines of the commit log and the last DUMP_TAIL_LINES lines of memcpy.dump (the test disassembly).

14.7 Output and per-iteration diffs

Every node result and piece of collateral is written to `out/`, each filename prefixed with the timestamp at the moment it is written (so files sort by when they were produced):

```
20260626_144501_implement.md
20260626_144502_test_build_memcpy.riscv
20260626_144502_test_build_memcpy.dump
20260626_144503_chisel_diff_attempt0.diff      # chipyard diff for this iteration
20260626_144503_chisel_diff_attempt0.json     # per-repo diff dict
20260626_145012_chisel_build_attempt0.stdout.txt
20260626_145230_verilator_run_attempt0.log
20260626_145231_feedback_attempt1.md
20260626_145950_debug_attempt1.md
20260626_153044_summary.json
```

Each iteration's Chisel diff is captured with `collect_diff` (in `helpers.py`) just before that attempt's build, so it reflects the exact source built from the `implement` node's work on attempt 0, and the cumulative `implement` + `debug` edits on later attempts.

GEM5-BOOM MICROARCHITECTURE ALIGNMENT

A CHIA case study that uses an LLM-in-the-loop to generate a [gem5](#) processor core model until it matches a target [BOOM](#) core. The full flow lives in `chia/examples/gem5_align`.

15.1 Overview

Architects routinely keep a fast, high-level microarchitectural simulator (e.g. [gem5](#)) and a cycle-exact RTL implementation (e.g. a [BOOM](#) core in [Chipyard](#)) of the *same* microarchitecture. The two drift apart: the [gem5](#) model mispredicts cycle counts because its configuration — and sometimes its C++ source — no longer reflects the RTL. **Alignment** is the work of closing that gap, and it's a huge lift, even for large engineering teams. It is nearly impossible for small teams and academics, and usually the simulators do not stay aligned, or are never aligned in the first place.

Our `gem5_align` chia workflow automates it as a search loop. Each iteration:

1. restores a *parent* [gem5](#) source + config state sampled from the best results so far,
2. asks an LLM to edit the [gem5](#) configuration and/or `src/` to better match [BOOM](#),
3. rebuilds [gem5](#),
4. runs a benchmark suite, and
5. compares [gem5](#) cycle counts against cached **Verilator golden** counts to get a per-benchmark `%diff`.

Results are written to a SQLite database (`alignment.db`); the next iteration samples its parent from the top entries, so the search keeps building on its strongest candidates. *N* iterations run concurrently — one per physical [gem5](#) node.

Note

Result

We ran alignment on medium [BOOM](#) for 10.5 days, and achieved a [gem5](#) core and configuration whose cycle counts were accurate to under 3% on average across our benchmarks! Read section 5 of our [arXiv paper](#) to hear more about our results.

15.2 How it works

15.2.1 The per-iteration flow

The head runs *N* iterations concurrently — one per physical [gem5](#) node. A single placement group with `STRICT_SPREAD` bundles pins each bundle to a distinct node, and a thread pool on the head drives one iteration per bundle. When an iteration finishes, the bundle is freed and the next iteration is dispatched onto it with a freshly sampled parent:

```

HEAD THREAD (one per bundle)
|
+-- sample parent uniformly from DB.top_k_entries(2)
+-- restore_gem5_state(parent.config, parent.diff)    [gem5 bundle]
+-- rebuild_gem5()                                   [gem5 bundle]
|
+-- align_node(...)                                  [llm worker]
|   (analyze parent's results, read BOOM source, edit config/source)
|
+-- rebuild_gem5()                                   [gem5 bundle]
+-- run_gem5_comparison()                            [gem5 bundle]
|   (debug_node loop on failures)
|
+-- persist IterationResult to DB + logs, dispatch next iteration

```

15.2.2 The Verilator golden cache

Alignment needs a ground truth to score against. `ensure_verilator_cache` builds the target Chipyard config once on a `chisel_build` node, runs every microbenchmark on `verilator_run` nodes, and caches the result (`<bench>.log + <bench>.out`) on the head. Subsequent runs reuse the cache, so the expensive RTL simulation happens only once:

```

def ensure_verilator_cache() -> bool:
    """If verilator results are not cached, build BUILD_CONFIG and run all benchmarks."""
    benchmarks = sorted(p.stem.replace(".verilator", "")
                        for p in UBENCH_BUILD.glob("*.verilator.riscv"))
    missing = [b for b in benchmarks
               if not (VERILATOR_CACHE / f"{b}.log").exists()
                   or not (VERILATOR_CACHE / f"{b}.out").exists()]
    if not missing:
        print(f"[verilator cache] All {len(benchmarks)} benchmarks cached. Skipping_
→build.")
        return True
    # else: build BUILD_CONFIG on a chisel node, dispatch every missing
    # benchmark to verilator nodes, and cache <bench>.log / <bench>.out
    ...

```

15.2.3 Distributing gem5 work across the cluster

Each `gem5` node is one `STRICT_SPREAD` bundle of a shared placement group, with a canonical `Gem5Node` pinned to it:

```

gem5_pg = placement_group([{"CPU": 1, "gem5": 1}] * N, strategy="STRICT_SPREAD")
ray.get(gem5_pg.ready())
bundle_nodes = [Gem5Node(placement_group=gem5_pg, bundle_index=i) for i in range(N)]

```

The `gem5` build / run / restore steps are ordinary CHIA functions tagged with a *fractional* `gem5` resource, so a `gem5` op and its co-located tool actors share a single `{CPU: 1, gem5: 1}` bundle rather than each grabbing a whole node:

```

@ChiaFunction(resources={"gem5": 0.5})
def rebuild_gem5() -> tuple[bool, str, float, str, str, str]:
    ...

```

The loop ships **this repo's** `chia` to every worker via Ray `py_modules`, so workers import the head's checkout regardless of what their Docker image baked in:

```

_RUNTIME_ENV = {
    "py_modules": [str(_CHIA_PKG)], # ship THIS repo's chia to every worker
    "excludes": ["**/__pycache__", "**/*.pyc"],
}
...
ray.init(address=os.environ.get("RAY_ADDRESS", "auto"), runtime_env=_RUNTIME_ENV)

```

15.2.4 The LLM in the loop

The aligning agent runs on an llm worker via *ClaudeCodeLLM*. It is given the per-benchmark cycle-count comparison table (gem5 vs Verilator %diff), the benchmark descriptions, and the run history. To diagnose *why* a benchmark is off, it also gets two key diagnostic signals:

- **O3PipeView pipeline traces** — gem5 runs under `--debug-flags=O3PipeView`, and the parent iteration's traces are staged on the worker so the agent can diff gem5 retire cycles against Verilator commit cycles at matching PCs;
- **performance counters** — BOOM's top-down (TMA) counters from the Verilator run (`divider_active`, `stq_full`, `dcache_miss`, `br_mispredict`, ...), which it cross-checks against the corresponding gem5 stats.txt counters to localize the mis-modeled mechanism.

Alongside these it has MCP tools to read the BOOM Chisel source, edit and rebuild the gem5 tree, and quick-run gem5 on a few benchmarks to test a hypothesis. It then proposes edits to the gem5 config and/or C++ source:

```

llm = ClaudeCodeLLM(
    model="claude-opus-4-6",
    timeout_seconds=3600,
    logging_name="align",
    resume_session=session_id is not None,
    extra_cli_args=["--effort", "max"],
)

```

A separate `debug_node` resumes the *same* CLI session if a rebuild or run fails, so the agent can iteratively fix its own changes within one iteration.

15.2.5 Targeting a different configuration

The target is the one thing you change to retarget the whole flow. In `config.py`, `BUILD_CONFIG` is the full Chipyard config class (built on the chisel node and surfaced to the LLM) and `CONFIG_SLUG` suffixes on-disk artifacts so multiple targets can coexist:

```

BUILD_CONFIG = "SmallBoomV3HumanCommitLogTMAConfig" # full Chipyard config class
CONFIG_SLUG  = "smallboom"                          # suffixes on-disk artifacts

```

15.3 Setup

These steps mirror the example's `README.md`. Run them from `<repo>/chia` unless noted.

1. **Head conda env** — run from the example directory:

```

cd examples/gem5_align
conda env create -f env.yml
conda activate gem5-align

```

Only the head needs this environment; workers get chia via Ray `py_modules` and run the Docker images named in `cluster.yaml`. If you rename the env, update `cluster.yaml` to match.

2. Cluster — in `cluster.yaml`, set `provider.head_ip` (the host running chia up / the Ray head) and each node type's `compatible_ips`. The required worker counts (`max_workers`):

| Node type | Workers | Role |
|---------------|---------|--|
| llm | 6 | aligning/debugging LLM (light) |
| chisel_build | 1 | builds the verilator golden cache (heavy — Chipyard) |
| verilator_run | 4 | runs verilator goldens |
| gem5_worker | 6 | builds + runs gem5 |

`compatible_ips` is the **pool** of hosts a type's workers may run on, not a 1:1 list — multiple workers (and multiple node types) co-locate on one host as separate containers, so you need fewer hosts than the worker total. `head_ip` is a single string; `compatible_ips` is a list (`${ENV_VAR}` is expanded):

```
provider:
  head_ip: 10.0.0.10
available_node_types:
  gem5_worker:
    compatible_ips: ["10.0.0.11", "10.0.0.12"]    # inline, block, or ${VAR}
```

Every listed host must be reachable via the SSH credentials in the top-level `auth` section (`ssh_user`, plus optional `ssh_private_key` path), have Docker, and run an SSH agent. The llm hosts bind-mount your Claude Code config via the `-v <dir>:/home/ray/.claude` run option. All four node types are required. See [Cluster Configuration Reference](#) for the full schema.

3. Benchmarks — fetch the examples/benchmarks submodule, then compile the ubench suite the loop runs (with the step-1 `gem5-align` env active — it provides the `riscv64-unknown-elf` toolchain):

```
git submodule update --init examples/benchmarks
cd examples/benchmarks/ubench && ./compile.sh    # -> build/<bench>.{gem5.elf,verilator.
↪riscv}
```

Without `build/` populated, the verilator cache step finds zero benchmarks and silently skips (see the note above).

4. Config — in `config.py`, set `BUILD_CONFIG` + `CONFIG_SLUG` together to your target. Optional env overrides: `GEM5_ALIGN_BENCH_ROOT`, `GEM5_ALIGN_LOG_DIR`.

5. Bring up the cluster — from `<repo>/chia` with the env active:

```
chia up examples/gem5_align/cluster.yaml
```

This can take a while on the first run, since the Chipyard Docker image is large to pull. On slow links the pull may time out; raise the pull timeout and retry (progress is saved if you restart quickly).

6. Launch the alignment job — `GEM5_ALIGN_VERILATOR_CACHE` is **required** and names the verilator golden cache on the head (an existing dir to reuse, or a fresh writable dir to generate on the first run). The entrypoint runs under Ray's job manager and does **not** inherit your shell, so pass it via `--runtime-env-json` rather than `export`. Run from the example dir so `--working-dir` uploads its files:

```
cd examples/gem5_align
chia job submit --working-dir . \
  --runtime-env-json '{"env_vars": {"GEM5_ALIGN_VERILATOR_CACHE": "/abs/path/on/head"}}' \
↪ \
  -- python gem5_align_loop.py
```

Add `GEM5_ALIGN_BENCH_ROOT` / `GEM5_ALIGN_LOG_DIR` to `env_vars` only if overriding their defaults.

7. Tear down — when the run is done:

```
chia down examples/gem5_align/cluster.yaml
```

Outputs land in `GEM5_ALIGN_LOG_DIR` (default `examples/gem5_align/align_loop_logs-<slug>/`): per-iteration `iter_N/` artifacts, `alignment.db`, and TensorBoard metrics/. Re-running resumes from a non-empty `alignment.db`.

RISC-V ISA EXTENSION IMPLEMENTATION

A CHIA case study that uses an LLM-in-the-loop to implement a RISC-V ISA extension in a [BOOM](#) core and prove it correct against [Spike](#), the golden ISA simulator. The full flow lives in `chia/examples/riscv_extensions`.

16.1 Overview

Proposing and evaluating microarchitectural improvements is a fundamental part of computer architecture research. Cross-stack approaches often propose new ISA extensions backed by additional microarchitectural features. Designing the RTL is usually the easy half, leaving thorough verification and PPA analysis (performance and ASIC quality-of-results collection) as the difficult task.

`riscv_extensions` automates both halves as a loop. For one extension, each run:

1. resets BOOM and lets an LLM implement the extension in Chisel, guided by the ratified spec that is given as an input;
2. evaluates on **directed tests** — one program per instruction in the extension, run differentially against Spike;
3. checks functional correctness on the full **base-ISA regression** suite, so the edit can't break anything else;
4. **stress-tests** with programs that `riscv-dv` generates — a random mix of the extension's instructions and ordinary code — co-simulated against Spike in lockstep for several million instructions; and
5. **synthesizes** the BoomTile with and without the extension implementation to measure the area/timing (PPA) cost of the extension.

Spike is the oracle throughout. It is compiled *into* the simulator (`cospike`) and checks every committed instruction, so a mismatch is caught the cycle it happens — not inferred from a final result. A divergence in the stress phase hands the failing trace to a debug LLM, which fixes the RTL; then the verify-then-stress process restarts, and a fresh clean stress run is the bar to converge.

Results accumulate as per-extension **sweeps** in the log DB: each sweep stores the BOOM diff that implements the extension, the LLM transcripts, per-test sim logs, the PPA delta against the unmodified core, and the generated stimulus.

Note

Result

We implemented and verified three extensions on a MegaBOOM core — Bit-Manipulation (Zba/Zbb/Zbc/Zbs), scalar Cryptography (Zbk*/Zkn*), and Zicond — each proven against Spike. They deliver ~5.6% and ~3.5% speedups (plus up to 10× on OpenSSL crypto), with no timing regressions and only modest area overhead in SkyWater 130nm. Read more in our [arXiv paper](#).

16.2 How it works

16.2.1 The per-run pipeline

`run_vext_loop()` drives one extension from a clean tree to a converged, synthesized result. Implementing and testing are one step — a regression failure feeds straight back to the LLM — and the stress phase only runs once both the directed tests and the regression suite pass:

```
DRIVER (one extension -> one sweep)
|
+-- reset BOOM + build the per-instruction directed tests
|   (from specs/<ext>/instructions.json)
|
+-- IMPLEMENT                                     [llm worker]
|   implement-LLM edits BOOM Chisel -> build the cospike sim ->
|   run the directed tests vs Spike; iterate until all pass
|
+-- REGRESSION  run the full base-ISA riscv-test suite on the DUT
|
+-- STRESS TEST                                     [gen + cosim nodes]
|   riscv-dv generates random programs mixing the extension;
|   stream them through lockstep cosims (Spike vs BOOM).
|   divergence -> debug-LLM fixes the RTL -> restart verify + stress
|
+-- PPA synth: BoomTile area/slack, baseline vs implemented [vlsi worker]
+-- archive the sweep (diff, transcripts, sim logs, PPA) to the DB
```

16.2.2 Differential verification against Spike

Every test — directed or random — is judged the same way: run it on the DUT with Spike riding inside the simulator, and fail at the first architectural disagreement. `CosimNode` runs one ELF in lockstep; the embedded Spike’s ISA string comes from the DUT’s own config, so the oracle always decodes exactly what the core claims to implement. No expected output is ever hand-encoded — the golden model *is* the check.

16.2.3 Directed tests: every instruction, once

The directed tests are built from `specs/<ext>/instructions.json`, the extension’s closed instruction set (generated from `riscv-opcodes`). `isa_tests` emits one minimal program per instruction — a handful of executions over random operands — and cross-compile each to an ELF. Coverage is 100% by construction: the table *is* the instruction set, so every instruction is exercised and checked against Spike.

16.2.4 The random stress test

Directed tests prove each instruction in isolation; the stress phase proves it in context. `riscv-dv` generates large random programs that weave the extension’s instructions through ordinary code — hazards, loops, memory traffic — and streams them through parallel lockstep cosims. Generators fill a pool on the database node from the run’s start; cosims drain it, marking each test pending -> passed. A single divergence resets the pool, so after the debug LLM fixes the RTL the whole batch re-runs and must pass clean.

16.2.5 Generating the extension’s instructions

`riscv-dv` can generate `bitmanip`, but has no generator for scalar crypto or `Zicond`. Each extension therefore names a `riscv-dv custom target` it generates against:

```
EXTENSIONS = {
    "bitmanip": Extension(name="bitmanip", isa_suffix="_zba_zbb_zbc_zbs", ...),
    "crypto": Extension(..., isa_suffix="_zbkb_zbkc_zbkx_zknd_zkne_zknh",
                        dv_target="riscv_dv_target_crypto"),
    "zicond": Extension(..., isa_suffix="_zicond",
                        dv_target="riscv_dv_target_zicond"),
}
```

For crypto and Zicond the target defines the missing instructions as riscv-dv custom (RV64X) instructions, which the generator node installs into riscv-dv before generating so they mix into the random stream like any ALU op; bitmanip uses riscv-dv's native Zb* support. The SV generation flow needs an Xcelium license, so the flow also ships a small set of committed prebuilt programs per extension: `--prebuilt-stress` seeds the pool from those instead of generating, letting the stress phase run with no license.

16.2.6 Distributing work across the cluster

Each extension pipeline holds one Chipyard build host for the duration of a run via a placement group, and the LLM's editor tool is pinned to the same bundle so its edits land in that host's BOOM tree. Generation, cosim, and synthesis fan out onto their own node types by resource tag — `gen_to_pool` onto the riscv-dv generators, cosims onto `verilator_run` nodes, BoomTile synthesis onto a `vlsi` worker. The loop ships **this repo's** chia and example packages to every worker via Ray `py_modules`, so workers run the head's checkout regardless of what their Docker image baked in.

16.2.7 The LLM in the loop

Both the implementing and debugging agents run on an llm worker via [ClaudeCodeLLM](#), with a bash tool to edit BOOM and a spec tool to read the ratified text. Sessions resume across turns so an agent can iterate on its own changes:

```
ClaudeCodeLLM(
    model=LLM_MODEL,
    system_message=open(PROMPTS_DIR / prompt_file).read(),
    resume_session=True, projects_cwd=None,
    extra_cli_args=LLM_EXTRA_ARGS,          # ["--effort", "max"]
)
```

The implement agent (`prompts/system.md`) writes the extension; on a stress divergence a debug agent (`prompts/debug.md`) gets the failing lockstep trace and repairs the RTL.

16.2.8 Adding an extension

One prerequisite: **Spike and the toolchain must already implement the extension.** Spike is the oracle, and the directed tests are assembled with `-march=...<ext>`, so the standard tools have to know the instructions — the flow verifies a BOOM implementation, it doesn't invent the ISA.

Then there are three things to add.

1. Register it in `constants.py` with its ISA-string suffix (and a custom `dv_target` only if riscv-dv can't generate it):

```
"zicond": Extension(name="zicond", isa_suffix="_zicond",
                    dv_target="riscv_dv_target_zicond"),
```

2. Add the spec + instruction table under `specs/<ext>/`:

```
specs/<ext>/instructions.json    # the closed instruction set (from riscv-opcodes)
specs/<ext>/spec.md             # OR spec.pdf - the spec tool renders a PDF as-is
```

3. Add stress collateral only if riscv-dv can't already generate the extension. Bitmanip uses riscv-dv's native Zb*, so there's nothing to add. Crypto and Zicond don't exist in riscv-dv, so each ships a custom target — copy an existing `riscv_dv_target_<ext>/` and edit the four files that matter:

```
riscv_dv_target_<ext>/
riscv_core_setting.sv          # enable the RV64X custom group
testlist.yaml                  # the random program shapes
isa/custom/riscv_custom_instr_enum.sv # one enum name per instruction
isa/custom/rv64x_instr.sv      # DEFINE_CUSTOM_INSTR per instruction
isa/custom/riscv_custom_instr.sv # render each to assembly (convert2asm)
```

(The copy also carries `isa/custom/rv32x_instr.sv`, a stub — our cores are 64-bit.) Point the registry entry's `dv_target` at the new directory.

Everything else — directed tests, the regression suite, stress generation, cosim, synthesis — is driven off the registry entry.

16.3 Setup

Run these from `<repo>/chia` unless noted.

Note

No licenses? Still runnable.

Two stages lean on commercial tools — random generation (Xcelium) and PPA synthesis (Cadence Genus on the open-source Sky130 PDK) — but you need neither to run the example. Pass `--prebuilt-stress` to feed the stress phase from prebuilt binaries instead of generating, and `--no-synth` to skip the area/timing step. The implement + verify loop runs fully with both off.

The prebuilt binaries are vendored as a git submodule; fetch them once with `git submodule update --init examples/riscv_extensions/prebuilt_stress`.

1. Head conda env — only the head needs it (this installs the `chia` command used below); workers get `chia` via Ray `py_modules` and the cluster's Docker images:

```
conda create -n chia_env python=3.10.19 && conda activate chia_env
pip install -e .
```

2. Cluster — `examples/riscv_extensions/clusters/cluster.yaml` is the on-prem template; set `provider`, `head_ip` and each node type's `compatible_ips`. The node types:

| Node type | Resource | Role |
|-------------|--------------|---|
| llm | llm | implement / debug LLM (Docker chia-claude-code) |
| head_local | head_local | the loop's read-only tools (spec / status / knowledge) |
| database | database | durable sweep store + the stress test pool |
| build | chipyard | resets BOOM, applies the LLM's edits, builds the cospike DUT |
| riscv_build | riscv_build | cross-compile the directed tests to ELFs |
| cosim | verilator | lockstep Spike-vs-BOOM co-simulation |
| gen | dv / xcelium | riscv-dv random generation (Xcelium; skip with <code>--prebuilt-stress</code>) |
| vlsi | VLSI / Syn | BoomTile synthesis (skip with <code>--no-synth</code>) |

The llm container bind-mounts your Claude Code config (`-v ${HOME}/.claude:/home/ray/.claude`). See [Cluster Configuration Reference](#) for the full schema.

```
chia up examples/riscv_extensions/clusters/cluster.yaml
```

3. Run one extension — `single_loop.py` implements and verifies a single extension and archives it to the DB. Run the file directly (it puts the example packages on `sys.path`):

```
chia job submit --working-dir . -- \
python examples/riscv_extensions/single_loop.py --extension bitmanip
```

Add `--no-synth` on a cluster without a synth node, and `--prebuilt-stress` to run the stress phase from committed binaries instead of generating (no Xcelium):

```
chia job submit --working-dir . -- \
python examples/riscv_extensions/single_loop.py \
--extension zicond --no-synth --prebuilt-stress
```

4. Run several in parallel — `multi_loop.py` fans out one pipeline per extension and summarizes their PPA:

```
chia job submit --working-dir . -- \
python examples/riscv_extensions/multi_loop.py --extensions bitmanip crypto zicond
```

5. Tear down — when the run is done:

```
chia down examples/riscv_extensions/clusters/cluster.yaml
```

Each run archives a sweep under the log DB (`<ext>/sweep_<N>/`): the converged BOOM diff, per-iteration LLM transcripts, directed and stress sim logs, the baseline-vs-implemented PPA summary, and the generated `.S` stimulus.

BOOM CRITICAL-PATH TIMING OPTIMIZATION

A CHIA case study that uses an LLM-in-the-loop to raise the clock frequency of a **BOOM** core by reshaping its Chisel to shorten the critical path — without sacrificing instructions-per-cycle (IPC). The full flow lives in `chia/examples/timing_opt`.

17.1 Overview

The maximum frequency of a synthesized core is set by its **critical path** — the longest register-to-register logic delay. Shortening it is expert, tedious work: read a synthesis timing report, find the gates on the worst path, and restructure the RTL (re-pipeline, rebalance muxing, precompute, ...) to cut delay — all while holding cycle behavior fixed, since a faster clock that costs you IPC may be a net loss. That trade-off is the **Iron Law** of processor performance (time/program = instructions × cycles/instruction × time/cycle): the win that matters is the *product* of frequency and IPC, not frequency alone.

`timing_opt` automates this as a search loop. Each iteration:

1. loads a *parent* design variant (its generated Verilog and Genus timing report) from a SQLite store,
2. asks an LLM to grep the timing report and edit the BOOM Chisel to shorten the BoomTile critical path,
3. rebuilds the design,
4. **synthesizes** BoomTile and **runs the Verilator suite in parallel** — synthesis measures the new worst-case slack, Verilator gates correctness and estimates IPC impact, and
5. records the result as a new *child* branch in a tree of variants.

Results accumulate in a SQLite database (`timing.db`), a **multi-branch tree**: each branch stores its diff against the base RTL, generated Verilog, the Genus timing report, synthesized area, per-benchmark top-down (TMA) counters, and logs. You pick which parent to optimize each turn, and the flow produces one child per invocation — so the search keeps building on its strongest candidates.

Note**Result**

Over 15 iterations of the flow, we yield more than a 2x increase in frequency at only 3.3% IPC loss, for a net Iron-Law performance improvement of 1.97x, in the Skywater 130nm process! Read section 5 of our [arXiv paper](#) to hear more about our results.

17.2 How it works

17.2.1 The per-iteration pipeline

`run_improve_timing_loop()` runs one parent $\rightarrow$ child pass. It stages the parent exactly (so the tree reproduces it), lets the LLM edit, then validates the edit with a build, a parallel synth + Verilator step, and a debug-retry loop on failure:

```
DRIVER (one parent -> one child branch)
|
+-- load parent diff / generated Verilog / timing report from DB
+-- acquire chipyard placement group + chipyard_bash tool [chipyard node]
+-- stage + reset: write parent Verilog, reset chipyard, re-apply parent diff
|
+-- /improve_timing [llm worker]
|   (grep the staged timing report, edit Chisel, A/B-test
|   candidate edits with the timing_experiment tool)
|
+-- build all thread variants (build-debug retry loop) [chisel node]
|
+-- PARALLEL:
|   dispatch BoomTile synthesis (async) [vlsi worker]
|   run Verilator suite [verilator nodes]
|   on Verilator failure: cancel synth, debug_failure, rebuild, retry
|
+-- collect synthesis result once Verilator is clean
+-- persist child branch: reports, area, syn_obj tarball, TMA counters,
    produced timing report (worst-slack columns), parent-vs-child summary
```

On any failure a `finally` block reverts chipyard to the parent diff and records the status; on success the edits are left in place (already persisted).

17.2.2 The timing_experiment A/B tool

A full BoomTile synthesis is expensive, so the LLM does not pay for one to test each idea. `TimingExperimentTool` is an MCP tool that runs a cheap **sub-block** Genus synthesis on the edited vs. unmodified RTL so the agent can A/B-test an edit before committing to it. A small sub-block synth can outlast an MCP HTTP round-trip, so the tool uses a **start / poll** split:

- `rebuild_verilog()` re-elaborates Chisel to Verilog (`make verilog`, no C++ build) and caches it;
- `list_modules()` / `list_modules_parent()` enumerate valid `vlsi_top` values on the edited / unmodified RTL;
- `start_synth_child(vlsi_top, ...)` / `start_synth_parent(vlsi_top, ...)` dispatch the two sub-block synths and return handles in sub-seconds — issue both for a parallel A/B comparison;

- `synth_status(handle, max_wait_seconds)` polls an in-flight synth, returning `running` or the full area + worst-slack summary on completion.

Each experiment is logged to the `llm_experiments` table by a head-pinned `ExperimentLogger` actor, since the tool's worker cannot see the head's disk.

17.2.3 Distributing work across the cluster

The build/Verilator host (Chipyard) is held for one iteration via a placement group, and the `chipyard_bash` tool the LLM drives is pinned to the same bundle:

```
pg = placement_group(["CPU": 1, "chipyard": 1], strategy="STRICT_PACK")
ray.get(pg.ready())
pg_opts = {"scheduling_strategy": PlacementGroupSchedulingStrategy(
    placement_group=pg, placement_group_bundle_index=0)}
chipyard_bash = BashTool(
    name=bash_name, work_dir=CHIPYARD_PATH, task_options=pg_opts,
    timeout_seconds=600,
)
```

Synthesis runs on a separate `vlsi` worker (requesting the VLSI / Syn resources) and the Verilator suite on the `verilator_run` nodes, **at the same time**. Synthesis measures worst-case slack; Verilator is primarily a correctness gate (and an IPC-degradation estimate). If Verilator fails, the in-flight synth is cancelled, a shared-session `debug_failure` agent repairs the build, and the step retries up to `--max-debug-retries` times.

The loop ships **this repo's** `chia` (plus the example packages) to every worker via Ray `py_modules`, so workers import the head's checkout regardless of what their Docker image baked in:

```
RUNTIME_ENV = {
    "working_dir": str(_REPO_ROOT),
    "py_modules": [
        str(_REPO_ROOT / "chia"),
        str(_REPO_ROOT / "examples" / "common"),
        str(_REPO_ROOT / "examples" / "sky130_vlsi"),
        str(_REPO_ROOT / "examples" / "timing_opt"),
    ],
    "excludes": ["/DB/", "verilatorbins/ubench/", "__pycache__", ...],
}
```

17.2.4 The LLM in the loop

The optimizing agent runs on an `llm` worker via [ClaudeCodeLLM](#). It is handed the staged timing report (too large to inline in the prompt) and two MCP tools: `chipyard_bash` to grep the report and edit Chisel, and the `timing_experiment` A/B tool above:

```
llm = ClaudeCodeLLM(
    model=model, # default: claude-opus-4-8
    timeout_seconds=timeout_seconds,
    log_dir="/tmp/ray/llm_logs",
    logging_name="improve_timing",
    extra_cli_args=["--effort", "max"],
)
return llm.prompt(prompt_text, tools)
```

Three prompt variants ship in `prompts/` to steer the Iron-Law trade-off:

- `improve_timing.md` (default) — **IPC-neutral** edits only: reshape logic to cut the critical path without changing cycle behavior;
- `improve_timing_ironlaw.md` — allow IPC-trading moves when they win the iron-law product (frequency × IPC); pass via `--prompt-file`;
- `improve_timing_ironlaw_noab.md` — the iron-law variant for runs **without** the `timing_experiment A/B` tool; pair it with `--no-experiment-tool`.

17.2.5 The seed flow

With an empty DB there is no parent to optimize, so `main()` first runs `seed_flow()`: it resets chipyard to the **unmodified** base RTL (empty diff), builds, synthesizes BoomTile, runs Verilator for the baseline TMA counters, and stores the result as the baseline branch — the first DB entry and the root of the variant tree. No LLM editing step.

17.3 Setup

Note that running this flow requires creating a logical worker environment with Genus. We do not want to expose details of how to do this for the commercial tool publicly, but if you have Genus licenses, you should feel free to reach out to us for help setting this up.

These steps mirror the example’s `README.md`. Run them from `<repo>/chia` unless noted. Because the synthesis tool (we used Cadence Genus on the open-source Sky130 PDK) and its collateral are commercial, you must supply your own vlsi synthesis worker.

1. Head conda env — only the head needs it; workers get chia via Ray `py_modules` and the cluster’s Docker images. The env is named `timing_loop`; fill in its `- -e /path/to/chia` line with your checkout first:

```
conda env create -f examples/timing_opt/env.yml
conda activate timing_loop
```

2. Fill in the stubs — the example ships obvious placeholders (`/path/to/...`, `CHANGE_ME_...`, `${VAR}`) you must replace before the flow runs end-to-end. See the **Paths to fill in** checklist in the `README`; the key ones are the synthesis-tool binary and PDK collateral in `sky130_vlsi/tools-chia.yml`, the timing-report relpaths and collateral paths in `constants.py` (or the matching `TIMING_OPT_*` env vars), and the head IP, EC2 key, and vlsi worker in `timing_cluster.yaml`.

3. Benchmarks — fetch the Verilator test binaries (the suite reads `asmtests/` and `embench/`; `dramsim_ini/` ships alongside):

```
git submodule update --init examples/timing_opt/verilatorbins
```

4. Bring up the cluster — `chia up` expands `${HEAD_IP}`, `${USER}` The reference topology is seven workers (2 verilator_run + 2 chisel_build + 1 llm + 2 vlsi):

```
export HEAD_IP=10.0.0.10
chia up examples/timing_opt/timing_cluster.yaml
```

The first bring-up is slow — it pulls the large Chipyard / Verilator images. See [Cluster Configuration Reference](#) for the full schema.

5. Seed the baseline — with an empty DB the flow builds + synthesizes the unmodified RTL and stores it as the baseline branch. `TIMING_OPT_DB_DIR` is **required** and must be a stable absolute head path; the `chia job submit` entrypoint runs under Ray’s job manager and does **not** inherit your shell, so pass it (and any non-default collateral paths) via `--runtime-env-json` rather than `export`. Run from the example dir so `--working-dir .` uploads it:

```
cd examples/timing_opt
chia job submit --working-dir . \
  --runtime-env-json '{"env_vars": {"TIMING_OPT_DB_DIR": "/abs/path/on/head/timing_opt_DB
↪"}}' \
  -- python improve_timing.py --seed-only
```

6. Run a timing-optimization iteration — pick a parent branch to optimize; each invocation produces one child (`<parent>_timing_v<N>`, auto-incremented). Reuse the same `--runtime-env-json` block so the DB path stays consistent:

```
chia job submit --working-dir . \
  --runtime-env-json '{"env_vars": {"TIMING_OPT_DB_DIR": "/abs/path/on/head/timing_opt_DB
↪"}}' \
  -- python improve_timing.py --branch baseline
```

(With an empty DB you can skip step 5 — the loop auto-seeds the baseline first, then optimizes it in the same job. Re-run with the same `--branch` to grow the tree wider with another sibling.)

7. Inspect results — the reporting script prints worst slack / achievable frequency per branch against a target period:

```
python examples/timing_opt/scripts/perf_table.py \
  --db /abs/path/on/head/timing_opt_DB/timing.db --target <ns>
```

8. Tear down — when the run is done:

```
chia down examples/timing_opt/timing_cluster.yaml
```


AGENTIC ARCHITECTURAL DISCOVERY WITH EVOLUTIONARY CODING AGENTS

A CHIA case study that sets up an agentic discovery flow using evolutionary coding agents to discover novel hardware data prefetchers for [ChampSim](#) — a widely-used microarchitectural simulator for memory system research. The full flow lives in the [evolve-flows](#) repository under `examples/alphaevolve-champsim-simple`.

18.1 Overview

Architects spend significant amounts of time and resources in designing microarchitectural blocks such as data prefetchers, cache replacement policies, and branch predictors to extract performance gains in a processor. Recently, ArchAgent¹ demonstrated the ability of evolutionary coding agents to automatically discover novel cache replacement policies for last-level caches in processors in a matter of days. These discovered policies were more performant and discovered much faster than prior human-designed state-of-the-art policies.

However, designing, debugging, and operating such flows comes with significant user burden. This case study demonstrates how CHIA addresses these challenges. It implements and deploys a complete ArchAgent-like evolutionary discovery flow for prefetcher discovery — from AlphaEvolve-driven candidate generation through parallel ChampSim evaluation — in a small amount of code by composing CHIA’s library nodes. The flow is designed to be a starting point: the same pattern scales to more sophisticated evaluation cascades (RTL simulation, FPGA emulation, physical design) using CHIA’s extensible node library.

A hardware data prefetcher is a microarchitectural block that predicts which cache lines a program will access next and fetches them before they are needed, hiding memory latency. Designing a good prefetcher is hard as prefetches must be accurate and timely while covering a substantial portion of the program’s memory accesses to derive meaningful gains. Different workloads exhibit different access patterns (stride, pointer-chasing, spatial, temporal), and a prefetcher that helps one workload may hurt another by polluting the cache with useless lines. IPC (instructions per cycle) is an end-to-end metric capturing a given application’s performance on a processor. The standard metric to evaluate prefetchers is **geomean IPC** across a suite of benchmarks where higher is better.

`alphaevolve-champsim-simple` automates prefetcher design as an evolutionary discovery loop. In each iteration:

1. an evolutionary coding agent proposes a candidate C++ prefetcher module (a `struct` implementing the ChampSim prefetcher API),
2. the candidate is **compiled** into a ChampSim binary on a dedicated build worker,
3. the binary is **run in parallel** against a suite of benchmark traces — one run per trace, fanned out across a pool of simulation workers, and
4. a **result mapper** aggregates per-trace IPC into a single fitness score (geomean IPC) that feeds back to the evolutionary search.

¹ Gupta et al. “ArchAgent: Agentic AI-driven Computer Architecture Discovery.” [arXiv](#); [OpenReview](#).

Note**Goal**

This case study showcases CHIA’s ability to compose evolutionary discovery flows from reusable building blocks. The `EvolverNode` wraps various evolutionary coding agents; the `ChampSimNode` handles build and simulation of candidates; the cluster config manages worker lifecycle and resource scheduling. The same three-function interface (`build_fn`, `run_fn`, `result_mapper_fn`) plugs into any CHIA evaluator backend, so swapping the simulator — or the evolutionary agent — is a small configuration and code edit, not a complete rewrite.

18.2 How it works

18.2.1 The evolutionary discovery loop

Each iteration of the flow generates a candidate C++ prefetcher, compiles it into a ChampSim simulator, runs it against a set of benchmark traces in parallel, and feeds the aggregated performance metrics back as a fitness score. The loop repeats until a stopping criterion is met (iteration limit, manual stop, or convergence).

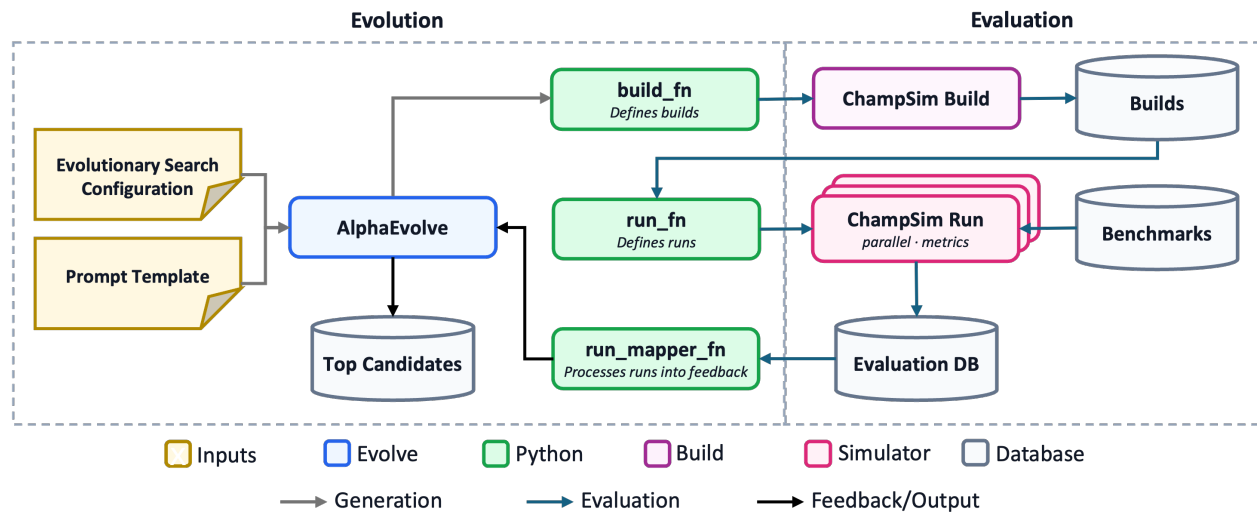


Fig. 1: Architecture of the evolutionary prefetcher discovery loop. The evolution side generates candidates using an evolutionary coding agent such as AlphaEvolve and receives fitness feedback; the evaluation side compiles each candidate, runs it against benchmark traces in parallel, and aggregates metrics into fitness feedback.

Note**Simplified example**

This example is a scaled down version of the flow described in ArchAgent: 3 SPEC CPU 2017 ChampSim traces, 5 simulation workers, and 10 search iterations — enough to demonstrate the architecture and see IPC improvements in under an hour.

18.2.2 The three-function evaluator interface

The heart of the CHIA integration is the `ChiaEvaluator`, which defines the flow's evaluation pipeline through three functions. This same interface is used by all CHIA evolutionary discovery flows, regardless of the underlying simulator:

- **build_fn** — takes a candidate program (source code) and produces a compiled artifact. Here it dispatches a ChampSim build onto a `champsim_build` worker:

```
def _build(self, program_solution: str) -> Any:
    return self._build_node.build_champsim.options(
        resources={"champsim_build": 1.0},
        num_cpus=8,
    ).chia_remote(
        self._champsim_root,
        program_solution,
        self._module_name,
        cache_level=self._cache_level,
        timeout_s=1800,
        incremental=True,
    )
```

- **run_fn** — takes the compiled binary and a workload, then runs the simulation. Each trace is dispatched to a separate `champsim` worker, so traces run in parallel up to the cluster's worker count:

```
def _run(self, *, workload: str) -> Any:
    binary = _eval_binary.get()
    return self._run_node.run_champsim.chia_remote(
        binary=binary,
        trace=workload,
        warmup_instructions=self._warmup,
        simulation_instructions=self._sim_instr,
        timeout_s=3600,
    )
```

- **result_mapper_fn** — transforms raw simulator output into the metrics dict the search backend consumes. For ChampSim this extracts IPC, MPKI, prefetch accuracy, and coverage, then computes a geometric mean IPC as the combined fitness score.

The `ChiaEvaluator` orchestrates these three functions: it calls `build_fn` once per candidate, fans out `run_fn` across all configured workloads in parallel via CHIA's resource scheduler, collects the results, and calls `result_mapper_fn` to produce the score. Build failures (invalid C++ is expected from LLM-generated code) score `0.0` and the search continues — the evaluator never crashes on a bad candidate.

18.2.3 EvolverNode: pluggable evolutionary agents

The `EvolverNode` wraps [SkyDiscover](#) — a flexible framework for AI-driven scientific and algorithmic discovery with support for evolutionary coding agents such as `AdaEvolve`, `OpenEvolve`, `EvoX`, and `GEPA`. CHIA adds an [AlphaEvolve](#) backend via the AlphaEvolve Google Cloud API. The `search.type` field in the config YAML selects which agent drives candidate generation; the evaluation pipeline stays the same regardless.

18.2.4 Distributing work across the cluster

The cluster uses three logical worker types, each tagged with a custom CHIA resource so the scheduler routes tasks to the right container:

| Node type | Workers | Role |
|-----------------------------|---------|---|
| <code>champsim_build</code> | 1 | compiles candidates into ChampSim binaries (Docker, 8 CPUs, <code>champsim_build</code> resource) |
| <code>champsim</code> | 5 | runs simulations against traces in parallel (Docker, <code>champsim</code> resource); one trace per worker |
| <code>evolver</code> | 1 | hosts the <code>EvolverNode</code> Ray actor that drives the search loop (conda, <code>evolver</code> resource) |

The build worker is pinned to 8 CPUs (`--cpuset-cpus=0-7`) for fast compilation. The five run workers execute traces concurrently — with three traces per iteration, all three run in parallel with two workers spare for the next iteration's overlap.

Each `ChampSimNode` is created with `require_colocated=False`, meaning builds and runs are dispatched to **any** available worker with the required resource, not pinned to a specific container. CHIA's resource scheduler handles the assignment:

```
build_node = ChampSimNode(require_colocated=False)
run_node = ChampSimNode(require_colocated=False)
```

The `EvolverNode` itself is a **detached** Ray actor — it survives the launching process, which enables the `stop` and `status` subcommands to connect from a separate terminal and interact with a running search. This is a standard Ray pattern that CHIA leverages for long-running search lifecycle management:

```
evolver = EvolverNode.options(
    name="adaevolve-prefetcher-simple-evolver",
    lifetime="detached",
    resources={"evolver": 1.0},
    runtime_env=RUNTIME_ENV,
).remote()
```

18.2.5 The seed program

The search starts from a **seed program** — a minimal next-line prefetcher that prefetches the cache line immediately following each L2C access. This gives the search a non-zero baseline IPC to improve upon, and ensures the first iteration exercises the full build → run → map pipeline before the LLM generates any candidates:

```
#include <stdint>
#include "address.h"
#include "modules.h"

struct evolved_pf : public champsim::modules::prefetcher {
```

(continues on next page)

(continued from previous page)

```

using prefetcher::prefetcher;

uint32_t prefetcher_cache_operate(champsim::address addr, champsim::address ip,
                                  uint8_t cache_hit, bool useful_prefetch,
                                  access_type type, uint32_t metadata_in) {
    champsim::block_number pf_addr{addr};
    prefetch_line(champsim::address{pf_addr + 1}, true, metadata_in);
    return metadata_in;
}

uint32_t prefetcher_cache_fill(champsim::address addr, long set, long way,
                                uint8_t prefetch, champsim::address evicted_addr,
                                uint32_t metadata_in) {
    return metadata_in;
}
};

```

The system prompt (shared by both search backends) describes the ChampSim prefetcher API, optimization objectives (IPC, prefetch accuracy, coverage, MPKI), and techniques to explore (stride detection, spatial locality, temporal correlation, best-offset, Markov chains, adaptive throttling). The LLM is constrained to pure C++17 with no external libraries.

18.3 Setup

These steps apply to both search backends. Run them from the example directory (`examples/alphaevolve-champsim-simple`) unless noted.

1. Prerequisites — this flow requires:

- A conda environment with chia installed
- ChampSim traces for your benchmarks of choice. We use 3 SPEC17 traces from the [4th Data Prefetching Championship \(DPC-4\) trace collection](#).
- AlphaEvolve prerequisites:
 - a GCP project with AlphaEvolve enabled and a Gemini Enterprise license (trial available) — we suggest working through the [Install and Configure AlphaEvolve](#) page
 - a gcloud credentials file: `run gcloud auth application-default login`
- AdaEvolve prerequisites:
 - an LLM API key (Gemini, OpenAI, Anthropic, DeepSeek, Mistral, or Cohere)

2. Install evolve-flows — clone the `evolve-flows` repository and install it along with SkyDiscover into your conda environment:

```

git clone --recurse-submodules git@github.com:ucb-bar/evolve-flows.git
cd evolve-flows
conda activate your_env
pip install -e ./skydiscover
pip install -e .

```

3. Configure traces and output — in `run_flow.py`, update `TRACE_DIR` to point at the directory containing your trace files, set the `WORKLOADS` list to the traces you want to evaluate against, and optionally update `DEFAULT_OUTPUT_BASE` to your desired output location:

```
TRACE_DIR = "/path/to/traces"
WORKLOADS = [
    os.path.join(TRACE_DIR, "trace_1.champsimtrace.xz"),
    os.path.join(TRACE_DIR, "trace_2.champsimtrace.xz"),
    os.path.join(TRACE_DIR, "trace_3.champsimtrace.xz"),
]
DEFAULT_OUTPUT_BASE = "./results"
```

We use a single-server cluster for this case study. For a multi-server cluster, we recommend using a networked file system or a CHIA database node for these directories for simplicity.

The simulation parameters (5M warmup + 25M simulation instructions per trace) are set in the same file and can be adjusted for longer or shorter runs.

4. Cluster config — in `cluster.yaml`, set `provider.head_ip` to your head machine's hostname or IP, and update `compatible_ips` for each node type to list the machines those workers should run on. Update the `auth` section with your SSH credentials:

```
provider:
  head_ip: your-head-machine

auth:
  ssh_user: your-username
  ssh_private_key: /path/to/your/ssh/key

available_node_types:
  champsim_build:
    compatible_ips: [your-head-machine]
    docker:
      image: "ghcr.io/ucb-bar/chia-champsim:latest"
      pull_before_run: true
      run_options:
        - -v /path/to/traces:/path/to/traces:ro
    # ...
  champsim:
    compatible_ips: [your-head-machine]
    docker:
      image: "ghcr.io/ucb-bar/chia-champsim:latest"
      pull_before_run: true
      run_options:
        - -v /path/to/traces:/path/to/traces:ro
    # ...
  evolver:
    compatible_ips: [your-head-machine]
    # ...
```

All three worker types can run on a single machine. The `champsim_build` and `champsim` workers use the `ghcr.io/ucb-bar/chia-champsim:latest` Docker image, which `chia` up pulls automatically (`pull_before_run: true`). Update the `-v` run option to mount your trace directory into the containers. See [Cluster Configuration Reference](#) for the full schema.

5. Bring up the cluster:

```
conda activate your_env
cd /path/to/evolve-flows
```

(continues on next page)

(continued from previous page)

```
chia up examples/alphaevolve-champsim-simple/cluster.yaml
```

Verify at the Ray dashboard (<http://localhost:8265>) — you should see 7 workers with resources `champsim_build: 1, champsim: 5, evolver: 1`.

18.3.1 Running with AlphaEvolve (primary path)

AlphaEvolve uses Google’s cloud API for candidate generation. Ensure you have completed the AlphaEvolve prerequisites from Step 1:

- A GCP project with AlphaEvolve enabled and a Gemini Enterprise license (trial available) — work through the [Install and Configure AlphaEvolve](#) page
- Application default credentials: run `gcloud auth application-default login`

6a. Set up AlphaEvolve config — in `config_alphaevolve.yaml`, update the `alphaevolve:` section to match your GCP project:

```
alphaevolve:
  project_id: "YOUR_GCP_PROJECT"
  engine_id: "alpha-evolve-experiment-engine"
  credentials_file: "application_default_credentials.json"
```

7a. Launch the search:

```
cd examples/alphaevolve-champsim-simple
python run_flow.py --config config_alphaevolve.yaml
```

`config_alphaevolve.yaml` sets `search.type: "alphaevolve"` and configures evolutionary search parameters.

18.3.2 Running with AdaEvolve (alternative path)

AdaEvolve runs entirely locally — no GCP project or external credentials beyond an LLM API key are needed. This makes it a good starting point, and demonstrates CHIA’s ability to swap search backends with a config change.

6b. Set the LLM API key — SkyDiscover’s included backends (AdaEvolve, EvoX, etc.) use an OpenAI-compatible interface and support multiple LLM providers. The default config uses Gemini, but you can switch by changing the model names in `config_adaevolve.yaml` and exporting the corresponding API key:

| Provider | Model name prefix | Environment variable |
|-----------|----------------------|----------------------|
| Gemini | gemini-... (default) | GEMINI_API_KEY |
| OpenAI | openai/gpt-... | OPENAI_API_KEY |
| Anthropic | anthropic/claude-... | ANTHROPIC_API_KEY |
| DeepSeek | deepseek/... | DEEPSEEK_API_KEY |
| Mistral | mistral/... | MISTRAL_API_KEY |
| Cohere | cohere/... | COHERE_API_KEY |

For example, to use Gemini (the default):

```
export GEMINI_API_KEY="your-api-key"
```

7b. Launch the search:

```
cd examples/alphaevolve-champsim-simple
python run_flow.py --config config_adaevolve.yaml
```

config_adaevolve.yaml sets search.type: "adaevolve" and configures the LLM models, population parameters, and search hyperparameters. The evaluation pipeline is identical — only the candidate generation differs.

18.3.3 Monitoring and stopping

Because the EvolverNode is a detached Ray actor, you can query and control a running search from a separate terminal:

```
# In a separate terminal (same conda env, same directory):
python run_flow.py status      # prints state, iteration, best_score
python run_flow.py stop       # graceful stop after current iteration
```

The launching terminal polls the actor every 30 seconds and incrementally saves the best program to best_prefetcher.cc in the output directory whenever the score improves.

Tear down — when the run is done:

```
cd /path/to/evolve-flows
chia down examples/alphaevolve-champsim-simple/cluster.yaml
```

18.3.4 Outputs

The output directory (printed at launch) contains:

- best_prefetcher.cc — the best evolved C++ prefetcher source
- chia_eval_log.jsonl — per-evaluation build/run details (one JSON line per candidate, including per-trace IPC, MPKI, prefetch accuracy, and coverage)

18.3.5 What success looks like

- Cluster comes up with all 7 workers
- First iteration completes (seed program builds, runs on all traces, returns a non-zero baseline IPC)
- Subsequent iterations generate new candidates via the LLM — some fail to compile (expected; scored 0.0), others build and run
- best_prefetcher.cc contains valid C++ prefetcher code
- chia_eval_log.jsonl has entries with combined_score > 0
- python run_flow.py status and python run_flow.py stop work from separate terminals

18.3.6 Troubleshooting

- **Ray connection refused:** check ray status on the head machine, or re-run chia up
- **Build failures:** check chia_eval_log.jsonl for failure_stage: build entries — LLM-generated C++ can sometimes fail to compile, which is expected (build failures score 0.0 and the search continues)
- **Auth errors (AlphaEvolve):** ensure you have run gcloud auth application-default login and that GOOGLE_APPLICATION_CREDENTIALS points to a valid credentials file (see prerequisites)
- **Early termination (AlphaEvolve):** the AlphaEvolve controller has an idle_timeout_s that stops the search if no new candidates are generated or evaluated and the evaluation queue is empty for that duration. If your evaluations take a long time (e.g. 20+ minutes per candidate), the timeout can fire between batches — after

evaluations drain and before the remote API produces the next candidate. Increase `idle_timeout_s` in the `alphaevolve:` config section to at least the worst-case gap (we default to 1800s). Set to `0` to disable entirely, but note that the controller will then never self-terminate if the backend goes quiet.

- **API key errors (AdaEvolve):** ensure the API key for your configured provider is exported (e.g. `GEMINI_API_KEY`, `OPENAI_API_KEY`) in the shell that runs `python run_flow.py` — it must match the `api_key` field in `config_adaevolve.yaml`

AUTONOMOUS CIRCT ISSUE SOLVING

A CHIA case study that uses an LLM-in-the-loop to triage the open-issue backlog of **CIRCT** — the MLIR-based hardware compiler infrastructure — and, for each candidate, drive a sequence of agents through **assess** → **reproduce** → **fix** → **verify** → (**regression repair**) → **writeup** inside a real CIRCT checkout. The full flow lives in `chia/examples/circt_issue_solver`.

19.1 Overview

Large open-source projects are facing a new burden: AI tools have enabled a significant increase in outside contributions to large open source GitHub repositories, including new issue reports and pull requests, but has lowered the overall quality of the contributions. As noted in the LLVM project’s AI tool use policy, when these contributions are done entirely by AI in an unprincipled way and without human review, this “extracts work from [maintainers] in the form of design and code review”. CHIA cannot make a contributor review their AI’s work, but it can force the AI to make improvements in a principled, structured way, that leads naturally to high quality bug fixes and PRs.

`circt_issue_solver` shows this. We take a multi-step approach to fixing an issue which ensures that fixes are only proposed for real bugs, and only for issues where discussion has converged on a clear solution. Furthermore, we run validation programmatically, including the entire regression test suite, and a reproduction script that confirms that the bug has gone from unfixed to fixed based on the LLMs changes. The pipeline works as follows:

First, the head node **triages** open issues down to a sample of plausible candidates, then runs one independent pipeline per candidate across a pool of CIRCT containers. Each pipeline:

1. **assesses** the issue — is this actually a bug, and are *both* the bug and the correct behavior clear enough to act on autonomously? If not, it logs the reason and skips;
2. **reproduces** it — writes `.circtissues/repro.sh` with the contract *exit 0 iff the bug is fixed*, and skips the issue if it doesn’t actually reproduce on the pinned tree;
3. **fixes** it — edits `/workspace/circt`, rebuilds, reruns the repro, and adds a lit regression test;
4. **verifies** deterministically — rebuilds, reruns the repro, and runs the full lit gate;
5. on a regression, gets **one repair turn** to fix the broken tests without un-fixing the bug; and
6. writes the **PR description it would submit**.

The output is local: a candidate diff and the PR writeup, persisted to `issue_logs/issue_<N>/` and a row in a SQLite database (`issues.db`). A second flow reads **PR review feedback** (reviewer comments *and* failing CI) and produces an updated diff plus the replies it would post. Crucially, **neither flow writes to GitHub — both only read**. A human reviews and submits.

Note

Result

We worked with the maintainers of CIRCT to target our flow at a manageable number of issues. It correctly assessed 16 issues, 5 of which were reproducible bugs with clear solutions. It solved all 5. Of those 5, 2 were not eligible for AI assisted PRs (because they were labelled as “good first issues”). We submitted PRs for the other 3 and they have all been upstreamed. Read section 5 of our [arXiv paper](#) to hear more about our results.

19.2 How it works

19.2.1 The per-issue pipeline

Triage runs on the head; each surviving candidate is fanned out as one `run_issue_remote` task pinned to a single CIRCT container. Within that container the phases run in sequence, and any phase can short-circuit the pipeline (`not_a_bug` / `unclear` / `no_repro`) so a fix attempt is only ever spent on an issue that has cleared the gates before it:

```
HEAD (triage, read-only GithubIssuesNode)
|
+-- list the open backlog, drop: already attempted, feature requests,
|    repro-less issues, and issues with an open PR attached
+-- sample max_issues candidates at random
|
+-- fan out one run_issue_remote per candidate across the circt slots
    |
    v
CIRCT WORKER (one issue per container, pinned)
|
+-- git reset --hard <tag> + warm incremental build
+-- ASSESS      [llm]  bug? AND is the correct behavior clear?  -> skip: not_a_bug / unclear
+-- REPRODUCE   [llm]  write .circtissues/repro.sh (exit 0 iff fixed)  -> skip: no_repro
+-- FIX         [llm]  edit /workspace/circt, rebuild, rerun repro, add a lit test
+-- VERIFY      [deterministic, no LLM]  rebuild, rerun repro, run the full lit gate
+-- REGRESSION  [llm]  only if repro is green but the suite went red - one repair turn
+-- WRITEUP     [llm]  the PR description it WOULD submit
|
+-- return result dict -> HEAD persists issue_logs/issue_<N>/ + a row in issues.db
```

Each LLM phase is a fresh, stateless Claude `--print` session: the context a later phase needs is inlined into its prompt (the `repro.sh` into the fix prompt; the diff and verdict into the writeup), rather than carried via `--resume`.

19.2.2 Triage on the head

Triage is a cheap heuristic filter over the open backlog using the read-only `GithubIssuesNode`. It lists the most recent open issues *without* fetching per-issue comments (the filter only needs title/body/labels), keeps issues that carry a fenced code block plus either a tool command or a failure signal (crash/assert/miscompile), and drops feature requests, already-attempted issues, and anything with an open PR attached. There is deliberately **no label gate** — whether an issue is really a bug is left to the per-issue assess phase, which reads the issue *and* the source:

```
def has_repro(issue) -> bool:
    """A self-contained repro: a fenced code block + a tool command or a
    failure signal (crash/assert/miscompile)."""
    body = issue.body or ""
```

(continues on next page)

(continued from previous page)

```
return "" in body and (bool(_CMD_RE.search(body)) or bool(_SIGNAL_RE.
→search(body)))
```

The qualifying set is shuffled and sampled, and only the chosen survivors are re-fetched with their comments attached — so the expensive per-issue requests are paid only for the handful that will actually run.

19.2.3 The deterministic verify gate

To keep fixes principled, the fix is judged by a **verify step that uses no LLM** — programmatic orchestration confirms the change works as well as the agent claims, so the agent cannot talk its way past it or mark its own work fixed. Verify captures the diff, rebuilds the tool targets, reruns the repro (which exits 0 only when the bug is fixed), and runs the whole lit suite minus the categories that are red on the unmodified image (test/CAPI and Tools/circt-tblgen need binaries this SDK-only build doesn't ship). An issue is only marked fixed when the repro is green **and** the full lit gate is green:

```
def circt_lit_gate_paths() -> list[str]:
    """Every top-level test/<category> except the baseline-red ones, so a
    gate failure means a real regression, not a missing-binary artifact."""
    ...

# status = "fixed" iff repro green AND full lit gate green; else "attempted"
```

If the repro flips green but the suite goes red, the pipeline fires a single **regression-repair** turn — the failing tests and their output inlined — to mend the regression without un-fixing the bug, then re-verifies. Because the gate runs the whole suite (not just the touched dialect), it catches collateral damage anywhere in the tree and never vacuously passes.

19.2.4 Distributing work across the cluster

Each per-issue task is a CHIA function that holds one whole circt slot, so exactly one issue runs per CIRCT container. Within the task, every LLM phase is dispatched onto an llm worker (1.0 per call, so the cluster's llm slots cap prompt concurrency) while the bash / build / lit MCP servers stay on the CIRCT worker and are reached over HTTP — the heavy CIRCT checkout never moves:

```
@ChiaFunction(resources={"circt": 1})
def run_issue_remote(issue_md: str, number: int, cfg: dict, ...) -> dict:
    ...
    cli = get(llm.prompt.options(resources={"llm": 1.0})
             .chia_remote(llm, prompt, tools))
```

The flow ships **this repo's** chia — plus the head-side circt_util.py and issue_task.py — to every worker via Ray py_modules, so edits to the flow reach workers on the next submit with no image rebuild:

```
_CHIA_PKG = FLOW_DIR.parent.parent / "chia"
_PY_MODULES = [str(FLOW_DIR / "circt_util.py"),
               str(FLOW_DIR / "issue_task.py"),
               str(_CHIA_PKG)]
...
ray.init(address="auto",
         runtime_env={"py_modules": _PY_MODULES, "excludes": _RUNTIME_ENV_EXCLUDES})
```

The general CIRCT build/test primitives and the *BuildTool* / *LitTool* MCP wrappers live in the chia package (chia.chipyard.circt), so they ride along with it.

19.2.5 The LLM in the loop

Each phase runs on an llm worker via [ClaudeCodeLLM](#), under a shared system prompt that casts it as a senior MLIR/CIRCT engineer and pins the rules that keep a fix honest — make the smallest root-cause change, never disable a test or special-case the repro, and treat LLVM/MLIR (the prebuilt SDK) as out of scope rather than hacking around it. A fresh session is built per phase:

```
llm = ClaudeCodeLLM(  
    model=cfg["model"], system_message=cfg["system_prompt"],  
    timeout_seconds=cfg["timeouts"][phase],  
    extra_cli_args=["--effort", "max"],  
    resume_session=True, projects_cwd=None,  
)
```

Alongside the prompt the agent gets MCP tools that execute inside the CIRCT container: a [BashTool](#) to read, edit, and run shell commands in `/workspace/circt`; an async `BuildTool` that starts a ninja rebuild and is polled to completion; and an async `LitTool` that runs a lit regression set. The async build/lit tools return immediately and are polled, so a long build can't stall the transport.

The assess phase is the one that most shapes quality, and it is where the “principled” commitment starts. It separates “is this a bug?” from “is the correct behavior clear?” — a crash is an obvious defect, but what the tool *should* do instead is often a design decision (e.g. reject the input vs. extend the code to handle it). The phase is asked to **enumerate the materially different ways a maintainer could reasonably resolve the issue**; if more than one is defensible and nothing in the issue, spec, or docs singles one out, it returns `UNCLEAR` and the pipeline skips the issue rather than producing a confidently-wrong patch. That guard is exactly what caught issue #8508 in the study (a transform run on an `extmodule` DUT where “diagnose and refuse” and “extend to support it” were both defensible — see `issue_logs/issue_8508/`): the bug was crystal clear, but the *fix* was a maintainer’s design call, so the flow correctly declined to guess.

19.2.6 The review flow

A companion flow (`review_loop.py` → `review_task.py`) closes the human loop. Given a PR:ISSUE pair, it reconstructs the PR’s state on the pinned tree (re-applying the PR’s current diff fetched from GitHub via [GithubPullsNode](#)), then runs **triage** → (if **actionable**) **fix** → **verify** → **replies** over the reviewer comments *and* any failing CI checks. A PR that is simply red in CI — with no human comments — is enough to trigger a round. It produces an updated diff and the author replies it would post; as with the issue flow, nothing is written back to GitHub.

19.2.7 Principled fixes and maintainer friendliness

The two commitments from the overview are not afterthoughts — they are wired into the pipeline and into how the flow was actually run, and together they are what let every submitted PR be upstreamed.

Fixes are principled by construction. The shared system prompt holds the agent to the smallest root-cause change and forbids the shortcuts that make an automated “fix” worthless — or worse, harmful — to a reviewer: disabling or weakening a test, special-casing the repro, or hacking around an LLVM/MLIR root cause it cannot legitimately build. Two gates enforce this rather than trusting the agent’s word:

- the **assess phase** (above) refuses to act unless the bug *and* its correct resolution are unambiguous, by enumerating the reasonable resolutions and bailing to `UNCLEAR` when more than one stands — so a fix is only ever attempted where there is a single clear answer;
- the **verify phase** (above) runs with no LLM in the loop at all: programmatic orchestration rebuilds, reruns the repro, and runs the full regression suite, so a change is only ever called *fixed* when it provably is.

The human — and the maintainers — stay in charge. Neither flow writes to GitHub; every diff and PR writeup is a *proposal*. In the study a human reviewed each change in detail and hand-wrote the pull request before anything reached upstream CIRCT. The load on the maintainers was bounded just as deliberately: the flow was run on a small,

randomly chosen set of issues — in close coordination with the CIRCT maintainers on both the *quantity* and the *quality* of contributions — rather than firing patches at the whole backlog. Issues labelled *good first issue* were deliberately left for human newcomers, in keeping with the project’s norms and the broader LLVM concern that unprincipled AI contributions mostly extract design- and code-review effort from maintainers. When the submitted PRs drew review feedback, that feedback was addressed with the **review flow** above — the same principled, human-in-the-loop machinery — before resubmitting. See our [arXiv paper](#) for the full study.

19.2.8 Targeting a different repository

The repo is the one thing you change to retarget both flows. In `config.py`:

```
GITHUB_REPO = "llvm/circt" # read by both flows
```

It defaults to `llvm/circt`; the only supported change is pointing it at a CIRCT **fork** (e.g. to review PRs on your own fork) — the rest of the flow still assumes CIRCT’s build and lit conventions.

Note

The `chia-circt` image is pinned at **firtool-1.148.0**. Issues fixed upstream after that tag won’t reproduce, so the reproduce gate correctly marks them `no_repro` and the pipeline moves on. Likewise, a root cause that lives in LLVM/MLIR (the prebuilt SDK / `llvm` submodule) is out of scope — only CIRCT’s own tree is buildable here, so the agent reports such cases instead of working around them.

19.3 Setup

These steps mirror the example’s `README.md`. Run them from the example directory (`<repo>/chia/examples/circt_issue_solver`) unless noted.

1. Head conda env — only the head needs this; workers get `chia` via Ray `py_modules` and run the Docker images named in `cluster.yaml`:

```
conda env create -f env.yml
conda activate circtissues
```

2. Cluster — `cluster.yaml` is single-machine by default: the head plus four containers on one host, read from CHIA_HEAD. The required worker counts (`min_workers` / `max_workers`):

| Node type | Workers | Role |
|---------------------------|---------|---|
| <code>circt_llm</code> | 2 | runs the assessing / fixing / reviewing LLM (light); image <code>chia-claude-code</code> |
| <code>circt_worker</code> | 2 | owns a <code>/workspace/circt</code> checkout, builds + runs lit (heavy); image <code>chia-circt</code> |

Each `circt_llm` container bind-mounts your Claude Code config into the container. If your credentials live somewhere other than `~/.claude`, edit the mount in `cluster.yaml` to match:

run_options:

```
- "-v ~/.claude:/home/ray/.claude" # mount your Claude config into the container
```

Scale up by raising the per-type and cluster-wide `min/max_workers` and adding IPs to `compatible_ips`. The default ports (GCS 6379, dashboard 8265) mean only one CHIA cluster should be up per host at a time. See [Cluster Configuration Reference](#) for the full schema.

3. GitHub token — both flows read issues/PRs through an authenticated client. Set a token with read access to the repo (if using public CIRCT this step can be skipped):

```
export GITHUB_TOKEN=...           # read access to GITHUB_REPO
export CHIA_HEAD=$(hostname)      # host to bring the cluster up on
```

4. Config — in `config.py`, leave `GITHUB_REPO` at `llvm/circt` or point it at a CIRCT fork.

5. Bring up the cluster — from the example directory with the env active:

```
chia up cluster.yaml
```

The first run pulls the `chia-circt` image, which is large; on slow links the pull may time out, so raise the pull timeout and retry.

6. Run the issue flow — `fix_issues_submit.sh` wraps `chia job submit` so the driver's logs land in the Ray dashboard (<http://localhost:8265>) and in `chia job logs <id>`:

```
./fix_issues_submit.sh --max-issues 2           # triage + attempt 2 candidates
./fix_issues_submit.sh --issue 10568            # one specific issue, skip triage
NO_WAIT=1 ./fix_issues_submit.sh --max-issues 5 # detach; watch the dashboard
```

7. (Optional) Review flow — feed it a PR number paired with its issue number; the PR's current diff and feedback are fetched from GitHub:

```
./review_submit.sh --pr 10648:7388
```

8. Tear down — when the run is done:

```
chia down cluster.yaml
```

Outputs land in `issue_logs/issue_<N>/`: the candidate `fix.diff`, the `pr_writeup.md` it would submit, a `verdict.json` (status, repro/build/lit results, diff counts), the saved repro, and per-phase LLM transcripts (`llm_<phase>.md` / `.jsonl`) — plus one row per attempt in `issues.db`. Review-flow outputs land under `review_logs/issue_<N>_pr_<M>/` (the updated diff and the replies it would post). Re-running skips issues already recorded in `issues.db`.

BASE RUNTIME

API reference for `chia.base`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

20.1 Bypass

General-purpose bypass for ChiaFunction tasks.

When bypass is active, a function is still dispatched through Ray (testing cluster scheduling, placement groups, resource allocation) but the real computation is replaced with pre-recorded data.

20.1.1 How it works

The bypass check happens on the caller side (in `chia_remote()`). When a function is bypassed:

1. The caller computes the mock data using the registered provider.
2. Instead of dispatching the real function, it dispatches a trivial `_bypass_return(data)` function with the **same scheduling options** (placement group, resources, etc.).
3. The worker receives the pre-computed data and just returns it.

20.1.2 Usage

```
from chia.base.bypass import Bypass

# Create a bypass instance - always, as part of loop setup.
# If yaml_path is None, bypass does nothing (all functions run normally).
bypass = Bypass(yaml_path=args.bypass_config)

# Register project-specific providers that build the correct dataclasses.
bypass.set_provider("my_function", my_provider_fn)

# Optionally gate the bypass on a runtime condition (default: always on).
bypass.set_cond("my_function", my_cond_fn)

# Done - the ChiaFunction trampoline checks the active bypass automatically.
```

20.1.3 YAML format

```
bypass:
  func_name:
    bypass: true
    data: /path/to/file.json  # optional

# shorthand (bypass, no data):
simple_function: true

# not listed = not bypassed (default, no error)
```

20.1.4 Provider signature

The provider runs on the worker (same scheduling as the real function):

```
def my_provider(tag, data_path, *args, **kwargs) -> ReturnType
```

- tag: the `_chia_tag` from the caller (str or None)
- data_path: the YAML data field (str or None)
- *args, **kwargs: the original function arguments

Resolution when a function is bypassed:

1. Provider registered -> `provider(tag, data_path, *args, **kwargs)`
2. Only data_path -> returns file contents (default file provider)
3. Neither -> error

20.1.5 Condition gate

A function may also register a *condition* via `set_cond`. It runs on the caller as the last gate in `is_bypassed` — after the bypass flag, the provider/data check, and the tag patterns — and decides whether the bypass actually happens:

```
def my_cond(tag, data_path, *args, **kwargs) -> bool
```

A falsy return means the call runs for real instead of being bypassed. With no condition registered the default is to bypass (True). Useful to gate on runtime state, e.g. only replay from the cache when the value is present.

`chia.base.bypass.get_active_bypass()`

Return the active Bypass instance, or None if bypass is not in use.

class `chia.base.bypass.Bypass`(*yaml_path*: str | None = None)

Bases: object

Controls which ChiaFunction tasks return pre-recorded data.

Create one instance per loop run, always, as part of setup:

```
bypass = Bypass(yaml_path=args.bypass_config)
```

If `yaml_path` is None, bypass does nothing — `is_bypassed()` returns False for all functions. No errors, no special handling.

If `yaml_path` is a path, the YAML is parsed and functions listed with `bypass: true` will be bypassed (if they also have a provider or data path).

The constructor registers itself as the active bypass instance. The ChiaFunction trampoline checks this automatically.

`file_server()`

Return the bypass file server actor handle, creating it if needed.

Custom providers can use this to read arbitrary files from the node that owns this Bypass:

```
def my_provider(tag, data_path, *args, **kwargs):
    server = get_active_bypass().file_server()
    text = ray.get(server.get_text.remote(data_path))
    ...
```

`set_provider(func_name: str, provider: Callable) → None`

Register a provider for *func_name*.

The provider runs on the worker (not the head node) and replaces the real function when it is bypassed. It is called as:

```
provider(tag, data_path, *args, **kwargs) -> return_value
```

- *tag*: the `_chia_tag` from the caller (str or None)
- *data_path*: the YAML data field for this function (str or None)
- *args, **kwargs*: the original function arguments, passed through

`set_cond(func_name: str, cond: Callable) → None`

Register a bypass condition for *func_name*.

The condition is an additional gate, evaluated **on the caller** as the last step of `is_bypassed()` — after the bypass flag, the provider/data check, and the tag patterns. It is called as:

```
cond(tag, data_path, *args, **kwargs) -> bool
```

with the same arguments as the provider (`set_provider()`). If it returns a falsy value the call is **not** bypassed and runs for real; if it returns truthy the bypass proceeds. When no condition is registered for a function the default is True (no extra gate).

Use it to make the bypass decision depend on runtime state, e.g. only replay from the cache when the value is actually present:

```
def cache_hit(tag, data_path, *args, **kwargs):
    return ray.get(get_active_cache().has.remote(tag))

bypass.set_cond("run_verilator_test", cache_hit)
```

property active: bool

True if any functions are configured for bypass.

`is_bypassed(func_name: str, tag: str | None = None, *args, **kwargs) → bool`

Should *func_name* be bypassed on this call?

Returns True only when:

1. `bypass[name]` is True (from YAML)
2. There is a provider or a `data_path`

3. (if a `tags` list is configured) `tag` matches a pattern
4. (if a condition is registered) the condition returns truthy

The `tag` parameter identifies a specific call (e.g. “`iter0_opt2`”). If the YAML has a `tags` list for this function, only calls whose tag matches a pattern are bypassed. If no `tags` list, all calls are bypassed regardless of tag.

The optional `*args`, `**kwargs` are the original call arguments; they are forwarded to a registered condition (see `set_cond()`). The condition is the **last** gate: it runs only after tag matching has already passed, and a falsy return means the call runs for real. With no condition registered the default is to bypass (`True`). A condition may invoke remote calls (e.g. a cache lookup), so callers that use `is_bypassed()` as a cheap predicate should be aware it can do work when a condition is registered.

Functions not listed in the YAML return `False` (not bypassed). If no YAML was loaded, always returns `False`.

get_provider_info(*func_name: str*) → tuple[Callable, str | None]

Return (provider, data_path) for dispatching on the worker.

Resolution:

1. Provider registered -> (provider, data_path_or_None)
2. Only data_path (no provider) -> (_default_file_provider, data_path)
3. Neither -> `KeyError`

The provider is executed on the worker, not here. This method only looks up what to dispatch.

get_state() → dict

Return a serializable dict of the bypass state, including providers.

Providers and conditions are serialized via cloudpickle (Ray’s default). This works for module-level functions and most common cases. Lambdas/closures capturing unpicklable objects will fail with a clear error.

The file server actor handle is included so workers can read files from the node that owns this Bypass.

load_state(*state: dict*) → None

Restore bypass state from a dict (from `get_state()`).

20.2 Cache

Head-node output cache for ChiaFunction tasks.

A small key/value store, pinned to the head node as a Ray actor, that pickles arbitrary Python objects keyed by a string tag, with an LRU byte budget and a warm-start scan from disk. It has two halves that work together with the `chia.base.bypass` mechanism:

- **Write is automatic.** A function marked `cache: true` in the YAML has its real-run output written to the cache automatically by ChiaFunction (via an `ObjectRefCallback` that fires when `get()` resolves the result), keyed by the call’s `_chia_tag`. No user code is needed.
- **Read is manual, via bypass.** The cache does *not* auto-serve. To replay a cached value, register a bypass provider that reads it back:

```
def cache_provider(tag, data_path, *args, **kwargs):
    hit, value = ray.get(get_active_cache().read.remote(tag))
    if not hit:
        raise KeyError(f"cache miss for tag {tag!r}")
```

(continues on next page)

(continued from previous page)

```

    return value

bypass.set_provider("run_verilator_test", cache_provider)

```

Cache = write-through populate; bypass = read path. They share the tag.

20.2.1 Usage

```

from chia.base.cache import start_cache

# Call once on the driver after ray.init(). Idempotent.
start_cache(size=4, units="GB", cache_dir_path="/data/chia_cache",
            yaml_path=args.bypass_config)

```

20.2.2 YAML format (same file as bypass, parallel cache: section)

```

cache:
  run_verilator_test:
    cache: true
    tags: ["iter.*"]  # optional; mirror bypass tag patterns

# shorthand (cache, no tags):
build_megaboom: true

```

`chia.base.cache.start_cache(size: float, cache_dir_path: str, units: str = 'B', yaml_path: str | None = None, namespace: str | None = None)`

Create (or find) the head-pinned cache actor. Idempotent.

Call from the driver after `ray.init()`. Mirrors `chia.trace.profiler.start_collector()`.

The actor is created **detached** so it is reachable from workers in a *different* Ray job (e.g. a bypass provider running on a remote worker) — a plain named actor is only visible within its creating job. Pair it with an explicit `namespace` so cross-job lookups resolve it. `stop_cache` still tears it down; cross-run reuse remains the on-disk pickles + warm start, not a lingering live actor.

Parameters

- **size** – Numeric budget; multiplied by `UNITS[units]` to get bytes.
- **cache_dir_path** – Directory for the pickle files (on the head node).
- **units** – One of `UNITS` (default "B").
- **yaml_path** – Optional path to the bypass/cache YAML; the `cache:` section opts functions into automatic caching.
- **namespace** – Optional Ray namespace for the named actor.

Returns

The Cache actor handle.

`chia.base.cache.stop_cache(namespace: str | None = None) → None`

Kill the cache actor. Idempotent.

Cross-run reuse comes from the on-disk pickles + warm-start scan, not from a persisted live actor, so killing the actor loses nothing on disk.

`chia.base.cache.get_active_cache(namespace: str | None = None)`

Return the cache actor handle, or None if no cache was started.

Always resolves the current named actor via `ray.get_actor` — no process-local handle is cached. A worker (e.g. inside a bypass provider or a nested dispatch) reaches the cache with no state threading, and a cache that was restarted/replaced is picked up automatically rather than serving a stale, dead handle.

`chia.base.cache.is_cached(func_name: str, tag: str | None = None) → bool`

Should `func_name`'s output be cached on this call?

True only when the function is configured `cache: true` and, if `tags: patterns` are present, `tag` `re.fullmatch`es one. Mirrors :meth:`chia.base.bypass.Bypass.is_bypassed`. The config is read from the actor (the source of truth) so it never goes stale across a cache restart; only tagged dispatches reach this path (see ``_maybe_wrap_cache).`

20.3 Chia Function

class `chia.base.ChiaFunction.ChiaWrapped(*args, **kwargs)`

Bases: `Protocol[P, R]`

Protocol describing the object returned by `@ChiaFunction()`.

Includes `__get__` overloads so that mypy treats instances of this protocol as descriptors. When accessed on a class (`obj is None`), the full `ParamSpec` is preserved. When accessed on an *instance* (bound-method position), the return type falls back to `ChiaWrapped[... , R]` because Python's type system cannot express "P minus the first argument". This avoids false positives on method calls while keeping full typing for standalone decorated functions.

class `chia.base.ChiaFunction.ChiaFunction(**kwargs)`

Bases: `object`

Decorator that creates both a local and `ray.remote` version of a function.

Usage:

```
@ChiaFunction(resources={"db1": 1.0})
def my_function(x, y):
    return x + y

# Local call:
my_function(1, 2)

# Remote call (returns ObjectRef):
ref = ChiaCallRemote(my_function, 1, 2)
result = get(ref)

# Or attribute-based:
ref = my_function.chia_remote(1, 2)
```

For bound methods on a class, the decorator uses a trampoline to serialize `self` and dispatch via `ray.remote`:

```
class MyTool:
    @ChiaFunction()
    def do_work(self, x):
        return x * 2
```

(continues on next page)

(continued from previous page)

```
tool = MyTool()
ref = tool.do_work.chia_remote(tool, x=5)
```

Accept any keyword arguments supported by `ray.remote().options()`.

Common options include `resources`, `num_cpus`, `num_gpus`, `num_returns`, `max_retries`, `retry_exceptions`, `memory`, `scheduling_strategy`, `max_calls`, `runtime_env`, `name`, `namespace`, `lifetime`, etc. See Ray documentation for the full list.

`chia.base.ChiaFunction.ChiaCallRemote`(*func*: ~`chia.base.ChiaFunction.ChiaWrapped`[~*P*, ~`chia.base.ChiaFunction.R`], **args*: ~`typing.~P`, ***kwargs*: ~`typing.~P`) → `ObjectRef`[*R*]

Call the `ray.remote` version of a `@ChiaFunction`-decorated function.

Returns a `ray.ObjectRef`[*R*] where *R* is the wrapped function's return type, so `get(...)` can infer the final value type.

class `chia.base.ChiaFunction.ObjectRefCallback`(*ref*, *callback*: `Callable`[`[Any]`, `Any`])

Bases: `object`

An `ObjectRef` paired with a callback that `get()` runs on resolution.

Lets a producer attach a post-resolution hook to the ref it hands back, so callers can stay async and simply `get()` it — the callback runs at fetch time and its return becomes `get`'s result — without passing `callback=` themselves. `ClaudeCodeLLM.prompt` returns one of these (carrying `_sync_transcript`) when `resume_session=True`.

`.ref` exposes the underlying `ObjectRef` for `ray.wait` / `chia_wait()`.

class `chia.base.ChiaFunction.ChiaActorHandle`(*actor*)

Bases: `object`

Thin proxy over a Ray actor handle that adds `chia_remote` to each method.

Built by `chia_actor()`. Attribute access returns a `_ChiaActorMethod`, so `handle.method.chia_remote(...)` dispatches the call and `get()` resolves it — matching the `@ChiaFunction` surface. `remote` still works, so existing `handle.method.remote(...)` call sites are unaffected.

Note: unlike a real `@ChiaFunction` dispatch, `chia_remote` here is a plain alias for Ray's `remote` — no profiling, bypass, or cache wrapping is applied; the call goes straight to the actor. Use `actor` to recover the raw Ray handle (e.g. for `ray.kill` or identity checks).

property actor

The underlying raw Ray actor handle.

`chia.base.ChiaFunction.chia_actor`(*handle*)

Wrap a Ray actor *handle* so its methods accept `chia_remote` and resolve via `get()`.

Lets actor calls share the dispatch surface used by `@ChiaFunction`:

```
store = chia_actor(some_actor)
n = get(store.size_bytes.chia_remote())
get(store.write.chia_remote(key, value))
```

`chia_remote` here is a plain alias for Ray's `remote` — it does NOT run the profiling / bypass / cache machinery that `ChiaWrapped.chia_remote` does; the call goes straight to the actor. `remote` still works, so pre-existing call sites are unchanged. Recover the raw handle via `handle.actor` (e.g. for `ray.kill`). Idempotent: wrapping an already wrapped handle returns it unchanged.

```
chia.base.ChiaFunction.get(ref: ObjectRefCallback, * (Keyword-only parameters separator (PEP 3102)),
    timeout: float | None = None, callback: Callable[[Any], Any] | None = None,
    _use_object_store: bool = False) → Any
```

```
chia.base.ChiaFunction.get(ref: Sequence[ObjectRef[R]], *, timeout: float | None = None, callback:
    Callable[[Any], Any] | None = None, _use_object_store: bool = False) →
    List[R]
```

```
chia.base.ChiaFunction.get(ref: Sequence[ObjectRef[Any]], *, timeout: float | None = None, callback:
    Callable[[Any], Any] | None = None, _use_object_store: bool = False) →
    List[Any]
```

```
chia.base.ChiaFunction.get(ref: ObjectRef[R], *, timeout: float | None = None, callback: Callable[[Any],
    Any] | None = None, _use_object_store: bool = False) → R
```

```
chia.base.ChiaFunction.get(ref: Sequence[CompiledDAGRef], *, timeout: float | None = None, callback:
    Callable[[Any], Any] | None = None, _use_object_store: bool = False) →
    List[Any]
```

```
chia.base.ChiaFunction.get(ref: CompiledDAGRef, *, timeout: float | None = None, callback:
    Callable[[Any], Any] | None = None, _use_object_store: bool = False) → Any
```

Wrapper around `ray.get()` for use in Chia loops.

Delegates to `ray.get()` and post-processes the result through the Chia profiler. Overloads mirror `ray.get()` so a single `ObjectRef[R]` returns `R` and a sequence of them returns `List[R]`.

`callback`: when given, it is invoked with the resolved value (after the profiler unwrap) and its return value is what `get` returns. This keeps dispatch async — the ref is produced non-blocking by `chia_remote` — while running a post-resolution side effect/transform only when the value is fetched (e.g. `get(ref, callback=fn)`).

A `ObjectRefCallback` carries its own callback, so `get(orc)` runs it without an explicit `callback=`. If both are present they compose: the ref's own callback first, then the explicit callback.

```
chia.base.ChiaFunction.chia_cancel(ref, force=False)
```

Cancel a running ChiaFunction task, killing its subprocesses first.

Looks up any subprocess PIDs spawned by the task, kills them on the correct remote nodes (using process group kill for `start_new_session` subprocesses), then calls `ray.cancel()`.

20.4 Chia Wait

`ray.wait` wrapper with stuck-PENDING_NODE_ASSIGNMENT detection + retry.

Ray's owner-side `NormalTaskSubmitter` sends `RequestWorkerLease` RPCs without a client-side timeout. If the chosen raylet is wedged but TCP-reachable (silent), the lease slot stays full forever and downstream tasks of the same scheduling key get throttled at the owner. `chia_wait` detects the pattern (PENDING_NODE_ASSIGNMENT older than `pending_timeout` while the cluster has free resources) and optionally cancels + resubmits.

Use a `TrackedRef` to pair an `ObjectRef` with the closure that can re-dispatch it. `chia_wait` mirrors `ray.wait`'s (`ready`, `pending`) return shape but operates on `TrackedRefs`.

```
class chia.base.chia_wait.TrackedRef(ref: ObjectRef, submit_fn: Callable[[], ObjectRef] | None = None,
    label: str = "", submitted_at: float = 0.0, retries: int = 0)
```

Bases: `object`

`ObjectRef` + resubmit closure + submission timestamp.

`submit_fn` must accept no arguments and produce a fresh `ObjectRef` equivalent to the original submission (same function, args, options). Capture call-site state via `functools.partial` or a closure.

```
chia.base.chia_wait.chia_wait(tracked: list[TrackedRef], *, num_returns: int = 1, timeout: float | None =
    None, pending_timeout: float | None = None, retry: bool = False,
    max_retries: int = 1, require_demand_absent: bool = False,
    min_free_fraction: float = 0.5, cancel_on_stuck: bool = False, print_logs:
    bool = False) → tuple[list[TrackedRef], list[TrackedRef]]
```

Drop-in replacement for `ray.wait()` that handles stuck-PENDING tasks.

Returns (ready, pending) lists of `TrackedRef` after at most `num_returns` complete or `timeout` seconds elapse — same semantics as `ray.wait`. Additionally, when `pending_timeout` is set, any pending `TrackedRef` whose submission age exceeds `pending_timeout` is classified as “stuck” iff:

- (a) Ray state API reports `state == PENDING_NODE_ASSIGNMENT`, AND
- (b) `ray.available_resources()` covers the task’s required resources (i.e. it could schedule somewhere if Ray were healthy), AND
- (c) for every required resource `r`, `available[r] / cluster_total[r] >= min_free_fraction` (cluster-side starvation check — distinguishes a wedged raylet, where the resource is sitting unused, from a busy cluster where tasks are merely waiting on the owner-side per-class budget), AND
- (d) (when `require_demand_absent=True`) the cluster has no pending demand reported for this task’s resource fingerprint.

For each stuck `TrackedRef`:

- If `retry=True`, `tr.submit_fn` is set, and `tr.retries < max_retries`: `ray.cancel(tr.ref, force=True)` (best effort), then call `tr.submit_fn()` for a fresh ref. Mutate `tr` in place (`ref/submitted_at/retries++`). The old ref is orphaned — its rate-limit slot may stay burnt until the wedged raylet’s TCP eventually resets.
- Else: log a warning and leave `tr` in pending unchanged.

The returned pending list contains the same `TrackedRef` objects passed in (possibly with mutated `ref/submitted_at` after retry); callers can keep parallel bookkeeping keyed on identity.

20.5 Chia KV Store

Detached, head-node-pinned Ray actor that stores arbitrary key/value pairs for the cluster to read and write live.

Use this whenever a pipeline needs a small, operator-adjustable piece of shared state — e.g. concurrency knobs, feature flags, or rate limits — without rebuilding the detached-actor scaffolding each time.

Because the actor is `lifetime="detached"`, its values persist across pipeline restarts. Any Ray client on the cluster can look it up by name and namespace:

```
from chia.base.chia_kv_store import get_or_create_kv_store, query_kv

handle = get_or_create_kv_store(
    name="my_pipeline_knobs", namespace="my_pipeline",
    num_parallel=2, stage="warmup",
)
n = query_kv(handle, "num_parallel", default=2)
```

An operator can adjust values live from any machine with Ray access:

```
python -m chia.base.chia_kv_store \
    --name my_pipeline_knobs --namespace my_pipeline --set num_parallel 4
```

(continues on next page)

(continued from previous page)

```
python -m chia.base.chia_kv_store \
    --name my_pipeline_knobs --namespace my_pipeline --get num_parallel
```

`chia.base.chia_kv_store.get_or_create_kv_store(name: str, namespace: str, **defaults: Any) → ActorHandle`

Attach to the detached actor if one already exists, otherwise create it pinned to the current Ray node.

The actor is scheduled with `NodeAffinitySchedulingStrategy` bound to whatever node id `ray.get_runtime_context().get_node_id()` returns at call time. When invoked from a pipeline driver on the head node, that pins the actor to the head.

Parameters

- **name** – Ray actor name.
- **namespace** – Ray actor namespace.
- ****defaults** – Initial values used only when creating a fresh actor. Ignored when attaching to an existing actor.

Returns

A Ray ActorHandle to the ChiaKVStore.

`chia.base.chia_kv_store.query_kv(handle, key: str, default: Any = None, timeout: float = 60.0) → Any`
Read key from the actor with a hard timeout.

Intended for hot loops that should never wedge on a broken actor: any exception (actor dead, timeout, serialization error) or a None handle falls back to `default`.

Parameters

- **handle** – ActorHandle returned by `get_or_create_kv_store()`. May be None — treated the same as an unreachable actor.
- **key** – Key to read.
- **default** – Value returned on any error, timeout, or missing key.
- **timeout** – Hard timeout in seconds for the underlying `ray.get`.

Returns

The stored value, or `default` on any failure.

20.6 Colocated

`chia.base.colocated` — placement-group plumbing for path-based node classes.

Any subsystem whose artifacts live on a worker’s filesystem (a `gem5` checkout, a `hammer obj_dir`, etc) can pin a family of `@ChiaFunction` members to one placement-group bundle and guarantee they land on the same worker.

class `chia.base.colocated.PinnedChiaFn(fn, scheduling_opts: dict)`

Bases: `object`

Exposes a `@ChiaFunction` with `.chia_remote` pre-pinned to a placement-group bundle. Resource requirements are carried over unchanged by `.options()`; with no scheduling opts it delegates to the raw function so the caller’s own placement applies.

(Lifted from `chia.simulators.gem5._PinnedChiaFn`; `gem5` can migrate here.)

options(***overrides*)

Layer extra Ray options on top of the node's pinning.

```
class chia.base.colocated.ColocatedNode(placement_group=None, require_colocated: bool = True, *,
    bundle_index: int = 0, reserve_bundle: dict | None = None,
    pg_strategy: str = 'STRICT_PACK', wait_for_pg: bool = True,
    pg_ready_timeout_s: float | None = None)
```

Bases: `object`

Base for node classes whose member ChiaFunctions must share one worker.

Subclasses declare:

- **_MEMBER_FNS**: names of class-level `@ChiaFunction` members that `__init__` re-binds into per-instance `PinnedChiaFn` wrappers, so `node.<fn>.chia_remote(...)` lands on this node's bundle while `Subclass.<fn>.chia_remote(...)` (the class attribute) stays unpinned.
- **_DEFAULT_BUNDLE**: resource shape of a self-reserved bundle.

Pinning never alters a member's resource demands — the placement group only changes where they are satisfied from (the bundle's reservation instead of the node's free pool). A bundle that cannot satisfy every member is therefore allowed: construction logs a warning naming the members that don't fit (derived from each member's own `@ChiaFunction` options), and dispatching one of them later raises `ValueError` at submission.

Placement is decided once at construction:

- **placement_group** given -> pin members to **bundle_index** of it (the node will NOT release a PG it did not create).
- **none** + **require_colocated=True** -> reserve a 1-bundle **_DEFAULT_BUNDLE** PG (owned + released by `close()`).
- **none** + **require_colocated=False** -> no pinning; the caller schedules each `.chia_remote / .options(...)` call itself.

Usable as a context manager so a self-reserved PG is released on exit.

Set up placement and bind the member functions.

Parameters

- **placement_group** – an existing Ray `PlacementGroup` to schedule onto. If given, **require_colocated** is moot (placement is already fixed) and this node will not remove the PG on close.
- **require_colocated** – when no PG is given, reserve one so all members co-locate. When `False`, leave placement to the caller.
- **bundle_index** – which bundle of the (given or reserved) PG to pin to.
- **reserve_bundle** – resource shape of a self-reserved bundle (default **_DEFAULT_BUNDLE**). A bundle too small for some members is allowed — those members just can't dispatch through this node (a construction-time warning lists them).
- **pg_strategy** – placement strategy for a self-reserved PG.
- **wait_for_pg** – block on `pg.ready()` for a self-reserved PG so the node is usable immediately.
- **pg_ready_timeout_s** – optional timeout for that wait.

property **placement_group**

The PG members are pinned to (`None` when the caller handles placement).

property owns_placement_group: bool

True iff this node reserved its PG and will release it on close().

property task_options: dict

Scheduling opts to co-locate an actor (e.g. a ChiaTool) with this node's bundle. Empty when the node has no placement group (`require_colocated=False`) — there is no shared placement to inherit, so an actor given these opts would not be pinned.

close() → None

Release the PG iff this node reserved it. Idempotent.

20.7 Dispatch Proxy

Head-relayed task dispatch for workers on reverse-tunneled nodes.

Ray's ownership model makes the submitting worker the *owner* of a task: the owner negotiates worker leases directly with the target raylet and serves the result object's data. On a reverse-tunneled worker that breaks for any task whose demands place it on a LAN node — chia's tunnels are hub-and-spoke (tunneled worker <-> head only; see `cluster/tunnel.py`), LAN raylet/worker ports are random and may be firewalled from tunneled machine, so the owner's lease RPC gets “no route to host” and the task sits in `PENDING_NODE_ASSIGNMENT` forever even when capacity is free.

The fix: when `chia_remote` is called on a worker that cannot directly reach the task's target node — a tunneled worker dispatching to a LAN node, OR a LAN worker dispatching to a tunneled node (`local -> remote`) — the dispatch is relayed through a `DispatchProxy` actor pinned to the head node. The proxy — not the originating worker — owns the inner task, so every RPC leg rides a link that already exists. The head is the only node with a bidirectional path to every spoke, so it is the universal relay. `should_proxy` decides this per-dispatch and fail-safe: it relays unless it can *prove* every node the task could land on is directly reachable (see the reachability analysis below):

```
tunneled worker -> proxy actor call ..... pinned head worker ports (reverse tunnel +  
↔DNAT,  
                                the same path ProfileCollectorActor uses)  
proxy <- args from tunneled worker ..... head-side -L object-manager forward  
proxy -> LAN raylet lease ... plain LAN traffic  
result -> tunneled worker ..... the proxy awaits the value and returns it, so  
                                the result object lives on the head; tunneled worker  
                                fetches it via the DNAT'd head object manager
```

Costs and limits:

- The head relays the data — args and results each cross the network twice.
- `num_returns > 1` is not supported (falls back to direct dispatch, with a warning, which may hang if the task lands on a LAN node).
- `ObjectRefs` nested inside proxied args are re-splatted as top-level args of the inner task; if such a ref is *owned by a tunneled worker*, the LAN executor cannot reach that owner and the inner task will hang. Pass values, not refs, into nested dispatches from tunneled worker-resident tasks.
- The proxy inherits the creating job's `runtime_env`, so it is per-job (`chia_dispatch_proxy_<job_id>`) and detached; it idles at `num_cpus=0` until the cluster is torn down.

`chia.base.dispatch_proxy.should_proxy(opts: dict | None = None) → bool`

True when this dispatch must be relayed through the head `DispatchProxy`.

Relays iff the task could land on a node the current worker cannot reach directly (see `_directly_reachable()`). This both enables LAN-origin dispatches to tunneled nodes (`local -> remote`) and lets provably-direct dispatches skip the single-actor relay. The head never relays.

`chia.base.dispatch_proxy.get_dispatch_proxy()`

Get-or-create this job's head-pinned DispatchProxy actor.

20.8 PID Registry

PID tracking and cancellation for ChiaFunction remote tasks.

Tracks subprocess PIDs spawned during @ChiaFunction remote execution and provides `chia_cancel()` to kill those process trees before cancelling the Ray task.

The Popen hook is scoped per-task via `threading.local` so concurrent Ray tasks on the same worker don't interfere with each other.

class `chia.base.pid_registry.PidRegistryActor`

Bases: `object`

Ray actor that maps task IDs to subprocess PIDs.

Created lazily on first registration, looked up by workers and the driver via `_get_registry()`.

kill_all(*grace: float = 25.0*) → `int`

Kill all tracked subprocess PIDs across all nodes.

Dispatches remote kill tasks and waits for completion. Each kill sends SIGTERM, polls for the process to exit, and only escalates to SIGKILL if it is still alive after `grace` seconds. Returns the number of PIDs targeted.

`chia.base.pid_registry.chia_cancel(ref: ObjectRef, force: bool = False)` → `None`

Cancel a running ChiaFunction task, killing its subprocesses first.

Looks up any subprocess PIDs spawned by the task, kills them on the correct remote nodes (using process group kill for `start_new_session` subprocesses), then calls `ray.cancel()`.

Parameters

- **ref** – The `ObjectRef` returned by `chia_remote()`.
- **force** – Passed through to `ray.cancel()`. If `True`, the Ray worker is killed; if `False` (default), a `TaskCancelledError` is raised cooperatively.

20.9 LLM Call

class `chia.base.llm_call.QueryResult`(*result: str, returncode: int, stderr: str, stream_result: str, success: bool = False*)

Bases: `object`

Structured result from prompting an LLM or agent.

Parameters

- **result** (*str*) – The final response from the LLM or agent
- **returncode** (*int*) – The returncode from running the prompt
- **stderr** (*str*) – The stderr output from running the prompt (clis only)
- **stream_result** (*str*) – The full transcript of all turns of the LLM or agent
- **success** (*bool*) – Whether the prompt completed successfully

```
class chia.base.llm_call.LLMCallBase(system_message: str, dangerously_skip_permissions=<object
                                     object>, config=<object object>)
```

Bases: ABC

Polymorphic base container for generic LLM and agent configuration traits and behavior. Easy to switch between different backing providers, servers, and CLIs

```
abstractmethod prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send a prompt to this LLM

Parameters

- **user_message** (*str*) – Message used to prompt the LLM
- **tools** (*Optional[List[ChiaTool]]*) – Tools available to the LLM during the call

20.10 MCP tool servers

20.10.1 Chia Tool

```
class chia.base.tools.ChiaTool.ToolInfo(name: 'str', port: 'int', node_id: 'str')
```

Bases: object

```
class chia.base.tools.ChiaTool.ChiaTool(name: str, task_options: Dict | None = None, logging_level=10)
```

Bases: object

Base class for MCP tool servers deployed onto Ray workers.

Subclasses define a `setup()` method, which calls:

```
``self.mcp.add_tool(self.method, name=...)`` (one or more times)
```

to register functions as tools with instances of this `ChiaTool`.

Subclass can shut down the tool server with:

```
``self.stop()``  
- Tells the actor to shut down uvicorn, then kills the actor.  
- Because start and stop run in the same actor process, the  
  uvicorn server reference is always reachable.
```

The resulting MCP endpoint is at:

```
http://{self.hostname}:{self.port}/{self.name}/mcp
```

Example subclass:

```
class BashTool(ChiaTool):  
    def setup(self):  
        self.mcp.add_tool(self.run_command, name=f"{self.name}_run_command")  
  
    def run_command(self, command: str) -> str:  
        ...
```

Alternatively, instead of a `setup` method a subclass can define an `__init__` method which must do the following:

```
def __init__(self, name, task_options):
    super().__init__(name, task_options=task_options)
    # Registers fns with self.mcp.add_tool
    super().__post_init__()
```

Initializes ChiaTool with a name and optional resource requirements.

setup(*args, **kwargs)

Hook for the auto-constructed style — override to register tools and set instance state, *instead of* writing `__init__`.

A subclass that defines `setup` and no `__init__` is given an `__init__` automatically (see `__init_subclass__()`) that runs `ChiaTool.__init__` before and `__post_init__` after `setup`, so the contract with the subclass can't be written incorrectly. Inside `setup` the base `__init__` has already run, so `self.name` / `self.mcp` are available:

```
class BashTool(ChiaTool):
    def setup(self, work_dir="/"):
        self.work_dir = work_dir
        self.mcp.add_tool(self.run_command, name=f"{self.name}_run_command")
```

Positional/keyword args from the constructor (other than `name`, `task_options`, and `logging_level`, which the base consumes) are forwarded here. Multi-level subclasses override `setup` and call `super().setup(...)` to chain.

stop()

Stop the tool's MCP server and clean up resources.

`chia.base.tools.ChiaTool.resolve_tool_url(url: str) → str`

Rewrite a tool URL so it is routable from the current node.

On tunnelled EC2 workers `CHIA_TOOL_ADVERTISE_HOST` and `CHIA_TOOL_RELAY_HOST` are set. Tool URLs advertise the head's real IP (`CHIA_TOOL_ADVERTISE_HOST`) which is only directly reachable from the local network. EC2 workers must connect via a reverse-tunnel relay instead, so this function replaces the host portion of the URL with the relay loopback (`CHIA_TOOL_RELAY_HOST`).

On non-tunnelled nodes (or when the env vars are absent) the URL is returned unchanged.

`chia.base.tools.ChiaTool.start_router(tool: ChiaTool, ip_address: str, base_port: int = 8000, max_tries: int = 100) → int`

Start MCP tool servers using a local uvicorn instance.

Uses a plain uvicorn server (background thread) instead of Ray Serve so that multiple nodes can each host their own independent tool servers without route-prefix conflicts.

Tries ports starting from `base_port`, skipping any that are already in use (e.g. host-networked Docker containers sharing the same port space).

Returns the port that was successfully bound.

`chia.base.tools.ChiaTool.stop_router(tool_name: str) → bool`

Stop a running uvicorn server by tool name.

Looks up the server in the worker-side `_active_servers` registry, signals it to exit, waits for the thread, and releases the port. Returns True if the server was found and stopped.

20.10.2 Chia Tool Template

```
class chia.base.tools.ChiaToolTemplate.ChiaToolTemplate(name, *args, task_options=None,
                                                         logging_level=10, **kwargs)
```

Bases: [ChiaTool](#)

Initializes ChiaTool with a name and optional resource requirements.

setup()

Setup function for the CHIA tool

hello_world_tool(name: str) → str

Example tool method that takes a name as input and returns a greeting string.

Parameters

name (str) – The name to include in the greeting.

goodbye_world_tool(name: str) → str

Example tool method that takes a name as input and returns a goodbye string.

Parameters

name (str) – The name to include in the goodbye message.

20.10.3 Bash Tool

```
class chia.base.tools.BashTool.BashTool(name: str, work_dir: str = '/', timeout_seconds: int = 120,
                                         task_options: Dict | None = None)
```

Bases: [ChiaTool](#)

MCP tool that executes bash commands in its deployed container.

When constructed with `task_options`, the tools are setup on machines using a setup function which is called remotely with those `task_options`.

Initializes ChiaTool with a name and optional resource requirements.

run_command(command: str) → str

Run a bash command and return combined stdout/stderr.

20.10.4 Async Bash Tool

```
class chia.base.tools.AsyncBashTool.AsyncBashTool(name: str, work_dir: str = '/', env: Dict[str, str] |
                                                    None = None, task_options: Dict | None = None)
```

Bases: [AsyncJobTool](#)

MCP bash tool for commands that may run for minutes, built on [AsyncJobTool](#).

Like `chia.base.tools.BashTool.BashTool` it runs shell commands in a working directory, but a single command here may run long enough (a full build, a test suite) that a *synchronous* MCP call loses its response on the streamable-HTTP transport and hangs the agent. So this tool never blocks the transport for long: `<name>_run` starts the command in the background and waits only a short, bounded slice for it to finish; if it is still running it returns `done=False` and the agent polls `<name>_run_status` until `done=True`.

Fast commands therefore complete inside the single `run` call (they finish within the initial wait); only genuinely long commands require a poll. One command runs at a time per tool instance — start a command, then poll it to completion before starting the next (a second `run` while one is in flight returns `started=False`).

Result dict on completion: `{exit_code: int, output: str}` (combined stdout+stderr, tail-truncated). Mirrors `BashTool`'s combined-stream behaviour. `work_dir` and `env` are plain attributes and pickle normally; the job/threading primitives are dropped + recreated by `AsyncJobTool`.

Initializes ChiaTool with a name and optional resource requirements.

run(*command: str, wait_seconds: int = 60*) → dict

Run a bash command in the BACKGROUND, waiting up to *wait_seconds* (capped at 120) for it to finish.

If it finishes in time, returns *done=true* with {*exit_code*, *output*}. If it is still running, returns *done=false* — then poll *<name>_run_status* until *done=true*. One command at a time: if a command is already running, returns *started=false* and you should poll *status* instead of starting another.

run_status(*wait_seconds: int = 60*) → dict

Wait up to *wait_seconds* (capped at 120) for the in-flight command, then report. *done=true* splices in {*exit_code*, *output*}.

20.10.5 Async Job Tool

class chia.base.tools.AsyncJobTool.**AsyncJobTool**(*name: str, task_options: Dict | None = None*)

Bases: [ChiaTool](#)

Base for MCP tools that run a long job in the background, then poll for it.

Why this exists: a single MCP tool call that blocks for minutes holds the streamable-HTTP connection open with no traffic for the whole job. That transport can lose the response for such a long synchronous call, stranding the result and hanging the agent forever (observed on real runs). The fix is to never block the transport for long: a *start* call kicks the work off in a daemon thread and returns immediately, and a *status* call long-polls in short, bounded chunks (*<=120s*) until *done=true*.

Subclasses register their own start/status tool methods that delegate to *_job_start()* and *_job_status()*:

```
class BuildTool(AsyncJobTool):
    def __init__(self, name, task_options=None):
        super().__init__(name, task_options=task_options)
        self.mcp.add_tool(self.build, name=f"{name}_build")
        self.mcp.add_tool(self.build_status, name=f"{name}_build_status")
        super().__post_init__()

    def build(self, targets=None) -> dict:
        return self._job_start(lambda: do_build(targets))

    def build_status(self, wait_seconds: int = 60) -> dict:
        return self._job_status(wait_seconds)
```

One job runs at a time per tool instance; calling *_job_start* while a job is in flight returns {"started": False, "running": True} rather than starting a second. The work callable must return a dict; on *done=true* its keys are spliced into the status result.

The job's threading primitives are not picklable, but ChiaTool serializes the tool object to the server actor (and to the LLM worker), so they are dropped in *__getstate__* and recreated in *__setstate__*.

Initializes ChiaTool with a name and optional resource requirements.

20.10.6 Util

chia.base.tools.util.**make_router_lifespan**(*mcp_instances: List[FastMCP]*)

A lifespan which runs all MCP session managers concurrently.

This is useful when multiple MCP ASGI apps are mounted under one FastAPI ingress app (e.g. /gem5, /chia). Mounting should be done outside the lifespan; this only manages session manager lifetimes.

CHIPYARD NODES

API reference for the Chipyard library nodes in `chia.chipyard`. These pages are generated automatically from the docstrings in the source, so they stay in sync with the code.

21.1 State definitions

class `chia.chipyard.state_def.BuildTarget(*values)`

Bases: `str`, `Enum`

Which Chipyard simulator flavor `ChiselBuildNode` builds. Maps to make targets in `chipyard/sims/verilator`.

VERILATOR

Default fast Verilator sim (make in `sims/verilator`). No waveform capability; binary `simulator-<pkg>.harness-<config>`.

VERILATOR_DEBUG

Waveform/VCD-capable Verilator sim (make `debug`). Binary suffixed `-debug`. Required for `wf_scopes` (build-time spatial trace filter) and runtime wave windows / VCD capture.

FIRESIM_METASIM_VERILATOR

A FireSim metasim (`VFireSim`) built under `sims/firesim/sim`; binary name uses the FireSim quintuplet `<PLATFORM>-firesim-<DESIGN>-<config>-<PLATFORM_CONFIG>`.

class `chia.chipyard.state_def.WaveWindow(pc: int, cyc: int, n: int = 1)`

Bases: `object`

One PC-triggered waveform-capture window.

Waveform collection is captured on the Nth retired commit of `pc` happens and dumps for `cyc` BOOM-core cycles thereafter. Each window fires at most once; overlapping windows merge into a single contiguous dump segment.

class `chia.chipyard.state_def.BuildArtifact(name: str, simulator_binary_content: bytes, simulator_binary_name: str, config: str, config_package: str, target: BuildTarget, success: bool, stdout: str, stderr: str, returncode: int, generated_src_files: list[tuple[str, str]]=<factory>, runtime_libs: list[tuple[str, bytes]]=<factory>, golden_model_digest: str = "")`

Bases: `object`

Result of a `ChiselBuildNode.build()`: a compiled simulator + metadata.

The compiled simulator is shipped by value (the ELF bytes travel inside this object).

name

Free-form label carried from `ChiselBuildNode.name` (default "chipyard") for downstream identification.

Type

str

simulator_binary_content

Raw ELF bytes of the compiled simulator binary. Empty (b"") when the build failed or timed out.

Type

bytes

simulator_binary_name

Filename of the simulator, e.g. "simulator-chipyard.harness-MegaBoomV3Config".

Type

str

config

Chipyard CONFIG the binary was built for (e.g. "MegaBoomV3Config").

Type

str

config_package

Chipyard CONFIG_PACKAGE (Scala package, e.g. "chipyard").

Type

str

target

The *BuildTarget* that produced this binary.

Type

chia.chipyard.state_def.BuildTarget

success

True iff the build's make exited 0 and the binary exists.

Type

bool

stdout

Captured stdout of the build command.

Type

str

stderr

Captured stderr of the build command.

Type

str

returncode

Exit code of the build command; -1 on timeout.

Type

int

generated_src_files

(filename, contents) pairs of generated .v/.sv and .top.mems.conf collateral: populated only when the build was run with collect_generated_src=True.

Type

list[tuple[str, str]]

runtime_libs

(filename, contents) shared libraries staged beside the binary at run time - libriscv.so above all. Populated by a dynamic golden_model build; empty otherwise.

Type

list[tuple[str, bytes]]

golden_model_digest

sha256 of the libriscv the simulator was built against; "" when unknown.

Type

str

```
class chia.chipyard.state_def.RiscvBuildArtifact(binary_name: str, binary_content: bytes, target: str,
                                                success: bool, stdout: str, stderr: str, returncode:
                                                int, dump: str = "")
```

Bases: object

Result of a RiscvBuildNode.build(): a cross-compiled RISC-V ELF.

binary_name

Output filename, e.g. "hello.riscv" or "hello".

Type

str

binary_content

Raw ELF bytes of the produced binary. Empty (b"") when the build failed or timed out.

Type

bytes

target

Which toolchain/output convention was used: "verilator" (baremetal) or "linux" (userspace).

Type

str

success

True iff make exited 0 and a non-empty binary was produced.

Type

bool

stdout

Captured stdout of the build command.

Type

str

stderr

Captured stderr of the build command (includes a timeout note on expiry).

Type
str

returncode

Exit code of the build command; -1 on timeout.

Type
int

dump

objdump -D disassembly of the ELF; "" unless build() was called with include_dump=True.

Type
str

```
class chia.chipyard.state_def.ProgramBuildArtifact(files: dict[str, bytes], success: bool, stdout: str,
                                                    stderr: str, returncode: int)
```

Bases: object

Outputs of a RiscvBuildNode.build_program().

files

Collected outputs, keyed by task-dir-relative path; empty on failure or if nothing matched outputs. Shipped by value, so scope outputs to what downstream nodes need.

Type
dict[str, bytes]

success

True iff the build command exited 0.

Type
bool

stdout

Captured stdout of the build command.

Type
str

stderr

Captured stderr (includes a timeout note on expiry).

Type
str

returncode

Exit code of the build command; -1 on timeout.

Type
int

```
class chia.chipyard.state_def.RiscvObjdumpArtifact(binary_name: str, dump: str, target: str, success:
                                                    bool, stdout: str, stderr: str, returncode: int)
```

Bases: object

Result of a RiscvObjdumpNode.dump(): a RISC-V ELF disassembly.

binary_name

Filename of the input ELF.

Type

str

dump

The disassembly (stdout of the `objdump` invocation); "" on failure.

Type

str

target

Which toolchain prefix was used: "verilator" (riscv64-unknown-elf-) or "linux" (riscv64-unknown-linux-gnu-).

Type

str

success

True iff `objdump` exited 0 and produced non-empty output.

Type

bool

stdout

Captured stdout of the `objdump` invocation (same content as `dump`).

Type

str

stderr

Captured stderr (includes a timeout note on expiry).

Type

str

returncode

Exit code of the `objdump` invocation; -1 on timeout.

Type

int

```
class chia.chipyard.state_def.RunResult(test_binary_name: str, log: str, out: str, returncode: int,
                                         success: bool, vcd_s3_path: str = "", vcd_size_bytes: int = 0,
                                         vcd_path: str = "", vcd_node_id: str = "", out_s3_path: str = "",
                                         log_s3_path: str = "", wave_windows: list[WaveWindow] =
                                         <factory>)
```

Bases: object

Result of a `VerilatorRunNode.run()`: captured simulator output.

test_binary_name

Filename of the input test ELF that was run.

Type

str

log

Simulator stdout captured to `<stem>.log` (program / HTIF output).

Type

str

out

the .out file, contains `spike-dasm` disassembly of the simulator's stderr (the human-readable committed-instruction trace).

Type

str

returncode

Exit code of the simulator process.

Type

int

success

True iff `returncode == 0`.

Type

bool

vcd_s3_path

Full s3://... URI of the uploaded VCD; "" when no waveform was captured or no upload occurred (the VCD is too large to return inline).

Type

str

vcd_size_bytes

Size of the uploaded/kept VCD; 0 when neither.

Type

int

vcd_path

Worker-side path of the kept VCD (`run(keep_waveform=True)` moves it out of the task dir so it survives cleanup). Together with `vcd_node_id` this is the claim ticket for [chia.chipyard.verilator_run_node.collect_waveform\(\)](#), the S3-free transfer path. "" when not kept.

Type

str

vcd_node_id

Ray node id (hex) of the worker holding `vcd_path`; collection tasks are pinned to it. "" when not kept.

Type

str

out_s3_path

s3://... URI of the uploaded .out; "" when not uploaded.

Type

str

log_s3_path

s3://... URI of the uploaded .log; "" when not uploaded.

Type

str

wave_windows

Echo of the [WaveWindow](#) list the run was configured with.

Typelist[[chia.chipyard.state_def.WaveWindow](#)]

```
class chia.chipyard.state_def.CosimResult(elf_name: str, match: bool, matched: int, completed: bool,
                                         first_divergence: dict | None, sim_cycles: int | None,
                                         failing_trace_gz: bytes | None, returncode: int = 0, crashed:
                                         bool = False)
```

Bases: object

Result of a CosimNode.run(): the lockstep cospike verdict for one ELF.

elf_name

Filename of the ELF that was co-simulated.

Type

str

match

True iff the DUT agreed with Spike (no divergence aborted the run).

Type

bool

matched

Committed instructions Spike checked before the run stopped.

Type

int

completed

True iff the program reached its end-of-test marker.

Type

bool

first_divergence

{line, spike, dut} describing the first mismatch, or None on a match.

Type

dict | None

sim_cycles

Simulation cycles executed, or None if not reported.

Type

int | None

failing_trace_gz

gzip'd tail window around the abort (DUT + Spike commit logs interleaved); None on a match.

Type

bytes | None

```
class chia.chipyard.state_def.SpikeResult(test_binary_name: str, isa: str, log: str, commit_log: str,
                                         returncode: int, success: bool)
```

Bases: object

Result of running a RISC-V ELF on the Spike ISA simulator.

test_binary_name

Filename of the input ELF that was executed.

Type

str

isa

The --isa=<...> string Spike ran with (e.g. "rv64gc").

Type

str

log

Spike stdout, i.e. the program / HTIF output.

Type

str

commit_log

Committed-instruction trace. Empty (""), unless the run requested it via `log_commits=True`.

Type

str

returncode

Exit code of the Spike process.

Type

int

success

True iff the run succeeded. For self-checking riscv-tests this is exit code 0 reported over HTIF.

Type

bool

```
class chia.chipyard.state_def.SpikeBuildArtifact(success: bool, lib_name: str, lib_content: bytes,  
digest: str, spike_bin: str, stdout: str, stderr: str,  
returncode: int, static_archives: list[tuple[str,  
bytes]]=<factory>, tools: list[tuple[str,  
bytes]]=<factory>, headers_tar: bytes = b",  
source_sha: str = "", source_dirty: bool = False)
```

Bases: object

Result of a `SpikeBuildNode.build()`: Spike as a shipped artifact, a build input to `ChiselBuildNode` and a runtime input to the worker that runs the simulator.

success

True iff make produced the library and it is newer than the sources it was built from.

Type

bool

lib_name

Filename of the shared library, i.e. "libriscv.so".

Type

str

lib_content

Raw bytes of the shared library, stripped unless the build was run with `strip=False`. Empty (b'') on failure.

Type

bytes

static_archives

(filename, contents) pairs of the archives a static link of the golden model needs - `libriscv.a` plus the libraries it does not contain: softfloat above all (libriscv.a carries ~900 undefined softfloat references; the *shared* library only looks self-contained because it absorbed them at its own link time), and `disasm/fdt`. For `ChiselBuildNode(static_golden_model=True)` from a container other than the one that built spike. Empty unless the build was run with `collect_static=True`: these run to ~300MB, and a build that links in the container it built in reads them off disk.

Type

list[tuple[str, bytes]]

tools

(filename, contents) pairs of spike executables, notably `spike-dasm`: the run worker disassembles the DUT trace with it, so a custom instruction stays un-disassembled unless the matching one travels with the model. Empty unless `collect_tools=True`.

Type

list[tuple[str, bytes]]

headers_tar

gzipped tar of `$RISCV/include/riscv`, needed only to stage this model into a *different* container than the one that built it (cospike compiles against these headers). Empty unless `collect_headers=True`.

Type

bytes

source_sha

`git rev-parse HEAD` of the `riscv-isa-sim` checkout, or "" when it is not a git tree.

Type

str

source_dirty

True iff that checkout has uncommitted changes - the normal case while a loop is editing spike, and the reason `source_sha` alone does not identify a model.

Type

bool

digest

sha256 of `lib_content`. This, not the SHA, is the golden model's identity: it is what a *BuildArtifact* records and what makes two runs comparable.

Type

str

spike_bin

Path to the spike build directory's library, in-container.

Type

str

stdout

Captured stdout of the build command.

Type

str

stderr

Captured stderr of the build command.

Type

str

returncode

Exit code of the build command; -1 on timeout.

Type

int

class chia.chipyard.state_def.**TortureMode**(*values)

Bases: str, Enum

How a RISC-V Torture run generates and checks tests.

SINGLE

Generate one test, run it, and diff signatures (make rgentest).

OVERNIGHT

Generate-and-test loop until N failures or T minutes elapse (make rnight).

REPLAY

Take a pre-supplied test.S, run it, and diff signatures (make rtest).

class chia.chipyard.state_def.**TortureTestRun**(name: str, success: bool, test_s: str, test_dump: str, spike_sig: str, rtlsim_sig: str, pseg_test_s: str | None = None)

Bases: object

Artifacts produced for one generated Torture test, pass or fail.

A test passes when the Spike reference signature matches the DUT (RTL simulation) signature.

name

Basename of the test, e.g. "test" or "test_1714571234".

Type

str

success

True iff the Spike signature matched the DUT signature.

Type

bool

test_s

The generated assembly source (.S).

Type

str

test_dump

objdump output, if Torture produced one (else "").

Type

str

spike_sig

The Spike reference signature.

Type

str

rtlsim_sig

The DUT (RTL simulation) signature.

Type

str

pseg_test_s

The narrowed program segment from `testrun`'s seek mode. Populated for failures only; `None` otherwise.

Type

str | None

```
class chia.chipyard.state_def.TortureResult(name: str, config: str, config_package: str, mode:
    TortureMode, success: bool, num_tests: int, num_failures:
    int, tests: list[TortureTestRun], stdout: str, stderr: str,
    returncode: int, build_artifact: BuildArtifact | None =
    None)
```

Bases: object

Result of a RISC-V Torture run against a built simulator.

name

Free-form label carried from the run for downstream identification.

Type

str

config

Chipyard CONFIG the simulator was built for (e.g. "MegaBoomV3Config").

Type

str

config_package

Chipyard CONFIG_PACKAGE (Scala package, e.g. "chipyard").

Type

str

mode

The `TortureMode` the run used.

Type

`chia.chipyard.state_def.TortureMode`

success

True iff all signatures matched and the run completed.

Type

bool

num_tests

Number of tests run — 1 for SINGLE/REPLAY, N for OVERNIGHT.

Type

int

num_failures

Number of tests whose signatures mismatched.

Type

int

tests

Per-test artifacts. Includes passing and failing tests for SINGLE/REPLAY; failing tests only for OVERNIGHT.

Typelist[*chia.chipyard.state_def.TortureTestRun*]**stdout**

Captured stdout of the Torture run.

Type

str

stderr

Captured stderr of the Torture run.

Type

str

returncode

Exit code of the Torture run.

Type

int

build_artifactThe *BuildArtifact* that produced the simulator. Populated by `torture_from_config`; None otherwise.**Type***chia.chipyard.state_def.BuildArtifact* | None

```
class chia.chipyard.state_def.FireMarshalArtifact(archive: bytes, img_name: str, bin_name: str,
                                                  json_name: str, success: bool, stdout: str, stderr:
                                                  str, returncode: int)
```

Bases: object

Result of a `FireMarshalNode.build_image()`: a bootable workload packaged as one portable archive.

The rootfs, boot binary, and FireSim workload descriptor are packed into a single sparse-aware gzip tar (`archive`) and carried by value, so they move between nodes over Ray with no shared filesystem and no S3. For very large rootfs images, prefer chia's worker-pinned chunked streaming (as `verilator_run_node` uses for VCDs) over inlining here.

archive`tar --sparse -czf of the .img + -bin + .json; b"" on failure. Unpack with tar -xzf.`

Type
bytes

img_name

rootfs member filename (<name>.img).

Type
str

bin_name

boot-binary member filename (<name>-bin).

Type
str

json_name

FireSim workload-descriptor member filename (<name>.json).

Type
str

success

True iff the archive was produced.

Type
bool

stdout

Captured stdout of the compose/package.

Type
str

stderr

Captured stderr (includes a timeout note on expiry).

Type
str

returncode

Exit code; -1 on timeout.

Type
int

```
class chia.chipyard.state_def.FireMarshalBaseArtifact(archive: bytes, img_name: str, bin_name: str,
                                                    success: bool, stdout: str, stderr: str,
                                                    returncode: int)
```

Bases: object

Result of a `FireMarshalNode.build_base()`: FireMarshal's br-base (rootfs + boot binary) as a sparse-tar archive, to feed `FireMarshalNode.build_image()`'s `base=`.

Built once (slow); dispatch early and/or cache so it runs at most once.

archive

tar --sparse -czf of br-base.img + br-base-bin; b"" on failure.

Type
bytes

img_name

rootfs member filename (`br-base.img`).

Type

str

bin_name

boot-binary member filename (`br-base-bin`).

Type

str

success

True iff the base was built and packaged.

Type

bool

stdout

Captured stdout of the build/package.

Type

str

stderr

Captured stderr (includes a timeout note on expiry).

Type

str

returncode

Exit code; -1 on timeout.

Type

int

21.2 Chisel build

```
class chia.chipyard.chisel_build_node.ChiselBuildNode(chipyard_path: str, config: str,
                                                         config_package: str = 'chipyard', target:
                                                         BuildTarget = BuildTarget.VERILATOR,
                                                         make_jobs: int = 32, timeout_seconds: int =
                                                         600, extra_make_args: dict = {},
                                                         collect_generated_src: bool = False,
                                                         clean_sim: bool = False, clean: bool = True,
                                                         golden_model: SpikeBuildArtifact | None =
                                                         None, static_golden_model: bool = False,
                                                         bundle_runtime_libs: bool | None = None,
                                                         wf_scopes: list[str] | tuple[str, ...] = (),
                                                         clean_wf_stamp: bool = True, logging_level:
                                                         int = 10, name: str = 'chipyard')
```

Bases: object

Elaborates a Chipyard config into a Verilator simulator binary.

Wraps the Chipyard `sims/verilator` (and FireSim `sims/firesim/sim`) Make flow: it shells out to make inside the appropriate `sims` directory to run the Chisel generator and Verilator, then reads the resulting simulator

ELF back into a `BuildArtifact` so it can be shipped to a `VerilatorRunNode` for execution. By default runs on cluster nodes tagged with `chipyard=1` resource.

Configure a single Chipyard Verilator build.

Parameters

- **chipyard_path** – Absolute path to the Chipyard checkout (the `chia_artifact` branch) on the build node. Set to `/home/ray/chipyard` when using the CHIA chipyard container. The Make flow runs inside `<chipyard_path>/sims/verilator` (or, for FireSim metasims, `<chipyard_path>/sims/firesim/sim`).
- **config** – Chipyard CONFIG: the Chisel Config class name that parametrizes the SoC to elaborate (e.g. `"RocketConfig"`, `"MegaBoomV3Config"`). Passed to Make as `CONFIG=<config>` and becomes the suffix of the produced binary's name.
- **config_package** – Chipyard CONFIG_PACKAGE: the Scala package the Config class lives in (passed as `CONFIG_PACKAGE=`). Defaults to `"chipyard"`; the generated binary is named `simulator-<config_package>.harness-<config>`.
- **target** – Which simulator to build (see `BuildTarget`): `VERILATOR` (fast, no waveforms), `VERILATOR_DEBUG` (make debug; VCD/waveform-capable, required for `wf_scopes` and runtime wave windows), or `FIRESIM_METASIM_VERILATOR` (a FireSim metasim `VFireSim` binary built under `sims/firesim/sim`).
- **make_jobs** – Value passed to `make -j` controlling build parallelism (number of concurrent compile jobs).
- **timeout_seconds** – Wall-clock limit for the main make build. On expiry the build is killed and a failed `BuildArtifact` (`returncode=-1`) is returned. The preceding clean steps have their own fixed 600s timeouts.
- **extra_make_args** – Additional KEY=value Make variables to append to the build command (e.g. `{"VERILATOR_THREADS": 4}`). For `FIRESIM_METASIM_VERILATOR` these may include `DESIGN / PLATFORM / PLATFORM_CONFIG` and are merged over the FireSim defaults. Note: passing `WF_SCOPES` here conflicts with `wf_scopes` (below).
- **collect_generated_src** – If True, after a successful build read every generated `.v/.sv` file (skipping `TestDriver.v` and any DPI-C files) plus the `.top.mems.conf` from the `gen-collateral` directory into `BuildArtifact.generated_src_files`.
- **clean_sim** – If True, run `make clean-sim` before building. This removes the simulator binaries but leaves the cached `chipyard.jar` and generated FIRRTL/Verilog (`gen_dir`) in place: so it can relink against stale Verilog.
- **clean** – If True (default), run `make clean` before building. This also clears the Chisel-generator cache (`chipyard.jar`) and `gen_dir`, guaranteeing the build reflects the current Scala sources. Slower but correct.
- **golden_model** – A `SpikeBuildArtifact` to link against; staged into `$RISCV` before make, recorded as `golden_model_digest` and, when the link is dynamic, bundled into `runtime_libs`. None links whatever the image installed.
- **static_golden_model** – Compile `libriscv` into the simulator for one hermetic binary. Pass `clean_sim=True` with it, or make will not relink for a new library.
- **bundle_runtime_libs** – Carry non-system shared libraries in `BuildArtifact.runtime_libs` for the run node to stage. Defaults to True only for a dynamic `golden_model` build.

- **wf_scopes** – chia_artifact-specific *spatial* waveform filter: a list of module-hierarchy scope paths (e.g. "TestHarness.chiptop0.system...") restricting which modules Verilator traces into the VCD. Joined with spaces into the WF_SCOPES Make variable. At most `_MAX_WF_SCOPES` (8) paths, and only valid with `target=VERILATOR_DEBUG` (the Makefile wires the filter only into the debug model).
- **clean_wf_stamp** – If True and `wf_scopes` is set, delete `sims/verilator/.wf_scopes.stamp` before building so Make re-applies the scope filter even when it would otherwise consider nothing changed (mirrors `run_wave.sh`).
- **logging_level** – Python logging level for this node's logger.
- **name** – Label copied into the resulting `BuildArtifact.name`; purely for downstream identification/traceability.

build() → *BuildArtifact*

Run the Chipyard Make flow and return the compiled simulator.

Takes no arguments: every input comes from the attributes set in `__init__`. Selects the `sims` directory, binary name and make command line from `self.target/self.config/self.config_package` (plus `self.extra_make_args`), optionally clears the `WF_SCOPES` stamp and runs `make clean/clean-sim`, then invokes the build under `self.timeout_seconds`.

Returns

On success, carries the simulator ELF bytes plus `config/config_package/target` and (if `collect_generated_src`) the generated source files. On a build failure or timeout, `success=False` with empty binary content and the captured `stdout/stderr` and `returncode` (-1 on timeout).

Return type

BuildArtifact

21.3 Spike build

Builds Spike's libriscv into a shippable artifact.

Spike is the golden model a cospike simulator checks each instruction against. This node builds the tree as it finds it and returns the library by value, the same contract *ChiselBuildNode* has for Chisel. The output feeds `ChiselBuildNode(golden_model=...)`, which links against it and bundles it into the simulator's artifact, so run workers need no spike installed.

Dynamic linking is the default; it lets spike and the RTL iterate independently. For a hermetic binary use `ChiselBuildNode(static_golden_model=True, clean_sim=True)` - `clean_sim` because make does not relink for a new library on its own.

```
class chia.chipyard.spike_build_node.SpikeBuildNode(chipyard_path: str, riscv_path: str | None =
    None, spike_rel: str =
    'toolchains/riscv-tools/riscv-isa-sim', make_jobs:
    int = 16, strip: bool = True, configure_args:
    list[str] | None = None, build_static: bool =
    False, collect_static: bool = False, collect_tools:
    bool = False, collect_headers: bool = False,
    install: bool = True, timeout_seconds: int =
    1800, logging_level: int = 10)
```

Bases: `object`

Builds libriscv from the riscv-isa-sim checkout inside a chipyard tree.

Configure one libriscv build.

Parameters

- **chipyard_path** – Chipyard checkout holding the spike sources.
- **riscv_path** – Toolchain prefix (\$RISCV). Defaults to the RISCV env var, then <chipyard_path>/conda-env/riscv-tools.
- **spike_rel** – riscv-isa-sim path relative to chipyard_path.
- **make_jobs** – make -j parallelism.
- **configure_args** – Extra ./configure arguments, e.g. ["--with-isa=rv64gcv"] or ["CXXFLAGS=-O0 -g"]. When given, configure is re-run and the library fully rebuilt. Default None keeps the existing build configuration.
- **strip** – Strip a copy before shipping (~177MB -> ~8MB).
- **build_static** – Also build STATIC_ARCHIVES for ChiselBuildNode(static_golden_model=True).
- **collect_static** – Carry the archives (~300MB) in the artifact, for linking in a different container.
- **collect_tools** – Also carry spike and spike-dasm.
- **collect_headers** – Also carry \$RISCV/include/riscv as a tar, for compiling cospike in a different container.
- **install** – Copy the library into \$RISCV/lib, where chipyard's link step looks.
- **timeout_seconds** – Timeout per subprocess.

build() → *SpikeBuildArtifact*

Rebuild libriscv from the current sources and return it by value.

The rebuild is unconditional (make is incremental, so a current tree costs seconds); skipping it is how patched sources end up paired with a stale library. Returns success=False with the captured output on failure - treat that as fatal.

chia.chipyard.spike_build_node.stage_golden_model(*artifact*: *SpikeBuildArtifact*, *riscv_path*: *str*, *logger*: *Logger* | *None* = *None*) → *None*

Write a travelled SpikeBuildArtifact into \$RISCV so a build in another container can link it. Redundant when spike was built in the same container with install=True.

21.4 CIRCT

General-purpose CIRCT chia nodes — firtool lowering, circt-opt invocation, custom-pass rebuild, ninja builds, lit runs, plus async MCP tool wrappers — usable from any chia loop.

These primitives wrap the CIRCT toolchain (firtool, circt-opt) plus chipyard's Chisel elaboration target so that any CHIA user can compose a CIRCT-driven flow without re-implementing subprocess glue. They run as @ChiaFunction tasks on a worker with the circt resource (advertised by the chia-circt image, which ships /opt/circt-sdk and a source tree at /workspace/circt). `chisel_elaborate_to_chirrtl` additionally needs Java + sbt; pair it with a chia-chipyard worker or layer chipyard's build deps into chia-circt if you need it co-located.

Usage example:

```
fir = chisel_elaborate_to_chirrtl(chipyard_path, "RocketConfig")["fir_text"]
hw  = firtool_lower_chirrtl_to_hw(fir, repl_seq_mem=True)["hw_mlir"]
sv  = circt_opt_lower_hw_to_verilog(hw)["verilog"]
```

```
chia.chipyard.circt.firtool_lower_chirrtl_to_hw(chirrtl_text: str, repl_seq_mem: bool = True,
                                                extra_args: tuple[str, ...] = (), timeout_seconds: int =
                                                600) → dict[str, str]
```

Lower CHIRRTL FIRRTL text to HW-dialect MLIR via firtool.

Wraps `firtool --ir-hw [--repl-seq-mem --repl-seq-mem-file=<path>]` against `/opt/circt-sdk/bin/firtool`. With `repl_seq_mem=True` the sequential memories are split off into a `.mems.conf` file (the same file Chipyard's MacroCompiler consumes) and that text is returned in the `mems_conf` key; otherwise it is the empty string.

Parameters

- **chirrtl_text** – Contents of a `.fir` (CHIRRTL) file.
- **repl_seq_mem** – Pass `--repl-seq-mem --repl-seq-mem-file=...` so seq-mems become external instances + a `mems.conf` sidecar.
- **extra_args** – Additional CLI flags appended verbatim to the firtool command line (e.g. `("--disable-all-randomization",)`).
- **timeout_seconds** – subprocess timeout.

Returns

Dict with keys `hw_mlir` (transformed MLIR text), `mems_conf` (empty if `repl_seq_mem=False`), `stdout`, `stderr`, `returncode`, and `success`.

Example:

```
out = firtool_lower_chirrtl_to_hw(open("Top.fir").read())
assert out["success"]; print(out["hw_mlir"][:200])
```

```
chia.chipyard.circt.circt_opt_run(mlir_text: str, pass_pipeline: str, extra_args: tuple[str, ...] = (),
                                  timeout_seconds: int = 600) → dict[str, str]
```

Run `circt-opt --pass-pipeline='<pipeline>'` on MLIR text.

Thin wrapper that prefers `/workspace/circt/build/bin/circt-opt` (so custom passes linked via `rebuild_circt_opt_with_custom_pass()` are picked up) and falls back to `/opt/circt-sdk/bin/circt-opt`.

Parameters

- **mlir_text** – Input MLIR (any dialect `circt-opt` understands).
- **pass_pipeline** – Pipeline string passed verbatim to `--pass-pipeline` (e.g. `"builtin.module(hw.module(canonicalize))"`).
- **extra_args** – Additional CLI flags appended after the pipeline.
- **timeout_seconds** – subprocess timeout.

Returns

Dict with keys `stdout` (transformed MLIR), `stderr`, `returncode`, and `success`.

Example:

```
out = circt_opt_run(hw_mlir, "builtin.module(canonicalize)")
print(out["stdout"])
```

```
chia.chipyard.circt.circt_opt_lower_hw_to_verilog(hw_mlir_text: str, extra_args: tuple[str, ...] = (),
                                                    timeout_seconds: int = 600) → dict[str, str]
```

Lower HW-dialect MLIR to SystemVerilog via firtool's `--ir-hw -> sv` path.

Convenience wrapper that runs firtool with `--format=mlir` to consume HW MLIR and emit Verilog with the standard lowering options applied (mirrors what chipyard's flow uses by default).

Parameters

- **hw_mlir_text** – HW-dialect MLIR (as produced by `firtool_lower_chirrtl_to_hw()`).
- **extra_args** – Additional CLI flags appended verbatim.
- **timeout_seconds** – subprocess timeout.

Returns

Dict with keys `verilog` (emitted SV text on success, empty otherwise), `stdout`, `stderr`, `returncode`, `success`.

Example:

```
out = circt_opt_lower_hw_to_verilog(hw_mlir)
print(out["verilog"][:200])
```

```
chia.chipyard.circt.rebuild_circt_opt_with_custom_pass(pass_cpp_path: str, td_snippet_path: str |
    None = None, transforms_subdir: str =
    'lib/Dialect/HW/Transforms',
    passes_td_path: str =
    'include/circt/Dialect/HW/Passes.td',
    num_cpus: int = 16, timeout_seconds: int =
    3600) → dict[str, str]
```

Drop a user `.cpp` into the CIRCT source tree, patch `Passes.td`, rebuild `circt-opt`.

1. Copy `pass_cpp_path` into `/workspace/circt/<transforms_subdir>/`.
2. Append the filename to the matching `CMakeLists.txt` if not present.
3. If `td_snippet_path` is given, splice it into `Passes.td` right before the closing `#endif` guard (idempotent).
4. `ninja -C /workspace/circt/build circt-opt` (with `-j ninja_jobs` when nonzero, else `ninja`'s default).

Parameters

- **pass_cpp_path** – Absolute host path to the `.cpp` to inject.
- **td_snippet_path** – Optional `.td` snippet to splice into `Passes.td`. If `None`, `Passes.td` is left untouched (useful when the pass already has its own `.td` definition).
- **transforms_subdir** – Subdirectory under `/workspace/circt/` where the `.cpp` should land (and its `CMakeLists.txt` lives). Defaults to the HW Transforms dir.
- **passes_td_path** – Path under `/workspace/circt/` to the `Passes.td` file to patch.
- **num_cpus** – `-j` value for `ninja`. Default 16 keeps headroom on a 32-core host. Pair with `.options(num_cpus=N)` at the call site if you also want Ray to reserve N CPUs per dispatch.
- **timeout_seconds** – subprocess timeout for the `ninja` build.

Returns

Dict with keys `success` (`"True"/"False"`), `log` (combined `stdout/stderr` from `ninja`), and `binary_path` (the rebuilt `circt-opt` location on success, empty otherwise).

Example:

```
r = rebuild_circt_opt_with_custom_pass("/work/passes/MyPass.cpp",
                                       "/work/my_pass_def.td")
print(r["success"], r["binary_path"])
```

`chia.chipyard.circt.list_circt_passes(category: str | None = None, timeout_seconds: int = 60) → list[str]`

List available circt-opt passes, optionally filtered by dialect prefix.

Runs `circt-opt --help` and parses out lines beginning with `--` that look like pass flags (single-token, no spaces, not a built-in option).

Parameters

- **category** – Optional pass-name prefix to filter by (e.g. "hw-", "firrtl-"). None returns every parsed pass.
- **timeout_seconds** – subprocess timeout.

Returns

Sorted list of pass names (without the leading `--`).

Example:

```
hw_passes = list_circt_passes(category="hw-")
print(hw_passes[:5])
```

`chia.chipyard.circt.chisel_elaborate_to_chirrtl(chipyard_path: str, config: str, config_package: str = 'chipyard', num_cpus: int = 16, timeout_seconds: int = 1800) → dict[str, str]`

Elaborate a chipyard Chisel config and return its CHIRRTL .fir text.

Drives the chipyard make target that produces the .fir and .anno.json for a given CONFIG/CONFIG_PACKAGE pair, so the output is ready to pipe straight into `firtool_lower_chirrtl_to_hw()`. Runs in `<chipyard>/sims/verilator` (the canonical entry point that re-exports firrtl).

Parameters

- **chipyard_path** – Path to the chipyard installation.
- **config** – Chisel config name (e.g. "RocketConfig").
- **config_package** – Scala package holding the config (default "chipyard").
- **num_cpus** – -j for the make invocation. Default 16 keeps headroom on a 32-core host. Pair with `.options(num_cpus=N)` at the call site if you also want Ray to reserve N CPUs per dispatch.
- **timeout_seconds** – subprocess timeout.

Returns

Dict with keys `fir_text` (CHIRRTL contents on success, empty otherwise), `anno_json` (annotations JSON, empty otherwise), `stdout`, `stderr`, `returncode`, `success`.

Example:

```
r = chisel_elaborate_to_chirrtl("/scratch/chipyard", "RocketConfig")
print(r["fir_text"][:200])
```

`chia.chipyard.circt.circt_ninja_build(targets: tuple[str, ...] = ('circt-opt',), num_cpus: int = 16, timeout_seconds: int = 1800) → dict`

Build CIRCT *targets* with `ninja -C /workspace/circt/build [-j N]`.

Generalises the build step of `rebuild_circt_opt_with_custom_pass()` to any target set (circt-opt, firtool, arcilator, ...). Incremental: only the touched objects + the affected tools relink.

Runs ninja in its OWN process group (`start_new_session`) and SIGKILLs the whole group on timeout. Without this, a lingering child (clang/lld) that keeps the stdout pipe open wedges `communicate()` forever — even after ninja exits. Returns `{success: bool, returncode: int, log_tail: str}`.

```
chia.chipyard.circt.circt_warm_build(targets: tuple[str, ...] = ('circt-opt', 'firtool'), num_cpus: int = 16,
                                     timeout_seconds: int = 5400) → dict
```

Idempotent per-container warm-up: ensure `lit` is installed and `targets` are built, then drop a sentinel so repeat calls are no-ops.

The chia-circt image bakes only `circt-opt`; warming the other tool targets here is cheap (shared dialect libs are already built). Safe to call at the start of every task — only the first call on a given container does work.

Returns `{success: bool, warmed: bool, log_tail: str}` (`warmed=False` means the sentinel already existed).

```
chia.chipyard.circt.circt_run_lit(test_paths: tuple[str, ...], extra_args: tuple[str, ...] = (),
                                   timeout_seconds: int = 1800, filter_out: str = "") → dict
```

Run `lit` on build/test path(s) (relative to `/workspace/circt/build`).

Resolves the `py_worker lit` (installed by `circt_warm_build()`), falls back to `PATH`. `FileCheck/not/count` come from the SDK on `PATH`. Empty `test_paths` → success with zero tests (nothing to regress). `filter_out` is a regex passed to `lit's --filter-out` to skip matching test names.

Returns `{success: bool, passed: int, failed: int, failures: list[str], log_tail: str}`.

```
class chia.chipyard.circt.BuildTool(name: str, num_cpus: int = 16, task_options=None)
```

Bases: `AsyncJobTool`

MCP tool: rebuild CIRCT tool target(s) with ninja, ASYNCHRONOUSLY.

Initializes ChiaTool with a name and optional resource requirements.

```
build(targets: list[str] | None = None) → dict
```

Start a ninja build of CIRCT target(s) in the BACKGROUND and return IMMEDIATELY (does NOT wait). Then poll `<name>_build_status` until it returns `done=true` to get `{success, returncode, log_tail}`.

Parameters

targets – e.g. `[“circt-opt”]` (default) or `[“firtool”]`.

```
build_status(wait_seconds: int = 60) → dict
```

Await the in-flight build (blocks up to `wait_seconds`, capped at 120). Returns `done=true` with `{success, returncode, log_tail}` once finished.

```
class chia.chipyard.circt.LitTool(name: str, task_options=None)
```

Bases: `AsyncJobTool`

MCP tool: run `lit` regression tests, ASYNCHRONOUSLY (same rationale as `build` — a whole-dialect `lit` run can take minutes).

Initializes ChiaTool with a name and optional resource requirements.

```
run_lit(test_paths: list[str]) → dict
```

Start a `lit` run on build/test path(s) (e.g. `[“test/Dialect/Comb”]`) in the BACKGROUND and return IMMEDIATELY. Then poll `<name>_lit_status` until it returns `done=true` to get `{success, passed, failed, failures, log_tail}`.

lit_status(*wait_seconds: int = 60*) → dict

Await the in-flight lit run (blocks up to *wait_seconds*, capped at 120). Returns *done=true* with the parsed lit result once finished.

21.5 Macro compiler

Chipyard MacroCompiler glue for SRAMs.

Parses the Chipyard/firtool `.top.mems.conf` into `SRAMSpecs`, generates the MacroCompiler MDF JSON library describing each macro, and runs Chipyard's `tapeout.jar` MacroCompiler to remap the synflop SRAMs onto those macros.

class `chia.chipyard.compiler.SRAMSpec`(*name: str, depth: int, width: int, ports: str, mask_gran: int | None, num_rw_ports: int = 0, num_read_ports: int = 0, num_write_ports: int = 0, parsed_ports: list[tuple[str, bool]]=<factory>*)

Bases: object

`chia.chipyard.compiler.rename_ports`(*ports_str: str*) → tuple[list[tuple[str, bool]], int, int, int]

Parse a comma-separated port string into named ports with counts.

Mirrors Chipyard `ConfReader.renamePorts()`. Each token becomes a (prefix, masked) tuple with incrementing per-type counters.

Example: “write,write,read” -> [(“W0”, False), (“W1”, False), (“R0”, False)], 1, 2, 0)

`chia.chipyard.compiler.parse_mems_conf`(*content: str*) → list[`SRAMSpec`]

Parse a `.top.mems.conf` file into a list of `SRAMSpec`.

`chia.chipyard.compiler.generate_macrocompiler_lib`(*sram_specs: list[SRAMSpec], cell_prefix: str = 'mapped_'*) → str

Generate MDF JSON library for MacroCompiler from SRAM specs.

Each SRAM gets a library entry named `<cell_prefix><name>` with exact-match depth/width/ports/mask_gran so MacroCompiler maps 1:1 without splitting.

`chia.chipyard.compiler.remap_with_macrocompiler`(*mems_conf_content: str, macrocompiler_lib_json: str, chipyard_path: str*) → str | None

Run MacroCompiler to remap synflop SRAMs to library macros.

Runs on a chipyard node where Java + `tapeout.jar` are available. This is the only function in this module that touches a Chipyard resource (it shells out to `java -cp <chipyard>/classpath_cache/tapeout.jar tapeout.macros.MacroCompiler`), so it is the only one decorated with `@ChiaFunction(resources={"chipyard": 1})`. The generators above (`parse/lib/stub/assemble`) are pure Python and stay plain callables.

Parameters

- **mems_conf_content** – Contents of `.top.mems.conf`
- **macrocompiler_lib_json** – MDF JSON library with SRAM entries
- **chipyard_path** – Path to chipyard installation (for `tapeout.jar`)

Returns

Remapped `.top.mems.v` content, or None on failure.

```
chia.chipyard.compiler.assemble_generated_src_with_macros(generated_src_files: list[tuple[str, str]], remapped_mems_v: str, macro_stubs: list[tuple[str, str]])
→ list[tuple[str, str]]
```

Swap in the MacroCompiler-remapped .top.mems.v and add macro stubs.

Library-agnostic: works for any MacroCompiler remap output (CACTI, SRAM22, etc.), since it only matches on module names, not the macro contents.

Parameters

- **generated_src_files** – Original generated_src from build.
- **remapped_mems_v** – MacroCompiler output instantiating the library macros.
- **macro_stubs** – List of (filename, content) blackbox stub modules for the mapped macros.

Returns

Modified generated_src with the remapped .top.mems.v swapped in, any gen-collateral .sv files that redefine the remapped modules removed, and the macro stubs appended. The .top.mems.conf is left untouched.

```
chia.chipyard.compiler.generate_macro_stubs(sram_specs: list[SRAMSpec], cell_prefix: str = 'mapped_') → list[tuple[str, str]]
```

Generate blackbox Verilog stubs for mapped SRAM macros.

Each stub declares the port interface matching the MacroCompiler instantiation (and the Liberty cell) with no body. Library-agnostic: the cell_prefix selects the macro family (e.g. "cacti_", "sram22_").

21.6 Chipyard / Hammer

chia.chipyard.chipyard_hammer — Chipyard VLSI (hammer) makefile nodes.

Chipyard wraps hammer-vlsi behind make in <chipyard>/vlsi: the buildfile target elaborates the design and generates \$(OBJ_DIR)/ hammer.d, a make fragment that is -include`d to provide the flow targets (`syn, par, drc, lvs, power, redo-<action>, ...). Everything is parameterized by make variables — CONFIG, tech_name, TOOLS_CONF / TECH_CONF / INPUT_CONFS, VLSI_TOP, OBJ_DIR, HAMMER_EXTRA_ARGS, ... — so one generic target runner covers the whole flow. [ChipyardHammerNode.make\(\)](#) is that runner.

The chipyard checkout, its generated RTL, and the OBJ_DIR are PATH-BASED on the worker, so chained targets (buildfile -> syn -> par) and report fetches ([ChipyardHammerNode.collect\(\)](#)) must land on the SAME worker. [ChipyardHammerNode](#) enforces that via a placement group (see [chia.base.colocated.ColocatedNode](#) for the given/reserved/no-PG construction modes). [ChipyardHammerNode.<fn>.chia_remote\(...\)](#) (the class attribute) is the raw, unpinned form for callers that handle placement themselves.

Like [chia.chipyard.chisel_build_node](#), this module assumes the worker's environment is already chipyard-ready (the chipyard docker images source the env in their setup commands); use the env argument for per-call overrides.

This module is deliberately independent of [chia.vlsi.hammer](#), which wraps bare hammer-vlsi CLI calls: here hammer is an implementation detail behind chipyard's Makefile.

```
class chia.chipyard.chipyard_hammer.ChipyardHammerResult(success: bool, returncode: int, target: str, vlsi_dir: str, obj_dir: str | None, stdout: str, stderr: str, listing: dict[str, int])=<factory>
```

Bases: object

```
class chia.chipyard.chipyard_hammer.ChipyardHammerCollectResult(base_dir: str, files: dict[str, str], skipped: dict[str, int], listing: dict[str, int])
```

Bases: `object`

```
class chia.chipyard.chipyard_hammer.ChipyardHammerNode(placement_group=None, require_collocated:
                                                    bool = True, *, bundle_index: int = 0,
                                                    reserve_bundle: dict | None = None,
                                                    pg_strategy: str = 'STRICT_PACK',
                                                    wait_for_pg: bool = True,
                                                    pg_ready_timeout_s: float | None = None)
```

Bases: `CollocatedNode`

Chipyard VLSI make / collect primitives sharing one placement.

The members are `@staticmethod @ChiaFunction(resources={"chipyard": 1})`; `__init__` re-binds each into a per-instance pinned form so `node.<fn>.chia_remote(...)` lands on this node's bundle:

```
with ChipyardHammerNode() as node:    # reserves {"CPU": 1, "chipyard": 1}
    obj_dir = "/scratch/vlsi-build/run1"
    bf = get(node.make.chia_remote(
        "/home/ray/chipyard", "buildfile", config="RocketConfig",
        obj_dir=obj_dir, make_vars={"tech_name": "sky130"}))
    syn = get(node.make.chia_remote(
        "/home/ray/chipyard", "syn", config="RocketConfig",
        obj_dir=obj_dir, make_vars={"tech_name": "sky130"}))
    rpts = get(node.collect.chia_remote(
        obj_dir, ["syn-rundir/reports/**"]))    # same worker, guaranteed
```

Set up placement and bind the member functions.

Parameters

- **placement_group** – an existing Ray PlacementGroup to schedule onto. If given, `require_collocated` is moot (placement is already fixed) and this node will not remove the PG on close.
- **require_collocated** – when no PG is given, reserve one so all members co-locate. When False, leave placement to the caller.
- **bundle_index** – which bundle of the (given or reserved) PG to pin to.
- **reserve_bundle** – resource shape of a self-reserved bundle (default `_DEFAULT_BUNDLE`). A bundle too small for some members is allowed — those members just can't dispatch through this node (a construction-time warning lists them).
- **pg_strategy** – placement strategy for a self-reserved PG.
- **wait_for_pg** – block on `pg.ready()` for a self-reserved PG so the node is usable immediately.
- **pg_ready_timeout_s** – optional timeout for that wait.

```
static make(chipyard_path: str, target: str, config: str | None = None, obj_dir: str | None = None,
            make_vars: dict[str, str] | None = None, jobs: int = 1, env: dict[str, str] | None = None,
            timeout_seconds: int = 86400) → ChipyardHammerResult
```

Run one `make -C <chipyard>/vlsi <target>` on a worker.

Parameters

- **chipyard_path** – Chipyard checkout root on the worker.

- **target** – Any vlsi Makefile target: “buildfile”, “syn”, “par”, “drc”, “lvs”, “power”, “redo-syn”, “clean”, ... (the flow targets exist after “buildfile” has generated hammer.d).
- **config** – Chipyard CONFIG (e.g. “RocketConfig”). Convenience for `make_vars={"CONFIG": ...}`.
- **obj_dir** – Hammer build directory, passed as `OBJ_DIR=` and used for listing. When None, chipyard derives its default (`vlsi/build/<long_name>-<TOP>`) and listing is empty. Pass the same value across buildfile/syn/par calls.
- **make_vars** – Any other vlsi Makefile variables, e.g. `{"tech_name": "sky130", "VLSI_TOP": "ChipTop", "INPUT_CONFS": "a.yml b.yml", "HAMMER_EXTRA_ARGS": "-p x.yml"}`. Appended last, so they win over config/obj_dir.
- **jobs** – make -j level (elaboration in “buildfile” benefits; the hammer flow targets manage their own parallelism).
- **env** – Extra environment variables layered over the worker’s (assumed chipyard-ready) environment.
- **timeout_seconds** – Wall-clock limit for the subprocess.

static collect(*base_dir: str, patterns: list[str], max_bytes_per_file: int | None = None*) → *ChipyardHammerCollectResult*

Fetch text files from a previous make’s OBJ_DIR (or any directory) on this worker.

Dispatch via the pinned instance member (`node.collect.chia_remote`) so it lands on the worker that owns the files — an unpinned call may not.

Parameters

- **base_dir** – Directory a previous target ran in (typically the obj_dir passed to [make\(\)](#)).
- **patterns** – Globs relative to base_dir (** is recursive), e.g. `["syn-rundir/reports/**", "*.log"]`. Files matched by multiple patterns appear once.
- **max_bytes_per_file** – When set, files over this size are recorded in `skipped` instead of shipped through the object store — protects against a glob accidentally matching a netlist. None (and 0, the falsy edge) means no cap: everything matched is shipped.

21.7 RISC-V build

Build RISC-V programs inside the `chia-riscv-cross` image.

`build_program` runs an arbitrary build command (bash/make/cmake) over caller-supplied files and collects requested outputs as bytes. `build` is a thin single-source-C/asm-to-ELF wrapper over it using the harness Makefile at `/opt/riscv-harness/Makefile`. Both run on `riscv_build` workers and never raise on build failure — callers branch on the artifact’s success.

class `chia.chipyard.riscv_build_node.RiscvBuildNode`(*timeout_seconds: int = 300, logging_level: int = 10*)

Bases: `object`

Cross-compiles RISC-V programs via the `chia-riscv-cross` toolchain.

Parameters

- **timeout_seconds** – Wall-clock limit per build command; on expiry the build returns `returncode=-1` (never raises).
- **logging_level** – Logging level for this node’s logger.

build_program(*input_files*: dict[str, bytes], *command*: list[str] | str, *work_dir*: str, *outputs*: list[str] | None = None, *cleanup_task_dir*: bool = True) → *ProgramBuildArtifact*

Run command over *input_files* in a task dir and collect outputs.

Parameters

- **input_files** – Files to drop into the task dir, keyed by relative path (nested paths allowed; parent dirs are created).
- **command** – Build command — a list is exec'd; a str runs via a shell.
- **work_dir** – Base dir; a uuid task subdir is created under it per call.
- **outputs** – Path/glob patterns (relative to the task dir) to read back as bytes; a directory pattern is walked recursively.
- **cleanup_task_dir** – Remove the task dir after collecting outputs.

Returns

ProgramBuildArtifact with the collected files and the command's success/stdout/stderr/returncode (-1 on timeout).

build(*source_content*: bytes, *program_name*: str, *work_dir*: str, *target*: Literal['verilator', 'linux'] = 'verilator', *extra_cflags*: str = "", *extra_ldflags*: str = "", *include_dump*: bool = False, *cleanup_task_dir*: bool = True, *lang*: Literal['c', 'asm'] = 'c') → *RiscvBuildArtifact*

Cross-compile one C/asm source into a RISC-V ELF via the harness Makefile.

Parameters

- **source_content** – Raw source bytes.
- **program_name** – Base name for the source, PROGRAM=, and output binary.
- **work_dir** – Base dir; a uuid task subdir is created under it per call.
- **target** – "verilator" (baremetal <name>.riscv) or "linux" (userspace <name>); selects the toolchain prefix.
- **extra_cflags** – Forwarded as EXTRA_CFLAGS= to the harness Makefile.
- **extra_ldflags** – Forwarded as EXTRA_LDFLAGS=.
- **include_dump** – Also build the dump target and return the disassembly.
- **cleanup_task_dir** – Remove the task dir after the build.
- **lang** – "c" -> .c, "asm" -> .S.

Returns

RiscvBuildArtifact with the ELF bytes, *binary_name*, *target*, optional dump, and success/std streams/returncode.

Raises

ValueError – If *target* or *lang* is unrecognized.

21.8 RISC-V objdump

Disassemble a RISC-V ELF with *objdump*.

Environment this node needs:

- riscv64-unknown-elf-objdump on PATH (target=verilator).
- riscv64-unknown-linux-gnu-objdump on PATH (target=linux).

Both are provisioned by the *chia-riscv-cross* Docker image (*dockerfiles/RiscvCrossDockerfile*).

Where it sits in the pipeline:

Binary bytes (from `RiscvBuildNode`, `S3`, or anywhere else) get passed into `RiscvObjdumpNode.dump(target="verilator" | "linux")`, which shells out to `<TOOL_PREFIX>objdump <flags> <binary>`.

`RiscvObjdumpNode.dump` returns `RiscvObjdumpArtifact` (dump + target + success + stdout, err + rc)

This is the standalone counterpart to `RiscvBuildNode.build(include_dump=True)`: use the build flag when you're compiling and want both outputs in one step, use this node when you already have an ELF and just want its disassembly.

```
class chia.chipyard.riscv_objdump_node.RiscvObjdumpNode(timeout_seconds: int = 120, logging_level:
                                                         int = 10)
```

Bases: `object`

Disassembles a RISC-V ELF with `objdump`.

Parameters

- **timeout_seconds** – Wall-clock limit applied to each `objdump` invocation in `dump()`. On expiry the dump returns `returncode=-1` (never raises); defaults to 120s.
- **logging_level** – Python logging level for this node's logger.

dump(*binary_content*: bytes, *binary_name*: str, *work_dir*: str, *target*: Literal['verilator', 'linux'] = 'verilator', *objdump_flags*: str = '-D', *cleanup_task_dir*: bool = True) → *RiscvObjdumpArtifact*

Disassemble *binary_content* with `<TOOL_PREFIX>objdump`.

Parameters

- **binary_content** – Raw bytes of the RISC-V ELF to disassemble.
- **binary_name** – Filename to give the ELF on disk (echoed into the returned artifact for traceability).
- **work_dir** – Base directory; a uuid-namespaced task subdir is created under it so concurrent runs on one worker don't collide.
- **target** – "verilator" selects the `riscv64-unknown-elf-` prefix (baremetal ELF); "linux" selects `riscv64-unknown-linux-gnu-` (userspace ELF).
- **objdump_flags** – Flags forwarded verbatim to `objdump` (split on whitespace); defaults to "-D" (disassemble all sections).
- **cleanup_task_dir** – If True (default), remove the task dir after the disassembly is captured.

Returns

Carries the disassembly (dump), the echoed `binary_name` and `target`, and `stdout/stderr/returncode`. `success=False` (with empty dump) on failure or timeout.

Return type

RiscvObjdumpArtifact

Raises

ValueError – If `target` is not a recognized value.

21.9 RISC-V DV generation

Generate random RISC-V test ELF's with riscv-dv (*run.py* *-steps gen,gcc_compile*: generate assembly, then link with the riscv64-unknown-elf toolchain in chia-riscv-dv).

Backends (the *simulator* arg): *pyflow* is free (no license) but covers only the base ISA plus partial/draft bitmanip; *xlm/vcs/questa* run the full SV flow (bitmanip/vector/crypto) and each need their CAD tool plus a FlexLM seat, sourced from *cad_setup*.

```
class chia.chipyard.riscv_dv_gen_node.GenSpec(test: str, iterations: int = 1, instr_cnt: int | None = None,
                                              plusargs: dict[str, object]=<factory>, gen_timeout: int
                                              | None = None, seed: int | None = None, extra_args:
                                              tuple[str, ...]=())
```

Bases: object

One riscv-dv test to generate: a *testlist.yaml* entry name plus optional per-run overrides. *instr_cnt* and *plusargs* are appended as *--sim_opts*, overriding the entry's *gen_opts* without editing the yaml; *extra_args* is raw *run.py* CLI for anything not modelled here. A None plusarg value is a bare flag (+*foo*).

test

Name of the *testlist.yaml* entry to generate (the instruction mix, e.g. "riscv_arithmetic_basic_test", "riscv_b_ext_test"). Passed as *--test*.

Type

str

iterations

Number of test programs to generate (*--iterations*).

Type

int

instr_cnt

Optional override for the instruction count per program; emitted via *--sim_opts +instr_cnt=<n>* when set.

Type

int | None

plusargs

Extra generator plusargs emitted via *--sim_opts*. {*k*: *v*} becomes +*k=v*; a None value becomes a bare +*k*.

Type

dict[str, object]

gen_timeout

Optional per-generation timeout forwarded as *--gen_timeout* to *run.py*.

Type

int | None

seed

Optional fixed RNG seed (*--seed*) for reproducible generation; None lets riscv-dv pick.

Type

int | None

extra_args

Raw additional `run.py` CLI tokens, appended verbatim, for flags not modelled here.

Type

`tuple[str, ...]`

sim_opts() → str

Render `instr_cnt` + `plusargs` into a `--sim_opts` string.

Returns

A space-joined string of `+key[=value]` tokens (empty if neither `instr_cnt` nor `plusargs` is set).

```
class chia.chipyard.riscv_dv_gen_node.RiscvDvGenNode(riscv_dv_dir: str = '/home/ray/riscv-dv',
simulator: str = 'pyflow', target: str | None =
None, custom_target: str | None = None,
testlist: str | None = None, isa: str | None =
None, mabi: str | None = None,
user_extension_dir: str | None = None,
custom_instr_dir: str | None = None,
cad_setup: str = '/ecad/tools/vlsi.bashrc',
timeout_seconds: int = 900, logging_level: int
= 10)
```

Bases: `object`

Generates random RISC-V test ELF's with `riscv-dv` as cosim stimulus.

Thin pass-through to `riscv-dv`'s `run.py`: each flag is forwarded only when set. Use `GenSpec.extra_args` for any flag we don't surface.

Configs are presented as filepaths that are expected to exist in the node's environment:

`custom_target`: your core config (dir holding `riscv_core_setting.sv`)

`testlist`: your test catalog (a `testlist.yaml`)

`user_extension_dir`: your custom generator `.sv` (a `riscv-dv` `user_extension/` layout); referenced by class name from the testlist and compiled in during `gen`

`custom_instr_dir`: your custom-instruction `.sv` (an `isa/custom/` layout: `riscv_instr_name_t` enum entries + `instr` classes), overlaid into the generator's `src/isa/custom/` so they generate. Defaults to `<custom_target>/isa/custom/`. (NOT a `run.py` flag — the enum is sealed before the include hooks, so it must land in `src/`; see `_install_custom_instrs`.)

The rest map 1:1 to `run.py` flags: `simulator` (`pyflow=free`, `xlm/vcs/questa= CAD+FlexLM`), `target` (a built-in core-config name), `isa`, `mabi`.

Parameters

- **riscv_dv_dir** – Path to the `riscv-dv` checkout containing `run.py` (defaults to the location inside the `chia-riscv-dv` image).
- **simulator** – `riscv-dv` backend / `--simulator`: `"pyflow"` (no license; base ISA + partial bitmanip) or `"xlm"/"vcs"/"questa"` (SV flow; full bitmanip/vector/crypto, needs the CAD tool + a FlexLM seat sourced from `cad_setup`).
- **target** – Built-in core-config name (`--target`); `None` to omit.
- **custom_target** – Path to a directory holding your `riscv_core_setting.sv` (`--custom_target`). Shipped to the worker via the Ray working dir. Takes precedence over `target`.

- **testlist** – Path to a custom `testlist.yaml` (`--testlist`); overrides the target dir's default test catalog.
- **isa** – Target ISA string (`--isa`, e.g. "rv64gc"); None omits.
- **mabi** – Target ABI string (`--mabi`, e.g. "lp64d"); None omits.
- **user_extension_dir** – Path to a riscv-dv `user_extension/` layout with your custom generator `.sv` (`--user_extension_dir`), referenced by class name from the testlist.
- **custom_instr_dir** – Path to an `isa/custom/` layout (`riscv_instr_name_t` enum entries + instr classes) overlaid into the generator's `src/isa/custom/` before compile so the instructions generate. Defaults to `<custom_target>/isa/custom/`. NOT a `run.py` flag — the enum is sealed before the include hooks, so it must land in `src/` (see `_install_custom_instrs()`).
- **cad_setup** – Path to the `bashrc` sourced before SV-flow runs to put the CAD tool + FlexLM license on `PATH` (ignored for `pyflow`).
- **timeout_seconds** – Wall-clock limit per `run.py` invocation; on expiry the process group is killed (never raises).
- **logging_level** – Python logging level for this node's logger.

compile_generator(*build_dir: str, test: str*) → str

Compile the generator testbench once into `build_dir` (`riscv-dv -co`), so later `generate(sim_only=True)` runs skip the testbench recompile and just simulate against it. *test* is any entry in the active testlist. Returns `build_dir`, to pass to `generate()` as `build_dir`. Multiple `generate(sim_only=True)` runs can reuse one build concurrently.

Parameters

- **build_dir** – Directory to compile the generator testbench into (`--output` with `--co`); created if absent. This same path is later passed to `generate()` as `build_dir`.
- **test** – Any entry in the active testlist. `run.py` needs one to load the list, but the `--co` build itself is test-independent.

Returns

The `build_dir` path, ready to hand to `generate(sim_only=True)`.

generate(*spec: GenSpec, work_dir: str, cleanup_task_dir: bool = True, sim_only: bool = False, build_dir: str | None = None*) → list[tuple[str, bytes, str]]

Generate + compile *spec* and return [(name, elf_bytes, asm_text)] for every ELF produced. Never raises: on failure it returns the (possibly empty) list found.

`sim_only=True` skips the testbench compile (`riscv-dv -so`) and reuses the build from a prior `compile_generator()`, passed as `build_dir`. Multiple `sim_only` gens can share one `build_dir` concurrently without colliding.

Parameters

- **spec** – The `GenSpec` describing the test, iteration count, and any per-run overrides to generate.
- **work_dir** – Base directory; a uuid-namespaced task subdir is created under it per call for isolation.
- **cleanup_task_dir** – If True (default), remove the task dir after collecting ELFs (unlinks any symlinks; the shared build is left intact).
- **sim_only** – If True, skip the testbench compile (`--so`) and reuse a prebuilt generator from `build_dir`. Requires `build_dir`.

- **build_dir** – Path returned by `compile_generator()`; symlinked read-only into the task dir when `sim_only=True`. Must be local to this worker.

Returns

A list of (name, elf_bytes, asm_text) tuples — one per ELF riscv-dv produced (the sibling .S source, or "" if absent). Never raises: on failure returns the (possibly empty) list found.

Raises

ValueError – If `sim_only=True` but no `build_dir` was provided.

21.10 Torture

Run RISC-V Torture against a compiled Chipyard/BOOM RTL simulator and diff the Spike vs RTL architectural signatures.

```
class chia.chipyard.torture_run_node.TortureRunNode(chipyard_path: str, sbt_override: str = 'java
-Djava.security.manager=allow -Xmx2G -Xss8M
-jar sbt-launch.jar', timeout_seconds: int =
1800, logging_level: int = 10)
```

Bases: object

Runs RISC-V Torture against a compiled Chipyard/BOOM RTL simulator.

Torture randomly generates RISC-V assembly tests, runs each on both Spike (the ISA reference model) and the RTL simulator under test (the DUT), and diffs their architectural signatures. This node drives the make flow in `<chipyard_path>/tools/torture`: it writes the simulator binary from a `BuildArtifact` to disk, passes it as the Torture R_SIM, invokes the appropriate target for the requested `TortureMode`, and collects the generated test, disassembly, and Spike/RTL signatures for each run.

Because Torture writes into a shared `tools/torture/output` directory, concurrent runs on the same checkout are serialized with a file lock.

Configure a Torture runner against a Chipyard checkout.

Parameters

- **chipyard_path** – Absolute path to the Chipyard checkout on the build node. Torture is driven out of `<chipyard_path>/tools/torture` (with its shared output/ subdirectory). Path for the CHIA chipyard container is `/home/ray/chipyard`
- **sbt_override** – The SBT= command handed to make, overriding the Makefile's default launcher.
- **timeout_seconds** – Wall-clock timeout for the make subprocess (default 30 min). For `TortureMode.OVERNIGHT` the effective timeout is raised to at least `overnight_minutes * 60 + 600` so the make process outlives the overnight loop.
- **logging_level** – Logging level for this node's logger.

```
torture(artifact: BuildArtifact, mode: TortureMode = TortureMode.SINGLE, work_dir: str =
'/tmp/chia-torture', torture_config_file: str | None = None, overnight_minutes: int = 30,
overnight_max_failures: int = 1, replay_test_s: str | None = None) → TortureResult
```

Run Torture against a pre-built simulator and collect the results.

The simulator binary from `artifact` is written to a per-run task directory under `work_dir` and passed to Torture as R_SIM. The make target invoked depends on `mode`:

- `TortureMode.SINGLE` → `make rgentest` — generate one test, run it on the DUT and Spike, and diff signatures.

- `TortureMode.OVERNIGHT` → `make rnight` — loop generating and running tests until `overnight_max_failures` failures accumulate or `overnight_minutes` elapse; failing tests are saved under `failedtests/`.
- `TortureMode.REPLAY` → `make rtest` — run a caller-supplied assembly test (`replay_test_s`) instead of generating one.

If `artifact.success` is `False` the run is skipped and an unsuccessful `TortureResult` is returned immediately. Runs are serialized on a per-checkout file lock.

Parameters

- **artifact** – Compiled RTL simulator to test, from a `ChiselBuildNode` build. Its ELF bytes become the Torture `R_SIM` (the DUT); Spike is the implicit reference model.
- **mode** – Which Torture flow to run — see `TortureMode`.
- **work_dir** – Base directory under which a fresh per-run task directory (random 8-hex name) is created to hold the simulator binary and collected artifacts.
- **torture_config_file** – Optional Torture generator config, passed as `-C <file>` (Torture's `--config`). Controls the generated instruction mix / sequences. `None` uses Torture's default.
- **overnight_minutes** – `OVERNIGHT` only — how long the generate-and-test loop runs, passed as `-m <minutes>`. Also extends the make subprocess timeout. Ignored in other modes.
- **overnight_max_failures** – `OVERNIGHT` only — stop after this many failing tests, passed as Torture's `-t <count>` threshold. Ignored in other modes.
- **replay_test_s** – `REPLAY` only — the assembly source to replay. Written to `replay.S` and passed as `TEST=` (Torture's `-a`). Required for `REPLAY`; raises `ValueError` if missing.

Returns

A `TortureResult` with overall success, the test/failure counts, per-test artifacts (`TortureTestRun`), and the raw make `stdout/stderr/returncode`. `build_artifact` is left `None` here; `torture_from_config()` populates it.

21.11 Verilator run

class `chia.chipyard.verilator_run_node.VerilatorRunNode(logging_level: int = 10)`

Bases: `object`

Runs one test ELF on a prebuilt chipyard Verilator simulator.

Construct a run node.

Parameters

logging_level – Python logging level for this node's logger. The node is otherwise stateless at construction time; per-run state (task dir, binary path) is created in `_setup` when `run` is invoked.

run(*artifact*: [BuildArtifact](#), *test_binary_content*: `bytes`, *test_binary_name*: `str`, *work_dir*: `str`, *plusargs*: `dict` = `{}`, *timeout_cycles*: `int` | `None` = `None`, *timeout_seconds*: `int` | `None` = `None`, *drsim_ini_files*: `dict`[`str`, `bytes`] = `{}`, *capture_waveform*: `bool` = `False`, *verbose*: `bool` = `True`, *cleanup_task_dir*: `bool` = `True`, *numactl*: `bool` = `False`, *wave_windows*: `list`[[WaveWindow](#)] = `()`, *dump_all_waveform*: `bool` = `False`, *upload_to_s3*: `bool` = `False`, *s3_path*: `str` = `"`, *aws_access_key_id*: `str` = `"`, *aws_secret_access_key*: `str` = `"`, *aws_session_token*: `str` = `"`, *aws_region*: `str` = `"`, *keep_waveform*: `bool` = `False`) → [RunResult](#)

Run one test ELF on the prebuilt Chipyard Verilator simulator.

Writes the simulator and test binaries into an isolated per-task directory, assembles the simulator's `+plusarg` command line, executes it (piping the committed-instruction stderr through `spike-dasm`), and returns the captured output. By default, runs on nodes tagged with the `verilator_run` resource.

Parameters

- **artifact** – The `BuildArtifact` produced by `ChiselBuildNode.build()`. Supplies the simulator ELF bytes and its name; the binary is materialized into the task dir and marked executable.
- **test_binary_content** – Raw bytes of the RISC-V test ELF to run (the program the simulator loads over the front-end server / HTIF).
- **test_binary_name** – Filename to give that ELF on disk; its stem also names the `.log` and `.out` output files.
- **work_dir** – Base working directory. A unique 8-hex-char subdirectory is created under it per run so concurrent runs sharing the same container/work_dir don't collide.
- **plusargs** – Extra simulator `+plusargs` as a dict. Each entry is emitted as `+key` when the value is falsy, else `+key=value` (e.g. `{"+loadmem": path}` or `{"+verbose": ""}`).
- **timeout_cycles** – Simulated-cycle budget; passed as `+max-cycles=<n>`. The simulator self-terminates when reached. `None` omits the `plusarg` (no cycle limit).
- **timeout_seconds** – Wall-clock limit. On expiry the simulator process is killed and whatever was captured so far is returned.
- **dramsim_ini_files** – DRAMSim2 model config as `{filename: bytes}`. When non-empty they're written to a `dramsim_ini` dir and the sim is launched with `+dramsim +dramsim_ini_dir=<dir>` to use the cycle-accurate DRAM model instead of the simple memory.
- **capture_waveform** – If True, emit a VCD via `+vcdfile=<task>/<stem>.vcd`. Requires the simulator to have been built with `target=VERILATOR_DEBUG`. Auto-enabled if `wave_windows` or `dump_all_waveform` is set.
- **verbose** – If True, append `+verbose` (commit-log / verbose sim output).
- **cleanup_task_dir** – If True (default), delete the per-task directory after the run (and after any S3 upload) to bound disk usage.
- **numactl** – If True, prefix the simulator argv with the platform's `numactl` binding (from `get_numa_prefix()`) to pin it to a NUMA node.
- **wave_windows** – chia_artifact-specific *temporal* waveform filter — a list of `WaveWindow` PC-triggered dump windows. Each is emitted as `+wf_pc_<i>=<hex> +wf_n_<i> +wf_cyc_<i>`: dump for `cyc` testbench cycles starting at the `n`-th retired commit of `pc`. At most `_MAX_WAVE_WINDOWS` (64); each is validated for `pc>0`, `n>=1`, `cyc>0`.
- **dump_all_waveform** – If True, append `+wf_dump_all=1` to bypass the window filter and dump the entire run (combine with `wf_scopes` at build time to still bound it spatially).
- **upload_to_s3** – If True (and `s3_path` is set), upload the produced `.vcd` (if any), `.out` and `.log` to S3 before the task dir is cleaned up. A missing VCD is a soft skip.
- **s3_path** – Destination `s3://bucket/prefix` for uploads. Files land under `<s3_path>/<basename>`. Required when `upload_to_s3=True`.
- **aws_access_key_id** – Explicit AWS access key for the upload. If set, `aws_secret_access_key` must also be set; otherwise boto3's default credential chain (env vars / profile / instance role) is used.

- **aws_secret_access_key** – Explicit AWS secret key (see above).
- **aws_session_token** – Optional session token for temporary (STS) credentials; only used when explicit keys are given.
- **aws_region** – Optional AWS region name for the S3 client.
- **keep_waveform** – If True, keep the produced .vcd on this worker’s disk (moved to <work_dir>/waveforms/, surviving cleanup_task_dir) and record its path + this worker’s ray node id in the RunResult — the claim ticket for `collect_waveform()`, the S3-free transfer path. Auto-enables capture_waveform. Independent of upload_to_s3 (both may be set).

Returns

Captured log (simulator stdout) and out (spike-dasm disassembly of stderr), the process returncode/success, any S3 URIs for uploaded artifacts, the kept-waveform claim ticket (vcd_path/vcd_node_id, when keep_waveform), and an echo of the configured wave_windows.

Return type

RunResult

run_metasim(*artifact*: [BuildArtifact](#), *test_binary_content*: bytes, *test_binary_name*: str, *work_dir*: str, *fesvr_step_size*: int = 128, *plusargs*: dict = {}, *timeout_cycles*: int | None = None, *timeout_seconds*: int | None = None, *verbose*: bool = True, *cleanup_task_dir*: bool = True, *numactl*: bool = False) → *RunResult*

Run a FireSim metasim (VFireSim) binary.

Unlike run, this method:

- omits +loadmem, +dramsim, and +dramsim_ini_dir (metasim uses FASED memory modeling)
- omits waveform capture (metasim uses +waveformfile with a -debug build variant, not +vcdfile)
- always passes +fesvr-step-size (required for metasim)

chia.chipyard.verilator_run_node.read_waveform_chunk(*path*: str, *offset*: int, *length*: int) → bytes

Read length bytes at offset from a kept waveform on this worker.

The transfer primitive behind `collect_waveform()`; returns b"" at or past EOF. Dispatch pinned (node affinity) to the node in the claim ticket — an unpinned call may land on a worker without the file.

chia.chipyard.verilator_run_node.remove_waveform(*path*: str) → bool

Delete a kept waveform on this worker; False if already gone.

Idempotent — safe to call from best-effort cleanup sweeps.

chia.chipyard.verilator_run_node.collect_waveform(*dest_path*: str, *run_result*: [RunResult](#), *chunk_bytes*: int = 67108864, *remove_source*: bool = False) → int

Stream a kept waveform from its verilator worker to THIS machine’s disk.

Caller-side orchestrator, not a ChiaFunction (mirrors `HammerNode.collect_fs`): it runs wherever you call it — another worker (e.g. the power node) or the driver — writes *dest_path* on that machine’s filesystem, and pulls bytes via `read_waveform_chunk()` tasks pinned to the node recorded in *run_result*. Peak memory is ~`chunk\_bytes`, not the file size; transfers are peer-to-peer through the object store.

Parameters

- **dest_path** – Where to write on the calling machine (parent dirs are created).

- **run_result** – A `RunResult` from `run(keep_waveform=True)` — its `vcd_path/vcd_node_id` are the claim ticket.
- **chunk_bytes** – Transfer chunk size.
- **remove_source** – If True, delete the worker-side VCD after a complete, size-verified copy.

Returns

Bytes written to `dest_path`.

Raises

- **ValueError** – `run_result` carries no claim ticket.
- **RuntimeError** – the copied size disagrees with the size recorded at keep time (truncated/clobbered source).

21.12 Spike cosim

Spike<->Verilator lockstep co-simulation of one ELF via chipyard's Cospike.

Spike is compiled into the simulator (`WithCospike + WithTraceIO`) and checks every committed instruction in-process; the run aborts at the first architectural divergence. The embedded spike takes its ISA from the DUT's isaDTS, so this node needs no isa argument. Cosim requires a zero-initialized DUT (a `RANDOM=0` build predefine) so unspecified reset state matches spike's zeros; Cospike forwards the legitimately non-deterministic reads (counters, IDs, LR/SC, device loads).

On a mismatch the run is repeated with `+verbose` to capture the interleaved DUT/spike commit logs around the abort (the `ctx`-line debug window). Composes `VerilatorRunNode` for the simulation, adding the `cospike` plusargs, divergence parsing, and the verbose re-run.

ESP SUPPORT

CHIA drives **ESP** (Embedded Scalable Platforms, Columbia SLD) flows through `chia.esp`. Every ESP flow is a `make` target run from a per-board working directory; a single *EspWorkspaceNode* models one such workspace, with a typed member per flow (`configure`, `build`, `sim`, `accgen`, `accel`, `synth`, `fpga_program`, `fpga_run`) plus generic primitives (`make` / `put_file` / `remove` / `collect`). All members share one placement so the path-based state a flow leaves behind is visible to the next.

The notes below cover what a worker needs to run these flows correctly; the *api reference* follows.

22.1 The worker

An ESP worker is built on ESP's own container image, which supplies the OS dependencies, the RISC-V / SPARC cross-toolchains, and the socgen Python environment; CHIA is layered on top as a self-contained interpreter. Two consequences shape how everything else is set up:

- **No CAD tools live in the image.** Simulation, HLS, and bitstream flows need the host's tool installs bind-mounted into the container, with the matching license environment variables set. Only the license-free flows (configuration, software builds, the RTL accelerator flow) run with no mounts at all — a good first target when validating a new worker.
- **socgen needs the image's Python, not CHIA's.** ESP's socgen relies on pre-3.10 `tkinter` behavior (plus `Pmw`), so `configure` runs in the container even when other stages run elsewhere. Do not expect it to work against the CHIA interpreter.

Keep site-specific values — hostnames, install paths, license servers — out of committed cluster configuration. Parameterize the configuration with environment variables resolved at load time, and source the real values from an uncommitted file, so the configuration itself stays portable.

22.2 Simulation

`sim` runs a full-SoC RTL simulation. The worker needs the simulator (and, once per checkout, Vivado to compile the vendor simulation libraries) bind-mounted with its license environment. Three things that commonly trip a first run:

- Point `XILINX_VIVADO` at the real Vivado install root. ESP's image stubs it, which mangles the paths the simulation flow compiles.
- Judge success by a transcript match, not exit code — an RTL simulation can exit cleanly without the program passing. `sim` takes a `pass_pattern` for this (ESP's testbench prints a fixed end-of-run message).
- After reconfiguring the SoC, run `sim(clean=True)` once. The simulator's incremental rebuild fails on units compiled against the previous SoC map.

22.3 Vivado and the FPGA flow

Important

If you plan to run Vivado, keep the ESP checkout on the host and bind-mount it into the container, rather than synthesizing inside the container. Vivado’s multi-process synthesis (notably the DDR4 memory-controller IP) hangs inside ESP’s older container userland. The reliable arrangement:

- Keep the ESP checkout on the host and bind-mount it into the worker container at the **identical path**. Vivado writes absolute paths into its project, so the host and the container must agree on where the workspace lives; pass that path as `esp_root` to every member.
- Run `synth` on a worker that has Vivado and the host userland (a bare host worker, e.g. the cluster head), separate from the container that runs `configure` and `build`. Because both see the one shared workspace, they cooperate on the same tree — the container compiles the software and generates the RTL, the host synthesizes it.

Other FPGA-flow facts worth knowing:

- **Vendor IP versions are pinned per board.** ESP pins exact Xilinx IP versions in its board constraints. A Vivado whose catalog has moved on fails at IP creation; adjust the pin in your checkout, or synthesize with an era-matched Vivado. No single value suits every Vivado release.
- **Match the `hw_server` version.** Programming connects a Vivado client to a `hw_server` on the machine holding the JTAG cable, and the server only accepts clients at or below its own version. `synth` and `fpga_program` each take a `vivado_bin` override, so you can synthesize with one Vivado and program with an older, server-compatible one.
- **Programming is fully networked.** `fpga_program` reaches `hw_server` over TCP; the worker needs reachability to it (a direct route or a tunnel), never the cable itself.
- **The run leg is EDCL over UDP.** `fpga_run` loads the SoC over its Ethernet via ESPLink, so the worker needs a UDP route to the board’s IP (set in the SoC configuration). If that UDP is relayed, note that ESP’s EDCL only answers requests whose source port is the ESPLink port (a relay must preserve it), and that UDP cannot ride an SSH tunnel — a point-to-point board segment needs a relay on the cabled host or a scoped firewall opening.
- **The console is read over TCP.** `fpga_run` scrapes a TCP-exposed serial port for its `pass_pattern`. Identify which serial channel is the console, and at what baud, for your board.

22.4 Resources as license seats

CHIA models tool licenses and scarce hardware as Ray resources, so the scheduler does admission control instead of letting tools fail at checkout. Advertise capacity on the workers that can satisfy each, and give members the matching demand:

| Resource | Held by / meaning |
|--------------------------|---|
| <code>esp</code> | any ESP-capable worker (config, software, RTL flow) |
| <code>esp_xcelium</code> | simulator seats — capacity = concurrent sims admitted |
| <code>esp_vivado</code> | Vivado synthesis seats (advertise on the host worker) |
| <code>esp_fpga</code> | an attached board — capacity 1 serializes access |

22.5 API reference

Generated from the source docstrings, so it stays in sync with the code.

22.5.1 State definitions

`chia.esp.state_def` — result/artifact dataclasses for the ESP nodes.

ESP (Embedded Scalable Platforms, Columbia SLD; github.com/sld-columbia/esp) drives everything through make targets run from a per-board working directory `<esp_root>/socs/<board>/`, and that directory is the state: config, generated RTL, and software outputs accumulate in place, with later targets reading what earlier ones wrote. These artifacts therefore mostly describe what happened on the worker; small binaries ship by value and anything over a caller-set cap stays in the workspace.

```
class chia.esp.state_def.EspMakeResult(success: bool, returncode: int, target: str, work_dir: str, stdout:
                                     str, stderr: str, listing: dict[str, int]=<factory>)
```

Bases: object

One generic ESP make-target run.

```
class chia.esp.state_def.EspCollectResult(base_dir: str, files: dict[str, str], skipped: dict[str, int],
                                         listing: dict[str, int])
```

Bases: object

Text files fetched by value from a workspace.

```
class chia.esp.state_def.EspConfigResult(success: bool, returncode: int, board_dir: str, esp_config: str,
                                         stdout: str, stderr: str)
```

Bases: object

One headless socgen run (`make esp-config`).

```
class chia.esp.state_def.EspAccelSpec(name: str, flow: Literal['rtl', 'vivado', 'stratus', 'catapult']='rtl',
                                     device_id: str = "", answers_tail: list[str] = <factory>)
```

Bases: object

Inputs for one `accgen.sh` run (the script's prompts, in order).

property make_name: str

Name in per-accelerator make targets (`<make_name>-hls, ...`): the skeleton directory's basename, not the bare accelerator name.

property acc_dir: str

Skeleton directory relative to the ESP root.

to_answers() → str

Render the newline-separated stdin script `accgen.sh` consumes.

```
class chia.esp.state_def.EspAccgenResult(success: bool, returncode: int, acc_dir: str, listing: dict[str,
int], stdout: str, stderr: str)
```

Bases: object

One `accgen.sh` run: a generated accelerator skeleton.

```
class chia.esp.state_def.EspAccelResult(success: bool, returncode: int, name: str, action: str, stdout:
                                       str, stderr: str)
```

Bases: object

One per-accelerator make target (`<name>-hls, <name>-baremetal, ...`).

```
class chia.esp.state_def.EspSimResult(success: bool, returncode: int, board_dir: str, test_program: str |  
                                     None, pass_matched: bool | None, stdout: str, stderr: str)
```

Bases: object

One full-system RTL simulation (make xmsim).

```
class chia.esp.state_def.EspSynthResult(success: bool, returncode: int, board_dir: str, bitstream: str |  
                                       None, reports: dict[str, str], stdout: str, stderr: str)
```

Bases: object

One FPGA synthesis run (make vivado-syn).

```
class chia.esp.state_def.EspFpgaRunResult(success: bool, returncode: int, board_dir: str, pass_matched:  
                                         bool | None, uart: str, stdout: str, stderr: str)
```

Bases: object

One software run on the programmed FPGA (make fpga-run[-linux]) with the UART console captured over TCP.

```
class chia.esp.state_def.EspSoftArtifact(target: Literal['soft', 'linux'], cpu: str, board: str, success:  
                                         bool, binaries: dict[str, bytes], kept: dict[str, int],  
                                         soft_build_dir: str, missing: list[str], stdout: str, stderr: str,  
                                         returncode: int)
```

Bases: object

ESP software build outputs (make soft / make linux) for one SoC.

Outputs land in <board_dir>/soft-build/<cpu>/ on the worker; files at or under the build's inline cap ship by value in binaries, larger ones (normally just linux.bin) are recorded in kept and stay in the workspace.

22.5.2 Workspace node

chia.esp.esp_workspace — one node class for ESP make flows.

ESP exposes every flow as a make target run from a working directory — <esp_root>/socs/<board>/ for full-SoC targets, an accelerators/ subdir for per-accelerator ones. That directory is PATH-BASED state on the worker: later targets read what earlier ones wrote, so chained targets (esp-config -> soft -> sim) and file fetches must land on the SAME worker.

EspWorkspaceNode models exactly that: one instance = one workspace, pinned to one placement-group bundle. Its members are all stateless per-call functions — generic primitives (make / put_file / remove / collect) plus one typed member per ESP flow (configure, build, sim, accgen, accel; FPGA members to come). *EspWorkspaceNode*.<fn>.chia_remote (the class attribute) is the raw, unpinned form for callers that handle placement themselves.

Workers are assumed ESP-ready (toolchains on PATH); use the env argument for per-call overrides.

```
chia.esp.esp_workspace.board_dir(esp_root: str, board: str) → str
```

Absolute path of the per-board working directory socs/<board>.

```
chia.esp.esp_workspace.with_acc_tile(config_text: str, acc: str, row: int, col: int, impl: str =  
                                     'basic_dma64', has_l2: int = 0, vendor: str = 'sld') → str
```

Return config_text with tile (row, col) replaced by an accelerator.

Rewrites the TILE_<row>_<col> line (keeping its tile index) to ... acc <ACC> 0 0 0 <impl> <has_l2> <vendor> and points the matching POWER_ line's first field at the accelerator. socgen identifies accelerators by their tech-library subdirectory name UPPERCASED (an unknown name silently becomes an empty tile), so acc should be the make name (e.g. "chiatest_rtl") and is uppercased here. impl must be an implementation point whose DMA width matches the SoC's (64 for the RISC-V CPUs), or socgen filters it out.

Raises ValueError when the tile or power line is absent.

```
class chia.esp.esp_workspace.EspWorkspaceNode(placement_group=None, require_colocated: bool =
    True, *, bundle_index: int = 0, reserve_bundle: dict |
    None = None, pg_strategy: str = 'STRICT_PACK',
    wait_for_pg: bool = True, pg_ready_timeout_s: float |
    None = None)
```

Bases: [ColocatedNode](#)

One ESP workspace: all its flow members share one placement.

The members are @staticmethod @ChiaFunction(resources={"esp": 1}); __init__ re-binds each into a per-instance pinned form so node.<fn>.chia_remote(...) lands on this node's bundle:

```
with EspWorkspaceNode() as ws:           # reserves {"CPU": 1, "esp": 1}
    root, board = "/home/espuser/esp", "xilinx-vc707-xc7vx485t"
    cfg = get(ws.configure.chia_remote(
        root, board,
        esp_config_path=f"{root}/socs/defconfig/esp_{board}_defconfig"))
    soft = get(ws.build.chia_remote(root, board, cpu="ariane"))
    outs = get(ws.collect.chia_remote(
        board_dir(root, board), ["soft-build/**/*log"]))
```

Set up placement and bind the member functions.

Parameters

- **placement_group** – an existing Ray PlacementGroup to schedule onto. If given, require_colocated is moot (placement is already fixed) and this node will not remove the PG on close.
- **require_colocated** – when no PG is given, reserve one so all members co-locate. When False, leave placement to the caller.
- **bundle_index** – which bundle of the (given or reserved) PG to pin to.
- **reserve_bundle** – resource shape of a self-reserved bundle (default _DEFAULT_BUNDLE). A bundle too small for some members is allowed — those members just can't dispatch through this node (a construction-time warning lists them).
- **pg_strategy** – placement strategy for a self-reserved PG.
- **wait_for_pg** – block on pg.ready() for a self-reserved PG so the node is usable immediately.
- **pg_ready_timeout_s** – optional timeout for that wait.

```
static make(work_dir: str, target: str, make_vars: dict[str, str] | None = None, jobs: int = 1, env: dict[str,
    str] | None = None, timeout_seconds: int = 86400, list_dir: str | None = None) →
    EspMakeResult
```

Run one make -C <work_dir> <target> on a worker.

The generic escape hatch behind the typed members.

Parameters

- **work_dir** – Directory to run make in.
- **target** – Any ESP Makefile target: “esp-config”, “soft”, “linux”, “sim”, “vivado-syn”, “<acc>-hls”, ...

- **make_vars** – Makefile variables appended to the command line, e.g. {"TEST_PROGRAM": "./soft-build/ariane/baremetal/fft.exe"}.
- **jobs** – make -j level.
- **env** – Extra environment variables layered over the worker's.
- **timeout_seconds** – Wall-clock limit; the make tree is killed and `returncode=-1` returned on expiry.
- **list_dir** – When given, `listing` manifests this directory after the run. Opt-in because a full board dir is a large tree.

static put_file(*base_dir: str, relpath: str, content: bytes | str*) → *str*

Write content to <base_dir>/<relpath> on this worker.

Parent directories are created; *relpath* may not escape *base_dir* (`ValueError`). Dispatch via the pinned instance member so the write lands on the workspace's worker. Returns the absolute path written.

static remove(*base_dir: str, relpath: str*) → *bool*

Delete <base_dir>/<relpath> (file or directory tree).

Workspaces are stateful, and some generated state must be scrubbed rather than rebuilt over (e.g. stale HLS project outputs a re-run would silently re-install). *relpath* may not escape *base_dir* (`ValueError`). Returns whether the path existed.

static collect(*base_dir: str, patterns: list[str], max_bytes_per_file: int | None = None*) → *EspCollectResult*

Fetch text files from a previous make's working dir on this worker.

Dispatch via the pinned instance member so it lands on the worker that owns the files.

Parameters

- **base_dir** – Directory a previous target ran in.
- **patterns** – Globs relative to *base_dir* (** is recursive), e.g. ["soft-build/**/* .log", "socgen/esp/.esp_config"]. Files matched by multiple patterns appear once.
- **max_bytes_per_file** – When set, files over this size are recorded in `skipped` instead of shipped through the object store — protects against a glob accidentally matching `linux.bin`. `None` (and 0, the falsy edge) means no cap.

static configure(*esp_root: str, board: str, esp_config: str | None = None, esp_config_path: str | None = None, make_vars: dict[str, str] | None = None, env: dict[str, str] | None = None, timeout_seconds: int = 1800*) → *EspConfigResult*

Configure the SoC: write `.esp_config`, run `make esp-config`.

Headless socgen. The config is written where the board Makefile actually reads it — `socgen/esp/.esp_config` (`ESP_CFG_BUILD`) under the board dir; when that file is absent ESP silently seeds it from the board's `defconfig` (`socs/defconfig/esp-<board>-defconfig`), ignoring any config placed elsewhere. Exactly one of `esp_config` / `esp_config_path` must be given.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **esp_config** – Full text of the `.esp_config` to configure with.
- **esp_config_path** – Worker-side path of an existing config to copy in instead.
- **make_vars** – Extra Makefile variables for the `esp-config` run.

- **env** – Extra environment variables layered over the worker’s.
- **timeout_seconds** – Wall-clock limit; `returncode=-1` on expiry.

Returns

`EspConfigResult` echoing the config text actually used.

Raises

- **ValueError** – If neither or both config sources are given.
- **FileNotFoundError** – If `esp_config_path` does not exist on the worker, or `socs/<board>` is not a directory.

```
static build(esp_root: str, board: str, cpu: str = 'ariane', target: Literal['soft', 'linux'] = 'soft', smp: bool | None = None, make_vars: dict[str, str] | None = None, jobs: int = 16, env: dict[str, str] | None = None, timeout_seconds: int = 7200, inline_max_bytes: int = 16777216) → EspSoftArtifact
```

Build ESP software (`make soft / make linux`) and read back its canonical outputs.

License-free; requires a configured SoC (the `.esp_config` decides the CPU the cross-toolchain targets), so run `configure()` on this workspace first.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **cpu** – Processor tile the SoC was configured with (“ariane”, “ibex”, or “leon3”) — names the `soft-build/<cpu>` output directory. If that directory is missing after a successful make but exactly one `soft-build/*` subdir exists, that one is used (with a warning).
- **target** – “soft” (bare-metal `prom.bin + systest.bin`) or “linux” (`linux.bin`).
- **smp** – When set, passed as `SMP=1 / SMP=0` on the make command line, overriding the board Makefile’s setting. `None` leaves the Makefile default.
- **make_vars** – Any other Makefile variables; appended after `smp` so they win on conflict.
- **jobs** – make -j level.
- **env** – Extra environment variables layered over the worker’s.
- **timeout_seconds** – Wall-clock limit — `make linux` compiles a kernel + root fs, so give it hours, not minutes.
- **inline_max_bytes** – Outputs at or under this size ship by value in binaries; larger ones are recorded in kept and stay at `soft_build_dir` on the worker.

Returns

`EspSoftArtifact`; success iff make exited 0 AND every canonical output exists, else `missing` names what the build failed to produce.

Raises

ValueError – If `target` is not a recognized value.

```
static sim(esp_root: str, board: str, test_program: str | None = None, sim_input: str | None = None, pass_pattern: str | None = None, clean: bool = False, make_vars: dict[str, str] | None = None, jobs: int = 1, env: dict[str, str] | None = None, timeout_seconds: int = 14400) → EspSimResult
```

Run a full-system RTL simulation (`make xmsim`, Xcelium).

Compiles the configured SoC’s RTL into the workspace’s `xcelium/` work library (incremental on repeat runs) and simulates the bare-metal software. Requires `configure()` and a licensed Xcelium (plus Vivado for the one-time Xilinx simlib compile) on the worker; demands the `esp_xcelium` resource, so dispatch

through a node whose bundle includes it, e.g. `EspWorkspaceNode(reserve_bundle={"CPU": 1, "esp": 1, "esp_xcelium": 1})`.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **test_program** – Worker-side path of the bare-metal ELF to simulate, passed as `TEST_PROGRAM=` (the make dependency chain regenerates the boot srecs from it). `None` runs the default `systest`.
- **sim_input** – Full text for `xcelium/xmsim.in`, the simulator's batch input script. Defaults to `XMSIM_BATCH_INPUT` (severity settings + `run` + `exit`); override to bound the run (`run 10 ms`) or add tracing commands.
- **pass_pattern** – Regex searched in the console transcript; when given, success additionally requires a match (RTL sims can end with exit code 0 without the test passing).
- **clean** – Run `make xmsim-distclean` first, discarding the compiled work library. Needed after an SoC reconfiguration: Xcelium's incremental rebuild fails on units compiled against the previous socmap (DLCSMD checksum mismatches).
- **make_vars** – Any other Makefile variables.
- **jobs** – `make -j` level (RTL compile benefits).
- **env** – Extra environment variables layered over the worker's.
- **timeout_seconds** – Wall-clock limit for compile + simulate; the backstop for a testbench that never terminates.

Returns

`EspSimResult`; success iff `make` exited 0 and, when `pass_pattern` was given, it matched.

Raises

FileNotFoundError – If `socs/<board>` is not a directory.

static accgen(*esp_root*: str, *spec*: [EspAccelSpec](#), *overwrite*: bool = False, *env*: dict[str, str] | None = None, *timeout_seconds*: int = 600) → [EspAccgenResult](#)

Generate an accelerator skeleton (`tools/accgen/accgen.sh`).

`accgen` is an interactive prompt sequence; this drives it by piping `spec.to_answers()` on stdin (empty answers take its defaults). The skeleton lands in `<esp_root>/<spec.acc_dir>`: hardware implementations, the accelerator XML, and bare-metal/Linux software.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **spec** – The `accgen` prompt answers.
- **overwrite** – `accgen` refuses to regenerate an existing skeleton; `True` deletes `spec.acc_dir` first (idempotent reruns).
- **env** – Extra environment variables layered over the worker's.
- **timeout_seconds** – Wall-clock limit; `returncode=-1` on expiry.

Returns

`EspAccgenResult`; success iff `accgen` exited 0 and the skeleton directory exists.

```
static accel(esp_root: str, board: str, name: str, action: str, make_vars: dict[str, str] | None = None, jobs:
    int = 16, env: dict[str, str] | None = None, timeout_seconds: int = 3600) → EspAccelResult
```

Run one per-accelerator make target: `make <name>-<action>`.

Common actions: `hls` (synthesize the kernel — or, for the RTL flow, just package it — and install the implementation points into the tech library so socgen can instantiate the tile), `baremetal` (the generated self-test program, output under `soft-build/<cpu>/baremetal/`), `driver/app` (Linux pieces).

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **name** – The accelerator’s make name — its skeleton directory basename, e.g. “`chiatest_rtl`” (`EspAccelSpec.make_name`).
- **action** – Target suffix; unknown ones fail in make, not here.
- **make_vars** – Any other Makefile variables.
- **jobs** – make -j level.
- **env** – Extra environment variables layered over the worker’s.
- **timeout_seconds** – Wall-clock limit; `returncode=-1` on expiry.

```
static synth(esp_root: str, board: str, top: str = 'top', overwrite_project: bool = False, vivado_bin: str |
    None = None, make_vars: dict[str, str] | None = None, jobs: int = 1, env: dict[str, str] | None
    = None, timeout_seconds: int = 21600, report_max_bytes: int = 262144) → EspSynthResult
```

Synthesize the configured SoC to a bitstream (`make vivado-syn`).

A batch Vivado implementation run — hours of wall clock for the larger parts. The bitstream stays in the workspace (linked as `<top>.bit` in the board dir); the implementation reports Vivado writes alongside it ship back by value.

Demands only `esp_vivado` (a Vivado seat), not `esp` — synthesis needs Vivado and the generated RTL, not the ESP cross-toolchains, so it can run on a Vivado-only worker (e.g. bare host) separate from the container that runs `configure/build`. When it does, that worker and the container must see the workspace at the SAME path (Vivado writes absolute paths into its project), so pass the shared-workspace path as `esp_root` to every member.

ESP’s project-setup recipe re-runs on every invocation and asks interactively whether to overwrite an existing Vivado project — a headless run would hang on it forever, so the answer is piped in: “n” (reuse the project) unless `overwrite_project`.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **top** – The design’s top module (the board Makefile’s TOP), naming the bitstream link.
- **overwrite_project** – Regenerate the Vivado project instead of reusing it. Needed when the SoC’s source list changed (e.g. after adding an accelerator and reconfiguring); a plain RTL edit does not need it. Vivado projects don’t open across versions, so also pass it when `vivado_bin` changed.
- **vivado_bin** – When given, a Vivado bin dir prepended to PATH for this run — the synthesis Vivado’s IP catalog must carry the versions the board’s scripts pin, which may rule out the newest install.
- **make_vars** – Any other Makefile variables.

- **jobs** – make -j level (the Vivado run manages its own threads).
- **env** – Extra environment variables layered over the worker’s.
- **timeout_seconds** – Wall-clock limit; `returncode=-1` on expiry.
- **report_max_bytes** – Per-report cap; larger reports are skipped.

Returns

`EspSynthResult`; success iff `make` exited 0 AND the bitstream exists.

```
static fpga_program(esp_root: str, board: str, fpga_host: str = 'localhost', hw_server_port: int = 3121,  
                    vivado_bin: str | None = None, make_vars: dict[str, str] | None = None, env:  
                    dict[str, str] | None = None, timeout_seconds: int = 1800) → EspMakeResult
```

Program the FPGA with the workspace bitstream (`make fpga-program`).

Fully network-based: connects to a Xilinx `hw_server` at `fpga_host:hw_server_port` (which runs on whatever machine holds the JTAG cable) and streams the bitstream through it.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **fpga_host** – Host running `hw_server`, as reachable from the worker.
- **hw_server_port** – `hw_server`’s TCP port.
- **vivado_bin** – When given, a Vivado bin dir prepended to `PATH` for this run — `hw_server` only accepts clients at or below its own version, which may rule out the synthesis Vivado.
- **make_vars** – Any other Makefile variables.
- **env** – Extra environment variables layered over the worker’s.
- **timeout_seconds** – Wall-clock limit; `returncode=-1` on expiry.

```
static fpga_run(esp_root: str, board: str, uart_host: str, uart_port: int, esplink_ip: str, linux: bool =  
                False, dram_image: str | None = None, prom_image: str | None = None, pass_pattern: str  
                | None = None, uart_timeout_seconds: int = 600, esplink_port: int | None = None,  
                make_vars: dict[str, str] | None = None, env: dict[str, str] | None = None,  
                timeout_seconds: int = 1800) → EspFpgaRunResult
```

Load and start software on the programmed FPGA and watch the UART.

Loads PROM + a payload into the running SoC over its Ethernet (ESPLink, EDCL/UDP to `esplink_ip`), then watches the console for a verdict. The board executes asynchronously, so a TCP connection to the UART (a `ser2net`- or `socat`-style bridge at `uart_host:uart_port`) is opened BEFORE the load and read until `pass_pattern` matches or `uart_timeout_seconds` passes.

By default this runs `make fpga-run[-linux]`, which loads the generic `systest/Linux` image. To run a specific program — e.g. an accelerator’s bare-metal self-test, which the `make` targets can’t load — pass `dram_image: esplink` is built and the `reset/PROM/DRAM/reset` sequence is driven directly with that image.

Parameters

- **esp_root** – ESP checkout root on the worker.
- **board** – Board working-directory name under `socs/`.
- **uart_host** – Host of the TCP-exposed UART console.
- **uart_port** – Its TCP port.
- **esplink_ip** – The SoC’s EDCL IP (a `socgen` config field), as reachable from the worker.

- **linux** – Load `linux.bin` (`fpga-run-linux`) instead of the bare-metal `systest.bin`. Ignored when `dram_image` is set.
- **dram_image** – Worker-side path of a program image to load into DRAM instead of `systest` — the direct-esplink path.
- **prom_image** – PROM image for the direct path; defaults to the sole `soft-build/**/prom.bin` in the workspace.
- **pass_pattern** – Regex searched in the UART transcript; when given, success additionally requires a match.
- **uart_timeout_seconds** – How long to keep reading the console after the load, absent a match.
- **esplink_port** – Override the SoC's EDCL UDP port.
- **make_vars** – Any other Makefile variables.
- **env** – Extra environment variables layered over the worker's.
- **timeout_seconds** – Wall-clock limit for the load.

MODEL BACKENDS

NOTE: We provide a single function to interface with each of these providers even though some are agents, some are LLM providers, and some are on-premises LLM servers. This interface is an agentic one, and for the non-agents, we turn the model into a (very primitive) agent, by placing it into a query->tool call->query loop.

We plan to in the near future expose an interface to the non-agent models (providers and servers), which is, instead of a primitive agent, the interface you would use to build your own agents.

For most serious tasks that don't require on-premises LLM serving, we expect you will get better results using the agents (Claude Code, Codex, Copilot, Antigravity, or OpenCode), with your preferred provider for credentials for the agent, as opposed to using the specific node for your provider (e.g. Claude Code with Bedrock credentials instead of the bedrock node).

API reference for `chia.models`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

23.1 Claude

```
class chia.models.claude.ClaudeCodeQueryResult(result: str, returncode: int, stderr: str, stream_result: str, success: bool = False, session_transcript: bytes | None = None, session_transcript_path: str | None = None)
```

Bases: [QueryResult](#)

Derived QueryResult specialized for the Claude Code CLI backend.

Carries the on-disk `<session_id>.jsonl` transcript bytes back to the caller so a `resume_session=True` LLM can continue the same conversation on a different worker — written by `ClaudeCodeLLM._capture_transcript()` after a CLI run and consumed by `ClaudeCodeLLM._restore_transcript()` before the next `--resume` invocation.

```
exception chia.models.claude.ClaudeCodeError(node_id: str, error_type: str, exit_code: int = -1, raw_message: str = "")
```

Bases: `Exception`

Base for all Claude Code CLI errors.

Every subclass must implement `__reduce__` for Ray serialization.

```
exception chia.models.claude.RateLimitError(node_id: str, reset_time: datetime, raw_message: str = "", exit_code: int = -1)
```

Bases: [ClaudeCodeError](#)

Raised when the Claude CLI response indicates a usage-limit hit.

```
exception chia.models.claude.AuthenticationError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [ClaudeCodeError](#)

Raised when the CLI's auth token/API key is invalid or expired.

```
exception chia.models.claude.BillingError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [ClaudeCodeError](#)

Raised when the billing account has payment issues.

```
exception chia.models.claude.InvalidRequestError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [ClaudeCodeError](#)

Raised when the request is malformed (bad prompt, unsupported params, etc.).

```
exception chia.models.claude.ServerError(node_id: str, exit_code: int = -1, raw_message: str = "",
                                         retry_after: int | None = None)
```

Bases: [ClaudeCodeError](#)

Raised when Anthropic's API returns a server-side error (500/503).

```
exception chia.models.claude.MaxOutputTokensError(node_id: str, exit_code: int = -1, raw_message: str = "",
                                                    partial_text: str = "")
```

Bases: [ClaudeCodeError](#)

Raised when the LLM's response was truncated by the output token limit.

```
exception chia.models.claude.UnknownClaudeError(node_id: str, exit_code: int = -1, raw_message: str = "",
                                                  stderr: str = "")
```

Bases: [ClaudeCodeError](#)

Raised for unclassified CLI errors.

```
chia.models.claude.parse_rate_limit_reset(text: str) → datetime | None
```

Parse a Claude rate-limit message and return the UTC reset time.

Expected format: "You've hit your limit · resets 4pm (America/Los_Angeles)"

Returns None when no rate-limit message is found.

```
chia.models.claude.parse_rate_limit_event(event: dict) → datetime | None
```

Parse a rate_limit_event JSON object and return the UTC reset time.

Only triggers when rate_limit_info.status is "rejected" — the event is also emitted with other statuses as an informational notice, which should NOT be treated as a rate limit.

```
class chia.models.claude.ClaudeCodeLLM(model: str = 'claude-sonnet-4-6', system_message: str = "",
                                       timeout_seconds: int = 600, retries: int = 3, logging_name: str = 'claude_code',
                                       logging_level: int = 10, log_dir: str | None = None, resume_session: bool = False,
                                       projects_cwd: str | None = '/home/ray/.claude/projects/-home-ray-llm-env',
                                       extra_cli_args: List[str] | None = None, log_stream: bool = True, log_all: bool = False,
                                       backend: str = 'cli', api_key: str | None = None, max_tokens: int = 16000,
                                       thinking: str | None = 'adaptive', max_tool_iterations: int = 100,
                                       dangerously_skip_permissions: bool = True, config=<object object>)
```

Bases: [LLMCallBase](#)

Wraps the Claude Code CLI (`claude --print`) as an LLM backend.

Each call to `prompt()` spawns a `claude` subprocess that can optionally connect to MCP tool servers (e.g. `BashTool`).

prompt(*user_message: str, tools: List[ChiaTool] | None = []*) → *ClaudeCodeQueryResult*

Send *user_message* to Claude Code CLI and return the response.

Returns

ClaudeCodeQueryResult with `success=True` when the CLI ran cleanly, or `success=False` when every retry attempt failed (in which case `result` is empty and `returncode` is `-1`).

Raises

- *RateLimitError* – Usage limit hit — propagates immediately.
- *AuthenticationError* – Auth failure — propagates immediately.
- *BillingError* – Billing/payment issue — propagates immediately.
- *InvalidRequestError* – Malformed request — propagates immediately.
- *ServerError* – After all retries with exponential backoff.
- *MaxOutputTokensError* – After one retry attempt.

23.2 Bedrock

Amazon Bedrock LLM backend built on the boto3 Converse API.

BedrockLLM talks to **any** tool-capable chat model on Amazon Bedrock (Claude, Amazon Nova, Llama, Mistral, Command R, ...) It uses the Bedrock Runtime `converse` API, which normalises messages, system prompts, and tool use across model families, and runs the agentic tool loop client-side — executing each `ChiaTool`'s MCP server over HTTP exactly like *chia.models.claude.ClaudeCodeLLM*'s API backend.

WARNING: experimental. Only exercised by the tests in `chia/models/tests/test_bedrock.py` (mocked unit tests, plus opt-in live tests). Not validated in production.

Auth/config come from the standard AWS chain (env vars, shared profile, or IAM role); pass `region` or rely on `AWS_REGION` / `AWS_DEFAULT_REGION`. `boto3` is imported lazily, so importing this module does not require it.

exception `chia.models.bedrock.BedrockError`(*node_id: str, error_type: str, status_code: str = "", raw_message: str = ""*)

Bases: `Exception`

Base for all Bedrock backend errors.

exception `chia.models.bedrock.RateLimitError`(*node_id: str, reset_time: datetime, raw_message: str = "", status_code: str = ""*)

Bases: *BedrockError*

Throttling / quota exhaustion (`ThrottlingException` etc.).

exception `chia.models.bedrock.AuthenticationError`(*node_id: str, status_code: str = "", raw_message: str = ""*)

Bases: *BedrockError*

Invalid / expired / unauthorized AWS credentials.

exception `chia.models.bedrock.InvalidRequestError`(*node_id: str, status_code: str = "", raw_message: str = ""*)

Bases: [BedrockError](#)

Malformed request or unknown model (ValidationException etc.).

exception `chia.models.bedrock.ServerError`(*node_id: str, status_code: str = "", raw_message: str = ""*)

Bases: [BedrockError](#)

Transient service-side failure (5xx, model timeout, connection).

exception `chia.models.bedrock.MaxOutputTokensError`(*node_id: str, status_code: str = "", raw_message: str = "", partial_text: str = ""*)

Bases: [BedrockError](#)

The response was truncated at maxTokens.

exception `chia.models.bedrock.UnknownBedrockError`(*node_id: str, status_code: str = "", raw_message: str = ""*)

Bases: [BedrockError](#)

Unclassified Bedrock error.

class `chia.models.bedrock.BedrockLLM`(*model: str, system_message: str = "", timeout_seconds: int = 600, retries: int = 3, logging_name: str = 'bedrock_llm', logging_level: int = 10, log_dir: str | None = None, region: str | None = None, max_tokens: int = 16000, max_tool_iterations: int = 100, client_kwargs: dict | None = None, dangerously_skip_permissions=<object object>, config=<object object>*)

Bases: [LLMCallBase](#)

Bedrock Converse-API LLM backend with client-side MCP tool execution.

Returns the same `QueryResult` shape as the other backends so callers are interchangeable; `returncode` is synthesised (0 on success, -1 when every retry fails) and `stderr` is unused.

prompt(*user_message: str, tools: List[ChiaTool] | None = []*) → [QueryResult](#)

Send *user_message* via the Bedrock Converse API and return the response, retrying transient failures with the same policy the other backends use.

23.3 Vertex

Google Vertex AI LLM backend built on the google-genai SDK.

[VertexGeminiLLM](#) runs Google's **Gemini** models on Vertex AI and drives the agentic tool loop client-side — executing each ChiaTool's MCP server over HTTP, exactly like the Bedrock and Claude API backends.

Vertex has **no single unified API** across model families — Gemini goes through google-genai, Claude-on-Vertex through AnthropicVertex, and Llama/Mistral through OpenAI-compatible MaaS endpoints. Two separate classes are provided here, one for Gemini and one for MaaS.

WARNING: experimental. Only exercised by the tests in `chia/models/tests/test_vertex.py` (mocked unit tests, plus opt-in live tests). Not validated in production.

Auth/config: Vertex needs a GCP project + location and Application Default Credentials (`gcloud auth application-default login`, a service-account key via `GOOGLE_APPLICATION_CREDENTIALS`, or a workload identity). Pass `project/location` or rely on `GOOGLE_CLOUD_PROJECT` / `GOOGLE_CLOUD_LOCATION`. `google-genai` is imported lazily.

exception `chia.models.vertex.VertexError`(*node_id: str, error_type: str, status_code: int | None = None, raw_message: str = ""*)

Bases: `Exception`

Base for all Vertex backend errors.

exception `chia.models.vertex.RateLimitError`(*node_id: str, reset_time: datetime, raw_message: str = "", status_code: int | None = None*)

Bases: `VertexError`

Quota / rate exhaustion (HTTP 429, RESOURCE_EXHAUSTED).

exception `chia.models.vertex.AuthenticationError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: `VertexError`

Invalid / missing credentials or permission (HTTP 401 / 403).

exception `chia.models.vertex.InvalidRequestError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: `VertexError`

Malformed request or unknown model (HTTP 400 / 404).

exception `chia.models.vertex.ServerError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: `VertexError`

Transient service-side failure (HTTP 5xx).

exception `chia.models.vertex.MaxOutputTokensError`(*node_id: str, status_code: int | None = None, raw_message: str = "", partial_text: str = ""*)

Bases: `VertexError`

The response was truncated at max_output_tokens.

exception `chia.models.vertex.UnknownVertexError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: `VertexError`

Unclassified Vertex error.

exception `chia.models.vertex.ContentBlockedError`(*node_id: str, block_reason: str = "", raw_message: str = ""*)

Bases: `VertexError`

The model returned no usable content because the prompt or the response was blocked (safety, recitation, block-list, ...).

This is NOT an HTTP/API error — Gemini reports it as a 200-OK response whose candidate carries a blocking `finish_reason` (or whose `prompt_feedback` carries a `block_reason` with no candidates). Surfacing it as a typed error keeps a block from masquerading as a successful empty answer. Never automatically retried: re-sending the same prompt will be blocked again.

class `chia.models.vertex.VertexGeminiLLM`(*model: str, system_message: str = "", timeout_seconds: int = 600, retries: int = 3, logging_name: str = 'vertex_gemini', logging_level: int = 10, log_dir: str | None = None, project: str | None = None, location: str | None = None, max_tokens: int = 16000, max_tool_iterations: int = 100, client_kwargs: dict | None = None, dangerously_skip_permissions=<object object>, config=<object object>*)

Bases: [LLMCallBase](#)

Gemini-on-Vertex LLM backend with client-side MCP tool execution.

Returns the same `QueryResult` shape as the other backends so callers are interchangeable; `returncode` is synthesised (0 on success, -1 when every retry fails) and `stderr` is unused.

prompt(*user_message*: str, *tools*: List[[ChiaTool](#)] | None = []) → [QueryResult](#)

Send *user_message* via Gemini on Vertex and return the response, retrying transient failures with the same policy the other backends use.

class chia.models.vertex.**VertexGenericLLM**(*model*: str, *project*: str | None = None, *location*: str | None = None, ***kwargs*)

Bases: [OpenAICompatLLM](#)

Non-Gemini Vertex models (Llama, Mistral, ...) via the Vertex Model-as-a- Service **OpenAI-compatible** endpoint.

Vertex has no single unified API, so Gemini uses google-genai ([VertexGeminiLLM](#)) while the open/partner families are reached through the OpenAI-compatible MaaS endpoint — which is exactly what [OpenAICompatLLM](#) already speaks. The only Vertex-specifics are the endpoint URL (built from project/location) and auth: a GCP ADC bearer token that rotates, supplied via `token_provider`. Everything else — the agent loop, tool handling, error translation — is inherited unchanged.

`model` is the MaaS model id, e.g. `meta/llama-3.1-8b-instruct-maas`.

prompt(*user_message*: str, *tools*: List[[ChiaTool](#)] | None = []) → [QueryResult](#)

Send *user_message* via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

23.4 Antigravity

Google Antigravity CLI LLM backend.

`AntigravityLLM` wraps Google's Antigravity CLI (the `agy` binary, installed via `curl -fsSL https://antigravity.google/cli/install.sh | bash`) behind the same synchronous `prompt` shape as the other Chia LLM backends. It runs `agy --print --output-format stream-json` (non-interactive "print mode"): `agy` emits one NDJSON event per line — `init` (model, cwd, tools), `step_update` (user input, agent text, tool calls with their arguments and output, per-step token usage) and a final `result` (status, response text, conversation id, total usage). We parse those into a readable turn-by-turn transcript on `stream_result` and the final answer on `result`.

Auth is OAuth-only. agy signs in with a Google account ("Antigravity")

/ Gemini Code Assist) and stores a refresh token on disk; in a container it uses file-based token storage under the Gemini config dir. There is no API-key path.

Sessions the first call starts a conversation and every later call can pass `--conversation <id>` so the model keeps its memory. `agy` persists each conversation as a SQLite file `<gemini_dir>/antigravity-cli/conversations/<conversation_id>.db`; because `prompt` may land on a different `antigravity_creds` worker each call, we checkpoint and carry those bytes on [AntigravityQueryResult](#) (`session_transcript`) and re-paste them before the next run, exactly like Claude's `.jsonl` transcript. The `_session_tracked` wrapper (shared with the Claude backend) harvests them on `get()` so callers need no bookkeeping:

```
llm = AntigravityLLM(resume_session=True)
a = get(llm.prompt.chia_remote(llm, "remember X", tools))
b = get(llm.prompt.chia_remote(llm, "what was X?", tools))  # same conversation
```

Tools are exposed to agy as MCP servers. agy only reads them from the fixed path `~/.gemini/config/mcp_config.json` and waits for every listed server before answering, so each run gets a private HOME holding exactly its own tool list (with `~/.gemini/antigravity-cli` symlinked to the real one); the user's own `~/.gemini/config` is never modified and concurrent prompts with different tool sets don't interfere.

The system prompt is folded into the user message (print mode has no `--system-prompt` flag), mirroring [CodexLLM](#).

```
exception chia.models.antigravity.AntigravityError(node_id: str, exit_code: int = -1, raw_message:
                                                    str = "")
```

Bases: `Exception`

Base for Antigravity CLI errors. Subclasses are Ray-serializable.

```
exception chia.models.antigravity.RateLimitError(node_id: str, reset_time: datetime | None = None,
                                                    raw_message: str = "", exit_code: int = -1)
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.AuthenticationError(node_id: str, exit_code: int = -1,
                                                         raw_message: str = "")
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.BillingError(node_id: str, exit_code: int = -1, raw_message: str =
                                                    "")
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.InvalidRequestError(node_id: str, exit_code: int = -1,
                                                         raw_message: str = "")
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.ServerError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.MaxOutputTokensError(node_id: str, exit_code: int = -1,
                                                         raw_message: str = "")
```

Bases: [AntigravityError](#)

```
exception chia.models.antigravity.UnknownAntigravityError(node_id: str, exit_code: int = -1,
                                                            raw_message: str = "")
```

Bases: [AntigravityError](#)

```
chia.models.antigravity.parse_rate_limit_reset(text: str) → datetime | None
```

Parse a human reset time such as `resets 4pm (America/Los_Angeles)`.

```
class chia.models.antigravity.AntigravityQueryResult(result: str, returncode: int, stderr: str,
                                                       stream_result: str, success: bool = False,
                                                       session_transcript: bytes | None = None,
                                                       session_transcript_path: str | None = None,
                                                       conversation_id: str | None = None, usage: dict
                                                       | None = None, events: list = <factory>,
                                                       diagnostics: str = "")
```

Bases: [QueryResult](#)

`QueryResult` specialised for the Antigravity CLI.

```
class chia.models.antigravity.AntigravityLLM(model: str | None = None, system_message: str = "",
                                             timeout_seconds: int = 600, retries: int = 3,
                                             logging_name: str = 'antigravity', logging_level: int =
                                             10, log_dir: str | None = None, agy_bin: str = 'agy',
                                             work_dir: str | None = None, add_dirs: list[str] | None =
                                             None, gemini_dir: str | None = None,
                                             dangerously_skip_permissions: bool = True, sandbox:
                                             bool = False, extra_cli_args: list[str] | None = None,
                                             resume_session: bool = False, config=<object object>)
```

Bases: [LLMCallBase](#)

Wrap the Google Antigravity CLI (agy --print) as a Chia LLM backend.

prompt(user_message: str, tools: list[[ChiaTool](#)] | None = None) → [AntigravityQueryResult](#)

Send user_message to agy --print and return the response.

Returns an [AntigravityQueryResult](#) (success=True on a clean run; success=False with returncode=-1 when every retry failed). With resume_session=True the conversation transcript is restored onto this worker before the run and captured off it afterwards, and prompt.chia_remote returns an ObjectRefCallback that syncs both onto this instance on get() (see _sync_transcript).

23.5 Codex

Codex CLI LLM backend.

CodexLLM wraps codex exec behind the same synchronous prompt shape as the other Chia LLM backends. Chia MCP tools are passed as per-run Codex config overrides, so this backend does not mutate the user's persistent Codex configuration.

exception chia.models.codex.CodexError(node_id: str, exit_code: int = -1, raw_message: str = "")

Bases: [Exception](#)

Base for Codex CLI errors. Subclasses are Ray-serializable.

exception chia.models.codex.RateLimitError(node_id: str, reset_time: datetime | None = None,
 raw_message: str = "", exit_code: int = -1)

Bases: [CodexError](#)

exception chia.models.codex.AuthenticationError(node_id: str, exit_code: int = -1, raw_message: str =
 ")")

Bases: [CodexError](#)

exception chia.models.codex.BillingError(node_id: str, exit_code: int = -1, raw_message: str = "")

Bases: [CodexError](#)

exception chia.models.codex.InvalidRequestError(node_id: str, exit_code: int = -1, raw_message: str =
 ")")

Bases: [CodexError](#)

exception chia.models.codex.ServerError(node_id: str, exit_code: int = -1, raw_message: str = "")

Bases: [CodexError](#)

exception chia.models.codex.MaxOutputTokensError(node_id: str, exit_code: int = -1, raw_message: str =
 = ")")

Bases: [CodexError](#)

exception `chia.models.codex.UnknownCodexError`(*node_id*: str, *exit_code*: int = -1, *raw_message*: str = "")

Bases: `CodexError`

class `chia.models.codex.CodexQueryResult`(*result*: str, *returncode*: int, *stderr*: str, *stream_result*: str, *success*: bool = False, *session_id*: str | None = None, *session_state*: dict[str, bytes] | None = None, *session_state_paths*: tuple[str, ...] = ())

Bases: `QueryResult`

QueryResult specialized for the Codex CLI backend.

Codex stores resumable conversations under CODEX_HOME rather than in a portable JSONL transcript. The state fields carry the needed opaque files back to the caller so a later `codex exec resume <session_id>` - can run on a different Chia worker.

`chia.models.codex.parse_session_id`(*stdout*: str) → str | None

Extract a Codex session id from JSONL stdout.

`chia.models.codex.parse_rate_limit_reset`(*text*: str) → datetime | None

Parse a human reset time such as `resets 4pm (America/Los_Angeles)`.

class `chia.models.codex.CodexLLM`(*model*: str | None = None, *system_message*: str = "", *timeout_seconds*: int = 600, *retries*: int = 3, *logging_name*: str = 'codex', *logging_level*: int = 10, *log_dir*: str | None = None, *codex_bin*: str = 'codex', *work_dir*: str | None = None, *extra_cli_args*: list[str] | None = None, *sandbox*: str = 'workspace-write', *approval_policy*: str = 'never', *dangerously_bypass_approvals_and_sandbox*: bool = True, *skip_git_repo_check*: bool = True, *ephemeral*: bool = False, *ignore_rules*: bool = False, *profile*: str | None = None, *reasoning_effort*: str | None = None, *resume_session*: bool = False, *auto_compact_token_limit*: int | None = 200000, *config*=<object object>)

Bases: `LLMCallBase`

Wrap `codex exec` as a Chia LLM backend.

prompt(*user_message*: str, *tools*: list[`ChiaTool`] | None = None) → `CodexQueryResult`

Send *user_message* to `codex exec`.

23.6 Copilot

GitHub Copilot CLI LLM backend.

`CopilotLLM` wraps `copilot --prompt` (non-interactive mode) behind the same synchronous `prompt` shape as the other Chia LLM backends. Chia MCP tools are passed as a per-session `--additional-mcp-config` override, so this backend does not mutate the user's persistent Copilot configuration.

Auth. `copilot` signs in with a GitHub account (`copilot login`) or a token from `COPILOT_GITHUB_TOKEN` / `GH_TOKEN` / `GITHUB_TOKEN`; stored credentials live in the system credential store, or under `COPILOT_HOME` (default `~/.copilot`) when no credential store is available (e.g. in a container).

The system prompt is folded into the user message (non-interactive mode has no system-prompt flag), mirroring `CodexLLM`.

exception `chia.models.copilot.CopilotError`(*node_id*: str, *exit_code*: int = -1, *raw_message*: str = "")

Bases: `Exception`

Base for Copilot CLI errors. Subclasses are Ray-serializable.

```
exception chia.models.copilot.RateLimitError(node_id: str, reset_time: datetime | None = None,
                                              raw_message: str = "", exit_code: int = -1)
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.AuthenticationError(node_id: str, exit_code: int = -1, raw_message: str
                                                    = "")
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.BillingError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.InvalidRequestError(node_id: str, exit_code: int = -1, raw_message: str
                                                    = "")
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.ServerError(node_id: str, exit_code: int = -1, raw_message: str = "")
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.MaxOutputTokensError(node_id: str, exit_code: int = -1, raw_message:
                                                    str = "")
```

Bases: [CopilotError](#)

```
exception chia.models.copilot.UnknownCopilotError(node_id: str, exit_code: int = -1, raw_message: str
                                                    = "")
```

Bases: [CopilotError](#)

```
class chia.models.copilot.CopilotQueryResult(result: str, returncode: int, stderr: str, stream_result: str,
                                              success: bool = False, session_id: str | None = None,
                                              session_state: dict[str, bytes] | None = None,
                                              session_state_paths: tuple[str, ...] = ())
```

Bases: [QueryResult](#)

QueryResult specialized for the Copilot CLI backend.

Copilot stores resumable sessions under `COPILOT_HOME` (the `session-store.db` index plus a `session-state/<session_id>/` directory) rather than in a portable JSONL transcript. The state fields carry the needed opaque files back to the caller so a later `copilot --resume=<session_id>` can run on a different Chia worker.

```
chia.models.copilot.parse_session_id(stdout: str) → str | None
```

Extract a Copilot session id from JSONL stdout.

```
chia.models.copilot.parse_rate_limit_reset(text: str) → datetime | None
```

Parse a human reset time such as `resets 4pm (America/Los_Angeles)`.

```
class chia.models.copilot.CopilotLLM(model: str | None = None, system_message: str = "",
                                     timeout_seconds: int = 600, retries: int = 3, logging_name: str =
                                     'copilot', logging_level: int = 10, log_dir: str | None = None,
                                     copilot_bin: str = 'copilot', work_dir: str | None = None,
                                     extra_cli_args: list[str] | None = None, allow_tools: list[str] | None
                                     = None, deny_tools: list[str] | None = None, allow_all: bool = True,
                                     reasoning_effort: str | None = None, resume_session: bool = False,
                                     config=<object object>)
```

Bases: [LLMCallBase](#)

Wrap the GitHub Copilot CLI (`copilot --prompt`) as a Chia LLM backend.

prompt(*user_message: str, tools: list[ChiaTool] | None = None*) → *CopilotQueryResult*

Send *user_message* to copilot --prompt.

23.7 Opencode

opencode CLI LLM backend.

OpenCodeLLM wraps the opencode CLI (<https://opencode.ai>) as an LLM backend, opencode is provider-agnostic: the model is given as *provider/model* (e.g. *anthropic/claude-sonnet-4-6*) and opencode runs its own server-side agentic tool loop, so there is no client-side MCP loop here.

WARNING: experimental. Only exercised by the tests in *chia/models/tests/test_opencode.py* (mocked unit tests, plus opt-in live tests). Not validated in production. Auth is environment-driven: opencode uses its own stored credentials (*opencode auth login*) or provider env vars.

exception *chia.models.opencode.OpenCodeError*(*node_id: str, error_type: str, exit_code: int = -1, raw_message: str = ""*)

Bases: *Exception*

Base for all opencode CLI errors.

exception *chia.models.opencode.RateLimitError*(*node_id: str, reset_time: datetime, raw_message: str = "", exit_code: int = -1*)

Bases: *OpenCodeError*

The provider behind opencode reported a usage/rate limit.

exception *chia.models.opencode.AuthenticationError*(*node_id: str, exit_code: int = -1, raw_message: str = ""*)

Bases: *OpenCodeError*

opencode has no/invalid credentials for the selected provider.

exception *chia.models.opencode.BillingError*(*node_id: str, exit_code: int = -1, raw_message: str = ""*)

Bases: *OpenCodeError*

The provider account has a billing/payment problem.

exception *chia.models.opencode.InvalidRequestError*(*node_id: str, exit_code: int = -1, raw_message: str = ""*)

Bases: *OpenCodeError*

Malformed request — bad model string, invalid config, unknown agent, etc.

exception *chia.models.opencode.ServerError*(*node_id: str, exit_code: int = -1, raw_message: str = "", retry_after: int | None = None*)

Bases: *OpenCodeError*

Transient provider/server-side failure (5xx, overloaded, connection).

exception *chia.models.opencode.MaxOutputTokensError*(*node_id: str, exit_code: int = -1, raw_message: str = "", partial_text: str = ""*)

Bases: *OpenCodeError*

The response was truncated at the output token limit.

exception *chia.models.opencode.UnknownOpenCodeError*(*node_id: str, exit_code: int = -1, raw_message: str = "", stderr: str = ""*)

Bases: *OpenCodeError*

Unclassified opencode CLI error.

`chia.models.opencode.parse_session_id(stdout: str) → str | None`

Pull the opencode session id out of `run --format json` stdout.

Each event is `{type, sessionID, part:{sessionID, ...}}`; the opening `step_start` line reliably carries it. Falls back to a regex scan if the JSON shape changes.

`chia.models.opencode.parse_run_error(stdout: str) → dict | None`

Pull the first structured error out of `run --format json` stdout.

opencode emits `{"type":"error", "sessionID":..., "error":{"name, data}}` events for failures that happen before/around the model request — notably an unknown model id, which surfaces as `{"name":"UnknownError","data":{"message": "Model not found: ..."}}}`. These never reach the session export's `messages[].info.error` because no assistant message is ever created, so the run stream is the *only* place they appear (confirmed against opencode 1.15.13). Returns the first such `error` dict (`{name, data}`), or `None`. Genuine provider errors (e.g. `APIError 401`) appear in *both* the run stream and the export; the export copy is richer (full `responseHeaders`), so callers prefer it and use this only as a fallback.

```
class chia.models.opencode.AdditionalModelProvider(id: str, models: List[str] | Dict[str, dict], npm: str
                                                    = '@ai-sdk/openai-compatible', name: str | None
                                                    = None, base_url: str | None = None, api_key: str
                                                    | None = None, options: Dict[str, object] | None =
                                                    None)
```

Bases: `object`

A custom model provider to inject into the opencode config's `provider` block.

opencode is provider-agnostic: beyond its built-in providers you can declare a custom or self-hosted one — an OpenAI-compatible endpoint, a private gateway, a local vLLM/Ollama server, etc. — directly in the config under `provider.<id>`. Each entry names the AI-SDK loader package (`npm`), a display name, the SDK options (notably `base_url` / `api_key`), and the models the provider serves. See <https://opencode.ai/docs/providers> (Custom providers) for the schema.

Once declared, select one of its models by passing `model="<id>/<model-id>"` to *OpenCodeLLM* — e.g. `AdditionalModelProvider(id="my-vllm", ...)` exposes its models as `my-vllm/<model-id>`.

id

Provider key. Used both as the key under `provider` in the config and as the provider half of `provider/model`. Must be unique.

Type

`str`

models

Either a list of model-id strings (each expands to a bare `{}` entry) or a dict mapping model-id -> model config, e.g. `{"name": ..., "limit": {...}, "cost": {...}, "options": {...}}`.

Type

`List[str] | Dict[str, dict]`

npm

The AI-SDK package opencode loads to talk to this provider. Defaults to `@ai-sdk/openai-compatible`, which fits any OpenAI-compatible endpoint.

Type

`str`

name

Human-readable display name (defaults to `id` when omitted).

Type

`str` | `None`

base_url

Endpoint URL; written to `options.baseURL`. Optional if supplied via `options` instead.

Type

`str` | `None`

api_key

Credential; written to `options.apiKey`. May be a literal secret or an opencode `{env:NAME}` template that opencode expands at runtime (preferred — keeps the secret out of the on-disk config file).

Type

`str` | `None`

options

Extra SDK options merged into the provider's `options` block (e.g. `headers`, custom timeouts). On conflict these win over `base_url` / `api_key`.

Type

`Dict[str, object]` | `None`

to_config_entry() → `dict`

Render this provider as an opencode `provider.<id>` config value.

```
class chia.models.opencode.OpenCodeQueryResult(result: str, returncode: int, stderr: str, stream_result:
                                                str, success: bool = False, usage: dict | None = None,
                                                session_id: str | None = None)
```

Bases: [`QueryResult`](#)

`QueryResult` specialised for the opencode CLI.

`usage` holds the run's summed metrics from `opencode export` (the same dict pushed to the profiler as `_last_metadata`): `input_tokens`, `output_tokens`, `reasoning_tokens`, `cache_read`, `cache_write`, `cost_usd` (from opencode's model price table) and `num_turns`. `session_id` is opencode's id for the session the run created.

```
class chia.models.opencode.OpenCodeLLM(model: str | None = None, system_message: str = "",
                                         timeout_seconds: int = 600, retries: int = 3, logging_name: str =
                                         'opencode', logging_level: int = 10, log_dir: str | None = None,
                                         opencode_bin: str = 'opencode', agent_name: str = 'chia',
                                         work_dir: str | None = None, extra_cli_args: List[str] | None =
                                         None, additional_providers: List[AdditionalModelProvider] |
                                         None = None, dangerously_skip_permissions: bool = True,
                                         config: dict | None = None)
```

Bases: [`LLMCallBase`](#)

Wraps the opencode CLI as an LLM backend.

Each [`prompt\(\)`](#) call runs `opencode run` (to create a session and get its id) then `opencode export` (to read the assistant response + usage from opencode's local DB). Returns the same `QueryResult` shape as the other backends; `returncode` is the run exit code.

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` to opencode and return the response.

Returns

QueryResult with success=True when opencode ran cleanly, or success=False when every retry attempt failed.

Raises

- **RateLimitError** / **AuthenticationError** / **BillingError** / –
- *InvalidRequestError* – propagate immediately.
- *ServerError* – after all retries with exponential backoff.
- *MaxOutputTokensError* – after one retry attempt.

23.8 Openai Compat

OpenAI-compatible LLM backend built on the openai SDK Chat Completions API.

OpenAICompatLLM talks to **any** provider that implements the OpenAI Chat Completions wire format and drives the agentic tool loop client-side — executing each ChiaTool’s MCP server over HTTP, like the other backends.

It is deliberately *not* provider-specific. The OpenAI Chat Completions format is a de-facto multi-vendor standard, so the only provider-specific inputs are:

- `base_url` — selects the provider’s endpoint (default: OpenAI itself).
- `auth` — the credential for that endpoint.

The same loop/tool/parse/error code therefore covers OpenAI, Fireworks, Groq, OpenRouter, self-hosted vLLM/TGI, Vertex MaaS, etc. Each provider is configuration, not a new module.

Auth, by default, lives entirely in the environment (`OPENAI_API_KEY` / `OPENAI_BASE_URL`) — construct with no key and the SDK reads them, matching the other backends. For providers whose credential is a **rotating token** (Vertex MaaS GCP token, Azure AD), pass `token_provider` — a zero-arg callable that returns a fresh token; it’s invoked when the client is built.

WARNING: experimental. Only exercised by the tests in `chia/models/tests/test_openai_compat.py`. Not validated in production. `openai` is imported lazily, so importing this module does not require it.

```
exception chia.models.openai_compat.OpenAICompatError(node_id: str, error_type: str, status_code: int
                                                    | None = None, raw_message: str = "")
```

Bases: `Exception`

Base for all OpenAI-compatible backend errors.

```
exception chia.models.openai_compat.RateLimitError(node_id: str, reset_time: datetime, raw_message:
                                                    str = "", status_code: int | None = None)
```

Bases: *OpenAICompatError*

HTTP 429.

```
exception chia.models.openai_compat.AuthenticationError(node_id: str, status_code: int | None =
                                                         None, raw_message: str = "")
```

Bases: *OpenAICompatError*

HTTP 401 / 403.

```
exception chia.models.openai_compat.InvalidRequestError(node_id: str, status_code: int | None =
                                                         None, raw_message: str = "")
```

Bases: *OpenAICompatError*

HTTP 400 / 404.

exception `chia.models.openai_compat.ServerError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: [OpenAICompatError](#)

HTTP 5xx, connection, or timeout.

exception `chia.models.openai_compat.MaxOutputTokensError`(*node_id: str, status_code: int | None = None, raw_message: str = "", partial_text: str = ""*)

Bases: [OpenAICompatError](#)

The response was truncated at `max_tokens` (`finish_reason='length'`).

exception `chia.models.openai_compat.ContextLengthExceededError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: [InvalidRequestError](#)

HTTP 400 (or 413) whose body shows the prompt exceeded the context window.

Subclasses [InvalidRequestError](#) so it inherits never-retry semantics (re-sending the same oversized prompt cannot help) while letting callers detect the context-overflow case specifically. Distinct from [MaxOutputTokensError](#), which is about *output* truncation.

exception `chia.models.openai_compat.BillingError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: [OpenAICompatError](#)

Payment / quota problem (HTTP 402, or a 429 carrying `insufficient_quota`).

Never retried: spending more requests cannot restore quota or credit.

exception `chia.models.openai_compat.UnknownOpenAIError`(*node_id: str, status_code: int | None = None, raw_message: str = ""*)

Bases: [OpenAICompatError](#)

Unclassified OpenAI-compatible error.

class `chia.models.openai_compat.OpenAICompatLLM`(*model: str, system_message: str = "", timeout_seconds: int = 600, retries: int = 3, logging_name: str = 'openai_compat_llm', logging_level: int = 10, log_dir: str | None = None, base_url: str | None = None, api_key: str | None = None, token_provider: Callable[[], str] | None = None, max_tokens: int = 16000, max_tool_iterations: int = 100, client_kwargs: dict | None = None, dangerously_skip_permissions=<object object>, config=<object object>*)

Bases: [LLMCallBase](#)

OpenAI-compatible Chat Completions backend with client-side MCP tools.

Returns the same `QueryResult` shape as the other backends so callers are interchangeable; `returncode` is synthesised (0 on success, -1 when every retry fails) and `stderr` is unused.

prompt(*user_message: str, tools: List[ChiaTool] | None = []*) → [QueryResult](#)

Send *user_message* via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

23.9 Openai Providers

Lightweight per-provider presets over `OpenAICompatLLM`.

Each big OpenAI-compatible provider differs only in its endpoint (`base_url`), a default logging name, and the Ray resource its calls require. The providers are thin subclasses that set the first two as class defaults and **re-decorate** `prompt` with their own `@ChiaFunction(resources=...)` so that, when dispatched via `prompt.chia_remote(self, ...)`, each lands only on workers advertising that provider's credential resource. The decorated body just defers to `OpenAICompatLLM.prompt()`.

Auth is environment-driven (see `OpenAICompatLLM`). The SDK only reads `OPENAI_API_KEY` automatically — for non-OpenAI providers export the provider key as `OPENAI_API_KEY`, or pass `api_key=` / a `token_provider`.

```
class chia.models.openai_providers.OpenAILLM(model: str, **kwargs)
```

Bases: `_OpenAICompatProvider`

OpenAI itself (default endpoint).

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

```
class chia.models.openai_providers.FireworksLLM(model: str, **kwargs)
```

Bases: `_OpenAICompatProvider`

Fireworks AI.

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

```
class chia.models.openai_providers.GroqLLM(model: str, **kwargs)
```

Bases: `_OpenAICompatProvider`

Groq.

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

```
class chia.models.openai_providers.OpenRouterLLM(model: str, **kwargs)
```

Bases: `_OpenAICompatProvider`

OpenRouter (itself a multi-provider router).

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

```
class chia.models.openai_providers.NvidiaLLM(model: str, **kwargs)
```

Bases: `_OpenAICompatProvider`

NVIDIA's hosted API (build.nvidia.com / NIM).

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

23.10 Ollama

Ollama self-hosted LLM preset over `OpenAICompatLLM`.

`Ollama` serves open-weight models locally behind an **OpenAI-compatible** Chat Completions endpoint at `/v1`. From chia's point of view a self-hosted Ollama is therefore just another OpenAI-compatible provider: point `base_url` at the Ollama server and the entire `OpenAICompatLLM` loop/tool/error stack applies unchanged. This is the same pattern as the cloud presets in `chia.models.openai_providers`.

23.10.1 Auth

Ollama requires no credentials, but the `openai` SDK refuses to build a client with an empty `api_key`. We therefore default it to the conventional dummy value `"ollama"` (overridable via `api_key / token_provider / OPENAI_API_KEY`).

```
class chia.models.ollama.OllamaLLM(model: str, **kwargs)
```

Bases: `OpenAICompatLLM`

Self-hosted Ollama via its OpenAI-compatible `/v1` endpoint.

`model` is the Ollama model tag to serve, e.g. `"llama3.1:8b"` or `"qwen2.5:7b"` (it must already be pulled on the server — see the `OLLAMA_PULL` build/run knob in `dockerfiles/OllamaDockerfile`).

```
prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send `user_message` via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

23.11 vLLM

vLLM self-hosted LLM preset over `OpenAICompatLLM`.

`vLLM` serves open-weight models on GPUs behind an **OpenAI-compatible** Chat Completions endpoint (`vllm serve <model> → /v1` on port 8000). From chia's point of view a self-hosted vLLM is just another OpenAI-compatible provider: point `base_url` at the vLLM server and the whole `OpenAICompatLLM` loop/tool/error stack applies unchanged — the same pattern as `OllamaLLM`.

Default hosting to port 8200 (not vLLM's usual 8000): chia uses the low 8000s heavily — on workers using the SSH-tunnel fallback its SSH tunnels reserve 8000-8010 (`head_tool_port`), and ChiaTool MCP servers probe ports 8000-8099 (`start_router` base 8000 + 100 tries; Ray Serve's proxy also defaults to 8000). Since chia runs containers `--net=host`, vLLM on 8000 would collide. 8200 clears the tool range with margin and stays below the Ray dashboard (8265) and all tunnel/ray ranges.

23.11.1 One model per server

Unlike Ollama (which serves many models and switches per request), a vLLM server serves **exactly one model**, fixed when `vllm serve` is launched. So `model` here must match the model the target server was started with (its HF id, or the `--served-model-name` if one was set). Serving multiple models means running multiple vLLM servers/workers.

23.11.2 Auth

vLLM started without `--api-key` ignores the credential, but the `openai` SDK refuses to build a client with an empty `api_key`, so we default it to the dummy `"vllm"` (overridable via `api_key / token_provider / OPENAI_API_KEY` — set a real one if the server was started with `--api-key`).

class chia.models.vllm.VLLMLLM(model: str, **kwargs)

Bases: [OpenAICompatLLM](#)

Self-hosted vLLM via its OpenAI-compatible /v1 endpoint.

model is the model the target vLLM server is serving, e.g. "Qwen/Qwen2.5-3B-Instruct" (must match the server's launch model / --served-model-name — see the VLLM_MODEL knob in dockerfiles/VLLMDockerfile).

prompt(user_message: str, tools: List[[ChiaTool](#)] | None = []) → [QueryResult](#)

Send user_message via Chat Completions and return the response, retrying transient failures with the same policy the other backends use.

FIRESIM

API reference for `chia.firesim`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

The FireSim API is still under development and may change in future releases.

24.1 Config

FireSim-specific configuration dataclasses and YAML parsers.

```
class chia.firesim.config.FireSimBuildConfig(name: str, platform: str, target_project: str,  
                                             target_project_makefrag: str | None, design: str,  
                                             target_config: str, platform_config: str, fpga_frequency:  
float, build_strategy: str, bit_builder_recipe: str,  
                                             deploy_quintuplet: str | None = None, post_build_hook:  
str | None = None, metasim_customruntimeconfig: str |  
None = None, build_id: str | None = None, build_group:  
str | None = None, incremental_base_build_id: str | None  
= None, market: str = 'ondemand', enable_pr: bool =  
False, pr_module_name: str | None = None,  
pr_base_build_id: str | None = None, pr_partition_cell:  
str | None = None)
```

Bases: `object`

A single bitstream build recipe, parsed from `config_build_recipes.yaml`.

Mirrors one stanza of a FireSim `config_build_recipes.yaml` (the recipe name is the YAML key; the remaining fields come from that entry). Drives an FPGA bitstream build for a given target design + FPGA platform.

name

Recipe name (the YAML key); used as the build's base name (e.g. "midasexamples_gcd").

Type

str

platform

FireSim platform (e.g. "f1", "f2"); from the recipe's PLATFORM.

Type

str

target_project

Chipyard target project (TARGET_PROJECT, e.g. "firesim", or "midasexamples" for MIDAS example designs).

Type

str

target_project_makefrag

Optional makefrag override for the target project (TARGET_PROJECT_MAKEFRAG, e.g. "../.../generators/firechip/chip/src/main/makefrag/firesim"); None to use the default.

Type

str | None

design

Top-level design/harness module to build (DESIGN, e.g. "FireSim", or "GCD" for the MIDAS example).

Type

str

target_config

Chipyard target Scala config (TARGET_CONFIG) selecting the SoC configuration (e.g. "FireSimMegaBoomChiaBigCacheConfig", "FireSimRocketConfig").

Type

str

platform_config

FireSim platform config (PLATFORM_CONFIG) selecting host-side simulation options (e.g. "DefaultF2Config").

Type

str

fpga_frequency

Target FPGA clock frequency in MHz (from platform_config_args.fpga_frequency, e.g. 20).

Type

float

build_strategy

Vivado build strategy (from platform_config_args.build_strategy, e.g. "TIMING").

Type

str

bit_builder_recipe

Path to the FireSim bit-builder recipe YAML (bit_builder_recipe) selecting the platform's build flow (e.g. "bit-builder-recipes/f2.yaml").

Type

str

deploy_quintuplet

Optional override for the FireSim deploy quintuplet (PLATFORM-TARGET_PROJECT-DESIGN-TARGET_CONFIG-PLATFORM_CONFIG, e.g. "f2-firesim-FireSim-FireSimMegaBoomChiaBigCacheConfig-DefaultF2Config"); None to let FireSim derive it.

Type

str | None

post_build_hook

Optional path to a script run after the build completes.

Type

str | None

metasim_customruntimeconfig

Optional custom runtime config for metasim (software-simulation) builds.

Type

str | None

build_id

User-specified build ID (e.g. "megaboom-baseline"). If omitted, an auto-generated {name}-{timestamp}-{short_uuid} is used.

Type

str | None

build_group

Optional subdirectory under builds/ for organizing builds (e.g. builds/{build_group}/{build_id}/). If omitted, builds are stored flat under builds/{build_id}/.

Type

str | None

incremental_base_build_id

build_id of a previous build whose DCPs to use as Vivado incremental synthesis/implementation references (~50% faster); None for a full build.

Type

str | None

market

AWS instance market for the build host ("ondemand" or "spot").

Type

str

enable_pr

Whether to enable partial reconfiguration (PR) for this build.

Type

bool

pr_module_name

Reconfigurable module name when enable_pr is set.

Type

str | None

pr_base_build_id

build_id of the PR base build (its presence triggers the reconfigurable-module flow).

Type

str | None

pr_partition_cell

PR partition cell; auto-discovered if omitted.

Type

str | None

```
class chia.firesim.config.FireSimRunConfig(hw_config_name: str, build_ref: str | None = None, agfi: str | None = None, workload_name: str = "", num_sims: int = 1, instance_type: str = 'f2.12xlarge', plusarg_passthrough: str = "", terminate_on_completion: bool = True, driver_tarball_path: str | None = None, ami_id: str | None = None, market: str = 'ondemand', bootbinary_path: str | None = None, rootfs_path: str | None = None, result_id: str | None = None, driver_s3_path: str | None = None, bootbinary_s3_path: str | None = None, rootfs_s3_path: str | None = None)
```

Bases: object

Runtime configuration for an FPGA simulation run.

Describes a single FireSim run: which bitstream (AGFI) and simulation driver to deploy, which workload to run, and on what instances. The build artifacts (agfi, driver_s3_path) are typically auto-derived from build_ref via [resolve_build\(\)](#).

hw_config_name

Name of the hardware-database (hwdb) entry / hardware config to run (the config_hwdb.yaml key, e.g. "firesim_megaboom_chia_bigcache").

Type

str

build_ref

Build reference — path under builds/ on S3 identifying a build, of the form "{recipe}/{build_id}". Examples: "FireSimRocket-20260409-a1b2c3d4" (flat) or "experiments/FireSimRocket-20260409-a1b2c3d4" (grouped). When set, agfi and driver_s3_path are auto-derived via [resolve_build\(\)](#).

Type

str | None

agfi

AWS Global FPGA Image ID (e.g. "agfi-01234abcde") to program onto the FPGA; auto-derived from build_ref if unset.

Type

str | None

workload_name

Name of the workload to run (e.g. "spec17-intrate.json").

Type

str

num_sims

Number of parallel simulation slots to launch.

Type

int

instance_type

AWS instance type for the run host (e.g. "f2.12xlarge").

Type

str

plusarg_passthrough

Extra +plusarg flags passed through to the simulator.

Type

str

terminate_on_completion

Whether to terminate the run instance when the run finishes.

Type

bool

driver_tarball_path

Local path to the simulation driver tarball (legacy/fallback; prefer `driver_s3_path`).

Type

str | None

ami_id

Optional AMI ID for the run instance.

Type

str | None

market

AWS instance market for the run host ("ondemand" or "spot").

Type

str

bootbinary_path

Local path to the boot binary (legacy/fallback; prefer `bootbinary_s3_path`).

Type

str | None

rootfs_path

Local path to the root filesystem image (legacy/fallback; prefer `rootfs_s3_path`).

Type

str | None

result_id

Optional subdirectory under `results/` for organizing run results (e.g. `results/{result_id}/{suite_name}/`). If omitted, results are stored under `results/{suite_name}/{timestamp}/`.

Type

str | None

driver_s3_path

S3 URI of the simulation driver tarball (preferred — any worker node can use it), e.g. `"s3://firesim-chia-builds/builds/design/driver-bundle.tar.gz"`; auto-derived from `build_ref` when `resolve_build()` is called.

Type

str | None

bootbinary_s3_path

S3 URI of the boot binary (preferred over the local path).

Type

str | None

rootfs_s3_path

S3 URI of the root filesystem image (preferred over the local path).

Type

str | None

resolve_build(s3_bucket: str) → *FireSimRunConfig*

Populate agfi and driver_s3_path from build_ref if not already set.

Fetches build-info.json from S3 for the referenced build and uses it to fill in agfi and driver_s3_path. Explicit values for either field take precedence (they are not overwritten). A no-op if build_ref is unset or both derived fields are already populated.

Parameters

s3_bucket – S3 bucket name holding the build's build-info.json.

Returns

self, with agfi and driver_s3_path populated from the build referenced by build_ref where they were previously unset. Returned for chaining.

```
class chia.firesim.config.FireSimConfig(chipyard_path: str, deploy_path: str, build_recipes_file: str =
    'config_build_recipes.yaml')
```

Bases: object

Top-level FireSim configuration, parsed from the Chia cluster YAML.

chipyard_path

Filesystem path to the Chipyard checkout on the host.

Type

str

deploy_path

Filesystem path to the FireSim deploy directory (sim/firesim workspace where builds/runs are launched).

Type

str

build_recipes_file

Filename of the FireSim build-recipes YAML to load (defaults to "config_build_recipes.yaml").

Type

str

```
chia.firesim.config.load_build_recipes(yaml_path: str) → dict[str, FireSimBuildConfig]
```

Parse a config_build_recipes.yaml file into FireSimBuildConfig objects.

Parameters

yaml_path – Path to config_build_recipes.yaml.

Returns

Dict mapping recipe name to FireSimBuildConfig. Empty if the YAML is empty.

Raises

FileNotFoundError – If yaml_path does not exist.

`chia.firesim.config.load_hwdb(yaml_path: str) → dict[str, dict[str, Any]]`

Parse config_hwdb.yaml into a dict of hw config name -> properties.

Parameters

yaml_path – Path to config_hwdb.yaml.

Returns

Dict mapping hw config name to its hwdb properties (agfi, etc.); empty if the YAML does not parse to a mapping.

Raises

FileNotFoundError – If `yaml_path` does not exist.

`chia.firesim.config.fetch_build_info(s3_bucket: str, build_ref: str) → dict`

Fetch build-info.json from S3 for a build reference.

Parameters

- **s3_bucket** – S3 bucket name.
- **build_ref** – Path under builds/ identifying a build. Examples: “FireSimRocket-20260409-a1b2c3d4” (flat), “prefetcher-experiments/FireSimRocket-20260409-a1b2c3d4” (grouped), or “latest” / “mygroup/latest” for the most recent build.

Returns

agfi, driver_s3_path, build_id, etc.

Return type

Parsed build-info dict with keys

Raises

FileNotFoundError – If build-info.json does not exist at the expected path.

24.2 Chia Functions

@ChiaFunction wrappers for FireSim build and run operations.

These functions can be scheduled onto the firesim_manager node, or called directly for standalone use.

`chia.firesim.chia_functions.firesim_build_bitstream(build_recipe_name: str, aws_config_dict: dict, s3_bucket: str, source_overlay_dir: str | None = None, docker_image: str = 'chia-chisel-build', instance_type: str = 'z1d.2xlarge', build_recipes_file: str | None = None, build_config_dict: dict | None = None, results_dir: str | None = None, local_log_dir: str | None = None) → BitstreamBuildResult`

Build an FPGA bitstream using an ephemeral EC2 build host.

All build steps run on the EC2 instance — the head node does NOT need a chipyard installation. Chisel elaboration + driver build happen inside a Docker container (chia-chisel-build); Vivado runs natively on the host.

Parameters

- **build_recipe_name** – Name of the build recipe (used for tagging/logging).
- **aws_config_dict** – Dict representation of AWSConfig fields.
- **s3_bucket** – S3 bucket for AFI tarball storage.
- **source_overlay_dir** – Local dir with modified Chisel sources to overlay onto the container’s chipyard. If None, uses the Docker image as-is.

- **docker_image** – Docker image for Chisel elaboration.
- **instance_type** – EC2 instance type for the build host.
- **build_recipes_file** – Path to build recipes YAML (to look up recipe by name). Mutually exclusive with `build_config_dict`.
- **build_config_dict** – Dict representation of `FireSimBuildConfig` fields. If provided, used directly instead of looking up from recipes file.
- **results_dir** – Local directory to store build results. Optional.
- **local_log_dir** – Local directory to mirror the build host's logs into. Optional; if `None`, logs are not copied back to the head node.

Returns

`BitstreamBuildResult` with the AGFI/AFI (or raw bitstream path), the `hwdb_entry` and `build_ref` needed to launch runs, and the build log. On a recipe-resolution error (missing recipe, or neither `build_config_dict` nor `build_recipes_file` given) returns early with `success=False` and the reason in `build_log`.

```
chia.firesim.chia_functions.firesim_run_workload(run_config_dict: dict, aws_config_dict: dict,
                                                sim_timeout: int = 14400, results_dir: str | None =
                                                None, name: str = "", instance_prefix: str | None =
                                                None, terminate_on_failure: bool = True,
                                                local_log_dir: str | None = None, log_prefix: str =
                                                'firesim-run') → FireSimRunResult
```

Run an FPGA simulation workload using ephemeral F2 EC2 instances.

Parameters

- **run_config_dict** – Dict representation of `FireSimRunConfig` fields. If it carries a `build_ref`, the AGFI and driver S3 path are auto-derived from it via `resolve_build` against the AWS config's S3 bucket (falling back to "firesim-chia-builds").
- **aws_config_dict** – Dict representation of `AWSConfig` fields.
- **sim_timeout** – Maximum simulation runtime in seconds (default 14400 = 4h).
- **results_dir** – Local directory to store results. Optional.
- **name** – Name of the simulation instance. Optional.
- **instance_prefix** – Optional prefix applied to the launched EC2 instance name(s); `None` uses the runner's default naming.
- **terminate_on_failure** – If `False`, leave instances running when the simulation raises — for post-mortem SSH debugging.
- **local_log_dir** – Local directory to mirror the run's logs into. Optional.
- **log_prefix** – Filename/tag prefix for emitted log files (default "firesim-run").

Returns

`FireSimRunResult` with per-slot uartlogs, rootfs/sim outputs, the overall success status, and the wall-clock duration.

24.3 State Def

Result dataclasses for FireSim build and run operations.

```
class chia.firesim.state_def.BitstreamBuildResult(recipe_name: str, agfi: str | None, afi: str | None,
success: bool, build_log: str, hwdb_entry: str,
driver_s3_path: str | None = None, build_id: str =
", build_ref: str = ", bitstream_path: str | None =
None)
```

Bases: object

Result of a firesim_build_bitstream() FPGA bitstream build.

recipe_name

Name of the build recipe this result is for.

Type

str

agfi

AWS Global FPGA Image ID produced by the build (e.g. "agfi-..."); None on failure or for non-AGFI platforms (see bitstream_path).

Type

str | None

afi

AWS FPGA Image ID (the un-globalized "afi-..." handle); None when not produced.

Type

str | None

success

True iff the build completed and a usable image was produced.

Type

bool

build_log

Captured build log (also carries the failure reason on early recipe-resolution errors).

Type

str

hwdb_entry

The hardware-database YAML stanza for this bitstream, ready to paste/feed into a FireSim run config.

Type

str

driver_s3_path

S3 URI of the uploaded simulation driver tarball; None if not uploaded.

Type

str | None

build_id

Unique identifier assigned to this build.

Type

str

build_ref

"{recipe_name}/{build_id}" — the handle to pass as a run config's build_ref to auto-resolve the AGFI + driver for a run.

Type

str

bitstream_path

S3 URI of a raw .bit artifact, for platforms that don't produce an AGFI (e.g. `corigine_xb10`); None for AGFI-producing platforms (`f2/f1`).

Type

str | None

```
class chia.firesim.state_def.FireSimRunResult(workload_name: str, success: bool, uartlogs: dict[str, str]=<factory>, rootfs_outputs: dict[str, dict[str, str]]=<factory>, sim_outputs: dict[str, dict[str, str]]=<factory>, duration_seconds: float = 0.0)
```

Bases: object

Result of a `firesim_run_workload()` FPGA simulation run.

workload_name

Name of the workload that was run.

Type

str

success

True iff the simulation completed successfully.

Type

bool

uartlogs

Per-slot UART console output, mapping `slot_name` -> captured uartlog text.

Type

dict[str, str]

rootfs_outputs

Per-slot files copied back from each simulation's rootfs, mapping `slot_name` -> {`relative_filepath` -> `file_content`}.

Type

dict[str, dict[str, str]]

sim_outputs

Per-slot host-side simulation artifacts (memory_stats, autocounters, etc.), mapping `slot_name` -> {`relative_filepath` -> `file_content`}.

Type

dict[str, dict[str, str]]

duration_seconds

Wall-clock duration of the run, in seconds.

Type

float

API reference for `chia.vlsi`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

25.1 Hammer

`chia.vlsi.hammer` — Hammer-vlsi nodes.

`HammerNode.run()` wraps one `hammer-vlsi` CLI call. Hammer's CLI is uniform across actions (`syn`, `par`, `drc`, `lvs`, `sim`, `power`, and the `-to-` bridge actions): configs in via repeated `-p`, an `--obj_dir` for build output, an `-o` output config that feeds the next action, and the action name. One node therefore covers the whole flow, with the action as a parameter.

`obj_dir` is PATH-BASED: it lives on the worker that ran the action, so chained actions (`syn -> syn-to-par -> par`) and report fetches (`HammerNode.collect()`) must land on the SAME worker. `HammerNode` enforces that via a placement group (see `chia.base.colocated.ColocatedNode` for the given / reserved / no-PG construction modes). `HammerNode.<fn>.chia_remote(...)` (the class attribute) is the raw, unpinned form for callers that handle placement themselves.

This module knows nothing about technologies, tools, or sites — all of that arrives via the project's configs.

```
class chia.vlsi.hammer.HammerResult(success: bool, returncode: int, action: str, obj_dir: str, stdout: str,
                                     stderr: str, output: dict = <factory>, listing: dict[str,
                                     int]=<factory>)
```

Bases: `object`

```
class chia.vlsi.hammer.HammerCollectResult(obj_dir: str, files: dict[str, str], skipped: dict[str, int],
                                           listing: dict[str, int])
```

Bases: `object`

```
class chia.vlsi.hammer.HammerMatchResult(obj_dir: str, matches: list[tuple[str, int]], skipped: dict[str,
int])
```

Bases: `object`

```
class chia.vlsi.hammer.HammerCollectFsResult(obj_dir: str, dest_dir: str, copied: dict[str, int], skipped:
dict[str, int])
```

Bases: `object`

```
class chia.vlsi.hammer.HammerNode(placement_group=None, require_colocated: bool = True, *,
                                  bundle_index: int = 0, reserve_bundle: dict | None = None, pg_strategy:
                                  str = 'STRICT_PACK', wait_for_pg: bool = True, pg_ready_timeout_s:
                                  float | None = None)
```

Bases: `ColocatedNode`

Hammer run / collect primitives sharing one placement.

The members are `@staticmethod @ChiaFunction(resources={"hammer": 1}); __init__` re-binds each into a per-instance pinned form so `node.<fn>.chia_remote(...)` lands on this node's bundle:

```
with HammerNode() as node:    # reserves a {"CPU": 1, "hammer": 1} PG
    syn = get(node.run.chia_remote(
        "syn", configs=[...], obj_dir="/scratch/build/run1"))
    rpts = get(node.collect.chia_remote(
        syn.obj_dir, ["syn-rundir/reports/**"]))
    par = get(node.run.chia_remote("syn-to-par", ...)) # same worker
```

`collect` requires the default bundle's hammer slot, so a collect dispatched while a run executes on this bundle waits for it in the default case — it cannot read half-written reports.

Set up placement and bind the member functions.

Parameters

- **placement_group** – an existing Ray PlacementGroup to schedule onto. If given, `require_colocated` is moot (placement is already fixed) and this node will not remove the PG on close.
- **require_colocated** – when no PG is given, reserve one so all members co-locate. When False, leave placement to the caller.
- **bundle_index** – which bundle of the (given or reserved) PG to pin to.
- **reserve_bundle** – resource shape of a self-reserved bundle (default `_DEFAULT_BUNDLE`). A bundle too small for some members is allowed — those members just can't dispatch through this node (a construction-time warning lists them).
- **pg_strategy** – placement strategy for a self-reserved PG.
- **wait_for_pg** – block on `pg.ready()` for a self-reserved PG so the node is usable immediately.
- **pg_ready_timeout_s** – optional timeout for that wait.

```
static run(action: str, configs: list[str] | None = None, config_contents: dict[str, str] | None = None,
           obj_dir: str = 'build', extra_args: list[str] | None = None, hammer_bin: str = 'hammer-vlsi',
           timeout_seconds: int = 86400) → HammerResult
```

Run one `hammer-vlsi` action on a worker.

Parameters

- **action** – Any action `hammer-vlsi` accepts: “syn”, “par”, “syn-to-par”, ...
- **configs** – Paths to config YAML/JSON files that exist *on the worker* (baked into the image or mounted), passed as `-p` in order (later files override earlier ones).
- **config_contents** – filename -> YAML/JSON text, written into `obj_dir/configs/` on the worker and appended as `-p` after `configs`. This is how a flow ships configs (or a previous action's output) by value to a remote worker.
- **obj_dir** – Hammer build directory on the worker.
- **extra_args** – Extra CLI args, inserted before the action.
- **hammer_bin** – The hammer executable, or a custom CLIDriver script.
- **timeout_seconds** – Wall-clock limit for the subprocess.

static collect(*obj_dir*: str, *patterns*: list[str], *max_bytes_per_file*: int | None = None) → *HammerCollectResult*

Fetch text files from a previous action's *obj_dir* on this worker.

Dispatch via the pinned instance member (`node.collect.chia_remote`) so it lands on the worker that owns *obj_dir* — an unpinned call may not.

Parameters

- **obj_dir** – The build directory a previous action ran in.
- **patterns** – Globbs relative to *obj_dir* (** is recursive), e.g. ["syn-rundir/reports/**", "syn-rundir/*.log"]. Files matched by multiple patterns appear once.
- **max_bytes_per_file** – When set, files over this size are recorded in `skipped` instead of shipped through the object store — protects against a glob accidentally matching a netlist. None (and 0, the falsy edge) means no cap: everything matched is shipped.

static list_matches(*obj_dir*: str, *patterns*: list[str], *max_bytes_per_file*: int | None = None) → *HammerMatchResult*

Resolve *patterns* against *obj_dir* on the worker and return the manifest of matching files (relpath + size) without reading contents.

The planning half of `collect_fs()`: it tells the caller what to stream and how big each file is. Same glob/dedup/cap rules as `collect()`.

static read_chunk(*obj_dir*: str, *rel*: str, *offset*: int, *length*: int) → bytes

Read *length* bytes at *offset* from *obj_dir/rel* on the worker. The transfer primitive behind `collect_fs()`; returns b"" at or past EOF. *rel* is confined to *obj_dir*.

collect_fs(*obj_dir*: str, *patterns*: list[str], *dest_dir*: str, *max_bytes_per_file*: int | None = None, *chunk_bytes*: int = 16777216) → *HammerCollectFsResult*

Stream matching files from a previous action's *obj_dir* onto the filesystem of THIS (the calling) process, a chunk at a time.

Unlike `collect()`, which returns every file's contents in one object-store payload, this writes each file to *dest_dir* on the caller's local disk incrementally — peak memory is ~`chunk\_bytes`, not the size of the whole collection. Use it to pull large report trees / gate-level netlists back from the worker to a machine that does NOT share a filesystem with it.

This is a caller-side orchestrator, not a @ChiaFunction: it runs wherever you call it and writes to that machine's disk, pulling bytes from the *obj_dir* worker via the node's pinned `list_matches()` / `read_chunk()` members. It therefore needs a placement group so both members hit the one worker that owns *obj_dir* — construct the node with `require_colocated=True` or pass `placement_group=...`

Files keep their path relative to *obj_dir*: a match at *obj_dir/syn-rundir/reports/x.rpt* lands at *dest_dir/syn-rundir/reports/x.rpt*.

Parameters

- **obj_dir** – The build directory a previous action ran in (on the worker).
- **patterns** – Globbs relative to *obj_dir* (** recursive); same matching/dedup/cap rules as `collect()`.
- **dest_dir** – Destination directory on the calling machine; created as needed.
- **max_bytes_per_file** – When set, files over this size are recorded in `skipped` and not streamed. None (and 0) means no cap.
- **chunk_bytes** – Bytes per worker read — the memory bound per file.

25.2 SRAM characterization (CACTI)

25.2.1 CACTI Runner

Methods to run CACTI 7, and return characterization results.

```
class chia.vlsi.sram_cacti.cacti_runner.CACTIResult(access_time_ns: float, cycle_time_ns: float,  
                                                    read_energy_nj: float, leakage_power_mw:  
                                                    float, height_mm: float, width_mm: float,  
                                                    area_um2: float, full_out: str)
```

Bases: object

```
chia.vlsi.sram_cacti.cacti_runner.run_cacti(spec: SRAMSpec, technology_um: float = 0.13,  
                                             cacti_path: str = 'cacti') → CACTIResult | None
```

Run CACTI for a single SRAM spec and return parsed results.

Returns None if CACTI fails.

```
chia.vlsi.sram_cacti.cacti_runner.analytical_area_estimate(spec: SRAMSpec,  
                                                           cell_area_per_bit_um2: float = 1.0) →  
                                                           CACTIResult
```

Fallback area estimate when CACTI fails.

Uses a simple model: $\text{area} = \text{depth} * \text{width} * \text{cell_area_per_bit}$. Timing is estimated from simple RC scaling.

25.2.2 CACTI Macrocompiler

CACTI-specific adapter over the generic Chipyard MacroCompiler glue.

The generic MacroCompiler support — MDF library generation and the `tapeout.jar` remap step — lives in `chia.chipyard.macrocompiler`. This module just wraps it with the `cacti_` cell-name prefix used by the CACTI characterization flow.

```
chia.vlsi.sram_cacti.cacti_macrocompiler.generate_cacti_macrocompiler_lib(sram_specs:  
                                                                            list[SRAMSpec],  
                                                                            cell_prefix: str =  
                                                                            'cacti_') → str
```

Generate the MacroCompiler MDF library for CACTI-characterized SRAMs.

Thin wrapper over `generate_macrocompiler_lib()` that defaults the cell-name prefix to `cacti_`.

25.2.3 SRAM Characterize

Top-level orchestration: characterize SRAMs with CACTI and generate Liberty files.

Parses `.top.mems.conf` from `generated_src`, runs CACTI for large SRAMs, generates Liberty files, and optionally runs MacroCompiler to remap `synflop_ext` modules to CACTI-characterized library macros.

```
class chia.vlsi.sram_cacti.sram_characterize.CharacterizedSRAM(name: str, spec: SRAMSpec,  
                                                                result: CACTIResult,  
                                                                lib_contents: dict[str, str],  
                                                                lef_content: str)
```

Bases: object

A single CACTI-characterized SRAM and its generated collateral.

`to_lib_dict()` → dict

Legacy dict shape.

```
class chia.vlsi.sram_cacti.sram_characterize.CactiCharacterization(generated_src_files:
    list[tuple[str, str]], srams:
    list[~chia.vlsi.sram_cacti.sram_characterize.Ch
```

= <factory>)

Bases: object

Result of `characterize_top_mems_conf_with_cacti()`.

The `sram_libs/` `sram_names` views are derived from `srams` so the existing dict-based consumers keep working, while `srams` exposes the full per-SRAM CACTI result.

```
chia.vlsi.sram_cacti.sram_characterize.characterize_top_mems_conf_with_cacti(generated_src_files:
    list[tuple[str,
    str]],
    cacti_path: str,
    syn-
flop_threshold_bytes:
    int = 256,
    technology_um:
    float = 0.13,
    cell_prefix: str
    = 'cacti_',
    corners: Sequence[LibertyCorner]
    = (Liberty-
    Corner(name='ff',
    temperature=-
    40,
    voltage=1.95,
    pvt_name='PVT_1P95V_-
    40C',
    lib_suffix='ff_n40C_1v95'),
    Liberty-
    Corner(name='ss',
    tempera-
    ture=100,
    voltage=1.6,
    pvt_name='PVT_1P6V_100C',
    lib_suffix='ss_100C_1v60'),
    Liberty-
    Corner(name='tt',
    tempera-
    ture=25,
    voltage=1.8,
    pvt_name='PVT_1P8V_25C',
    lib_suffix='tt_025C_1v80'))
    → CactiChar-
    acterization
```

Firtool specific characterize SRAMs with CACTI and generate Liberty files.

Finds `.top.mems.conf` in `generated_src_files`, runs CACTI for each large SRAM, generates `.lib` files, and replaces `.top.mems.v` to remove synflop definitions for SRAMs that become library cells.

Small SRAMs (< `synflop_threshold_bytes`) keep their synflop implementations and are synthesized as flip-flop arrays by Genus.

Parameters

- **generated_src_files** – List of (filename, contents) from the build.
- **cacti_path** – Path to the CACTI binary.
- **synflop_threshold_bytes** – SRAMs smaller than this stay as synflops.
- **technology_um** – Technology node in microns (default 0.130 for Sky130).
- **cell_prefix** – Prefix on the Liberty/LEF cell name. Default `cacti_` disambiguates from chipyard's `.top.mems.v` synflop wrapper for the MacroCompiler remap step. Pass `""` when the caller's Verilog refers to the bare `_ext` module directly (e.g. `firtool -repl-seq-mem`), so Genus can link the `extmodule` reference to the Liberty cell without an intervening rewrite.
- **corners** – Liberty corners to characterize each SRAM at. Defaults to `SKY130_CORNERS` (`ff/ss/tt`). Each corner produces one entry in the per-SRAM `lib_contents` dict, keyed by its `lib_suffix`.

Returns

- `generated_src_files`: the (unmodified) input files
- `srams`: a `CharacterizedSRAM` per large SRAM (name, spec, CACTI result, per-corner Liberty contents, and LEF content)

It also exposes `.sram_libs` (legacy list-of-dicts) and `.sram_names` views.

Return type

A `CactiCharacterization` with

```
chia.vlsi.sram_cacti.sram_characterize.characterize_srams_with_cacti(sram_specs:
    list[SRAMSpec],
    generated_src_files:
    list[tuple[str, str]],
    cacti_path: str,
    synflop_threshold_bytes:
    int = 256, technology_um:
    float = 0.13, cell_prefix:
    str = 'cacti_', corners:
    Sequence[LibertyCorner]
    =
    (LibertyCorner(name='ff',
    temperature=-40,
    voltage=1.95,
    pvt_name='PVT_1P95V_-
    40C',
    lib_suffix='ff_n40C_1v95'),
    LibertyCorner(name='ss',
    temperature=100,
    voltage=1.6,
    pvt_name='PVT_1P6V_100C',
    lib_suffix='ss_100C_1v60'),
    LibertyCorner(name='tt',
    temperature=25,
    voltage=1.8,
    pvt_name='PVT_1P8V_25C',
    lib_suffix='tt_025C_1v80'))
    → CactiCharacterization
```

Characterize SRAM specs with CACTI and generate Liberty/LEF.

Splits `sram_specs` into tiny SRAMs (kept as synflops, below `synflop_threshold_bytes`) and large SRAMs (CACTI-characterized), runs CACTI for the large ones, and returns a `CactiCharacterization`. `generated_src_files` is passed through into the result unchanged.

See `characterize_top_mems_conf_with_cacti` for the variant that first parses the specs out of a `.top.mems.conf` in `generated_src_files`.

25.2.4 Liberty Gen

Generate minimal Liberty (.lib) files for SRAM macros from CACTI results.

Produces Liberty files with area, timing arcs, and pin definitions that match the Chisel-generated `_ext` module port names. Uses scalar delay model (no NLDM tables) — sufficient for synthesis area estimation and basic timing.

```
class chia.vlsi.sram_cacti.liberty_gen.LibertyCorner(name: str, temperature: int, voltage: float,  
                                                  pvt_name: str, lib_suffix: str)
```

Bases: object

```
chia.vlsi.sram_cacti.liberty_gen.generate_liberty(spec: SRAMSpec, result: CACTIResult, corner:  
                                                LibertyCorner | None = None, cell_prefix: str =  
                                                'cacti_') → str
```

Generate a Liberty .lib file for an SRAM macro at a specific corner.

The cell name and pin names match the Chisel-generated `_ext` module so Genus can use this as a drop-in library cell.

If corner is None, defaults to typical (tt_025C_1v80).

`cell_prefix` defaults to `cacti_` so the Liberty cell name doesn't collide with the synflop wrapper named `<spec.name>` in chipyard's `.top.mems.v` (MacroCompiler then rewrites synflop refs to point at `cacti_<name>`). Pass "" when the upstream Verilog references `<spec.name>` directly as an extmodule (e.g. firtool -repl-seq-mem) so the Liberty cell name lines up.

25.2.5 LEF Gen

Generate minimal LEF (.lef) files for SRAM macros from CACTI results.

Produces LEF files with macro dimensions, pin shapes, and obstruction layers matching the CACTI-characterized SRAM area. Pin names match the Liberty cell and Verilog blackbox stubs (`cacti_<name>`).

LEF is geometry-only (no timing) so ONE file is generated per SRAM, not per corner like Liberty.

Uses CLASS BLOCK BLACKBOX per LEF 5.7 spec – designed for blocks that contain a SIZE statement estimating total area with approximate pin locations.

```
chia.vlsi.sram_cacti.lef_gen.generate_lef(spec: SRAMSpec, result: CACTIResult, pin_layer: str =  
                                         'met4', obs_layers: list[str] | None = None, cell_prefix: str =  
                                         'cacti_') → str
```

Generate a minimal LEF file for an SRAM macro.

Parameters

- **spec** – SRAM specification (pin names, depth, width, ports).
- **result** – CACTI characterization result (physical dimensions).
- **pin_layer** – Metal layer name for pin geometry (default “met4” for Sky130).
- **obs_layers** – Metal layers for obstruction block. Defaults to [“met1”, “met2”, “met3”] for Sky130.

- **cell_prefix** – Prefix on the LEF MACRO name (must match Liberty cell name). See `generate_liberty` for the disambiguation rationale.

Returns

Complete LEF file content as a string.

DATABASES

API reference for `chia.database`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

26.1 Base

`chia.database.base` — shared machinery and contract for database nodes.

DatabaseNode is the abstract base for engine-specific database nodes (*SQLiteNode*, *PostgresNode*). It owns everything that is engine-agnostic:

- the three placement modes (placement group via *ColocatedNode*, or a hard NodeAffinity pin via `node_id= / pin_to_current_node=True`),
- the `_DbBoundChiaFn` re-binding loop that injects the node's locator (a file path, a DSN, ...) and default `connect_opts` into every member call,
- spawned-tool tracking and the tools-then-placement-group `close()`.

It also declares each common member as an abstract staticmethod, so the family is polymorphic: *DatabaseNode* cannot be instantiated, an engine subclass missing a member fails at construction (not at first dispatch), and engine-blind code can be written against the base:

```
def record(db: DatabaseNode, sql: str, params: tuple) -> None:
    get(db.execute.chia_remote(sql, params))
```

The subclasses override every stub with a real `@ChiaFunction`; the stubs carry the canonical signatures and semantics.

Portability caveat: the members share names, signatures, result types (*ExecResult*, `list[dict]`), and *DatabaseNode.transaction()*'s params-type dispatch — but SQL *text* is dialect-specific. At minimum the parameter placeholders differ (see each subclass's `paramstyle`: sqlite is `qmark` — `? / :name` — and postgres is `format` — `%s / %(name)s`), and DDL idioms often do too. Engine-blind code must either render SQL per `db.paramstyle` or stick to statements valid in both dialects.

Contract notes the subclasses follow (asserted by their tests, not by the base): write members are declared `max_retries=0` (Ray task replay would double-apply a committed-but-unreturned non-idempotent write), and read members open their connection with an engine-appropriate read-only guard.

```
class chia.database.base.ExecResult(rowcount: int, lastrowid: int | None = None, rows: list[dict] | None = None)
```

Bases: `object`

Outcome of one write statement, uniform across engines.

`rowcount` is the DBAPI `cursor.rowcount` (the total for executemany). `lastrowid` is sqlite-only (None for executemany and non-INSERT statements; always None on postgres — use `RETURNING` and `read rows` instead). `rows` carries fetched result rows when the statement returned any (e.g. postgres `INSERT ... RETURNING id`).

```
class chia.database.base.DatabaseNode(locator: str, placement_group=None, require_colocated: bool =
    True, *, node_id: str | None = None, pin_to_current_node: bool =
    False, bundle_index: int = 0, reserve_bundle: dict | None = None,
    pg_strategy: str = 'STRICT_PACK', wait_for_pg: bool = True,
    pg_ready_timeout_s: float | None = None, connect_opts: dict |
    None = None)
```

Bases: [ColocatedNode](#), ABC

Abstract base for engine-specific database nodes.

Subclasses must define:

- every abstract member below as a `@staticmethod @ChiaFunction` taking the engine’s locator as the explicit first argument,
- `paramstyle` — the PEP 249 placeholder style of the engine’s SQL,
- `spawn_query_tool` — the engine’s LLM-facing MCP tool factory,

and may extend `_MEMBER_FNS` with engine-specific members (e.g. `SQLiteNode` appends `wal_checkpoint`); the binding loop picks the extras up automatically.

Set up placement and bind the member functions.

Parameters

- **locator** — the engine-specific database address (an absolute file path for sqlite, a DSN/conninfo string for postgres), already validated/normalized by the subclass.
- / (`placement_group` / `require_colocated` / `bundle_index`) — `reserve_bundle` / `pg_strategy` / `wait_for_pg` / `pg_ready_timeout_s`: as in `ColocatedNode`.
- **node_id** — pin every member to this Ray node via a hard `NodeAffinitySchedulingStrategy` (`soft=False`) instead of a placement group — for state that already lives on a known machine. Mutually exclusive with `placement_group`.
- **pin_to_current_node** — convenience for `node_id=<caller's node>`. Requires an initialized Ray context.
- **connect_opts** — engine-specific connection defaults injected into every member call; per-call `connect_opts=` overrides.

paramstyle: `str = ''`

PEP 249 paramstyle of the SQL this node’s members expect (e.g. “qmark” for sqlite, “format” for postgres).

close() → None

Stop spawned tools, then release the PG iff this node reserved it.

Tools are stopped BEFORE the PG is removed — a tool’s server actor may be pinned to this PG, so tearing the PG down first would orphan it. Idempotent.

abstractmethod static init_schema(*locator: str, script: str, *, connect_opts: dict | None = None*) → None

Create the database (if creatable) and run a multi-statement DDL script. Idempotent `CREATE ... IF NOT EXISTS` scripts encouraged.

abstractmethod static executescript(locator: str, script: str, *, connect_opts: dict | None = None) → None

Run a multi-statement SQL script (no parameters).

abstractmethod static add_column_if_missing(locator: str, table: str, column: str, type_decl: str, *, connect_opts: dict | None = None) → bool

ALTER TABLE ... ADD COLUMN iff absent; True iff added. Identifiers are regex-validated before any SQL interpolation.

abstractmethod static schema(locator: str, *, connect_opts: dict | None = None) → str

Human-readable rendering of the current tables/indexes.

abstractmethod static execute(locator: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None) → [ExecResult](#)

Run one statement in its own committed transaction. Declared with `max_retries=0` (non-idempotent-replay guard).

abstractmethod static executemany(locator: str, sql: str, seq_of_params: list, *, connect_opts: dict | None = None) → [ExecResult](#)

DBAPI executemany in one committed transaction; `rowcount` is the total across all parameter sets.

abstractmethod static transaction(locator: str, ops: list, *, connect_opts: dict | None = None) → list[[ExecResult](#)]

Run ops atomically — all or nothing, in one transaction.

Each op is (sql, params) where the params TYPE selects the form: tuple/dict -> execute, list -> executemany, None -> bare execute. Any error rolls the whole batch back and re-raises.

abstractmethod static query(locator: str, sql: str, params: tuple | dict = (), *, limit: int | None = None, connect_opts: dict | None = None) → list[dict]

Rows as list[dict]; limit=N caps the fetch. Opened with the engine's read-only guard, so accidental writes raise.

abstractmethod static query_one(locator: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None) → dict | None

First row as a dict, or None.

abstractmethod static query_value(locator: str, sql: str, params: tuple | dict = (), *, default: Any = None, connect_opts: dict | None = None) → Any

First column of the first row, or default when there is no row. (An aggregate over an empty table yields a NULL row, returned as None, not default.)

abstractmethod spawn_query_tool(name: str, *, read_write: bool = False, **tool_kwargs)

Create this engine's LLM-facing MCP query tool (read-only SQL; opt-in single-statement writes with `read_write=True`). The returned tool is tracked and stopped by `close()`.

26.2 Postgres Node

chia.database.postgres_node — generic PostgreSQL client node for chia loops.

SQL placeholders are psycopg's `paramstyle = "format"` (%s positional, %(name)s named) — SQL written for SQLiteNode (? / :name) is NOT portable verbatim.

Placement: Colocation is NOT required for correctness — any worker can reach the server over TCP — so `require_colocated` defaults to False (members dispatch unpinned). The placement modes are still available for latency or topology reasons (`placement_group=`, `node_id=`, `pin_to_current_node=True`).

Server: this node does not manage one. The easy path:

```
docker run -d --name chia-pg -p 5432:5432 \
  -e POSTGRES_PASSWORD=chia postgres:16
db = PostgresNode("postgresql://postgres:chia@<host>:5432/postgres")
```

A managed PostgresServerNode(CollocatedNode) (initdb / pg_ctl with the data dir on one worker's local disk — where colocation genuinely matters for postgres) is deliberately deferred: it is blocked on distributing postgres binaries to worker images, and running the official docker image on the target host is usually the better answer.

Driver: psycopg (v3) via the optional extra `pip install 'chia[postgres]'`. All psycopg imports are lazy (inside function bodies), so importing this module — and constructing a node from a string DSN — works without the driver; only executing members (locally or on a worker) needs it.

Concurrency model: each `chia_remote` call opens a fresh connection. Defaults: `autocommit=True` with explicit `conn.transaction()` blocks for writes, `lock_timeout = 30s`, 30 s connect timeout. Override per node or per call via `connect_opts` (keys: `connect_timeout_s`, `lock_timeout_ms`, `statement_timeout_ms`, `application_name`). Read members run with `default_transaction_read_only = on` — a guard against accidental writes, not an adversarial boundary.

Write semantics: write members are declared `max_retries=0` (Ray task replay would double-apply a committed-but-unreturned non-idempotent write). `lastrowid` has no meaning on postgres — use `INSERT ... RETURNING id` and read `ExecResult.rows`:

```
res = get(db.execute.chia_remote(
    "INSERT INTO t (v) VALUES (%s) RETURNING id", ("x",)))
new_id = res.rows[0]["id"]
```

LLM tool read-only layering: `PostgresNode.spawn_query_tool()` connects with `default_transaction_read_only=on` by default - fine for cooperative LLMs. The real boundary is credentials: pass a GRANT SELECT-only role's DSN via `spawn_query_tool(..., dsn=readonly_dsn)`.

```
class chia.database.postgres_node.PostgresNode(dsn: str | dict, placement_group=None,
    require_collocated: bool = False, *, node_id: str | None
    = None, pin_to_current_node: bool = False,
    bundle_index: int = 0, reserve_bundle: dict | None =
    None, pg_strategy: str = 'STRICT_PACK',
    wait_for_pg: bool = True, pg_ready_timeout_s: float |
    None = None, connect_opts: dict | None = None)
```

Bases: [DatabaseNode](#)

Generic PostgreSQL client node over one server/database (by DSN).

All members are `@staticmethod @ChiaFunction` taking `dsn` as the explicit first argument; `__init__` rebinds them so instance calls inject `self.dsn` (and the node's `connect_opts`) automatically:

```
db = PostgresNode("postgresql://postgres:chia@dbhost:5432/postgres")
get(db.init_schema.chia_remote("CREATE TABLE IF NOT EXISTS t ..."))
get(db.execute.chia_remote("INSERT INTO t (v) VALUES (%s)", ("x",)))
rows = get(db.query.chia_remote("SELECT * FROM t"))
```

See the module docstring for the server story, paramstyle, and the read-only layering of the LLM tool.

Normalize dsn and defer to DatabaseNode.

Parameters

- **dsn** – libpq conninfo string (`postgresql://user:pw@host/db` or `host=... dbname=...`) or a dict of conninfo parts (dict form requires psycopg locally).

- **require_collocated** – defaults to False — postgres clients reach the server from any worker, so members dispatch unpinned unless a placement mode is requested.
- **connect_opts** – connection defaults injected into every member call (keys: `connect_timeout_s`, `lock_timeout_ms`, `statement_timeout_ms`, `application_name`); per-call `connect_opts=` overrides.

Remaining args are as in `DatabaseNode`.

paramstyle: `str = 'format'`

PEP 249 paramstyle of the SQL this node's members expect (e.g. "qmark" for sqlite, "format" for postgres).

spawn_query_tool(*name: str, *, read_write: bool = False, dsn: str | None = None, **tool_kwargs*) → *PostgresQueryTool*

Create a *PostgresQueryTool* for this node's database.

No placement is required — any worker can reach the server — so this works on an unpinned node (the tool actor is then scheduled wherever Ray likes). Pass `dsn=` to give the tool different credentials (e.g. a GRANT SELECT-only role — the real read-only boundary). The returned tool is tracked and stopped by `close()`. `tool_kwargs` forward to *PostgresQueryTool* (e.g. `row_limit`, `cell_char_limit`, `total_char_limit`, `schemas`).

static init_schema(*dsn: str, script: str, *, connect_opts: dict | None = None*) → None

Run a multi-statement DDL `script` in one transaction. Idempotent CREATE ... IF NOT EXISTS scripts are encouraged. (psycopg runs multi-statement strings only when no parameters are passed — which is the case here.)

static executescript(*dsn: str, script: str, *, connect_opts: dict | None = None*) → None

Alias-level sibling of *init_schema()* (postgres has no file/dirs to create, so the two are identical here; both exist to satisfy the `DatabaseNode` contract).

static add_column_if_missing(*dsn: str, table: str, column: str, type_decl: str, *, connect_opts: dict | None = None*) → bool

ALTER TABLE ... ADD COLUMN IF NOT EXISTS ; True iff added.

`table` may be schema-qualified (`myschema.t`; default schema `public`). Identifiers and `type_decl` are regex-validated before any SQL interpolation (and before the psycopg import, so validation errors don't depend on the driver).

static schema(*dsn: str, *, schemas: tuple = ('public',), connect_opts: dict | None = None*) → str

Human-readable table/index listing from `information_schema` and `pg_indexes` for the given schemas. A simple rendering, not `pg_dump` fidelity.

static execute(*dsn: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None*) → *ExecResult*

Run one statement in its own committed transaction. When the statement returns rows (RETURNING, or a SELECT), they are attached as `ExecResult.rows`

static executemany(*dsn: str, sql: str, seq_of_params: list, *, connect_opts: dict | None = None*) → *ExecResult*

`executemany` in one committed transaction; `rowcount` is the total across all parameter sets.

static transaction(*dsn: str, ops: list, *, connect_opts: dict | None = None*) → list[*ExecResult*]

Run `ops` atomically in one transaction.

Each `op` is (`sql`, `params`) where the `params TYPE` selects the form:

- tuple or dict -> `execute(sql, params)` (single statement)
- list -> `executemany(sql, params)` (bulk rows)

- None -> `execute(sql)` (no params)

NOTE: single-statement params MUST be a tuple/dict — a list means `executemany`. Any error rolls the whole batch back and re-raises (surfacing as a Ray task error at `get()`). Explicit `BEGIN/COMMIT` inside op SQL is unsupported. Returns one `ExecResult` per op (rows attached for ops that return them).

static query(*dsn: str, sql: str, params: tuple | dict = (), *, limit: int | None = None, connect_opts: dict | None = None*) → list[dict]

Rows as list[dict]. `limit=N` caps the fetch (protects the object store from unbounded SELECTs); default None fetches all. The session runs with `default_transaction_read_only = on`, so an accidental write raises `ReadOnlySqlTransaction`.

static query_one(*dsn: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None*) → dict | None

First row as a dict, or None.

static query_value(*dsn: str, sql: str, params: tuple | dict = (), *, default: Any = None, connect_opts: dict | None = None*) → Any

First column of the first row, or `default` when there is no row. (An aggregate over an empty table yields a NULL row, returned as None, not `default`.)

class `chia.database.postgres_node.PostgresQueryTool`(*name: str, dsn: str, task_options: dict | None = None, row_limit: int = 100, cell_char_limit: int = 4096, total_char_limit: int = 32768, read_write: bool = False, schemas: tuple = ('public',)*)

Bases: [ChiaTool](#)

MCP tool exposing SQL over a PostgreSQL database

`query/schema` connect with `default_transaction_read_only=on` in the session options — a weak guard against LLM-authored writes (SQL containing `SET default_transaction_read_only = off` flips it). For an adversarial boundary, construct the tool with a `GRANT SELECT-only` role's DSN. Errors are caught and returned as text; row and byte caps keep the tool result bounded even for an unfiltered `SELECT *`. With `read_write=True` an `execute` tool is additionally registered for single-statement writes.

This class is deliberately self-contained (lazy `psycopg` imports, no module-level helpers) so it can be copied verbatim into a driver's `__main__` as a by-value-pickling escape hatch for worker images whose chia install predates this module.

Initializes `ChiaTool` with a name and optional resource requirements.

query(*sql: str*) → str

Execute `sql` against the database in a read-only session.

Returns a pipe-delimited markdown-ish table with row/cell/byte caps and a trailing `NOTE:` line when a cap fires.

schema() → str

Table/index listing from `information_schema / pg_indexes` for the tool's configured schemas — live, so migrated columns appear automatically.

execute(*sql: str*) → str

Execute ONE write statement in its own committed transaction (registered only when `read_write=True`). Returns `OK: rowcount=N` (plus returned rows for `RETURNING`) or an error string.

26.3 Sqlite Node

`chia.database.sqlite_node` — generic colocated SQLite store for chia loops.

SQLiteNode is a *DatabaseNode* (and thus a *ColocatedNode*) whose member `@ChiaFunctions` all operate on one SQLite database file. Because every member is pinned to the same machine, they all see the same file — that colocation guarantee is what makes a plain on-disk SQLite database a safe shared store for a distributed chia loop. SQL placeholders are sqlite's `paramstyle = "qmark"` (? positional, `:name` named); see *PostgresNode* for the client-server sibling.

Placement modes (decided once at construction):

- `default / placement_group=...` — standard *ColocatedNode* pinning; a fresh DB lands wherever the bundle does and all members co-locate there.
- `node_id=... / pin_to_current_node=True` — hard *NodeAffinity* pin, for a DB file that already lives on a known machine (e.g. the head node's local disk, as in `gem5align's alignment.db`).

Concurrency model: each `chia_remote` call may run in a *different worker process* on the pinned machine, so every member opens a fresh connection. Defaults: WAL journal mode (readers never block the writer), 30 s busy timeout, `BEGIN IMMEDIATE` write transactions (concurrent writers queue on the lock instead of failing mid-transaction), `synchronous=NORMAL`, `foreign_keys=ON`. Override per node or per call via `connect_opts` (keys: `busy_timeout_s`, `wal`, `synchronous`, `foreign_keys`).

Warning

Never point `db_path` at NFS or other shared/network storage. WAL's shared-memory coordination only works between processes on one machine with a local filesystem — which is exactly what the colocation guarantee provides. On network filesystems WAL can corrupt the database.

Data guidance: rows travel through the Ray object store as `list[dict]` (BLOB columns round-trip as bytes). Store large artifacts as files and put *paths* in the DB; multi-MB blobs inflate the object store and every `get()`.

Write semantics: write members are declared with `max_retries=0` — Ray's task replay after a worker death would double-apply a committed-but- unreturned non-idempotent write. Callers that know a write is idempotent can opt back in via `node.execute.options(max_retries=...)`.

Example — `gem5align's AlignmentDB` expressed on this node:

```
db = SQLiteNode("/abs/path/alignment.db", pin_to_current_node=True)
get(db.init_schema.chia_remote(ITERATIONS_SCHEMA + BENCH_SCHEMA))
get(db.add_column_if_missing.chia_remote("iterations", "base_rev", "TEXT"))
tool = db.spawn_query_tool("align_db")           # read-only SQL for LLMs

# insert_iteration: INSERT + DELETE + bulk INSERT, atomically
get(db.transaction.chia_remote([
    ("INSERT OR REPLACE INTO iterations (...) VALUES (?, ...)", iter_row),
    ("DELETE FROM benchmark_results WHERE entry_id = ?", (entry_id,)),
    ("INSERT INTO benchmark_results (...) VALUES (?, ...)", bench_rows),
]))

max_iter = get(db.query_value.chia_remote(
    "SELECT MAX(iteration) FROM iterations", default=-1))
```

Composite reads with Python-side logic (an `AlignmentDB load_entry / best_per_benchmark`) belong in a domain subclass — append them to `_MEMBER_FNS` and the binding machinery picks them up:

```
class AlignmentDBNode(SQLiteNode):
    _MEMBER_FNS = SQLiteNode._MEMBER_FNS + ("load_entry",)

    @staticmethod
    @ChiaFunction(num_cpus=0.1)
    def load_entry(db_path: str, entry_id: str, *,
                  connect_opts: dict | None = None) -> dict | None:
        conn = _connect(db_path, read_only=True, connect_opts=connect_opts)
        ...
```

chia.database.sqlite_node.SQLiteExecResult

Back-compat alias — the original name for the shared result dataclass.

```
class chia.database.sqlite_node.SQLiteNode(db_path: str | PathLike, placement_group=None,
                                           require_colocated: bool = True, *, node_id: str | None =
                                           None, pin_to_current_node: bool = False, bundle_index: int
                                           = 0, reserve_bundle: dict | None = None, pg_strategy: str =
                                           'STRICT_PACK', wait_for_pg: bool = True,
                                           pg_ready_timeout_s: float | None = None, connect_opts:
                                           dict | None = None)
```

Bases: [DatabaseNode](#)

Colocated node exposing generic SQLite operations over one DB file.

All members are `@staticmethod @ChiaFunction` taking `db_path` as the explicit first argument; `__init__` re-binds them so instance calls inject `self.db_path` (and the node's `connect_opts`) automatically:

```
with SQLiteNode("/abs/path/state.db") as db:
    # reserves a PG
    get(db.init_schema.chia_remote("CREATE TABLE IF NOT EXISTS t ..."))
    get(db.execute.chia_remote("INSERT INTO t VALUES (?)", (1,)))
    rows = get(db.query.chia_remote("SELECT * FROM t"))
```

See the module docstring for placement modes, concurrency model, and the AlignmentDB mapping example.

Validate `db_path` and defer to [DatabaseNode](#).

Parameters

- **db_path** – absolute path of the database file *on the target machine* (a relative path would resolve against a Ray worker's cwd). Created on first write if absent.
- **connect_opts** – connection defaults injected into every member call (keys: `busy_timeout_s`, `wal`, `synchronous`, `foreign_keys`); per-call `connect_opts`=overrides.

Remaining args are as in [DatabaseNode](#).

paramstyle: `str = 'qmark'`

PEP 249 paramstyle of the SQL this node's members expect (e.g. “qmark” for sqlite, “format” for postgres).

spawn_query_tool(*name: str, *, read_write: bool = False, **tool_kwargs*) → [SQLiteQueryTool](#)

Create a [SQLiteQueryTool](#) co-located with this node.

Exposes read-only SQL (and, with `read_write=True`, single-statement writes) over MCP, pinned to the same machine as the members so the tool reads the same DB file. The returned tool is tracked and stopped by `close()`. `tool_kwargs` forward to [SQLiteQueryTool](#) (e.g. `row_limit`, `cell_char_limit`, `total_char_limit`).

static init_schema(*db_path: str, script: str, *, connect_opts: dict | None = None*) → None

Create parent dirs and the DB file, then run `script` via `executescript`. Idempotent CREATE TABLE IF NOT EXISTS-style scripts are encouraged; `executescript` semantics apply (the script may contain its own BEGIN/COMMIT).

static executescript(*db_path: str, script: str, *, connect_opts: dict | None = None*) → None

Run a multi-statement SQL script (no params — `sqlite3` limitation). Like `init_schema()` without the create-dirs step.

static add_column_if_missing(*db_path: str, table: str, column: str, type_decl: str, *, connect_opts: dict | None = None*) → bool

ALTER TABLE ADD COLUMN iff the column is absent; True iff added.

table/column must be plain identifiers and `type_decl` a benign type expression — they are interpolated into the SQL (identifiers cannot be parameters).

static schema(*db_path: str, *, connect_opts: dict | None = None*) → str

CREATE TABLE / CREATE INDEX statements from `sqlite_master`.

static execute(*db_path: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None*) → *ExecResult*

Run one statement in its own committed transaction.

static executemany(*db_path: str, sql: str, seq_of_params: list, *, connect_opts: dict | None = None*) → *ExecResult*

`executemany` in one committed transaction; `rowcount` is the total across all parameter sets.

static transaction(*db_path: str, ops: list, *, connect_opts: dict | None = None*) → list[*ExecResult*]

Run ops atomically: BEGIN IMMEDIATE; each op in order; COMMIT.

Each op is (sql, params) where the params TYPE selects the form:

- tuple or dict -> `execute(sql, params)` (single statement)
- list -> `executemany(sql, params)` (bulk rows)
- None -> `execute(sql)` (no params)

NOTE: single-statement params MUST be a tuple/dict — a list means `executemany`. Any `sqlite` error rolls the whole batch back and re-raises (surfacing as a Ray task error at `get()`). Explicit BEGIN/COMMIT inside op SQL is unsupported. Returns one *SQLiteExecResult* per op.

static query(*db_path: str, sql: str, params: tuple | dict = (), *, limit: int | None = None, connect_opts: dict | None = None*) → list[dict]

Rows as list[dict]. `limit=N` caps the fetch (protects the object store from unbounded SELECTs); default None fetches all. The connection is opened with `PRAGMA query_only=ON`, so an accidental write raises `OperationalError`.

static query_one(*db_path: str, sql: str, params: tuple | dict = (), *, connect_opts: dict | None = None*) → dict | None

First row as a dict, or None.

static query_value(*db_path: str, sql: str, params: tuple | dict = (), *, default: Any = None, connect_opts: dict | None = None*) → Any

First column of the first row, or default when there is no row (e.g. `query_value("SELECT MAX(iteration) FROM t", default=-1)` — note an aggregate over an empty table yields a NULL row, returned as None, not default).

```
static wal_checkpoint(db_path: str, mode: str = 'TRUNCATE', *, connect_opts: dict | None = None) → dict
```

PRAGMA wal_checkpoint(mode) — fold the -wal sidecar back into the main file so the DB is a single copyable file (snapshot / backup story). Returns {"busy", "log", "checkpointed"}.

```
class chia.database.sqlite_node.SQLiteQueryTool(name: str, db_path: str, task_options: dict | None = None, row_limit: int = 100, cell_char_limit: int = 4096, total_char_limit: int = 32768, read_write: bool = False)
```

Bases: [ChiaTool](#)

MCP tool exposing SQL over a SQLite DB file (generalized from gem5align's AlignmentDbQueryTool).

query/schema open sqlite3.connect(..., uri=True, mode=ro) — the hard safety boundary against LLM-authored SQL: any INSERT / UPDATE / DELETE / DDL raises `OperationalError`, which is caught and returned as text. Row and byte caps keep the tool result bounded even when the LLM writes an unfiltered SELECT *. With read_write=True an execute tool is additionally registered for single-statement writes.

WAL caveat: a mode=ro open can fail if the -shm sidecar needs recovery; the error string is returned to the LLM. A co-resident writer (the owning [SQLiteNode](#)) normally keeps the sidecars live.

This class is deliberately self-contained (stdlib sqlite3 only, no module-level helpers) so it can be copied verbatim into a driver's `__main__` as a by-value-pickling escape hatch for worker images whose chia install predates this module — see the module docstring.

Initializes ChiaTool with a name and optional resource requirements.

```
query(sql: str) → str
```

Execute sql read-only against the database.

Returns a pipe-delimited markdown-ish table. Rows capped at `self.row_limit`; cells truncated to `self.cell_char_limit` chars with ... marker; total result capped at `self.total_char_limit` bytes. A trailing NOTE: line flags any cap that fired so the caller can narrow the query.

```
schema() → str
```

Return CREATE TABLE / CREATE INDEX statements from the live DB.

Pulls from `sqlite_master` so newly-migrated columns appear automatically without updating the prompt.

```
execute(sql: str) → str
```

Execute ONE write statement in its own committed transaction (registered only when read_write=True). Returns OK: rowcount=N lastrowid=M or an error string.

GITHUB

API reference for `chia.github`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

27.1 Github Client

Shared read-only GitHub REST client plumbing.

Base class + typed exceptions used by the repo-bound nodes (*GithubIssuesNode* and *GithubPullsNode*). Bind to one repo at construction; all calls are synchronous and raise the typed exceptions below on failure (no in-band success: bool). Head-node only — not a Ray task.

exception `chia.github.github_client.GithubError`

Bases: `Exception`

Base class for all GitHub client errors.

exception `chia.github.github_client.GithubAuthError`

Bases: *GithubError*

401 Unauthorized — missing or bad token for a private resource.

exception `chia.github.github_client.GithubRateLimitError(reset_time: int, message: str = "")`

Bases: *GithubError*

403 with X-RateLimit-Remaining: 0. `reset_time` is a unix timestamp.

exception `chia.github.github_client.GithubNotFoundError`

Bases: *GithubError*

404 — repo, issue, or pull request not found.

exception `chia.github.github_client.GithubRequestError`

Bases: *GithubError*

422 or other 4xx that doesn't fall into auth / rate-limit / not-found.

exception `chia.github.github_client.GithubServerError`

Bases: *GithubError*

5xx after one retry, or a network timeout.

class `chia.github.github_client.GithubClient(repo: str, token: str | None = None, timeout_seconds: int = 30, logging_level: int = 10)`

Bases: `object`

Repo-bound read-only GitHub REST client.

Holds the session, auth header, and the retrying `_request()` helper. Subclasses add resource-specific read methods (issues, pulls, ...). Pass a `token` or set `GITHUB_TOKEN` in the environment for private repos and a higher rate limit.

27.2 Github Issues Node

Read-only GitHub issues client, exposed as a chia service-pattern node.

Bind to one repo at construction and expose a small, synchronous API. Errors are raised as typed exceptions (no in-band `success: bool`); see `GithubError` and its subclasses (defined in `chia.github.github_client` and re-exported here for backwards compatibility). For pull-request reviews/comments, see `chia.github.github_pulls_node.GithubPullsNode`.

```
class chia.github.github_issues_node.GithubIssuesNode(repo: str, token: str | None = None, state: str = 'open', timeout_seconds: int = 30, logging_level: int = 10)
```

Bases: `GithubClient`

Read-only client for one GitHub repo's issues.

Service-pattern node (head-node only, not a Ray task). Bind to one repo at construction; fetch via `recent()` or `get_issue()`. All calls are synchronous and raise typed exceptions on failure. Pull requests live on `chia.github.github_pulls_node.GithubPullsNode`.

recent(*n*: int = 1, *after*: int | None = None, *fetch_comments*: bool = True) → list[`GithubIssue`]

Return up to *n* most-recent issues (sorted by created desc).

With *after*=None, returns the *n* newest issues. With *after*=K, returns the *n* newest issues whose number is strictly less than K. Pull requests are excluded — to list PRs use `chia.github.github_pulls_node.GithubPullsNode.recent()`.

fetch_comments defaults to True (one extra paginated request per issue that has comments). Pass False to skip that N+1 cost when you only need list-payload fields (title/body/labels) — e.g. to filter a large pool cheaply, then re-fetch comments for the few survivors via `get_issue()`.

get_issue(*number*: int, *allow_pull_request*: bool = False) → `GithubIssue`

Fetch one ISSUE by number.

GitHub serves pull requests through the issues endpoint too (shared number space), so the endpoint CAN return a PR — but a PR-as-issue is a degraded view (no head/base/branch data) and almost never what an issues-node caller wants, so by default a PR number raises `GithubRequestError` instead of silently succeeding. Pass *allow_pull_request*=True to opt in (the result's *is_pull_request* field will be True). For a proper PR view use `chia.github.github_pulls_node.GithubPullsNode.get_pull()`.

get(*number*: int) → `GithubIssue`

Deprecated alias for `get_issue()` with *allow_pull_request*=True (the historical permissive behavior). Prefer `get_issue()`, or `GithubPullsNode.get_pull()` for pull requests.

linked_pull_requests(*number*: int, *open_only*: bool = False) → list[dict]

Pull requests attached to issue *number*, via its timeline.

A PR that references the issue (Fixes #N, or a manual link) leaves a cross-referenced timeline event whose *source* is a PR; we collect those. Each result is {"number": int, "state": str} (state is the PR's open/closed, deduped by number). *open_only* keeps just open PRs.

Note: relies on cross-reference events (the common case, incl. the closing-keyword link). A PR linked ONLY via the sidebar with no cross-reference may not appear.

27.3 Github Pulls Node

Read-only GitHub pull-request client, exposed as a chia service-pattern node.

Companion to `chia.github.github_issues_node.GithubIssuesNode`, sharing the same repo-bound, head-node-only conventions and the `chia.github.github_client.GithubClient` HTTP plumbing. Use this for pull-request metadata and — its main purpose — reading the REVIEW FEEDBACK on a PR (review summaries + inline line comments + conversation comments) so an agent can act on it like a PR author.

Errors raise the typed exceptions from `chia.github.github_client`.

```
class chia.github.github_pulls_node.GithubPullsNode(repo: str, token: str | None = None,
                                                    timeout_seconds: int = 30, logging_level: int = 10)
```

Bases: `GithubClient`

Read-only client for one GitHub repo's pull requests.

Bind to one repo at construction, then:

- `recent()` — list the newest PRs (open/closed/all, per call).
- `get_pull()` — PR metadata (branches, head sha, draft/merged, body).
- `pull_diff()` — the PR's current unified diff as raw text.
- `check_runs()` — CI check results (+ annotations for failures).
- `reviews()` — submitted review summaries.
- `review_comments()` — inline, line-anchored review comments.
- `conversation_comments()` — general (non-inline) conversation comments.
- `review_feedback()` — all of the above bundled into a `GithubPullFeedback` (with `to_markdown()` for LLM input).

recent(*n*: int = 1, *after*: int | None = None, *state*: str = 'open') → list[`GithubPull`]

Return up to *n* most-recent pull requests (sorted by created desc).

With *after*=None, the *n* newest PRs; with *after*=K, the *n* newest PRs whose number is strictly less than K. *state* is "open" / "closed" / "all" (per-call — this node binds no state at construction).

Note: the /pulls LIST payload omits the computed merged flag; merged on results from this method is derived from merged_at (equivalent for merged-vs-not).

get_pull(*number*: int) → `GithubPull`

Fetch one PULL REQUEST's metadata by number.

Unlike the shared issues endpoint, /pulls/{number} serves ONLY pull requests — a plain issue number 404s even though it exists in the shared number space. That 404 is re-raised with a clearer message, since “not found” usually means “exists, but as an issue”. For issues use `chia.github.github_issues_node.GithubIssuesNode.get_issue()`.

get(*number*: int) → `GithubPull`

Deprecated alias for `get_pull()`.

pull_diff(*number*: int) → str

The PR's CURRENT unified diff (head vs base), as raw text.

Uses the application/vnd.github.diff media type on the single-PR endpoint — this is the diff of the PR as it exists NOW (rebases and force-pushes included), which makes it the authoritative base

for reconstructing the PR's state. GitHub refuses very large diffs (~20k+ lines) with a 406, surfaced as `GithubRequestError`.

reviews(*number: int*) → list[[*GithubReview*](#)]

Submitted review summaries on PR *number* (oldest first).

review_comments(*number: int*) → list[[*GithubReviewComment*](#)]

Inline, line-anchored review comments on PR *number* (oldest first).

conversation_comments(*number: int*) → list[[*GithubComment*](#)]

General (non-inline) conversation comments on PR *number*.

PRs share the issue number space, so these come from the issues comments endpoint — the back-and-forth that isn't anchored to a line.

check_runs(*number: int, head_sha: str | None = None, annotations_for_failures: bool = True*) → list[[*GithubCheckRun*](#)]

CI check results on PR *number*'s head commit (latest run per check).

Uses the Checks API with `filter=latest` so a re-run replaces its earlier attempt. For FAILED runs (conclusion failure/timed_out), also fetches the run's file/line annotations — typically the compiler/test errors — capped at 50 per run. *head_sha* avoids a refetch of the PR when the caller already has it.

review_feedback(*number: int, include_conversation: bool = True, include_checks: bool = True*) → [*GithubPullFeedback*](#)

Bundle the PR + its reviews, inline comments, (optionally) the conversation comments, and (optionally) the CI check results into one `GithubPullFeedback`. A PR with no human feedback but FAILING CI is `NOT is_empty()` — red CI counts as feedback to act on.

27.4 State Def

```
class chia.github.state_def.GithubComment(author: str, created_at: str, body: str)
```

Bases: object

```
class chia.github.state_def.GithubIssue(number: int, title: str, state: str, author: str, created_at: str,
                                         updated_at: str, closed_at: str | None, body: str, labels: list[str],
                                         url: str, is_pull_request: bool, comments:
                                         list[chia.github.state_def.GithubComment] = <factory>)
```

Bases: object

to_markdown() → str

Render the issue + comments as one LLM-ready text blob.

```
class chia.github.state_def.GithubReview(author: str, state: str, body: str, submitted_at: str)
```

Bases: object

A submitted review on a PR (the summary, not the inline comments).

```
class chia.github.state_def.GithubReviewComment(author: str, path: str, line: int | None, body: str,
                                                  diff_hunk: str, in_reply_to: int | None, created_at:
                                                  str, id: int)
```

Bases: object

An inline (line-anchored) review comment on a PR's diff.

```
class chia.github.state_def.GithubPull(number: int, title: str, state: str, author: str, body: str, base_ref:
                                     str, head_ref: str, head_sha: str, draft: bool, merged: bool, url:
                                     str)
```

Bases: object

A pull request's metadata (separate from issue/PR-as-issue).

```
class chia.github.state_def.GithubCheckAnnotation(path: str, start_line: int, end_line: int, level: str,
                                                  message: str, title: str = "")
```

Bases: object

A file/line-anchored message a CI check attached to its run.

```
class chia.github.state_def.GithubCheckRun(name: str, status: str, conclusion: str, summary: str, url:
                                           str, annotations: list[GithubCheckAnnotation] =
                                           <factory>)
```

Bases: object

One CI check's latest result on a PR's head commit.

```
class chia.github.state_def.GithubPullFeedback(pull: GithubPull, reviews: list[GithubReview] =
                                                <factory>, review_comments:
                                                list[GithubReviewComment] = <factory>,
                                                conversation_comments: list[GithubComment] =
                                                <factory>, check_runs: list[GithubCheckRun] =
                                                <factory>)
```

Bases: object

Everything a PR author needs to act on a review round: the PR plus its reviews, inline review comments, general conversation comments, and the CI check results on its head commit.

is_empty() → bool

True if there is nothing to act on: no reviews/comments AND no failing CI checks (green/neutral CI alone is not feedback).

to_markdown() → str

Render the review feedback as one LLM-ready, reviewer-style blob.

Inline comments are numbered [C1], [C2], ... so an author can reference them when replying. Each shows its file, line, the diff hunk it anchors to, and the reviewer's text.

API reference for `chia.aws`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

28.1 Config

```
class chia.aws.config.AWSConfig(region: str = 'us-east-1', key_name: str = 'firesim', vpc_name: str =
    'firesim', security_group_name: str = 'for-farms-only-firesim', ssh_user: str
    = 'ubuntu', ssh_private_key: str | None = None, use_public_ip: bool =
    False, s3_bucket: str = 'firesim-chia-builds', subnet_id: str | None = None,
    aws_creds_dir: str | None = None)
```

Bases: `object`

AWS infrastructure configuration for FireSim operations.

```
class chia.aws.config.EC2InstanceConfig(instance_type: str, volume_size_gb: int = 200, market: str =
    'ondemand', spot_max_price: str = 'ondemand',
    spot_interruption_behavior: str = 'terminate', ami_id: str | None
    = None, tags: dict[str, str]=<factory>, user_data: str | None =
    None)
```

Bases: `object`

Configuration for a single EC2 instance type to launch.

28.2 EC2

Thin wrapper around AWS EC2 operations for launching/terminating instances.

Imports FireSim's `awstools` functions when available, falling back to direct `boto3` calls otherwise.

```
class chia.aws.ec2.EC2Instance(instance_id: str, private_ip: str, public_ip: str | None, instance_type: str)
```

Bases: `object`

Serializable representation of a launched EC2 instance.

```
chia.aws.ec2.get_default_ami(region: str | None = None) → str
```

Look up the F2/FPGA Developer AMI (Ubuntu) in the current (or given) region.

Uses the same AMI name pattern as FireSim's `awstools.get_f2_ami_name()`, with fallback attempts for hotfix version bumps.

```
chia.aws.ec2.launch_ec2_instances(aws_config: AWSConfig, instance_config: EC2InstanceConfig, count:
    int = 1, instance_name: str = 'chia-firesim') → list[EC2Instance]
```

Launch EC2 instances and return serializable `EC2Instance` objects.

Parameters

- **aws_config** – AWS/VPC configuration.
- **instance_config** – Instance type and market configuration.
- **count** – Number of instances to launch.

Returns

List of EC2Instance dataclasses (safe for Ray serialization).

```
chia.aws.ec2.wait_for_instances(instance_ids: list[str], region: str = 'us-east-1', timeout: float = 600) →  
list[EC2Instance]
```

Wait for instances to reach ‘running’ state and populate IP addresses.

Parameters

- **instance_ids** – List of EC2 instance IDs to wait on.
- **region** – AWS region.
- **timeout** – Maximum seconds to wait.

Returns

Updated list of EC2Instance with IP addresses filled in.

```
chia.aws.ec2.terminate_ec2_instances(instance_ids: list[str], region: str = 'us-east-1', dryrun: bool =  
False, wait: bool = True, wait_timeout: int = 600) → None
```

Terminate EC2 instances by ID.

Parameters

- **instance_ids** – Instance IDs to terminate.
- **region** – AWS region.
- **dryrun** – If True, validate the request without actually terminating.
- **wait** – If True, block until every instance reaches the *terminated* state. Ensures the caller can safely assume capacity/quota has been released before returning.
- **wait_timeout** – Max seconds to wait for termination (when wait=True).

28.3 Host

EphemeralEC2Host — combines an EC2 instance with Chia’s SSHClient for remote execution, with automatic cleanup via context manager.

```
class chia.aws.host.EphemeralEC2Host(ec2_instance: EC2Instance, aws_config: AWSConfig)
```

Bases: object

An ephemeral EC2 instance paired with an SSH connection.

Intended for use as a context manager so the instance is always terminated, even on exceptions:

```
with EphemeralEC2Host(ec2_inst, aws_cfg) as host:  
    host.wait_ready()  
    host.run("echo hello")
```

```
wait_ready(timeout: float = 300) → None
```

Block until SSH is reachable.

rsync_up(*local_path*: str, *remote_path*: str, *exclude*: list[str] | None = None, *filter_rules*: list[str] | None = None) → None

Upload files from the local machine to this host.

rsync_down(*remote_path*: str, *local_path*: str, *exclude*: list[str] | None = None) → None

Download files from this host to the local machine.

run(*cmd*: str, *timeout*: int = 300, *check*: bool = True, *retries*: int = 0, *retry_delay*: float = 5.0)

Run a command over SSH.

run_script(*commands*: list[str], *timeout*: int = 600, *check*: bool = True)

Run multiple commands in a single SSH session.

terminate() → None

Terminate this EC2 instance.

28.4 S3

S3 object-store client bound to one bucket, exposed as a chia service-pattern node.

binds to one bucket at construction and exposes a small, synchronous API. Errors are raised as typed exceptions (no in-band success: bool); see `S3Error` and its subclasses. Transient failures (5xx, throttling, network errors) are retried once before raising.

Usage:

```
from chia.aws.s3 import S3Node, S3NotFoundError

node = S3Node("my-bucket", region="us-west-2")
node.put_bytes("results/run1.json", b"{}".format(data))
data = node.get_bytes("results/run1.json")
node.upload_file("/tmp/waves.vcd", "waves/run1.vcd")
```

For anything beyond this surface (presigned URLs, multipart tuning, ...), drop down to the raw boto3 client via `node._client`.

exception `chia.aws.s3.S3Error`(*message*: str = "", *code*: str = "")

Bases: `Exception`

Base class for all S3 node errors. `code` is the AWS error code, if any.

exception `chia.aws.s3.S3AuthError`(*message*: str = "", *code*: str = "")

Bases: `S3Error`

Missing/invalid credentials, or access denied (401/403/AccessDenied).

exception `chia.aws.s3.S3NotFoundError`(*message*: str = "", *code*: str = "")

Bases: `S3Error`

Object or bucket does not exist (404/NoSuchKey/NoSuchBucket).

exception `chia.aws.s3.S3RequestError`(*message*: str = "", *code*: str = "")

Bases: `S3Error`

Non-transient client-side error (other 4xx, e.g. `BucketAlreadyExists`).

exception `chia.aws.s3.S3ServerError`(*message: str = "", code: str = ""*)

Bases: [S3Error](#)

5xx/throttling after one retry, or a network/timeout failure.

class `chia.aws.s3.S3ObjectInfo`(*key: str, size: int, last_modified: str, etag: str*)

Bases: `object`

Serializable metadata for one S3 object (one list result row).

class `chia.aws.s3.S3Node`(*bucket: str, region: str | None = None, profile: str | None = None, aws_access_key_id: str | None = None, aws_secret_access_key: str | None = None, aws_session_token: str | None = None, timeout_seconds: int = 60, logging_level: int = 10*)

Bases: `object`

Client for one S3 bucket.

Service-pattern node (head-node only, not a Ray task). Bind to one bucket at construction; all methods are synchronous and raise typed exceptions on failure. Credentials come from the default boto3 chain unless overridden — see `__init__` for the resolution order.

Bind to bucket. Credential resolution, in order of precedence:

1. Explicit keys — pass `aws_access_key_id` and `aws_secret_access_key` together (plus `aws_session_token` for temporary/STS credentials). Use this to ship credentials by value into an environment with no `~/.aws` or instance role, e.g. a docker worker:

```
key, secret, token = load_aws_creds(creds_dir)  # on the head node
node = S3Node(bucket,                          # on the worker
               aws_access_key_id=key,
               aws_secret_access_key=secret,
               aws_session_token=token or None)
```

Empty strings are treated as “not provided” (so a blank `load_aws_creds` tuple falls through to the default chain), but passing only one of key/secret, a token without both keys, or explicit keys together with `profile` raises `ValueError`.

2. `profile` — a named profile from `~/.aws`.
3. Neither — boto3’s default chain (env vars, `~/.aws`, instance metadata / IAM role). The right choice on the head node and on EC2.

Credentials are resolved lazily by boto3: nothing is validated here, and missing/invalid credentials surface as `S3AuthError` on the first call.

upload_file(*local_path: str | Path, key: str*) → None

Upload a local file to `key` (multipart for large files).

download_file(*key: str, local_path: str | Path*) → None

Download `key` to a local path, creating parent directories.

put_bytes(*key: str, data: bytes, content_type: str | None = None*) → None

Write data to `key`.

get_bytes(*key: str*) → bytes

Return the contents of `key`.

list(*prefix: str = "", max_keys: int | None = None*) → list[*S3ObjectInfo*]

List objects under **prefix**, up to **max_keys** (None = all).

Includes zero-byte .../ folder-marker keys if present; callers that don't want them should filter on **key.endswith("/")**.

exists(*key: str*) → bool

Return whether **key** exists. A 403 raises *S3AuthError* rather than reporting False.

delete(*key: str*) → None

Delete **key**. Idempotent — S3 does not error on a missing key.

ensure_bucket() → bool

Create the bound bucket if it does not exist.

Returns True if the bucket was created, False if it already existed. Raises *S3AuthError* if the bucket exists but is not accessible.

SIMULATORS

API reference for `chia.simulators`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

29.1 gem5

`chia.simulators.gem5` — gem5 build / run / source-state primitives.

Functions are PATH-BASED and co-located: a gem5 binary is far too large to ship through the object store the way chipyard ships its Verilator binary as bytes, so build / run / capture / restore all operate on a gem5 checkout on the worker's filesystem and must land on the SAME worker. [Gem5Node](#) enforces that via a placement group (see its docstring). Portability across workers is achieved by shipping the small source *diff* ([Gem5Node.capture_gem5_source_state\(\)](#)) and rebuilding, not by moving the binary.

Also defines [Gem5ToolServer](#), the LLM-facing MCP adapter over a [Gem5Node](#) (see [Gem5Node.spawn_tool\(\)](#)). Because the tool subclasses `ChiaTool`, importing this module pulls in the MCP/FastMCP stack.

```
class chia.simulators.gem5.Gem5Isa(*values)
```

Bases: `str`, `Enum`

```
class chia.simulators.gem5.Gem5Variant(*values)
```

Bases: `str`, `Enum`

```
class chia.simulators.gem5.Gem5BuildArtifact(binary_path: str, isa: str, variant: str, gem5_root: str,
                                              base_rev: str, success: bool, returncode: int,
                                              build_duration_s: float, stdout_tail: str, stderr_tail: str)
```

Bases: `object`

Result of an scons build. Path-based: `binary_path` lives on the worker and is only reachable from a run pinned to the same node.

```
class chia.simulators.gem5.Gem5RunResult(workload_name: str, status: str, returncode: int, outdir: str,
                                          num_cycles: int | None, sim_insts: int | None, sim_seconds:
                                          float | None, host_seconds: float | None, wall_s: float | None,
                                          stats: dict[str, float]=<factory>, stdout_tail: str = "",
                                          error_messages: str = "", stats_content: str | None = None,
                                          debug_trace: bytes | None = None)
```

Bases: `object`

Result of a single gem5 invocation on one workload.

```
class chia.simulators.gem5.Gem5SourceState(base_rev: str, source_diff: str, config_contents: str = "")
```

Bases: `object`

Portable snapshot of edits to a gem5 checkout (ship this, not the binary).

```
class chia.simulators.gem5.Gem5Node(placement_group=None, require_colocated: bool = True, *,
                                     bundle_index: int = 0, reserve_bundle: dict | None = None,
                                     pg_strategy: str = 'STRICT_PACK', wait_for_pg: bool = True,
                                     pg_ready_timeout_s: float | None = None)
```

Bases: object

gem5 build / run / source-state primitives sharing one placement.

The four core operations are @staticmethod @ChiaFunction(resources= {"gem5": 1.0}) members; `__init__` re-binds each into a per-instance `_PinnedChiaFn` so `node.<op>.chia_remote(...)` lands on this node's bundle. `Gem5Node.<op>.chia_remote(...)` (the class attribute) is the raw, unpinned form.

Co-location: because gem5 binaries stay on the worker filesystem, every member must run on the same node. Placement is decided once at construction:

- `placement_group` given -> pin members to `bundle_index` of it (the node will NOT release a PG it did not create).
- `none + require_colocated=True` -> reserve a 1-bundle {"gem5": 1, "CPU": 1} PG (owned + released by this node).
- `none + require_colocated=False` -> no pinning; the caller schedules each `.chia_remote / .options(...)` call itself.

`spawn_tool()` (opt-in) creates a `Gem5ToolServer` co-located on this node's bundle, exposing the same build/run logic to an LLM over MCP; spawned tools are stopped by `close()`.

Usable as a context manager so spawned tools are stopped and a self-reserved PG is released on exit.

Set up placement and bind the member functions.

Parameters

- **placement_group** – an existing Ray PlacementGroup to schedule onto. If given, `require_colocated` is moot (placement is already fixed) and this node will not remove the PG on close.
- **require_colocated** – when no PG is given, reserve one so all members co-locate. When False, leave placement to the caller.
- **bundle_index** – which bundle of the (given or reserved) PG to pin to.
- **reserve_bundle** – resource shape of a self-reserved bundle (default {"CPU": 1, "gem5": 1.0}); must provide `gem5 >=` each member's requirement (1.0).
- **pg_strategy** – placement strategy for a self-reserved PG.
- **wait_for_pg** – block on `pg.ready()` for a self-reserved PG so the node is usable immediately.
- **pg_ready_timeout_s** – optional timeout for that wait.

property placement_group

The PG members are pinned to (None when the caller handles placement).

property owns_placement_group: bool

True iff this node reserved its PG and will release it on `close()`.

property task_options: dict

Scheduling opts to co-locate an actor (e.g. a `ChiaTool`) with this node's bundle. Empty when the node has no placement group (`require_colocated=False`) — there is no shared placement to inherit, so an actor given these opts would not be pinned.

spawn_tool(*name: str, *, gem5_root: str, config_script: str, workloads, **tool_kwargs*)

Create a Gem5ToolServer co-located on this node's bundle.

Exposes this node's gem5 build/run/stats to an LLM over MCP, pinned to the same worker so the tool and `node.<op>.chia_remote(...)` operate on the same on-disk gem5 checkout. The returned tool is tracked and stopped by `close()`.

Requires a placement group — construct the node with `require_colocated=True` or pass `placement_group=...`; otherwise there is no shared placement for the tool to join.

`tool_kwargs` are forwarded to Gem5ToolServer (e.g. `config_args`, `isa`, `variant`, `stats_keys`, `excluded_workloads`, `timeouts`).

close() → None

Stop any spawned tools, then release the PG iff this node reserved it.

Tools are stopped BEFORE the PG is removed: a tool's `_ToolServerActor` is pinned to this PG, so tearing the PG down first would orphan it. Idempotent.

static build_gem5(*gem5_root: str, isa: str = 'RISCV', variant: str = 'opt', *, target: str | None = None, jobs: int | None = None, extra_scons_args: str = "", timeout_s: int = 3600*) → *Gem5BuildArtifact*

Incremental scons-build a gem5 binary in `gem5_root`.

Returns a path-based artifact; on failure `stderr_tail` carries the filtered compiler/linker diagnostics (so this doubles as a fast compile check). Records `base_rev` (git HEAD) for later diffing.

static run_gem5(*gem5_bin: str, config_script: str, outdir: str, *, workload_name: str | None = None, config_args: list[str] | None = None, gem5_args: list[str] | None = None, debug_flags: str | None = None, debug_file: str | None = None, stats_keys: dict[str, list[str]] | None = None, stats_block: int | str = 'last', capture_stats: bool = False, capture_debug_trace: bool = False, cwd: str | None = None, env: dict[str, str] | None = None, timeout_s: int = 3600*) → *Gem5RunResult*

Run a single gem5 invocation and parse cycle/instruction counters.

Builds the command line as:

```
gem5_bin [gem5_args] --outdir=<outdir> [--debug-flags=..] [--debug-file=..]
config_script [config_args]
```

`--kernel / --memory-backend` etc. are config-script arguments, so pass them via `config_args` (keeps this node agnostic to any specific config.py). After the run, reads `<outdir>/stats.txt` and fills `num_cycles / sim_insts / stats` (logical names from `stats_keys`, defaulting to `DEFAULT_STATS_KEYS`).

`stats_block` picks which dump to read (“last” = final cumulative dump; pass “first” for configs that reset stats at an ROI workbegin).

static capture_gem5_source_state(*gem5_root: str, *, base_rev: str | None = None, diff_paths: list[str] | None = None, config_path: str | None = None*) → *Gem5SourceState*

Capture a portable snapshot of the current edits in `gem5_root`.

Computes `git diff <base_rev> -- <diff_paths>`, first marking untracked files under those paths as intent-to-add so brand-new files (e.g. a new SimObject's .py/.cc/.hh) are included, then un-staging them to leave the index clean. Optionally snapshots `config_path`.

static restore_gem5_source_state(*gem5_root: str, state: Gem5SourceState, *, restore_paths: list[str] | None = None, config_path: str | None = None*) → tuple[bool, str]

Reset `gem5_root` to `state.base_rev` and apply `state.source_diff`.

`git checkout <base_rev> -- <paths> + git clean -fd <paths>` (to drop untracked leftovers from a prior apply that would collide with the next apply) + `git apply`, then optionally write `state.config_contents` to `config_path`. Caller is responsible for a subsequent `build_gem5`. Returns (ok, message).

static `parse_gem5_stats`(*stats_text*: str, *stats_keys*: dict[str, list[str]] | None = None, *stats_block*: int | str = 'last') → dict[str, float]

Map logical name -> value from `stats.txt` text.

`stats_keys` maps a logical name to ordered candidate gem5 stat names (first present wins); defaults to `DEFAULT_STATS_KEYS`. `stats_block` selects which Begin/End Simulation Statistics dump to read (“first” | “last” | int index, negatives allowed).

static `parse_gem5_stats_file`(*stats_path*: str, *stats_keys*: dict[str, list[str]] | None = None, *stats_block*: int | str = 'last') → dict[str, float]

Like `parse_gem5_stats()` but reads from a path; missing file -> {}.

static `truncate_gz_trace`(*path*: str, *max_decompressed_bytes*: int) → tuple[int, bool]

In-place head-truncate a gzipped trace to a decompressed-byte cap, ending on a line boundary; returns (retained_bytes, was_truncated). Stays valid gzip so downstream readers needn’t know it was trimmed.

static `summarize_o3_pipeview`(*trace_path*: str, *max_first*: int = 30, *max_slowest*: int = 10, *tick_per_cycle*: int = 1000, *reservoir_size*: int = 1000) → str

Stream an O3PipeView `--debug-file` trace into a compact markdown digest (per-stage wait-cycle stats, slowest instructions, first-N table).

O3-CPU-specific. Memory stays bounded (a few tens of KB) regardless of trace length: percentiles come from a reservoir sample, mean/max/count are exact. The raw trace on disk remains the authoritative artifact.

class `chia.simulators.gem5.Gem5ToolServer`(*name*: str, *gem5_root*: str, *config_script*: str, *workloads*: dict[str, str] | str, *, *gem5_bin*: str | None = None, *isa*: str = 'RISCV', *variant*: str = 'opt', *config_args*: list[str] | None = None, *kernel_flag*: str = '--kernel', *stats_keys*: dict[str, list[str]] | None = None, *stats_block*: int | str = 'last', *excluded_workloads*: set[str] | None = None, *workload_glob*: str = '*.elf', *max_workloads_per_call*: int = 5, *build_timeout_s*: int = 3600, *run_timeout_per_workload_s*: int = 3600, *expose*: tuple[str, ...] | None = None, *task_options*: dict | None = None)

Bases: [ChiaTool](#)

LLM-facing MCP tool server exposing [Gem5Node](#)’s build/run/stats.

A thin *adapter* over the node, not a reimplementation:

- Each method calls the corresponding [Gem5Node](#) function **locally** (a `@ChiaFunction` invoked directly runs in-process — no extra Ray hop), so one battle-tested build/run implementation serves both the programmatic node path and this agentic path.
- Operational arguments (`gem5_root`, `gem5_bin`, `config_script`, timeouts, ...) are bound at construction, NOT exposed to the LLM — the model only chooses *what to run*, never *where/how* the environment is wired.
- Results are rendered to compact text (the dataclasses are for code, not for a model to read), and long-running calls emit SSE keepalives so the CLI’s idle timer doesn’t abandon the stream and drop the result.

Co-location: the tool runs build/run against `gem5_root` on the worker's filesystem, so pin the actor to the same bundle as whatever bash tool the LLM edits source through, via `task_options` (exactly as the loop pins `QuickRunTool`). `Gem5Node.spawn_tool()` wires this up automatically.

LLM-facing methods:

- `{name}_build()` — incrementally rebuild gem5 from the current on-disk source and report OK / compiler errors (a fast compile check).
- `{name}_run(workloads, skip_build=False)` — build (unless skipping) and run 1..N workloads, returning a small cycles/insts/status table.
- `{name}_stats(workload, pattern)` — grep that workload's most recent `stats.txt` for a regex (drill into gem5 counters).
- `{name}_list_workloads()` — list the workloads available here.

Pass `expose=(...)` to register only a subset of the four tools — e.g. `expose=("build",)` makes this server a pure compile-check tool.

Initializes ChiaTool with a name and optional resource requirements.

async build(*ctx: Context*) → str

Incrementally rebuild gem5 from the current on-disk source and report the result.

Use after a meaningful source edit to catch compile errors *before* spending a run on a binary that won't build. Returns OK (<secs>) on a clean build, or FAIL ... followed by the relevant compiler/linker diagnostics. A TIMEOUT is inconclusive — the build did not finish in time, NOT proof your code is broken; narrow your edit or try again.

async run(*workloads: list[str], ctx: Context, skip_build: bool = False*) → str

Build (unless `skip_build`) and run 1..N workloads against the current source/config, returning a mark-down table of cycles + instructions + status.

Set `skip_build=True` to reuse the existing binary when you've only changed the gem5 *config*, not source. After this returns, call `{name}_stats("<workload>", "<regex>")` to drill into any workload's gem5 counters. Run at most `max_workloads_per_call` per call.

stats(*workload: str, pattern: str, max_lines: int = 40*) → str

Grep the most recent `run` of *workload* for `stats.txt` lines matching the regex *pattern* (capped at `max_lines`).

Use to read gem5 internal counters (e.g. `IntDiv|FuncUnit, iqFullEvents|blockedCycles, dcache.demand, IPC|CPI`).

list_workloads() → str

List the workloads available on this worker (excluded ones hidden).

ANALYSIS

API reference for `chia.analysis.sustainability`. These pages are generated from the docstrings in the source, so they stay in sync with the code.

Note that these sustainability analyses were chosen because, as far as we could tell, they did not have public codebases for performing these analyses. We exclude frameworks, which provide their own codebases publically, like [ACT](#) and [GreenSku framework](#), and instead direct people to those frameworks directly and encourage them to integrate CHIA with the already public codebases of those frameworks.

30.1 Focal

FOCAL: a first-order carbon model for comparing processor architectures.

Implements the Normalized Carbon Footprint (NCF) metric from Lieven Eeckhout, “FOCAL: A First-Order Carbon Model to Assess Processor Sustainability” (ASPLOS ‘24, <https://doi.org/10.1145/3620665.3640415>).

FOCAL weighs a design’s *embodied* footprint (proxied by chip area) against its *operational* footprint (proxied by power under a fixed-time use case, or by energy under a fixed-work use case) using a single embodied-to-operational weight `alpha` in $[0, 1]$. Comparing a `test` design against a `ref` baseline, the fixed-time NCF is:

$$\text{NCF}(\text{test}, \text{ref}) = \alpha * (\text{A}_{\text{test}} / \text{A}_{\text{ref}}) + (1 - \alpha) * (\text{P}_{\text{test}} / \text{P}_{\text{ref}})$$

where `A` is chip area and `P` is power consumption. $\text{NCF} < 1$ means the test design has a lower total carbon footprint than the reference (i.e. it is more sustainable); $\text{NCF} > 1$ means it is worse; $\text{NCF} == 1$ means the two are carbon-equal. The same expression holds for the fixed-work scenario with energy substituted for power.

`alpha` is the embodied-to-operational weight in $[0, 1]$. Paper scenarios: `alpha = 0.8` (embodied-dominated, e.g. mobile and datacenter hardware) and `alpha = 0.2` (operational-dominated, e.g. always-connected devices).

```
class chia.analysis.sustainability.focal.FocalComparison(ncf: float, embodied_ratio: float,
                                                         operational_ratio: float, alpha_ref: float,
                                                         test_pwr: float, ref_pwr: float, test_area:
                                                         float, ref_area: float)
```

Bases: `object`

FOCAL comparison of a `test` architecture against a `ref` baseline.

Computed under the fixed-time scenario, i.e. power is the operational proxy. (Pass energy in place of power to evaluate the fixed-work scenario; the NCF expression is identical.)

property more_sustainable: bool

True if the test design has a strictly lower carbon footprint.

property verdict: str

Human-readable verdict for the test design relative to ref.

property pct_change: float

Percent change in carbon footprint of test vs ref.

Negative means a reduction (more sustainable); e.g. -30.0 means the test design emits 30% less carbon than the reference.

property breakeven_alpha: float | None

Embodied weight alpha at which test and ref are carbon-equal.

Solves $NCF(\alpha) = 1$ for alpha at the current area and power ratios:

$$\alpha^* = (1 - P_{\text{test}}/P_{\text{ref}}) / (A_{\text{test}}/A_{\text{ref}} - P_{\text{test}}/P_{\text{ref}})$$

Returns None when the embodied and operational ratios are equal (NCF does not depend on alpha, so there is no crossing). A value outside [0, 1] means the verdict never flips within the valid weight range, i.e. the test design is uniformly more or less sustainable.

`chia.analysis.sustainability.focal.focal_compare(test_pwr: float, ref_pwr: float, test_area: float, ref_area: float, alpha_ref: float) → FocalComparison`

Compare a test architecture against a ref baseline with FOCAL.

Computes the Normalized Carbon Footprint (NCF) of the test design relative to the reference under the fixed-time scenario:

$$NCF = \alpha_{\text{ref}} * (\text{test_area} / \text{ref_area}) + (1 - \alpha_{\text{ref}}) * (\text{test_pwr} / \text{ref_pwr})$$

$NCF < 1$ means the test design has a lower total carbon footprint than the reference (more sustainable); $NCF > 1$ means it is worse.

Parameters

- **test_pwr** – Power of the test design (operational proxy, numerator). Substitute energy to evaluate the fixed-work scenario instead.
- **ref_pwr** – Power of the reference design (operational proxy, denominator).
- **test_area** – Chip area of the test design (embodied proxy, numerator).
- **ref_area** – Chip area of the reference design (embodied proxy, denominator).
- **alpha_ref** – Embodied-to-operational weight in [0, 1] (the FOCAL `alpha_E20`). 0.8 = embodied-dominated, 0.2 = operational-dominated.

Returns

A `FocalComparison` with the NCF, its embodied/operational components, and convenience verdicts.

Raises

ValueError – if a denominator is non-positive, an input is negative, or `alpha_ref` is outside [0, 1].

class `chia.analysis.sustainability.focal.FocalSweep`(*parameter: str, values: list[float], ncf: list[float], comparisons: list[FocalComparison], breakevens: list[float], test_pwr: float, ref_pwr: float, test_area: float, ref_area: float, alpha_ref: float*)

Bases: object

NCF of a test/ref pair as one FOCAL parameter is swept over a range.

property crosses_unity: bool

True if the verdict flips (NCF crosses 1) within the swept range.

class chia.analysis.sustainability.focal.**FocalBothRefs**(arch_a_ref: [FocalSweep](#), arch_b_ref: [FocalSweep](#))

Bases: object

Two alpha sweeps comparing arch A and arch B, each taken as the reference.

Holds the arch_a_ref sweep (A is the reference, so the test design is B) and the arch_b_ref sweep (B is the reference, so the test design is A). See `sweep_both_arches_as_ref` for why both directions matter.

chia.analysis.sustainability.focal.**focal_sweep**(test_pwr: float, ref_pwr: float, test_area: float, ref_area: float, alpha_ref: float, *, sweep: str = 'alpha_ref', values: Sequence[float] | None = None, num: int = 51, span: float = 2.0, plot_path: str | Path | None = None) → [FocalSweep](#)

Sweep one FOCAL parameter and report NCF across the range.

Holds four of the five FOCAL parameters fixed at the values passed in and varies the fifth (sweep), recomputing the test-vs-ref NCF at each point. This mirrors the paper's sensitivity analyses: sweeping alpha_ref shows how the verdict shifts between operational- and embodied-dominated regimes, while sweeping an area or power term traces NCF against that dimension.

Parameters

- **test_pwr** – Nominal test power.
- **ref_pwr** – Nominal reference power.
- **test_area** – Nominal test area.
- **ref_area** – Nominal reference area.
- **alpha_ref** – Nominal embodied-to-operational weight in [0, 1].
- **sweep** – Name of the parameter to vary; one of `SWEEPABLE_PARAMETERS`.
- **values** – Explicit values to sweep. If None, a default range is built: alpha_ref sweeps [0, 1]; any other parameter sweeps from nominal / span to nominal * span.
- **num** – Number of points in the default range (ignored if values given).
- **span** – Multiplicative half-range for non-alpha default sweeps.
- **plot_path** – If given, save a PNG of NCF vs the swept parameter there (requires matplotlib, imported lazily).

Returns

A [FocalSweep](#) with the swept values, the NCF at each, the per-point [FocalComparison](#) objects, and any NCF==1 break-even crossings.

Raises

ValueError – if sweep is not a FOCAL parameter, or a default range cannot be built because the nominal swept value is non-positive.

chia.analysis.sustainability.focal.**sweep_both_arches_as_ref**(arch_a_pwr: float, arch_a_area: float, arch_b_pwr: float, arch_b_area: float, *, values: Sequence[float] | None = None, num: int = 51, plot_path: str | Path | None = None) → [FocalBothRefs](#)

Sweep alpha comparing two architectures with *each* used as the reference.

The NCF metric is not symmetric: $NCF(X, Y) \neq 1 / NCF(Y, X)$ in general, because NCF is a weighted *sum* of the area and power ratios rather than a single ratio (in fact $NCF(X, Y) * NCF(Y, X) \geq 1$). The reference design is the denominator, so for the *same* alpha you can get a different NCF — and even a different verdict — depending on which architecture you pick as the reference. There is no privileged “baseline” here, so it is worth looking at both directions before concluding that one design is more sustainable.

This runs two alpha sweeps over the same grid: one with arch A as the reference (test design is B) and one with arch B as the reference (test design is A). Compare the two curves at a given alpha to see the asymmetry.

Parameters

- **arch_a_pwr** – Power of architecture A.
- **arch_a_area** – Chip area of architecture A.
- **arch_b_pwr** – Power of architecture B.
- **arch_b_area** – Chip area of architecture B.
- **values** – Explicit alpha values to sweep. If None, sweeps [0, 1].
- **num** – Number of alpha points in the default range (ignored if **values**).
- **plot_path** – If given, save a PNG overlaying both NCF curves vs alpha (requires matplotlib, imported lazily).

Returns

A FocalBothRefs holding the arch_a_ref and arch_b_ref sweeps.

30.2 PFAS

PFAS-in-semiconductor-manufacturing mask data for chip PFAS/carbon modeling.

All data in this module is derived from the following work, and full credit for the underlying analysis belongs to its authors:

Mariam Elgamal, Abdulrahman Mahmoud, Gu-Yeon Wei, David Brooks, and Gage Hills. “Modeling PFAS in Semiconductor Manufacturing to Quantify Trade-offs in Energy Efficiency and Environmental Impact of Computing Systems.” arXiv preprint arXiv:2505.06727 (2025).

Abstract: <https://arxiv.org/abs/2505.06727> PDF: <https://arxiv.org/pdf/2505.06727>

Any use of this node should cite this work.

The paper uses the number of lithography masks as a first-order proxy for the amount of PFAS (“forever chemicals”) used in chip manufacturing, because the PFAS-containing layers (photoresist, anti-reflective coatings, topcoats) scale with mask count – eq. (1): $\#PFAS_{litho} \sim \text{number of lithography masks}$.

Data provenance within the paper:

- **BEOL_METAL_STACK_TYPES** – lithography masks per BEOL metal-stack recipe. Mask counts follow Table II (“Number of lithography process steps and masks per metal line process”); the recipe names are the BEOL stack types in the Figure 5 legend, read top-to-bottom.
- **BEOL_PROCESSES_PER_NODE** – per-node BEOL metal-stack composition, decoded from the stacked bars of Figure 5.
- **PROCESSES_PER_NODE** – the full per-node stack: FEOL and MOL layers (counts from Figure 5) prepended to the BEOL layers above.

Any transcription errors here are ours, not the authors’.

The PFAS proxy (lithography masks * area/yield) is only meaningful when considered relative to other instances.

`chia.analysis.sustainability.pfas.num_lith_masks(node: str) → int`

Number of PFAS-containing lithography masks for node.

Sums the full per-node stack in PROCESSES_PER_NODE: each FEOL and MOL layer counts as one mask, and each BEOL metal-stack recipe is priced by BEOL_METAL_STACK_TYPES (paper eq. 1: #PFAS_litho ~ number of masks).

`chia.analysis.sustainability.pfas.get_PFAS_proxy(node: str, area: float, yield_: float) → float`

PFAS manufacturing proxy for node (paper eq. 2):

$$\text{PFAS_chip_manufacturing} = \text{\#PFAS_litho} * \text{Area} / \text{Yield}$$

where #PFAS_litho is `num_lith_masks(node)`. area is die area (in whatever unit you use consistently) and `yield_` is the manufacturing yield in (0, 1]. (`yield` is a Python keyword, hence the trailing underscore.)

Like the mask proxy itself, the result is only meaningful relative to other instances computed the same way. This is not an absolute quantity of PFAS.

LLM SERVERS, PROVIDERS, AND AGENT NODES

API reference for the LLM servers, providers, and agents library nodes in `chia.models`.

31.1 Base class

```
class chia.base.llm_call.QueryResult(result: str, returncode: int, stderr: str, stream_result: str, success:
                                     bool = False)
```

Bases: `object`

Structured result from prompting an LLM or agent.

Parameters

- **result** (*str*) – The final response from the LLM or agent
- **returncode** (*int*) – The returncode from running the prompt
- **stderr** (*str*) – The stderr output from running the prompt (clis only)
- **stream_result** (*str*) – The full transcript of all turns of the LLM or agent
- **success** (*bool*) – Whether the prompt completed successfully

```
class chia.base.llm_call.LLMCallBase(system_message: str, dangerously_skip_permissions=<object
                                     object>, config=<object object>)
```

Bases: `ABC`

Polymorphic base container for generic LLM and agent configuration traits and behavior. Easy to switch between different backing providers, servers, and CLIs

```
abstractmethod prompt(user_message: str, tools: List[ChiaTool] | None = []) → QueryResult
```

Send a prompt to this LLM

Parameters

- **user_message** (*str*) – Message used to prompt the LLM
- **tools** (*Optional[List[ChiaTool]]*) – Tools available to the LLM during the call

31.2 Providers

We provide the prompt interface for each of the following providers. Many of our interfaces for these providers are lightly tested using free tokens. In particular, error handling is likely fragile.

31.2.1 CLIs

Claude `chia.models.claude`

Codex `chia.models.codex`

Copilot `chia.models.copilot`

OpenCode `chia.models.opencode`

Antigravity `chia.models.antigravity`

31.2.2 Remote providers

Bedrock `chia.models.bedrock`

Vertex `chia.models.vertex`

OpenAI `chia.models.openai_providers` `chia.models.openai_compat`

OpenRouter (OpenAI compatible) `chia.models.openai_providers` `chia.models.openai_compat`

FireworkAI (OpenAI compatible) `chia.models.openai_providers` `chia.models.openai_compat`

Groq (OpenAI compatible) `chia.models.openai_providers` `chia.models.openai_compat`

31.2.3 On-prem servers

VLLM `chia.models.vllm`

Ollama `chia.models.ollama`

INDICES AND TABLES

- `genindex`
- `search`

PYTHON MODULE INDEX

C

chia.analysis.sustainability.focal, 245
chia.analysis.sustainability.pfas, 248
chia.aws.config, 233
chia.aws.ec2, 233
chia.aws.host, 234
chia.aws.s3, 235
chia.base.bypass, 115
chia.base.cache, 118
chia.base.chia_kv_store, 123
chia.base.chia_wait, 122
chia.base.ChiaFunction, 120
chia.base.colocated, 124
chia.base.dispatch_proxy, 126
chia.base.llm_call, 127
chia.base.pid_registry, 127
chia.base.tools.AsyncBashTool, 130
chia.base.tools.AsyncJobTool, 131
chia.base.tools.BashTool, 130
chia.base.tools.ChiaTool, 128
chia.base.tools.ChiaToolTemplate, 130
chia.base.tools.util, 131
chia.chipyard.chipyard_hammer, 155
chia.chipyard.chisel_build_node, 146
chia.chipyard.circt, 149
chia.chipyard.cosim_node, 167
chia.chipyard.macrocompiler, 154
chia.chipyard.riscv_build_node, 157
chia.chipyard.riscv_dv_gen_node, 160
chia.chipyard.riscv_objdump_node, 158
chia.chipyard.spike_build_node, 148
chia.chipyard.state_def, 133
chia.chipyard.torture_run_node, 163
chia.chipyard.verilator_run_node, 164
chia.database.base, 217
chia.database.postgres_node, 219
chia.database.sqlite_node, 223
chia.esp.esp_workspace, 172
chia.esp.state_def, 171
chia.firesim.chia_functions, 205
chia.firesim.config, 199
chia.firesim.state_def, 206
chia.github.github_client, 227
chia.github.github_issues_node, 228
chia.github.github_pulls_node, 229
chia.github.state_def, 230
chia.models.antigravity, 186
chia.models.bedrock, 183
chia.models.claude, 181
chia.models.codex, 188
chia.models.copilot, 189
chia.models.ollama, 197
chia.models.openai_compat, 194
chia.models.openai_providers, 196
chia.models.opencode, 191
chia.models.vertex, 184
chia.models.vllm, 197
chia.simulators.gem5, 239
chia.vlsi.hammer, 209
chia.vlsi.sram_cacti.cacti_macrocompiler, 212
chia.vlsi.sram_cacti.cacti_runner, 212
chia.vlsi.sram_cacti.lef_gen, 215
chia.vlsi.sram_cacti.liberty_gen, 215
chia.vlsi.sram_cacti.sram_characterize, 212

A

`acc_dir` (*chia.esp.state_def.EspAccelSpec* property), 171

`accel()` (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 176

`accgen()` (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 176

`active` (*chia.base.bypass.Bypass* property), 117

`actor` (*chia.base.ChiaFunction.ChiaActorHandle* property), 121

`add_column_if_missing()` (*chia.database.base.DatabaseNode* static method), 219

`add_column_if_missing()` (*chia.database.postgres_node.PostgresNode* static method), 221

`add_column_if_missing()` (*chia.database.sqlite_node.SQLiteNode* static method), 225

`AdditionalModelProvider` (class in *chia.models.opencode*), 192

`afi` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 207

`agfi` (*chia.firesim.config.FireSimRunConfig* attribute), 202

`agfi` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 207

`ami_id` (*chia.firesim.config.FireSimRunConfig* attribute), 203

`analytical_area_estimate()` (in module *chia.vlsi.sram_cacti.cacti_runner*), 212

`AntigravityError`, 187

`AntigravityLLM` (class in *chia.models.antigravity*), 187

`AntigravityQueryResult` (class in *chia.models.antigravity*), 187

`api_key` (*chia.models.opencode.AdditionalModelProvider* attribute), 193

`archive` (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 144

`archive` (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 145

`assemble_generated_src_with_macros()` (in mod-

ule *chia.chipyard.macrocompiler*), 154

`AsyncBashTool` (class in *chia.base.tools.AsyncBashTool*), 130

`AsyncJobTool` (class in *chia.base.tools.AsyncJobTool*), 131

`AuthenticationError`, 181, 183, 185, 187, 188, 190, 191, 194

`AWSConfig` (class in *chia.aws.config*), 233

B

`base_url` (*chia.models.opencode.AdditionalModelProvider* attribute), 193

`BashTool` (class in *chia.base.tools.BashTool*), 130

`BedrockError`, 183

`BedrockLLM` (class in *chia.models.bedrock*), 184

`BillingError`, 182, 187, 188, 190, 191, 195

`bin_name` (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 145

`bin_name` (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 146

`binary_content` (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 135

`binary_name` (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 135

`binary_name` (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), 136

`bit_builder_recipe` (*chia.firesim.config.FireSimBuildConfig* attribute), 200

`bitstream_path` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 208

`BitstreamBuildResult` (class in *chia.firesim.state_def*), 206

`board_dir()` (in module *chia.esp.esp_workspace*), 172

`bootbinary_path` (*chia.firesim.config.FireSimRunConfig* attribute), 203

`bootbinary_s3_path` (*chia.firesim.config.FireSimRunConfig* attribute), 203

`breakeven_alpha` (*chia.analysis.sustainability.focal.FocalComparison* property), 246

`build()` (*chia.chipyard.chisel_build_node.ChiselBuildNode* method), 148

`build()` (*chia.chipyard.circt.BuildTool* method), 153

`build()` (*chia.chipyard.riscv_build_node.RiscvBuildNode* method), 158

`build()` (*chia.chipyard.spike_build_node.SpikeBuildNode* method), 149

`build()` (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 175

`build()` (*chia.simulators.gem5.Gem5ToolServer* method), 243

`build_artifact` (*chia.chipyard.state_def.TortureResult* attribute), 144

`build_gem5()` (*chia.simulators.gem5.Gem5Node* static method), 241

`build_group` (*chia.firesim.config.FireSimBuildConfig* attribute), 201

`build_id` (*chia.firesim.config.FireSimBuildConfig* attribute), 201

`build_id` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 207

`build_log` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 207

`build_program()` (*chia.chipyard.riscv_build_node.RiscvBuildNode* method), 158

`build_recipes_file` (*chia.firesim.config.FireSimConfig* attribute), 204

`build_ref` (*chia.firesim.config.FireSimRunConfig* attribute), 202

`build_ref` (*chia.firesim.state_def.BitstreamBuildResult* attribute), 207

`build_status()` (*chia.chipyard.circt.BuildTool* method), 153

`build_strategy` (*chia.firesim.config.FireSimBuildConfig* attribute), 200

`BuildArtifact` (class in *chia.chipyard.state_def*), 133

`BuildTarget` (class in *chia.chipyard.state_def*), 133

`BuildTool` (class in *chia.chipyard.circt*), 153

`Bypass` (class in *chia.base.bypass*), 116

C

`CactiCharacterization` (class in *chia.vlsi.sram_cacti.sram_characterize*), 212

`CACTIResult` (class in *chia.vlsi.sram_cacti.cacti_runner*), 212

`capture_gem5_source_state()` (*chia.simulators.gem5.Gem5Node* static method), 241

`characterize_srams_with_cacti()` (in module *chia.vlsi.sram_cacti.sram_characterize*), 214

`characterize_top_mems_conf_with_cacti()` (in module *chia.vlsi.sram_cacti.sram_characterize*), 213

`CharacterizedSRAM` (class in *chia.vlsi.sram_cacti.sram_characterize*), 212

`check_runs()` (*chia.github.github_pulls_node.GithubPullsNode* method), 230

`chia.analysis.sustainability.focal` module, 245

`chia.analysis.sustainability.pfas` module, 248

`chia.aws.config` module, 233

`chia.aws.ec2` module, 233

`chia.aws.host` module, 234

`chia.aws.s3` module, 235

`chia.base.bypass` module, 115

`chia.base.cache` module, 118

`chia.base.chia_kv_store` module, 123

`chia.base.chia_wait` module, 122

`chia.base.ChiaFunction` module, 120

`chia.base.colocated` module, 124

`chia.base.dispatch_proxy` module, 126

`chia.base.llm_call` module, 127

`chia.base.pid_registry` module, 127

`chia.base.tools.AsyncBashTool` module, 130

`chia.base.tools.AsyncJobTool` module, 131

`chia.base.tools.BashTool` module, 130

`chia.base.tools.ChiaTool` module, 128

`chia.base.tools.ChiaToolTemplate` module, 130

`chia.base.tools.util` module, 131

`chia.chipyard.chipyard_hammer` module, 155

`chia.chipyard.chisel_build_node` module, 146

`chia.chipyard.circt` module, 149

`chia.chipyard.cosim_node` module, 167

`chia.chipyard.compiler` module, 154

[chia.chipyard.riscv_build_node](#)
 module, 157
[chia.chipyard.riscv_dv_gen_node](#)
 module, 160
[chia.chipyard.riscv_objdump_node](#)
 module, 158
[chia.chipyard.spike_build_node](#)
 module, 148
[chia.chipyard.state_def](#)
 module, 133
[chia.chipyard.torture_run_node](#)
 module, 163
[chia.chipyard.verilator_run_node](#)
 module, 164
[chia.database.base](#)
 module, 217
[chia.database.postgres_node](#)
 module, 219
[chia.database.sqlite_node](#)
 module, 223
[chia.esp.esp_workspace](#)
 module, 172
[chia.esp.state_def](#)
 module, 171
[chia.firesim.chia_functions](#)
 module, 205
[chia.firesim.config](#)
 module, 199
[chia.firesim.state_def](#)
 module, 206
[chia.github.github_client](#)
 module, 227
[chia.github.github_issues_node](#)
 module, 228
[chia.github.github_pulls_node](#)
 module, 229
[chia.github.state_def](#)
 module, 230
[chia.models.antigravity](#)
 module, 186
[chia.models.bedrock](#)
 module, 183
[chia.models.claude](#)
 module, 181
[chia.models.codex](#)
 module, 188
[chia.models.copilot](#)
 module, 189
[chia.models.ollama](#)
 module, 197
[chia.models.openai_compat](#)
 module, 194
[chia.models.openai_providers](#)
 module, 196
[chia.models.opencode](#)
 module, 191
[chia.models.vertex](#)
 module, 184
[chia.models.vllm](#)
 module, 197
[chia.simulators.gem5](#)
 module, 239
[chia.vlsi.hammer](#)
 module, 209
[chia.vlsi.sram_cacti.cacti_macrocompiler](#)
 module, 212
[chia.vlsi.sram_cacti.cacti_runner](#)
 module, 212
[chia.vlsi.sram_cacti.lef_gen](#)
 module, 215
[chia.vlsi.sram_cacti.liberty_gen](#)
 module, 215
[chia.vlsi.sram_cacti.sram_characterize](#)
 module, 212
[chia_actor\(\)](#) (in module [chia.base.ChiaFunction](#)), 121
[chia_cancel\(\)](#) (in module [chia.base.ChiaFunction](#)), 122
[chia_cancel\(\)](#) (in module [chia.base.pid_registry](#)), 127
[chia_wait\(\)](#) (in module [chia.base.chia_wait](#)), 122
[ChiaActorHandle](#) (class in [chia.base.ChiaFunction](#)), 121
[ChiaCallRemote\(\)](#) (in module [chia.base.ChiaFunction](#)), 121
[ChiaFunction](#) (class in [chia.base.ChiaFunction](#)), 120
[ChiaTool](#) (class in [chia.base.tools.ChiaTool](#)), 128
[ChiaToolTemplate](#) (class in [chia.base.tools.ChiaToolTemplate](#)), 130
[ChiaWrapped](#) (class in [chia.base.ChiaFunction](#)), 120
[chipyard_path](#) ([chia.firesim.config.FireSimConfig](#) attribute), 204
[ChipyardHammerCollectResult](#) (class in [chia.chipyard.chipyard_hammer](#)), 155
[ChipyardHammerNode](#) (class in [chia.chipyard.chipyard_hammer](#)), 156
[ChipyardHammerResult](#) (class in [chia.chipyard.chipyard_hammer](#)), 155
[chisel_elaborate_to_chirrtl\(\)](#) (in module [chia.chipyard.circt](#)), 152
[ChiselBuildNode](#) (class in [chia.chipyard.chisel_build_node](#)), 146
[circt_ninja_build\(\)](#) (in module [chia.chipyard.circt](#)), 152
[circt_opt_lower_hw_to_verilog\(\)](#) (in module [chia.chipyard.circt](#)), 150
[circt_opt_run\(\)](#) (in module [chia.chipyard.circt](#)), 150
[circt_run_lit\(\)](#) (in module [chia.chipyard.circt](#)), 153
[circt_warm_build\(\)](#) (in module [chia.chipyard.circt](#)), 153

- ClaudeCodeError, 181
 ClaudeCodeLLM (class in chia.models.claude), 182
 ClaudeCodeQueryResult (class in chia.models.claude), 181
 close() (chia.base.colocated.ColocatedNode method), 126
 close() (chia.database.base.DatabaseNode method), 218
 close() (chia.simulators.gem5.Gem5Node method), 241
 CodexError, 188
 CodexLLM (class in chia.models.codex), 189
 CodexQueryResult (class in chia.models.codex), 189
 collect() (chia.chipyard.chipyard_hammer.ChipyardHammerNode static method), 157
 collect() (chia.esp.esp_workspace.EspWorkspaceNode static method), 174
 collect() (chia.vlsi.hammer.HammerNode static method), 210
 collect_fs() (chia.vlsi.hammer.HammerNode method), 211
 collect_waveform() (in module chia.chipyard.verilator_run_node), 166
 ColocatedNode (class in chia.base.colocated), 125
 commit_log (chia.chipyard.state_def.SpikeResult attribute), 140
 compile_generator() (chia.chipyard.riscv_dv_gen_node.RiscvDvGenNode method), 162
 completed (chia.chipyard.state_def.CosimResult attribute), 139
 config (chia.chipyard.state_def.BuildArtifact attribute), 134
 config (chia.chipyard.state_def.TortureResult attribute), 143
 config_package (chia.chipyard.state_def.BuildArtifact attribute), 134
 config_package (chia.chipyard.state_def.TortureResult attribute), 143
 configure() (chia.esp.esp_workspace.EspWorkspaceNode static method), 174
 ContentBlockedError, 185
 ContextLengthExceededError, 195
 conversation_comments() (chia.github.github_pulls_node.GithubPullsNode method), 230
 CopilotError, 189
 CopilotLLM (class in chia.models.copilot), 190
 CopilotQueryResult (class in chia.models.copilot), 190
 CosimResult (class in chia.chipyard.state_def), 139
 crosses_unity (chia.analysis.sustainability.focal.FocalSweep property), 246
- ## D
- DatabaseNode (class in chia.database.base), 218
 delete() (chia.aws.s3.S3Node method), 237
 deploy_path (chia.firesim.config.FireSimConfig attribute), 204
 deploy_quintuplet (chia.firesim.config.FireSimBuildConfig attribute), 200
 design (chia.firesim.config.FireSimBuildConfig attribute), 200
 digest (chia.chipyard.state_def.SpikeBuildArtifact attribute), 141
 download_file() (chia.aws.s3.S3Node method), 236
 driver_s3_path (chia.firesim.config.FireSimRunConfig attribute), 203
 driver_s3_path (chia.firesim.state_def.BitstreamBuildResult attribute), 207
 driver_tarball_path (chia.firesim.config.FireSimRunConfig attribute), 203
 dump (chia.chipyard.state_def.RiscvBuildArtifact attribute), 136
 dump (chia.chipyard.state_def.RiscvObjdumpArtifact attribute), 137
 dump() (chia.chipyard.riscv_objdump_node.RiscvObjdumpNode method), 159
 duration_seconds (chia.firesim.state_def.FireSimRunResult attribute), 208
- ## E
- EC2Instance (class in chia.aws.ec2), 233
 EC2InstanceConfig (class in chia.aws.config), 233
 elf_name (chia.chipyard.state_def.CosimResult attribute), 139
 enable_pr (chia.firesim.config.FireSimBuildConfig attribute), 201
 ensure_bucket() (chia.aws.s3.S3Node method), 237
 EphemeralEC2Host (class in chia.aws.host), 234
 EspAccelResult (class in chia.esp.state_def), 171
 EspAccelSpec (class in chia.esp.state_def), 171
 EspAccgenResult (class in chia.esp.state_def), 171
 EspCollectResult (class in chia.esp.state_def), 171
 EspConfigResult (class in chia.esp.state_def), 171
 EspFpgaRunResult (class in chia.esp.state_def), 172
 EspMakeResult (class in chia.esp.state_def), 171
 EspSimResult (class in chia.esp.state_def), 171
 EspSoftArtifact (class in chia.esp.state_def), 172
 EspSynthResult (class in chia.esp.state_def), 172
 EspWorkspaceNode (class in chia.esp.esp_workspace), 173
 ExecResult (class in chia.database.base), 217
 execute() (chia.database.base.DatabaseNode static method), 219
 execute() (chia.database.postgres_node.PostgresNode static method), 221

`execute()` (*chia.database.postgres_node.PostgresQueryTool* method), 222
`execute()` (*chia.database.sqlite_node.SQLiteNode* static method), 225
`execute()` (*chia.database.sqlite_node.SQLiteQueryTool* method), 226
`executemany()` (*chia.database.base.DatabaseNode* static method), 219
`executemany()` (*chia.database.postgres_node.PostgresNode* static method), 221
`executemany()` (*chia.database.sqlite_node.SQLiteNode* static method), 225
`executescript()` (*chia.database.base.DatabaseNode* static method), 218
`executescript()` (*chia.database.postgres_node.PostgresNode* static method), 221
`executescript()` (*chia.database.sqlite_node.SQLiteNode* static method), 225
`exists()` (*chia.aws.s3.S3Node* method), 237
`extra_args` (*chia.chipyard.riscv_dv_gen_node.GenSpec* attribute), 160

F

`failing_trace_gz` (*chia.chipyard.state_def.CosimResult* attribute), 139
`fetch_build_info()` (in module *chia.firesim.config*), 205
`file_server()` (*chia.base.bypass.Bypass* method), 117
`files` (*chia.chipyard.state_def.ProgramBuildArtifact* attribute), 136
`FireMarshalArtifact` (class in *chia.chipyard.state_def*), 144
`FireMarshalBaseArtifact` (class in *chia.chipyard.state_def*), 145
`firesim_build_bitstream()` (in module *chia.firesim.chia_functions*), 205
FIRESIM_METASIM_VERILATOR (*chia.chipyard.state_def.BuildTarget* attribute), 133
`firesim_run_workload()` (in module *chia.firesim.chia_functions*), 206
`FireSimBuildConfig` (class in *chia.firesim.config*), 199
`FireSimConfig` (class in *chia.firesim.config*), 204
`FireSimRunConfig` (class in *chia.firesim.config*), 201
`FireSimRunResult` (class in *chia.firesim.state_def*), 208
`FireworksLLM` (class in *chia.models.openai_providers*), 196
`first_divergence` (*chia.chipyard.state_def.CosimResult* attribute), 139
`firtool_lower_chirrtl_to_hw()` (in module *chia.chipyard.circt*), 149
`focal_compare()` (in module *chia.analysis.sustainability.focal*), 246
`focal_sweep()` (in module *chia.analysis.sustainability.focal*), 247
`FocalBothRefs` (class in *chia.analysis.sustainability.focal*), 247
`FocalComparison` (class in *chia.analysis.sustainability.focal*), 245
`FocalSweep` (class in *chia.analysis.sustainability.focal*), 246
`fpga_frequency` (*chia.firesim.config.FireSimBuildConfig* attribute), 200
`fpga_program()` (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 178
`fpga_run()` (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 178

G

`Gem5BuildArtifact` (class in *chia.simulators.gem5*), 239
`Gem5Isa` (class in *chia.simulators.gem5*), 239
`Gem5Node` (class in *chia.simulators.gem5*), 239
`Gem5RunResult` (class in *chia.simulators.gem5*), 239
`Gem5SourceState` (class in *chia.simulators.gem5*), 239
`Gem5ToolServer` (class in *chia.simulators.gem5*), 242
`Gem5Variant` (class in *chia.simulators.gem5*), 239
`gen_timeout` (*chia.chipyard.riscv_dv_gen_node.GenSpec* attribute), 160
`generate()` (*chia.chipyard.riscv_dv_gen_node.RiscvDvGenNode* method), 162
`generate_cacti_macrocompiler_lib()` (in module *chia.vlsi.sram_cacti.cacti_macrocompiler*), 212
`generate_lef()` (in module *chia.vlsi.sram_cacti.lef_gen*), 215
`generate_liberty()` (in module *chia.vlsi.sram_cacti.liberty_gen*), 215
`generate_macro_stubs()` (in module *chia.chipyard.macrocompiler*), 155
`generate_macrocompiler_lib()` (in module *chia.chipyard.macrocompiler*), 154
`generated_src_files` (*chia.chipyard.state_def.BuildArtifact* attribute), 134
`GenSpec` (class in *chia.chipyard.riscv_dv_gen_node*), 160
`get()` (*chia.github.github_issues_node.GithubIssuesNode* method), 228
`get()` (*chia.github.github_pulls_node.GithubPullsNode* method), 229
`get()` (in module *chia.base.ChiaFunction*), 121
`get_active_bypass()` (in module *chia.base.bypass*), 116
`get_active_cache()` (in module *chia.base.cache*), 119
`get_bytes()` (*chia.aws.s3.S3Node* method), 236
`get_default_ami()` (in module *chia.aws.ec2*), 233

[get_dispatch_proxy\(\)](#) (in module `chia.base.dispatch_proxy`), 127
[get_issue\(\)](#) (`chia.github.github_issues_node.GithubIssuesNode` method), 228
[get_or_create_kv_store\(\)](#) (in module `chia.base.chia_kv_store`), 124
[get_PFAS_proxy\(\)](#) (in module `chia.analysis.sustainability.pfas`), 249
[get_provider_info\(\)](#) (`chia.base.bypass.Bypass` method), 118
[get_pull\(\)](#) (`chia.github.github_pulls_node.GithubPullsNode` method), 229
[get_state\(\)](#) (`chia.base.bypass.Bypass` method), 118
[GithubAuthError](#), 227
[GithubCheckAnnotation](#) (class in `chia.github.state_def`), 231
[GithubCheckRun](#) (class in `chia.github.state_def`), 231
[GithubClient](#) (class in `chia.github.github_client`), 227
[GithubComment](#) (class in `chia.github.state_def`), 230
[GithubError](#), 227
[GithubIssue](#) (class in `chia.github.state_def`), 230
[GithubIssuesNode](#) (class in `chia.github.github_issues_node`), 228
[GithubNotFoundError](#), 227
[GithubPull](#) (class in `chia.github.state_def`), 230
[GithubPullFeedback](#) (class in `chia.github.state_def`), 231
[GithubPullsNode](#) (class in `chia.github.github_pulls_node`), 229
[GithubRateLimitError](#), 227
[GithubRequestError](#), 227
[GithubReview](#) (class in `chia.github.state_def`), 230
[GithubReviewComment](#) (class in `chia.github.state_def`), 230
[GithubServerError](#), 227
[golden_model_digest](#) (`chia.chipyard.state_def.BuildArtifact` attribute), 135
[goodbye_world_tool\(\)](#) (`chia.base.tools.ChiaToolTemplate.ChiaToolTemplate` method), 130
[GroqLLM](#) (class in `chia.models.openai_providers`), 196

H

[HammerCollectFsResult](#) (class in `chia.vlsi.hammer`), 209
[HammerCollectResult](#) (class in `chia.vlsi.hammer`), 209
[HammerMatchResult](#) (class in `chia.vlsi.hammer`), 209
[HammerNode](#) (class in `chia.vlsi.hammer`), 209
[HammerResult](#) (class in `chia.vlsi.hammer`), 209
[headers_tar](#) (`chia.chipyard.state_def.SpikeBuildArtifact` attribute), 141
[hello_world_tool\(\)](#) (`chia.base.tools.ChiaToolTemplate.ChiaToolTemplate` method), 130

[hw_config_name](#) (`chia.firesim.config.FireSimRunConfig` attribute), 202
[idle_entry](#) (`chia.firesim.state_def.BitstreamBuildResult` attribute), 207
[id](#) (`chia.models.opencode.AdditionalModelProvider` attribute), 192
[img_name](#) (`chia.chipyard.state_def.FireMarshalArtifact` attribute), 145
[img_name](#) (`chia.chipyard.state_def.FireMarshalBaseArtifact` attribute), 145
[incremental_base_build_id](#) (`chia.firesim.config.FireSimBuildConfig` attribute), 201
[init_schema\(\)](#) (`chia.database.base.DatabaseNode` static method), 218
[init_schema\(\)](#) (`chia.database.postgres_node.PostgresNode` static method), 221
[init_schema\(\)](#) (`chia.database.sqlite_node.SQLiteNode` static method), 224
[instance_type](#) (`chia.firesim.config.FireSimRunConfig` attribute), 202
[instr_cnt](#) (`chia.chipyard.riscv_dv_gen_node.GenSpec` attribute), 160
[InvalidRequestError](#), 182, 183, 185, 187, 188, 190, 191, 194
[is_bypassed\(\)](#) (`chia.base.bypass.Bypass` method), 117
[is_cached\(\)](#) (in module `chia.base.cache`), 120
[is_empty\(\)](#) (`chia.github.state_def.GithubPullFeedback` method), 231
[isa](#) (`chia.chipyard.state_def.SpikeResult` attribute), 140
[iterations](#) (`chia.chipyard.riscv_dv_gen_node.GenSpec` attribute), 160

J

[json_name](#) (`chia.chipyard.state_def.FireMarshalArtifact` attribute), 145

K

L

[launch_ec2_instances\(\)](#) (in module `chia.aws.ec2`), 233
[lib_content](#) (`chia.chipyard.state_def.SpikeBuildArtifact` attribute), 140
[lib_name](#) (`chia.chipyard.state_def.SpikeBuildArtifact` attribute), 140
[LibertyCorner](#) (class in `chia.vlsi.sram_cacti.liberty_gen`), 215

[linked_pull_requests\(\)](#)
 ([chia.github.github_issues_node.GithubIssuesNode](#)
[method](#)), 228
[list\(\)](#) ([chia.aws.s3.S3Node](#) [method](#)), 236
[list_circt_passes\(\)](#) ([in module chia.chipyard.circt](#)),
 152
[list_matches\(\)](#) ([chia.vlsi.hammer.HammerNode](#) [static](#)
[method](#)), 211
[list_workloads\(\)](#) ([chia.simulators.gem5.Gem5ToolServer](#)
[method](#)), 243
[lit_status\(\)](#) ([chia.chipyard.circt.LitTool](#) [method](#)), 153
[LitTool](#) ([class in chia.chipyard.circt](#)), 153
[LLMCallBase](#) ([class in chia.base.llm_call](#)), 127
[load_build_recipes\(\)](#) ([in module chia.firesim.config](#)),
 204
[load_hwdb\(\)](#) ([in module chia.firesim.config](#)), 204
[load_state\(\)](#) ([chia.base.bypass.Bypass](#) [method](#)), 118
[log](#) ([chia.chipyard.state_def.RunResult](#) [attribute](#)), 137
[log](#) ([chia.chipyard.state_def.SpikeResult](#) [attribute](#)), 140
[log_s3_path](#) ([chia.chipyard.state_def.RunResult](#) [at-](#)
[tribute](#)), 138

M

[make\(\)](#) ([chia.chipyard.chipyard_hammer.ChipyardHammerNode](#)
[static method](#)), 156
[make\(\)](#) ([chia.esp.esp_workspace.EspWorkspaceNode](#)
[static method](#)), 173
[make_name](#) ([chia.esp.state_def.EspAccelSpec](#) [property](#)),
 171
[make_router_lifespan\(\)](#) ([in module](#)
[chia.base.tools.util](#)), 131
[market](#) ([chia.firesim.config.FireSimBuildConfig](#) [at-](#)
[tribute](#)), 201
[market](#) ([chia.firesim.config.FireSimRunConfig](#) [attribute](#)),
 203
[match](#) ([chia.chipyard.state_def.CosimResult](#) [attribute](#)),
 139
[matched](#) ([chia.chipyard.state_def.CosimResult](#) [attribute](#)),
 139
[MaxOutputTokensError](#), 182, 184, 185, 187, 188, 190,
 191, 195
[metasim_customruntimeconfig](#)
 ([chia.firesim.config.FireSimBuildConfig](#) [at-](#)
[tribute](#)), 201
[mode](#) ([chia.chipyard.state_def.TortureResult](#) [attribute](#)),
 143
[models](#) ([chia.models.opencode.AdditionalModelProvider](#)
[attribute](#)), 192
[module](#)
 [chia.analysis.sustainability.focal](#), 245
 [chia.analysis.sustainability.pfas](#), 248
 [chia.aws.config](#), 233
 [chia.aws.ec2](#), 233
 [chia.aws.host](#), 234
 [chia.aws.s3](#), 235
 [chia.base.bypass](#), 115
 [chia.base.cache](#), 118
 [chia.base.chia_kv_store](#), 123
 [chia.base.chia_wait](#), 122
 [chia.base.ChiaFunction](#), 120
 [chia.base.colocated](#), 124
 [chia.base.dispatch_proxy](#), 126
 [chia.base.llm_call](#), 127
 [chia.base.pid_registry](#), 127
 [chia.base.tools.AsyncBashTool](#), 130
 [chia.base.tools.AsyncJobTool](#), 131
 [chia.base.tools.BashTool](#), 130
 [chia.base.tools.ChiaTool](#), 128
 [chia.base.tools.ChiaToolTemplate](#), 130
 [chia.base.tools.util](#), 131
 [chia.chipyard.chipyard_hammer](#), 155
 [chia.chipyard.chisel_build_node](#), 146
 [chia.chipyard.circt](#), 149
 [chia.chipyard.cosim_node](#), 167
 [chia.chipyard.compilers](#), 154
 [chia.chipyard.riscv_build_node](#), 157
 [chia.chipyard.riscv_dv_gen_node](#), 160
 [chia.chipyard.riscv_objdump_node](#), 158
 [chia.chipyard.spike_build_node](#), 148
 [chia.chipyard.state_def](#), 133
 [chia.chipyard.torture_run_node](#), 163
 [chia.chipyard.verilator_run_node](#), 164
 [chia.database.base](#), 217
 [chia.database.postgres_node](#), 219
 [chia.database.sqlite_node](#), 223
 [chia.esp.esp_workspace](#), 172
 [chia.esp.state_def](#), 171
 [chia.firesim.chia_functions](#), 205
 [chia.firesim.config](#), 199
 [chia.firesim.state_def](#), 206
 [chia.github.github_client](#), 227
 [chia.github.github_issues_node](#), 228
 [chia.github.github_pulls_node](#), 229
 [chia.github.state_def](#), 230
 [chia.models.antigravity](#), 186
 [chia.models.bedrock](#), 183
 [chia.models.claude](#), 181
 [chia.models.codex](#), 188
 [chia.models.copilot](#), 189
 [chia.models.ollama](#), 197
 [chia.models.openai_compat](#), 194
 [chia.models.openai_providers](#), 196
 [chia.models.opencode](#), 191
 [chia.models.vertex](#), 184
 [chia.models.vllm](#), 197
 [chia.simulators.gem5](#), 239
 [chia.vlsi.hammer](#), 209

[chia.vlsi.sram_cacti.cacti_macrocompiler](#), 212
[chia.vlsi.sram_cacti.cacti_runner](#), 212
[chia.vlsi.sram_cacti.lef_gen](#), 215
[chia.vlsi.sram_cacti.liberty_gen](#), 215
[chia.vlsi.sram_cacti.sram_characterize](#), 212
[more_sustainable](#) ([chia.analysis.sustainability.focal.FocalComparison](#) property), 245

N

[name](#) ([chia.chipyard.state_def.BuildArtifact](#) attribute), 133
[name](#) ([chia.chipyard.state_def.TortureResult](#) attribute), 143
[name](#) ([chia.chipyard.state_def.TortureTestRun](#) attribute), 142
[name](#) ([chia.firesim.config.FireSimBuildConfig](#) attribute), 199
[name](#) ([chia.models.opencode.AdditionalModelProvider](#) attribute), 192
[npm](#) ([chia.models.opencode.AdditionalModelProvider](#) attribute), 192
[num_failures](#) ([chia.chipyard.state_def.TortureResult](#) attribute), 144
[num_lith_masks\(\)](#) (in module [chia.analysis.sustainability.pfas](#)), 249
[num_sims](#) ([chia.firesim.config.FireSimRunConfig](#) attribute), 202
[num_tests](#) ([chia.chipyard.state_def.TortureResult](#) attribute), 144
[NvidiaLLM](#) (class in [chia.models.openai_providers](#)), 196

O

[ObjectRefCallback](#) (class in [chia.base.ChiaFunction](#)), 121
[OllamaLLM](#) (class in [chia.models.ollama](#)), 197
[OpenAICompatError](#), 194
[OpenAICompatLLM](#) (class in [chia.models.openai_compat](#)), 195
[OpenAILLM](#) (class in [chia.models.openai_providers](#)), 196
[OpenCodeError](#), 191
[OpenCodeLLM](#) (class in [chia.models.opencode](#)), 193
[OpenCodeQueryResult](#) (class in [chia.models.opencode](#)), 193
[OpenRouterLLM](#) (class in [chia.models.openai_providers](#)), 196
[options](#) ([chia.models.opencode.AdditionalModelProvider](#) attribute), 193
[options\(\)](#) ([chia.base.colocated.PinnedChiaFn](#) method), 124
[out](#) ([chia.chipyard.state_def.RunResult](#) attribute), 137
[out_s3_path](#) ([chia.chipyard.state_def.RunResult](#) attribute), 138

[OVERNIGHT](#) ([chia.chipyard.state_def.TortureMode](#) attribute), 142
[owns_placement_group](#) ([chia.base.colocated.ColocatedNode](#) property), 125
[owns_placement_group](#) ([chia.simulators.gem5.Gem5Node](#) property), 340

P

[paramstyle](#) ([chia.database.base.DatabaseNode](#) attribute), 218
[paramstyle](#) ([chia.database.postgres_node.PostgresNode](#) attribute), 221
[paramstyle](#) ([chia.database.sqlite_node.SQLiteNode](#) attribute), 224
[parse_gem5_stats\(\)](#) ([chia.simulators.gem5.Gem5Node](#) static method), 242
[parse_gem5_stats_file\(\)](#) ([chia.simulators.gem5.Gem5Node](#) static method), 242
[parse_mems_conf\(\)](#) (in module [chia.chipyard_macrocompiler](#)), 154
[parse_rate_limit_event\(\)](#) (in module [chia.models.claude](#)), 182
[parse_rate_limit_reset\(\)](#) (in module [chia.models.antigravity](#)), 187
[parse_rate_limit_reset\(\)](#) (in module [chia.models.claude](#)), 182
[parse_rate_limit_reset\(\)](#) (in module [chia.models.codex](#)), 189
[parse_rate_limit_reset\(\)](#) (in module [chia.models.copilot](#)), 190
[parse_run_error\(\)](#) (in module [chia.models.opencode](#)), 192
[parse_session_id\(\)](#) (in module [chia.models.codex](#)), 189
[parse_session_id\(\)](#) (in module [chia.models.copilot](#)), 190
[parse_session_id\(\)](#) (in module [chia.models.opencode](#)), 192
[pct_change](#) ([chia.analysis.sustainability.focal.FocalComparison](#) property), 245
[PidRegistryActor](#) (class in [chia.base.pid_registry](#)), 127
[PinnedChiaFn](#) (class in [chia.base.colocated](#)), 124
[placement_group](#) ([chia.base.colocated.ColocatedNode](#) property), 125
[placement_group](#) ([chia.simulators.gem5.Gem5Node](#) property), 240
[platform](#) ([chia.firesim.config.FireSimBuildConfig](#) attribute), 199
[platform_config](#) ([chia.firesim.config.FireSimBuildConfig](#) attribute), 200

plusarg_passthrough (chia.firesim.config.FireSimRunConfig attribute), 202
 plusargs (chia.chipyard.riscv_dv_gen_node.GenSpec attribute), 160
 post_build_hook (chia.firesim.config.FireSimBuildConfig attribute), 200
 PostgresNode (class in chia.database.postgres_node), 220
 PostgresQueryTool (class in chia.database.postgres_node), 222
 pr_base_build_id (chia.firesim.config.FireSimBuildConfig attribute), 201
 pr_module_name (chia.firesim.config.FireSimBuildConfig attribute), 201
 pr_partition_cell (chia.firesim.config.FireSimBuildConfig attribute), 201
 ProgramBuildArtifact (class in chia.chipyard.state_def), 136
 prompt() (chia.base.llm_call.LLMCallBase method), 128
 prompt() (chia.models.antigravity.AntigravityLLM method), 188
 prompt() (chia.models.bedrock.BedrockLLM method), 184
 prompt() (chia.models.claude.ClaudeCodeLLM method), 183
 prompt() (chia.models.codex.CodexLLM method), 189
 prompt() (chia.models.copilot.CopilotLLM method), 190
 prompt() (chia.models.ollama.OllamaLLM method), 197
 prompt() (chia.models.openai_compat.OpenAICompatLLM method), 195
 prompt() (chia.models.openai_providers.FireworksLLM method), 196
 prompt() (chia.models.openai_providers.GroqLLM method), 196
 prompt() (chia.models.openai_providers.NvidiaLLM method), 196
 prompt() (chia.models.openai_providers.OpenAILLM method), 196
 prompt() (chia.models.openai_providers.OpenRouterLLM method), 196
 prompt() (chia.models.opencode.OpenCodeLLM method), 193
 prompt() (chia.models.vertex.VertexGeminiLLM method), 186
 prompt() (chia.models.vertex.VertexGenericLLM method), 186
 prompt() (chia.models.vllm.VLLM method), 198
 pseg_test_s (chia.chipyard.state_def.TortureTestRun attribute), 143
 pull_diff() (chia.github.github_pulls_node.GithubPullsNode method), 229
 put_bytes() (chia.aws.s3.S3Node method), 236
 put_file() (chia.esp.esp_workspace.EspWorkspaceNode static method), 174

Q

query() (chia.database.base.DatabaseNode static method), 219
 query() (chia.database.postgres_node.PostgresNode static method), 222
 query() (chia.database.postgres_node.PostgresQueryTool method), 222
 query() (chia.database.sqlite_node.SQLiteNode static method), 225
 query() (chia.database.sqlite_node.SQLiteQueryTool method), 226
 query_kv() (in module chia.base.chia_kv_store), 124
 query_one() (chia.database.base.DatabaseNode static method), 219
 query_one() (chia.database.postgres_node.PostgresNode static method), 222
 query_one() (chia.database.sqlite_node.SQLiteNode static method), 225
 query_value() (chia.database.base.DatabaseNode static method), 219
 query_value() (chia.database.postgres_node.PostgresNode static method), 222
 query_value() (chia.database.sqlite_node.SQLiteNode static method), 225
 QueryResult (class in chia.base.llm_call), 127

R

RateLimitError, 181, 183, 185, 187–189, 191, 194
 read_chunk() (chia.vlsi.hammer.HammerNode static method), 211
 read_waveform_chunk() (in module chia.chipyard.verilator_run_node), 166
 rebuild_circt_opt_with_custom_pass() (in module chia.chipyard.circt), 151
 recent() (chia.github.github_issues_node.GithubIssuesNode method), 228
 recent() (chia.github.github_pulls_node.GithubPullsNode method), 229
 recipe_name (chia.firesim.state_def.BitstreamBuildResult attribute), 207
 remap_with_macrocompiler() (in module chia.chipyard.macrocompiler), 154
 remove() (chia.esp.esp_workspace.EspWorkspaceNode static method), 174
 remove_waveform() (in module chia.chipyard.verilator_run_node), 166
 rename_ports() (in module chia.chipyard.macrocompiler), 154

REPLAY (*chia.chipyard.state_def.TortureMode* attribute), 142
 resolve_build() (*chia.firesim.config.FireSimRunConfig* method), 204
 resolve_tool_url() (in *module chia.base.tools.ChiaTool*), 129
 restore_gem5_source_state() (*chia.simulators.gem5.Gem5Node* static method), 241
 result_id (*chia.firesim.config.FireSimRunConfig* attribute), 203
 returncode (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 returncode (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 145
 returncode (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 146
 returncode (*chia.chipyard.state_def.ProgramBuildArtifact* attribute), 136
 returncode (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 136
 returncode (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), 137
 returncode (*chia.chipyard.state_def.RunResult* attribute), 138
 returncode (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 142
 returncode (*chia.chipyard.state_def.SpikeResult* attribute), 140
 returncode (*chia.chipyard.state_def.TortureResult* attribute), 144
 review_comments() (*chia.github.github_pulls_node.GithubPullsNode* method), 230
 review_feedback() (*chia.github.github_pulls_node.GithubPullsNode* method), 230
 reviews() (*chia.github.github_pulls_node.GithubPullsNode* method), 230
 RiscvBuildArtifact (class in *chia.chipyard.state_def*), 135
 RiscvBuildNode (class in *chia.chipyard.riscv_build_node*), 157
 RiscvDvGenNode (class in *chia.chipyard.riscv_dv_gen_node*), 161
 RiscvObjdumpArtifact (class in *chia.chipyard.state_def*), 136
 RiscvObjdumpNode (class in *chia.chipyard.riscv_objdump_node*), 159
 rootfs_outputs (*chia.firesim.state_def.FireSimRunResult* attribute), 208
 rootfs_path (*chia.firesim.config.FireSimRunConfig* attribute), 203
 rootfs_s3_path (*chia.firesim.config.FireSimRunConfig* attribute), 204
 rsync_down() (*chia.aws.host.EphemeralEC2Host* method), 235
 rsync_up() (*chia.aws.host.EphemeralEC2Host* method), 234
 rtlsim_sig (*chia.chipyard.state_def.TortureTestRun* attribute), 143
 run() (*chia.aws.host.EphemeralEC2Host* method), 235
 run() (*chia.base.tools.AsyncBashTool.AsyncBashTool* method), 131
 run() (*chia.chipyard.verilator_run_node.VerilatorRunNode* method), 164
 run() (*chia.simulators.gem5.Gem5ToolServer* method), 243
 run() (*chia.vlsi.hammer.HammerNode* static method), 210
 run_cacti() (in *module chia.vlsi.sram_cacti.cacti_runner*), 212
 run_command() (*chia.base.tools.BashTool.BashTool* method), 130
 run_gem5() (*chia.simulators.gem5.Gem5Node* static method), 241
 run_lit() (*chia.chipyard.circt.LitTool* method), 153
 run_metasim() (*chia.chipyard.verilator_run_node.VerilatorRunNode* method), 166
 run_script() (*chia.aws.host.EphemeralEC2Host* method), 235
 run_status() (*chia.base.tools.AsyncBashTool.AsyncBashTool* method), 131
 RunResult (class in *chia.chipyard.state_def*), 137
 runtime_libs (*chia.chipyard.state_def.BuildArtifact* attribute), 135
 S3AuthError, 235
 S3Error, 235
 S3Node (class in *chia.aws.s3*), 236
 S3NotFoundError, 235
 S3ObjectInfo (class in *chia.aws.s3*), 236
 S3RequestError, 235
 S3ServerError, 235
 schema() (*chia.database.base.DatabaseNode* static method), 219
 schema() (*chia.database.postgres_node.PostgresNode* static method), 221
 schema() (*chia.database.postgres_node.PostgresQueryTool* method), 222
 schema() (*chia.database.sqlite_node.SQLiteNode* static method), 225
 schema() (*chia.database.sqlite_node.SQLiteQueryTool* method), 226
 seed (*chia.chipyard.riscv_dv_gen_node.GenSpec* attribute), 160
 ServerError, 182, 184, 185, 187, 188, 190, 191, 194
 set_cond() (*chia.base.bypass.Bypass* method), 117

set_provider() (*chia.base.bypass.Bypass* method), 117
 setup() (*chia.base.tools.ChiaTool.ChiaTool* method), 129
 setup() (*chia.base.tools.ChiaToolTemplate.ChiaToolTemplate* method), 130
 should_proxy() (in module *chia.base.dispatch_proxy*), 126
 sim() (*chia.esp.esp_workspace.EspWorkspaceNode* static method), 175
 sim_cycles (*chia.chipyard.state_def.CosimResult* attribute), 139
 sim_opts() (*chia.chipyard.riscv_dv_gen_node.GenSpec* method), 161
 sim_outputs (*chia.firesim.state_def.FireSimRunResult* attribute), 208
 simulator_binary_content (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 simulator_binary_name (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 SINGLE (*chia.chipyard.state_def.TortureMode* attribute), 142
 source_dirty (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 141
 source_sha (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 141
 spawn_query_tool() (*chia.database.base.DatabaseNode* method), 219
 spawn_query_tool() (*chia.database.postgres_node.PostgresNode* method), 221
 spawn_query_tool() (*chia.database.sqlite_node.SQLiteNode* method), 224
 spawn_tool() (*chia.simulators.gem5.Gem5Node* method), 240
 spike_bin (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 141
 spike_sig (*chia.chipyard.state_def.TortureTestRun* attribute), 143
 SpikeBuildArtifact (class in *chia.chipyard.state_def*), 140
 SpikeBuildNode (class in *chia.chipyard.spike_build_node*), 148
 SpikeResult (class in *chia.chipyard.state_def*), 139
 SQLiteExecResult (in module *chia.database.sqlite_node*), 224
 SQLiteNode (class in *chia.database.sqlite_node*), 224
 SQLiteQueryTool (class in *chia.database.sqlite_node*), 226
 SRAMSpec (class in *chia.chipyard.macrocompiler*), 154
 stage_golden_model() (in module *chia.chipyard.spike_build_node*), 149
 start_cache() (in module *chia.base.cache*), 119
 start_router() (in module *chia.base.tools.ChiaTool*), 129
 static_archives (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 141
 stats() (*chia.simulators.gem5.Gem5ToolServer* method), 243
 stderr (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 stderr (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 145
 stderr (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 146
 stderr (*chia.chipyard.state_def.ProgramBuildArtifact* attribute), 136
 stderr (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 135
 stderr (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), 137
 stderr (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 142
 stderr (*chia.chipyard.state_def.TortureResult* attribute), 144
 stdout (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 stdout (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 145
 stdout (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 146
 stdout (*chia.chipyard.state_def.ProgramBuildArtifact* attribute), 136
 stdout (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 135
 stdout (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), 137
 stdout (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), 141
 stdout (*chia.chipyard.state_def.TortureResult* attribute), 144
 stop() (*chia.base.tools.ChiaTool.ChiaTool* method), 129
 stop_cache() (in module *chia.base.cache*), 119
 stop_router() (in module *chia.base.tools.ChiaTool*), 129
 success (*chia.chipyard.state_def.BuildArtifact* attribute), 134
 success (*chia.chipyard.state_def.FireMarshalArtifact* attribute), 145
 success (*chia.chipyard.state_def.FireMarshalBaseArtifact* attribute), 146
 success (*chia.chipyard.state_def.ProgramBuildArtifact* attribute), 136
 success (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), 135
 success (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), 137

- [success](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)
[success](#) (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), [140](#)
[success](#) (*chia.chipyard.state_def.SpikeResult* attribute), [140](#)
[success](#) (*chia.chipyard.state_def.TortureResult* attribute), [143](#)
[success](#) (*chia.chipyard.state_def.TortureTestRun* attribute), [142](#)
[success](#) (*chia.firesim.state_def.BitstreamBuildResult* attribute), [207](#)
[success](#) (*chia.firesim.state_def.FireSimRunResult* attribute), [208](#)
[summarize_o3_pipeview\(\)](#) (*chia.simulators.gem5.Gem5Node* static method), [242](#)
[sweep_both_arches_as_ref\(\)](#) (in module *chia.analysis.sustainability.focal*), [247](#)
[synth\(\)](#) (*chia.esp.esp_workspace.EspWorkspaceNode* static method), [177](#)
- ## T
- [target](#) (*chia.chipyard.state_def.BuildArtifact* attribute), [134](#)
[target](#) (*chia.chipyard.state_def.RiscvBuildArtifact* attribute), [135](#)
[target](#) (*chia.chipyard.state_def.RiscvObjdumpArtifact* attribute), [137](#)
[target_config](#) (*chia.firesim.config.FireSimBuildConfig* attribute), [200](#)
[target_project](#) (*chia.firesim.config.FireSimBuildConfig* attribute), [199](#)
[target_project_makefrag](#) (*chia.firesim.config.FireSimBuildConfig* attribute), [200](#)
[task_options](#) (*chia.base.colocated.ColocatedNode* property), [126](#)
[task_options](#) (*chia.simulators.gem5.Gem5Node* property), [240](#)
[terminate\(\)](#) (*chia.aws.host.EphemeralEC2Host* method), [235](#)
[terminate_ec2_instances\(\)](#) (in module *chia.aws.ec2*), [234](#)
[terminate_on_completion](#) (*chia.firesim.config.FireSimRunConfig* attribute), [203](#)
[test](#) (*chia.chipyard.riscv_dv_gen_node.GenSpec* attribute), [160](#)
[test_binary_name](#) (*chia.chipyard.state_def.RunResult* attribute), [137](#)
[test_binary_name](#) (*chia.chipyard.state_def.SpikeResult* attribute), [139](#)
[test_dump](#) (*chia.chipyard.state_def.TortureTestRun* attribute), [142](#)
[test_s](#) (*chia.chipyard.state_def.TortureTestRun* attribute), [142](#)
[tests](#) (*chia.chipyard.state_def.TortureResult* attribute), [144](#)
[to_answers\(\)](#) (*chia.esp.state_def.EspAccelSpec* method), [171](#)
[to_config_entry\(\)](#) (*chia.models.opencode.AdditionalModelProvider* method), [193](#)
[to_lib_dict\(\)](#) (*chia.vlsi.sram_cacti.sram_characterize.CharacterizedSRAM* method), [212](#)
[to_markdown\(\)](#) (*chia.github.state_def.GithubIssue* method), [230](#)
[to_markdown\(\)](#) (*chia.github.state_def.GithubPullFeedback* method), [231](#)
[ToolInfo](#) (class in *chia.base.tools.ChiaTool*), [128](#)
[tools](#) (*chia.chipyard.state_def.SpikeBuildArtifact* attribute), [141](#)
[torture\(\)](#) (*chia.chipyard.torture_run_node.TortureRunNode* method), [163](#)
[TortureMode](#) (class in *chia.chipyard.state_def*), [142](#)
[TortureResult](#) (class in *chia.chipyard.state_def*), [143](#)
[TortureRunNode](#) (class in *chia.chipyard.torture_run_node*), [163](#)
[TortureTestRun](#) (class in *chia.chipyard.state_def*), [142](#)
[TrackedRef](#) (class in *chia.base.chia_wait*), [122](#)
[transaction\(\)](#) (*chia.database.base.DatabaseNode* static method), [219](#)
[transaction\(\)](#) (*chia.database.postgres_node.PostgresNode* static method), [221](#)
[transaction\(\)](#) (*chia.database.sqlite_node.SQLiteNode* static method), [225](#)
[truncate_gz_trace\(\)](#) (*chia.simulators.gem5.Gem5Node* static method), [242](#)
- ## U
- [uartlogs](#) (*chia.firesim.state_def.FireSimRunResult* attribute), [208](#)
[UnknownAntigravityError](#), [187](#)
[UnknownBedrockError](#), [184](#)
[UnknownClaudeError](#), [182](#)
[UnknownCodexError](#), [188](#)
[UnknownCopilotError](#), [190](#)
[UnknownOpenAIErrror](#), [195](#)
[UnknownOpenCodeError](#), [191](#)
[UnknownVertexError](#), [185](#)
[upload_file\(\)](#) (*chia.aws.s3.S3Node* method), [236](#)
- ## V
- [vcd_node_id](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)

[vcd_path](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)
[vcd_s3_path](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)
[vcd_size_bytes](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)
[verdict](#) (*chia.analysis.sustainability.focal.FocalComparison* property), [245](#)
[VERILATOR](#) (*chia.chipyard.state_def.BuildTarget* attribute), [133](#)
[VERILATOR_DEBUG](#) (*chia.chipyard.state_def.BuildTarget* attribute), [133](#)
[VerilatorRunNode](#) (class in *chia.chipyard.verilator_run_node*), [164](#)
[VertexError](#), [184](#)
[VertexGeminiLLM](#) (class in *chia.models.vertex*), [185](#)
[VertexGenericLLM](#) (class in *chia.models.vertex*), [186](#)
[VLLMLLM](#) (class in *chia.models.vllm*), [197](#)

W

[wait_for_instances\(\)](#) (in module *chia.aws.ec2*), [234](#)
[wait_ready\(\)](#) (*chia.aws.host.EphemeralEC2Host* method), [234](#)
[wal_checkpoint\(\)](#) (*chia.database.sqlite_node.SQLiteNode* static method), [225](#)
[wave_windows](#) (*chia.chipyard.state_def.RunResult* attribute), [138](#)
[WaveWindow](#) (class in *chia.chipyard.state_def*), [133](#)
[with_acc_tile\(\)](#) (in module *chia.esp.esp_workspace*), [172](#)
[workload_name](#) (*chia.firesim.config.FireSimRunConfig* attribute), [202](#)
[workload_name](#) (*chia.firesim.state_def.FireSimRunResult* attribute), [208](#)